ABB 机器人实验台实验指导书

编著：文 丽　　李艳斌　　何 维等



湖南科技大学信息与电气工程学院

前言

工业机器人是面向工业领域的多关节机械手或多自由度的机器装置，它能自动执行工作，是靠自身动力和控制能力来实现各种功能的一种机器。它可以接受人类指挥，也可以按照预先编排的程序运行。

目前工业机器人由主体、驱动系统和控制系统三个基本部分组成。主体即机座和执行机构，包括臂部、腕部和手部。驱动系统包括动力装置和传动机构，用以使执行机构产生相应的动作；控制系统是按照输入的程序对驱动系统和执行机构发出指令信号，并进行控制。另外根据目前科学技术，工业机器人可增加视觉、触觉、力觉或语音功能，能在更复杂的环境下工作，通过算法能实现自我学习，自我编程，最终变为智能型工业机器人。

目前全球主要工业机器人品牌包括瑞典艾波比机器人（ABB），日本安川机器人（Yaskawa），德国库卡机器人（Kuka），日本发那科机器人（Fanuc），日本爱普生机器人（Epson），中国沈阳新松机器人（SIASUN）。其中 ABB 市场占有率最高，其次是 Fanuc，Yaskawa，和 Kuka。

本实验指导书以 ABB 工业机器人为主的工作站作为实验平台，编写书共 11 章节，主要包括实验装置介绍，机器人仿真系统基本使用实验，码垛实验仿真与实际操作实验，机器人各坐标系定义实验，机器人轨迹示教编程实验等 10 节实验内容，如有疏漏、错误之处，望批评指出！

目录

前言 2
第一章　实验装置介绍 4
1.1 系统介绍 4
1.2 设备介绍 6
1.3 编程介绍 17
1.4 注意事项及安全概述 17
第二章　机器人仿真系统基本使用实验 19
第三章　码垛实验仿真与实际操作实验 39
第四章　机器人各坐标系定义实验 52
第五章　机器人轨迹示教编程实验 77
第六章　基于 Socket 通讯控制机器人运动实验 85
第七章　手势识别控制机器人运动实验 105
第八章　语音识别控制机器人运动实验 115
第九章　基于视觉识别控制机器人定位抓取实验 125
第十章　基于 Profinet 通讯控制机器人速度实验 135
第十一章　简单码垛生产线实验 144
第十二章　组态王与 PLC 控制机器人实验 168
第十三章　工业相机物体识别码垛生产线实验 178
第十四章　视觉工程项目软件架构及源码说明 179
附件一 树莓派语音识别源代码 196
附件二 树莓派手势识别源代码 203

第一章 实验装置介绍

1.1 系统介绍

工业机器人工作站主要包括工业机器人、控制与通信系统、工作台架、视觉系统、气动系统、人机交互系统等组成，如图 1 所示：

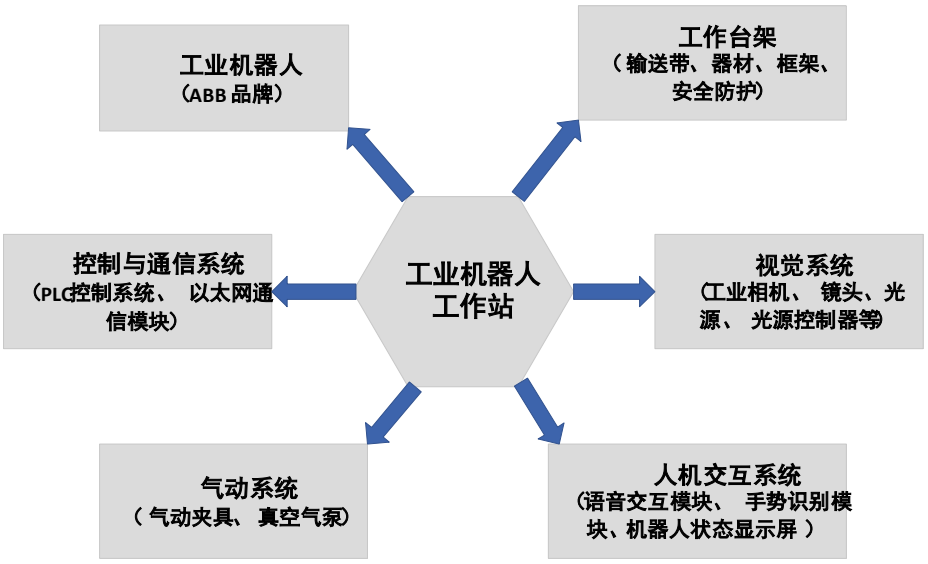


图 1.1 系统组成

其中，工业机器人为 ABB 品牌的 6 轴机器人；控制与通信系统主要包括 PLC 控制系统，以太网通信模块；气动系统包括气动夹具、真空气泵；人机交互系统包括语音交互模块、手势识别模块、机器人状态显示屏等组成。

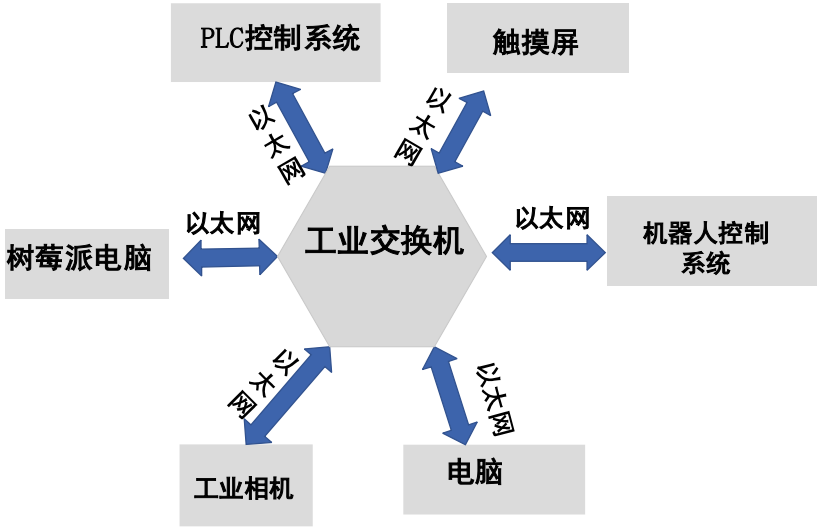


图 1.2 通信系统

工业机器人工作站主要通信为以工业交换机的以太网通信模式。PLC 控制系统与机器人控制系统通过以太网进行数据交换；树莓派电脑与机器人控制系统进行数据交换；工业相机控制系统与机器人控制系统进行数据交换；触摸屏与 PLC 控制系统、机器人控制系统进行数据交换；电脑终端与机器人控制系统、工业相机、树莓派电脑、PLC 控制系统、触摸屏等进行在线编程、修改程序、监控数据、存储数据等功能。

在以太网通信协议中，这些设备必须在同一个局域网网段范围内进行互相访问，每台设备都定义了独一无二的 IP 地址如 ABB 机器人的 IP 地址为：10.70.2. 252，网关为 10.70.2.1，子网掩码为 255.255.255.0 ,DNS:211.67.208.1，在配置过程中 DNS 可以不配置。那么电脑的配置：IP 地址为：10.70.2. 240，网关为 10.70.2.1，子网掩码为：255.255.255.0,DNS:211.67.208.1；在电脑访问每一台已知设备的 IP 时，要求电脑防火墙关闭，通过运行程序，能顺利 PING 通 IP 地址后，才能进行信息交换，否则无法进行数据交换。

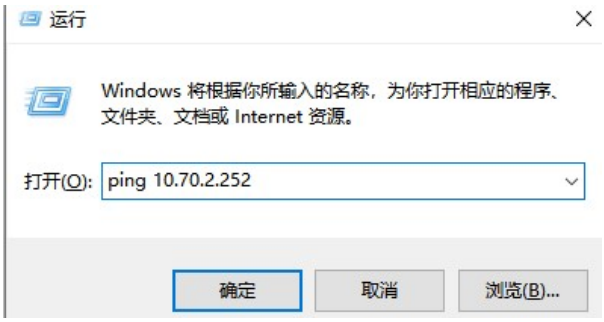


图 1. 3 电脑通信测试

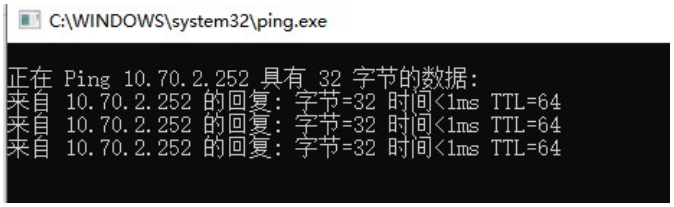


图 1.4 电脑与设备通信正常返回信息

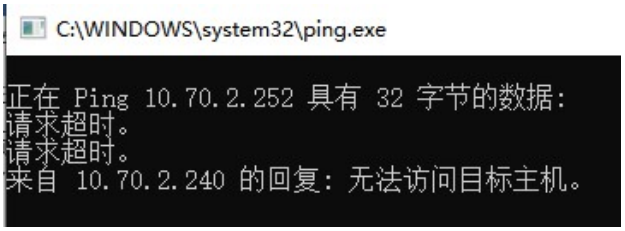


图 1.5 电脑与设备通信错误返回信息

1.2 设备介绍

工业机器人基础教学工作站型号 MFTW-IR 和 CHL-JC-02-A 都是由 ABB 工业机器人为主体控制的工作站。两种型号的工作站都包括工业机器人、PLC 模块、工业相机、光源、镜头、树莓派电脑、工作台、报警灯、急停按钮、安全光栅、空压机等，如图 1.6，图 1.7 所示。

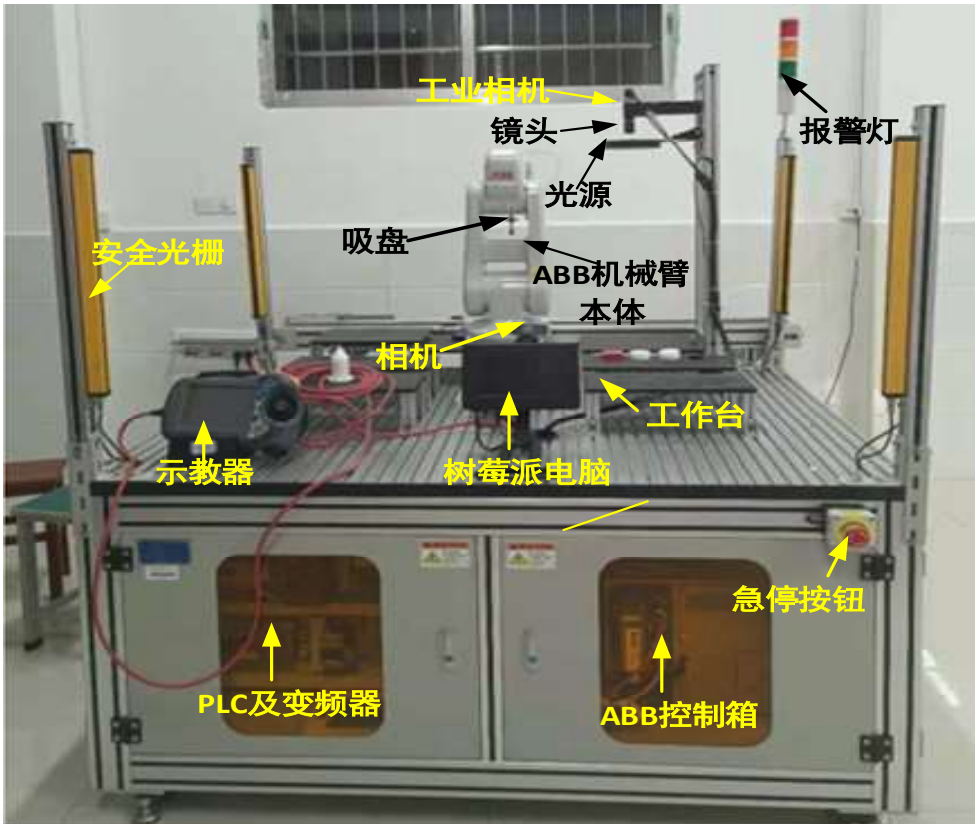


图 1.6　MFTW-IR 工作站实验台图 1

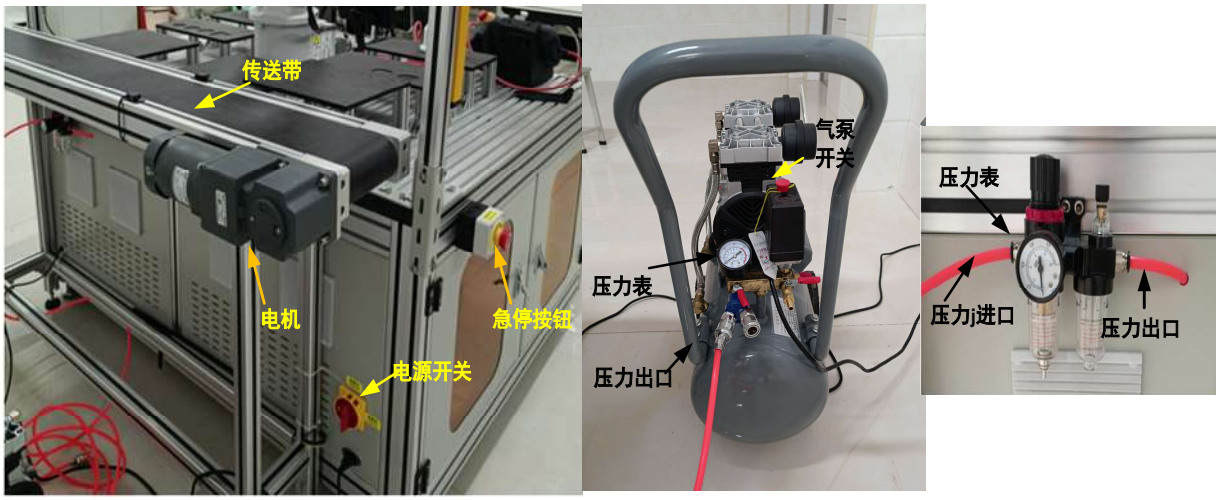


图 1.7　MFTW-IR 工作站实验台图 2

工业机器人基础教学工作站型号 MFTW-IR 和 CHL-JC-02-A 都设计有断路器进行电源的开启和关闭。

两种型号的工作站主要区别是：

1、MFTW-IR 工作站输送带配置的电机为交流 220V，电机速度的控制为变频器面板控制方式，变频器型号为 GD20-OR7G-S2（英威腾 GD20 系列）；CHL-JC-02-A 工作站输送带为步进电机型号为 57HS21A，通过触摸屏手动输入步进电机的速度，控制速度快慢型号为 DM542(蕾赛科技品牌)。

2、MFTW-IR 工作站工作台的尺寸与 CHL-JC-02-A 工作站工作台的尺寸不一样，码垛，堆放物体的托架尺寸也不一样。

1.2.1 工业机器人

工业机器人为 ABB 六轴工业机器人，PLC 控制系统中上位机软件通过网络连接的方式可以直接读写 PLC 模块中的数值，PLC 程序中与机器人建立了 socket 通讯可以直接发送数据来控制机器人移动，在 PLC 上位机软件中设置了权限，普通用户只能对机器人的第 1 轴，第 2 轴操作控制，登录管理员模式，可以对机器人 6 轴完全控制。工业机器人包括本体、控制柜、示教器三部分组成。

A、工业机器人本体

本体作为工作站的核心与动作执行器，其主要功能是完成控制系统所产生的一系列行为决策。系统运动准确、可靠、稳定，且在运行过程中，系统设计有多重防碰撞、防伤人的传感模块。此外，系统设置有紧急停止开关与柔性控制功能，防止机器人失控，保障系统安全，本体上主要安装 6 台伺服电机，减速器和皮带轮以及预留的管道和控制线路接口。

本体型号为 IRB120，如图 1.8 所示。该本体为柔性生产低负载常用本体，本体质量相对较轻，仅重 25kg，有效载荷负重 3kg，垂直腕 4kg，可满足实验室日常使用需求。机器人本体安全、可靠，防护等级达 IP30，可有效防止意外事故的发生。机器人手腕部分集成有 10 路数字信号，可实现各类传感器的安装，集成了 4 路空气气路，可有效实现气动控制。此外，系统设置有 2 通道安全回路监测和 3 位启动装置，系统对 EMC/EMI 具有辐射屏蔽功能。



图 1.8　IRB120 机器人本体

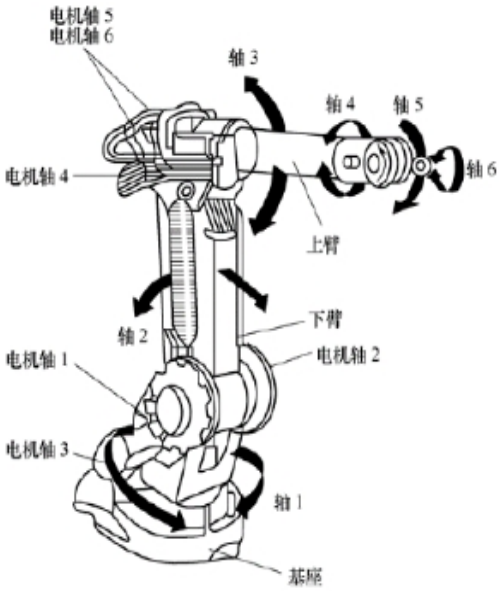


图 1.9 工业机器人的机械结构

机器人本体主要参数如下：

（1）物理特性

1. 机器人底座尺寸 180mm×180mm
2. 机器人高度 700mm
3. 重量 25kg
4. 工作范围 580mm；
5. 有效荷重 3kg；
6. 重复定位精度 0.01mm。

（2）电气特性

1. 电源电源 200-600V，50/60Hz；
2. 变压器额定功率 3.0kVA；
3. 功耗 0.25kW。

（3）运动特性

1. 1 轴：+165°~-160°，最大速度 250°/s；
2. 2 轴：+110°~-110°，最大速度 250°/s；
3. 3 轴：+70°~-90°，最大速度 250°/s；
4. 4 轴：+160°~-160°，最大速度 320°/s；
5. 5 轴：+120°~-120°，最大速度 320°/s；
6. 6 轴：+400°~-400°，最大速度 420°/s；

（4）使用环境

1. 运行环境温度+5℃~+45℃；
2. 运输与存储时温度-25℃~+55℃；
3. 使用时最高许可湿度 95%
4. 噪声水平小于 70dB

B、控制柜

工作站所选用的控制柜型号为 IRC5Compact，该控制柜中有控制计算机和伺服控制器。控制计算机通过对驱动系统的控制，使执行系统按照规定的要求进行工作。控制硬件主要包括工控机、多处理器系统、PCI 总线、大容量闪存盘、停电备用电源、USB 存储接口。控制软件为成熟可靠的实时 OS，采用高级 RAPID 编程语言进行编程，系统支持 PC- DOS 文件格式。吸引电源采用频率为 50-60Hz

单相 200V-600V 交流电，控制轴数为 6 轴，系统安全防护等级为 IP20。此外，系统还设置有安全紧急停机、带监测功能的双通道安全回路、3 位启动装置等、5 路电子限位开关安全输出、8 路安全输入与监测输出。输入输出 IO 口最多 4096 路信号、支持 24VDC 或继电器数字信号，2 路 0-10V 模拟信号，1 路 RS232 或 RS422 串口，10/100MB/S 以太网等。

控制柜外部如图 1.10 所示，控制柜上预留有各种插槽接口、按钮开关、指示灯。如：示教器连接插槽用来连接示教器和控制柜，模式转换开关可以切换控制机器人的两种方式即自动控制和手动控制，在示教编程和系统设置情况下，必须切换为手动模式条件下才能进行编辑和更改。紧急停止按钮是用于出现危险情况下，机器人停止工作的保护装置。

抱闸按钮是保证机器人的各轴姿态在不通电的情况下，电机通过合上抱闸，保证电机位置不变，保证机器人各轴不会受重力而跌落。如果在一些紧急情况下，不能通过示教器操作移动机器人，此时需要通过手动松抱闸来移动机器人。松抱闸前，必须做好安全防护。松抱闸前做好好吊装准备，以免松抱闸后机器人跌落砸伤人员。

1)若控制柜有电，则可以通过按机器人本体上抱闸按钮来移动机器人。

2)若此时控制柜无电，即控制柜无法给电机抱闸提供 24V 控制电，此时最快捷方法为：.打开电机盖，将电机的动力线插头拔下，根据电路图找到抱闸端子，直接提供 24V，抱闸即可打开。



图 1.10 机器人控制柜

C、示教器

编程示教器采用彩色图像界面触摸屏与操纵杆，示教器可热插拔，支持左右手操作。工业机器人程序输入方式有两种：第一种利用电脑软件编程输入程序。第二种脱离电脑利用示教器进行编程。

在 ABB 工业机器人系统中同样配备了示教器，在示教器上通过手动模式编程控制工业机器人的运动，现对其进行简单的介绍，如图 1.11 所示。



图 1.11　示教器



图 1.12 示教器的正确手持方式

A 为连接电缆，连接了示教器和控制柜。

B 为触摸屏。

C 为急停开关，当所控制的工业机器人出现了危险运动情况时可以通过急停开关紧急停止当前运动。

D 为手动操作摇杆，在手动操作模式下可以通过手动操作摇杆来进行机器人各个轴运动的控制。

E 为数据备份用 USB 接口，用于将备份好的数据在其他的存储设备之间传输。

F 为使能器按钮，有两个键程，第一个键程按下时可以在手动操控模式下控制电机开启。

G 为触摸屏用笔。

H 为示教器复位按钮。

对于惯用手为右手的人来说，应左手握示教器，四指按在使能器按钮上，右手进行屏幕和按钮的操作，其具体的正确的手持方式如图 1.12 所示。

1.2.2 通信系统

机器人通信系统集成在控制柜上，提供 RS232，以太网等通信方式，提供部分详细通信协议，用以太网通信编程可以直接控制机器人的动作并实时监测机器人的运动过程状态。

PLC 控制系统模块均采用西门子 200 SMART PLC，MFTW-IR 工作站配置 PLC 型号为 CPU SR20 ,扩展模块为 EMQR16 和 EMDR16,；CHL-JC-02-A 工作站配置 PLC 型号为 CPU ST40。

1.2.3 视觉系统

视觉系统包括 130 万像素工业相机、600 万像素工业镜头、环形光源及数字可编程光源控制器。视觉系统通过以太网与外界进行通信。

视觉系统包含一台工业相机、工业定焦镜头、环形光源和光源控制器，相机安装在机械臂末端，如图 1.13 所示。

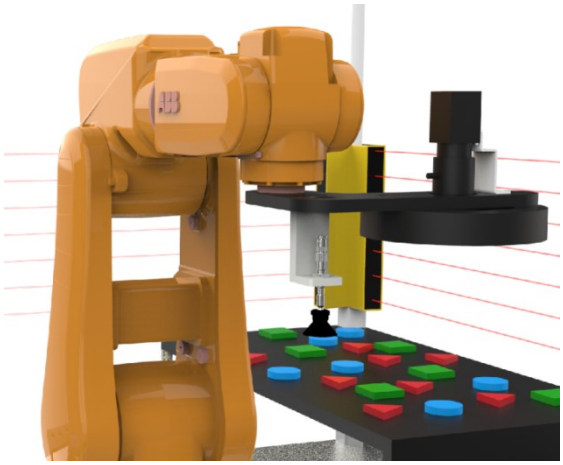


图 1. 13　视觉系统安装示意图

相机选择海康威视 130 万像素新款工业相机 MV-CA060-10GC，如图 1.14 所示。相机支持千兆以太网接口提供了 1Gbps 带宽，无需中继最大传输距离可到 100m，板上缓存达 128MB，可缓存多张图片，用于突发传输或者重传，支持自动曝光控制、用户自定义查找表 LUT、Gamma 校正等，可使用硬触发或者软触发的方式来完成多台相机或者相机与外部设备之间的同步。同时可以配合不同的曝光模式完成相机成像采集，彩色相机植入优异的图像插值算法，更好的颜色还原，并且支持自动白平衡，兼容 GigE Vision 协议 V1.2，无缝链接第三方软件。



图 1.14 工业相机 MV-CA060-10GC

相机参数如表 1 所示：

表 1　MV-CE013-50GC 参数表

参数类型	参数值
型号	MV-CE013-50GC
名称	130 万像素 1/3” CMOS 千兆以太网工业面阵相机
传感器型号	Sharp RJ33
像元尺寸	3.75 μm × 3.75 μm
靶面尺寸	1/3”
分辨率	1280 × 960
帧率	30 fps
动态范围	60 dB
信噪比	37 dB
增益范围	0 dB ～ 20 dB
曝光时间范围	34 μs ～ 1 sec
快门模式	支持自动曝光、手动曝光、一键曝光模式
数据接口	Gigabit Ethernet（1000 Mbit/s）兼容 Fast Ethernet（100 Mbit/s）
数字 I/O	6-pin Hirose 接头提供供电和 I/O:1 路光耦隔离输入(Line0)，1 路光耦隔离输出(Line1)，1 路双向可配置非隔离 I/O（Line2）
缓存容量	128 MB
黑白/彩色	彩色
接口	GigE
供电及功耗	电压范围 5～ 15 VDC；支持 PoE 供电，功耗＜2.6 W@12 VDC

温度	工作温度 0～50℃，储藏温度-30～70℃
外形尺寸	29 mm × 29 mm × 42 mm
重量	约 68g
镜头接口	C-Mount
软件	MVS 或者第三方支持 GigE Vision 协议软件

工业镜头选择海康威视 MVL-HF1628M-6MP 600 万 16mm 镜头，镜头具有超高清晰度，支持最小 2.4μm 像元，外形采用超紧凑设计，如图 14 所示，具有优秀的抗振能力和高低温性能，镜头光圈为 F2.8，最大靶面 1/1.8 英寸，可支持 3MP/1.3MP/3MP/6MP /10MP 相机，低畸变，支持超短工作距，并在整个区间内保持优秀光学性能，针对机器视觉光源和芯片特殊优化，表面多层宽带镀膜，保证可见光和近红外波段的高透过率，全系列量产，可选焦距范围 6-50mm。

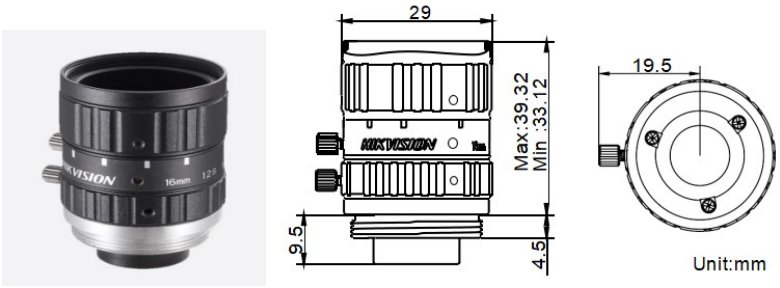


图 1.15 工业镜头 MVL-HF1628M-6MP

主要技术参数如下：

表 2　MVL-HF1628M-6MP 工业镜头参数

参数类型	参数值
型号	MVL-HF1628M-6MP
名称	固定焦距，手动光圈，六百万像素，FA 镜头
焦距	16mm
光圈	F2.8-F16
像面尺寸	Φ9mm(1/1.8”)
光学畸变	-0.38%
视场角	1/1.8”
外形尺寸	Φ29×35.36mm
最近摄距	0.1m

法兰后焦	17.526mm
滤镜螺纹	M27*0.5

光源选择 LED 环形光源 RN15060-W，其采用铝合金外壳、自然风冷、带漫射板。

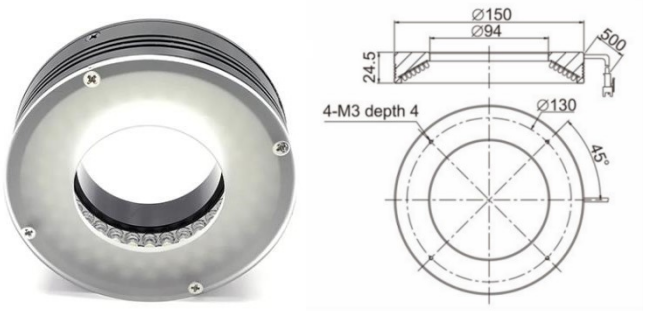


图 1. 16 环形光源 RN15060-W

主要技术参数如表 3 所示：

表 3　环形光源参数表

参数类型	参数值
产品型号	RN15060-W
产品类型	60 度环形光源
LED 类型	贴片 LED
颜色	红色
波长	--
色温	6600K
功率	14.4W
输入电压	DC 24V max.
线缆长度	0.5m
连接器型号	SMR-03V-B
连接器定义	1：正极（+）红色；2：留空；3：负极（—）黑色；
外壳材质	铝合金（表面氧化发黑处理）
冷却方式	自然风冷
外形尺寸	外径 150x 内径 94x 高 24.5mm
使用温度和	
湿度	温度: 0 to 40°C，湿度: 20 to 85%RH(非凝结)

1.2.4 输送带系统

输送带系统主要包括电机、输送带、光电等，电机可以通过上位机程序软件控制调速，完成多机器人物品传输。

工作站台上安装输送带，通过电机控制输送带的运转并进行调速，完成两套或四套工作机器人工作站之间的物料传输。考虑安全性，工业机器人工作站采用错位布置，因此，输送带模块置于两工作站正中间区域。

输送带模块主要包括输送带、调速电机、控制箱、变频器、光电传感器等，其中调速电机供电电压为单相 220V 交流电，最大功率为 200w，最大电流为 1.69A，调速范围为 0-180r/min。输送带采用铝合金边框，长宽为 3000mm×230mm，线速度控制在 0-0.5m/s。光电传感器器采用红外对射感应模块，输入电压为 DC12-24V，输出电流为 300mA，感应距离为 20-350cm，可以检测 1mm 以上任何除无色透明外的物体。光电编码器指向角为 3-5°，响应时间在 2ms 以内。



图 1. 17 输送带主要模块

1.2.5 气动系统

气动系统主要包括电磁阀、气动夹具与真空气泵等外设，用于气动夹具的拾取与释放等操作，系统在机器人本体末端安装气动夹具。

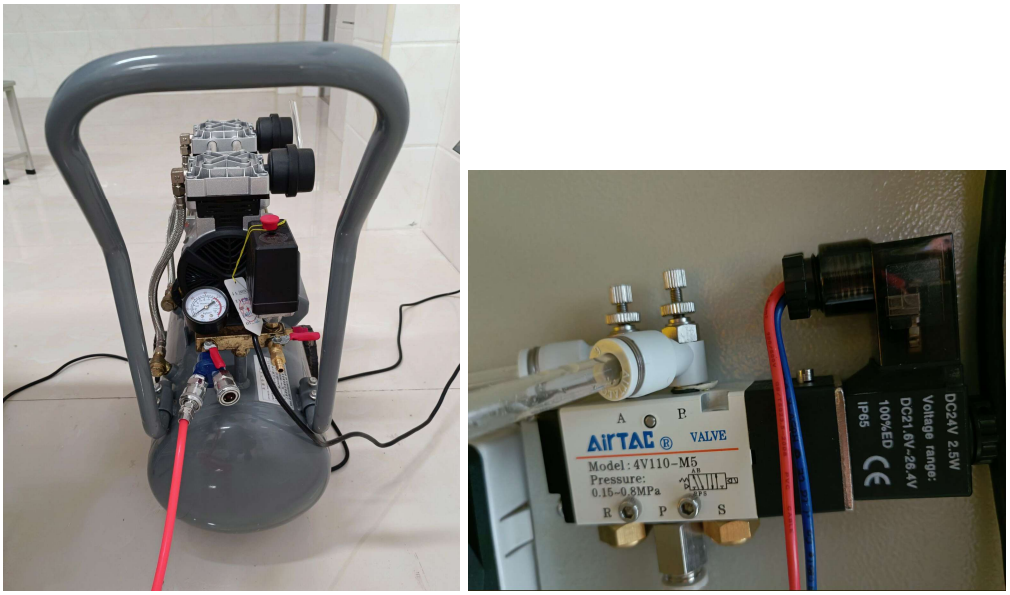


图 1. 18 气动系统主要元器件

1.2.6 安全系统及其他

为了防止机器人系统在学生无故进入机器人工作范围而造成意外伤害，系统增加红外光栅模块，即当系统运行时，红外光栅系统自动启动，当人为进入光栅范围，机器人将紧急停机并预警，保障系统整个实验的安全。此外，工作台架还主要包括用于显示机器人状态的高清显示屏、语音与图像识别模块、用于工业机器人基础实验的码垛、拆跺、轨迹跟踪、装配等所需模块与模具，以及工业机器人视觉检测所需的外设模块。工作台主要分为两大区域，即基础操作区与视觉自由实验区。基础实验区各工件卡位位置固定，主要完成基础示教的搬运、码垛、装配与轨迹跟踪，而视觉自由实验区为开放性实验区，实验者可以随意摆放工件，可以完成视觉标定、检测、识别与智能搬运等实验。

1.2.7 人机交互模块

人机交互模块主要包括三大模块，即实时显示模块，语音交互模块与手势图像识别模块。实时显示模块为工控 PC 机输出显示，系统配置机器人运动检测软件，实时查看机器人运行状态，动作类型等，为人机交互中的机器输出。语音交互模块配置自然语言处理识别模块，完成自然语言高质量输入与预处理，用户可根据自然语言处理算法与识别结果进行机器人显示与动作执行，完成人机交互。手势图像识别系统为单独模块，模块采用 Android 系统，配置 200W 摄像头，采用 7 寸液晶显示屏，分辨率为 1280×800，配置网络设备，实现系统内通信。

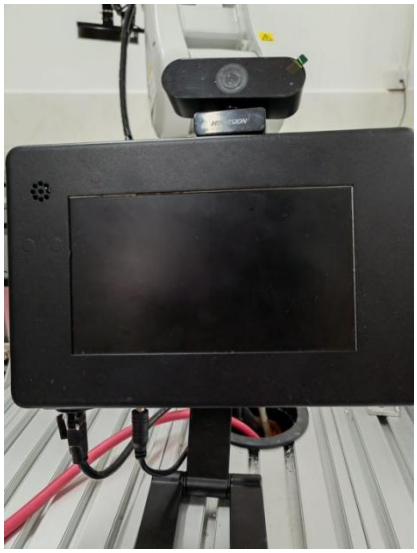


图 1. 19 手势识别与语音交互集成模块

1.3 编程介绍

ABB 机器人的编程分为电脑软件编程和示教器上进行编程。电脑软件编程需要安装 RobotStudio 和 RobotWare。RobotStudio 是一个集成机器人在线编程和离线仿真的软件，同时兼具了代码备份，参数配置还有系统制作功能。RobotWare 是机器人系统库文件的总成，在运行 RobotStudio 时，调用 RobotWare 里面的库文件实现机器人运动的各种功能。在安装 RobotStudio 软件版本时，软件中自动安装了相应版本的 RobotWare 软件。有时候则需要独立安装其他版本的软件。

RobotStudio 软件编程比较方便，在软件界面中可以根据需要选择各种型号的 ABB 模型库和工装夹具等，在没有实物机器臂的情况下完全可以通过仿真开发出来，节约了设计周期。在针对复杂编程时，示教器无法比拟。示教器采用高级 RAPID 编程语言进行编程，控制器的系统配置是出厂时固定的，不能随便修改。RAPID 编程语言属于动作语言，包含了一连串的控制机器人的指令。还包括主模块和程序模块的之间的相互调用功能。

RobotStudio 软件主要使用步骤：

打开 RobotStudio 桌面快捷键→新建→空工作站解决方案→ABB 模型库选型号→导入模型库选工装夹具→从布局创建系统（选 RobotWare 版本）→配置机器人系统库→打开虚拟示教器→切换手动模式→打开使能按钮（Enable）→定义 I/O 变量→定义程序数据变量、常量→程序编辑器编写程序模块（调用 RAPID 编程指令）→程序编辑器编写主程序（调用 RAPID 编程指令）→示教修改位置→调式单步运行→调试连续运行→切换自动运行

示教器编程主要使用步骤：

打开电源启动控制柜→切换到手动状态→打开控制面板→配置系统参数（通信配置等）→定义 I/O 变量→定义程序数据变量、常量→程序编辑器编写程序模块（调用 RAPID 编程指令）→程序编辑器编写主程序（调用 RAPID 编程指令）→示教修改位置→调式单步运行→调试连续运行→切换自动运行

1.4 注意事项及安全概述

机器人速度快，质量重并且力度很大。运动过程中的停顿或停止都会产生危险。即使可以预料到运动轨迹，但外部信号有可能会改变操作，会在没有任何警

告的情况下，产生料想不到的运动。因此，当机器人在运动或进入保护空间时，务必遵循所有的安全条例。

1.4.1 自身安全

为了安全操作机器人系统，请遵循下列原则：

1. 操作机器人前，需要确认“紧急停止”按钮功能是否正常。
2. 如果在保护空间内有工作人员，请手动操作机器人系统。
3. 当进入保护空间时，请准备好机器人示教器或急停，以便随时控制或停止机器人。
4. 在示教与维护作业中，绝不允许操作人员在自动运动模式下进入机器人工作范围内，决不允许其他无关人员进入机器人运动范围内。
5. 注意液压、气压系统以及带点部件。即使断电，这些电路上的残余电量也很危险。
6. 机器人通电后，保持实验台关闭，且身体的任何部位不要进入机器人运动范围之内。
7. 机器人运动时始终从机器人的前方进行观察，不要背对机器人进行作业，以便机器人工作不正常时，及时按下工作台、控制柜或示教器上的急停按钮。
8. 始终按预先制定好的操作程序进行操作。
9. 机器人工作时，操作人员注意查看机器人电缆状况，防止其缠绕在机器人上，同时严禁在控制柜内随便放置配件、工具、杂物等。
10. 机器人工作时，严禁操作者离开现场，以确保意外情况的及时处理。

1.4.2 安全工具

机器人工作站配备了急停开关、安全光幕等。紧急停止和安全保护光幕机制受到监控，以便在控制器检测到任何故障时停止机器人。

1.4.3 维护控制器时确保电源关闭

高压作业可能会产生致命性后果。触碰高压可能会导致心跳停顿、灼烧或其它严重伤害。为了避免这些伤害，请务必在作业前关闭控制器上主电源开关或整个设备断电。

第二章　机器人仿真系统基本使用实验

实验目的：学习 RobotStudio 软件的基本使用方法。

实验要求：掌握 RobotStudio 软件基本使用方法，完成简单基本动作指令。

实验步骤：

1、鼠标双击打开电脑桌面 RobotStudio 6.06 或 RobotStudio6.08 软件图标，界面打开如下图所示，如果界面不是图中所示，点击任务栏的“文件”选项：



图 2.1　RobotStudio 6.06 软件打开界面

2、工作站选择创建“空工作站解决方案”，解决方案名称处进行方案的名称更改，方案的名称只能是英文和数字的组合，中文名称软件系统不兼容，同时可以在位置处进行选择需要保存解决方案文件的路径，更改好名称和文件路径后，点击下方的创建按钮，一个新的空工作站解决方案创建完成，如下图所示：

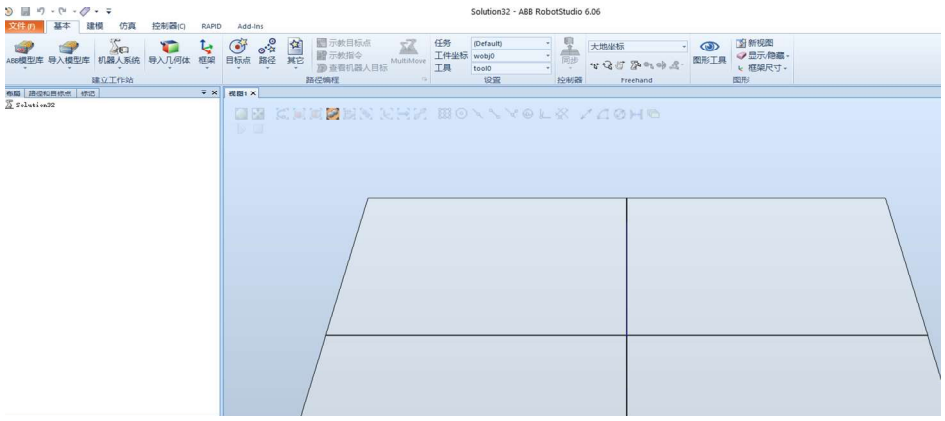


图 2.2 新创建的空工作站解决方案

3.点击任务栏“ABB 模型库”选择型号为 IRB 120 机器人添加到解决方案，添加后的机器人如图所示：

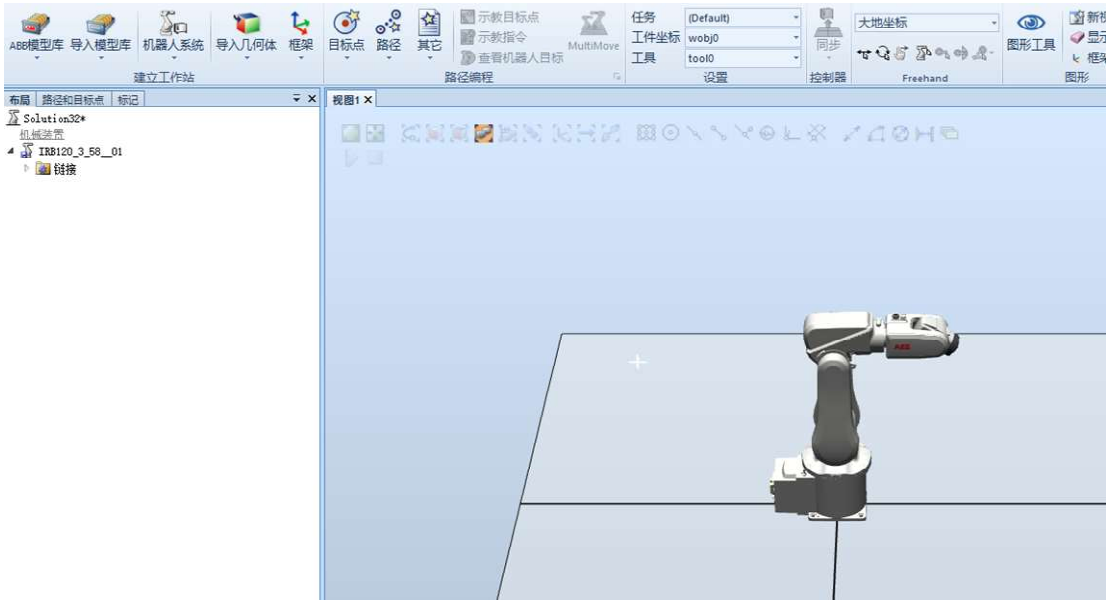


图 2.3 解决方案添加机器人模型

4.点击任务栏“机器人系统”选择从布局创建系统，如下图所示：

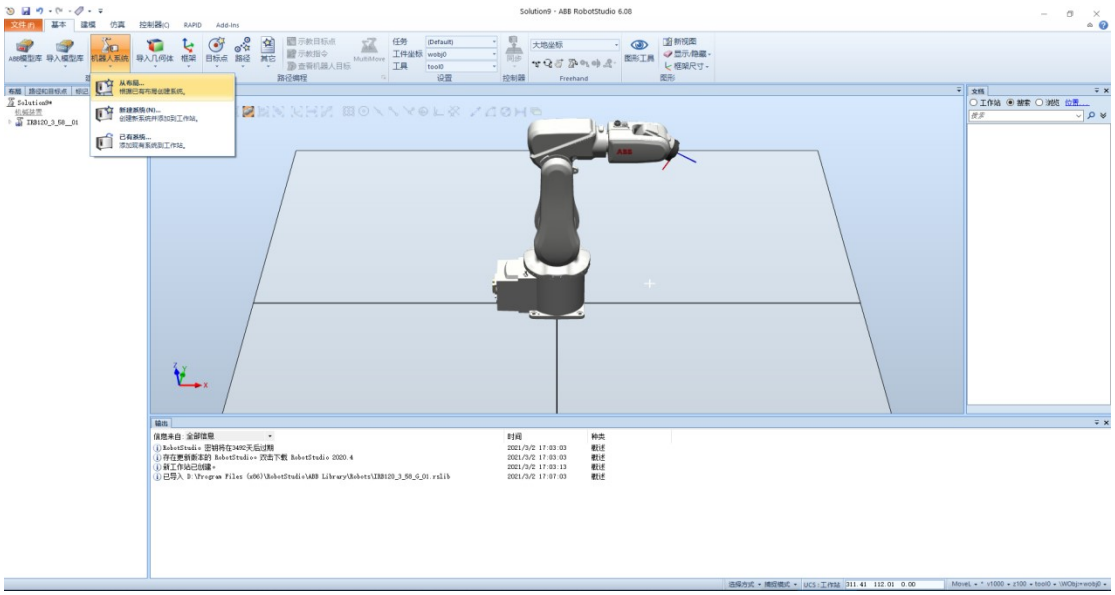


图 2.4 创建机器人系统

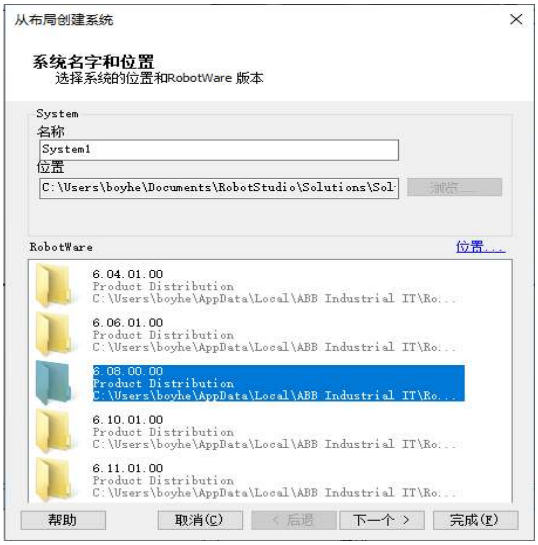


图 2.5 RobotWare 系统版本选择

可以在名称处进行系统名称的更改，RobotWare 版本选择 6.06.00.00 版本，点击下一个，机械装置选项保持默认继续下一个，过程如下图所示：

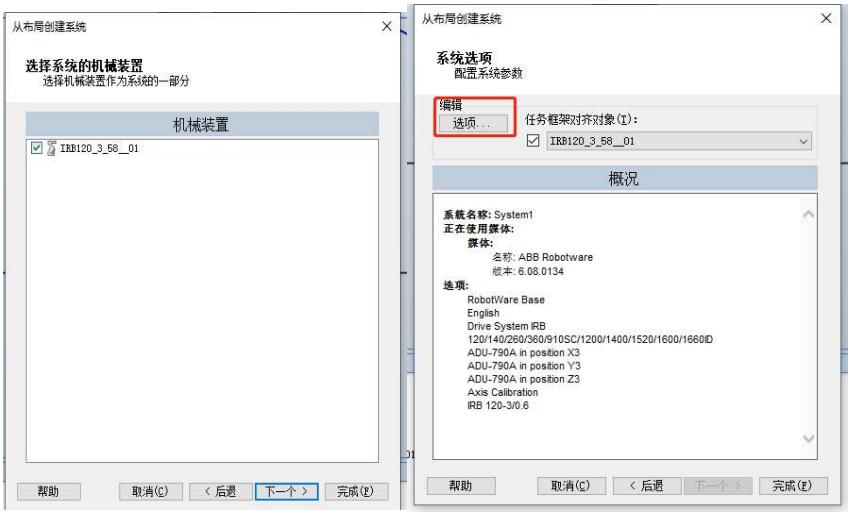


图 2.6 选项功能选择

在系统选项界面，选择编辑处下方的“选项”打开后，将以下功能选项勾选，Default Language 勾选 Chinese，Industrial Networks 勾选 709-1 DeviceNet Master/Slave、969-1 PROFIBUS Controller，Motion Events 勾选 608-1 World Zones，Communication 勾选 616-1 PC Interface、617-1 FlexPendant Interface，Engineering Tools 勾选 623-1 Multitasking，Functional Safety 勾选 996-1 Safety Module、1125-2 SafeMove Pro，其余的选项功能不用勾选，以上选项是日常学习和工业开发中最常用的功能选项。勾选完毕后点击确定，如下图所示：

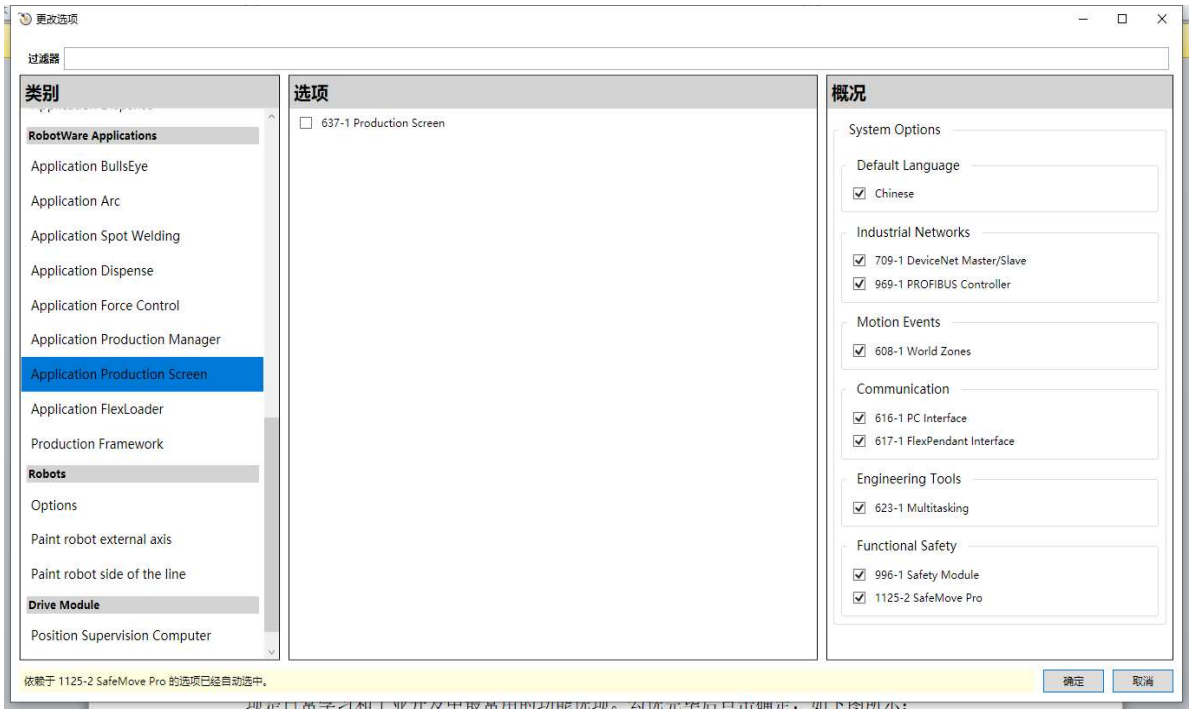


图 2.7 选择创建的机器人系统的选项功能

点击完确定后，机器人功能选项选择完毕，点击完成，机器人系统自动启动，启动完成后，一个含有机器人控制系统的解决方案就创建成功，如下图所示：

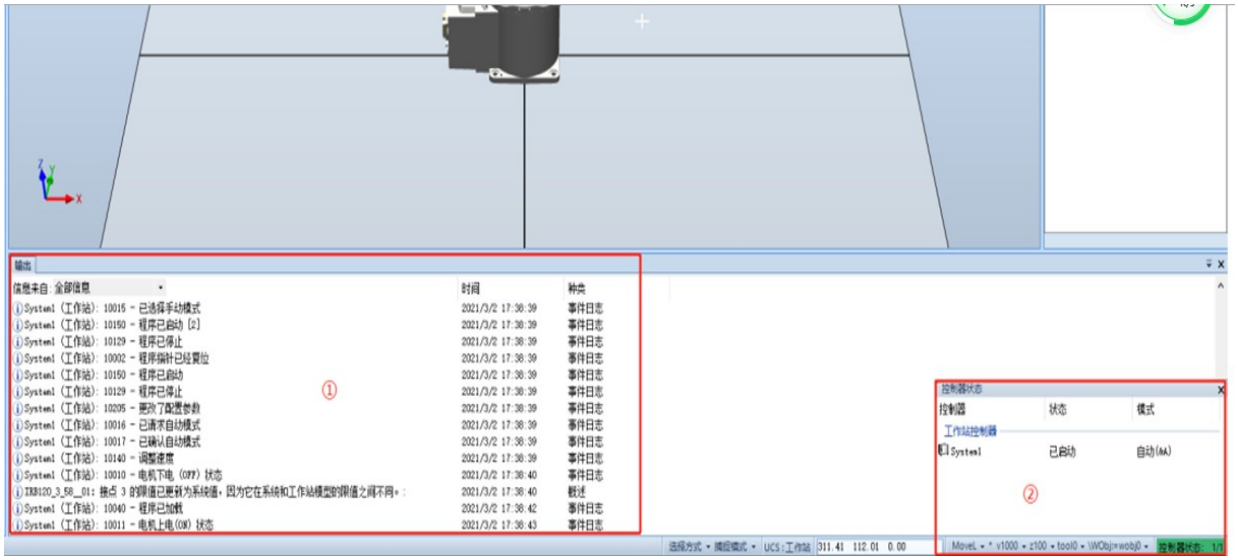


图 2.8 创建完毕的机器人控制系统

创建完成后，机器人默认的状态为“自动运行”模式，可以通过红框“②”看控制器的状态，通过红框“①”输出信息来看机器人系统创建的每一步输出信息是否正确。

创建完机器人系统后就可以通过创建系统的虚拟示教器进行控制机器人移动，操作方法如下：点击任务栏“控制器”选项，点击选项下面的“示教器”功能，选择“虚拟示教器”如下图所示：

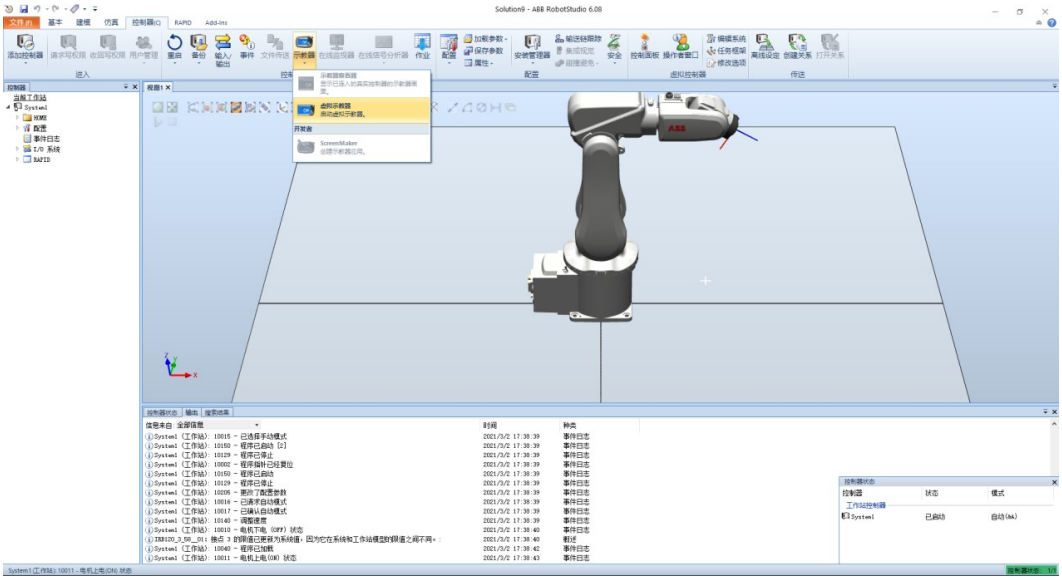


图 2.9 启动虚拟示教器

虚拟示教器打开界面如下图所示：

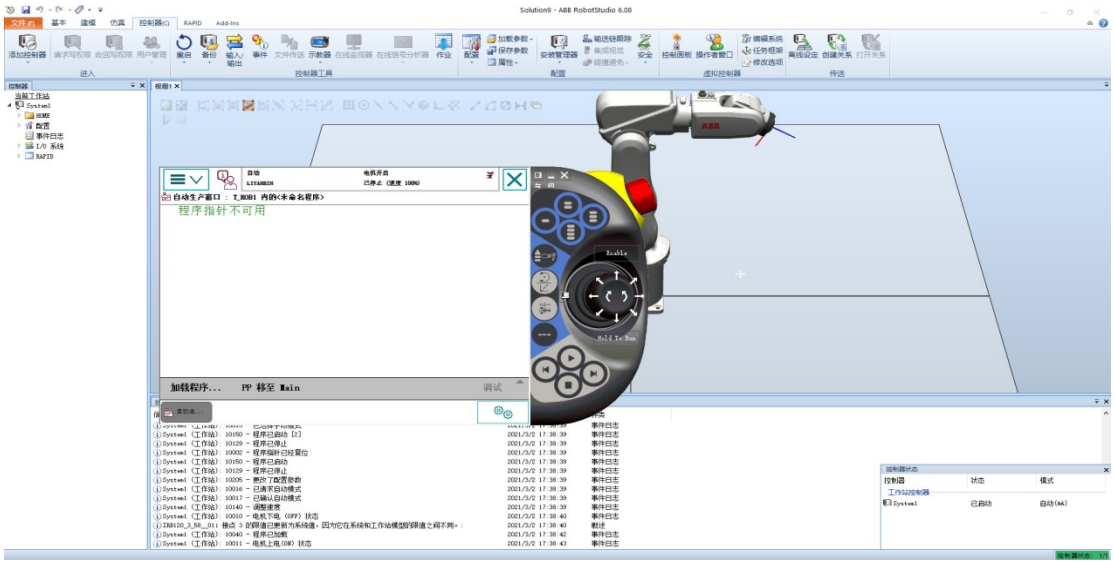


图 2.10 虚拟示教器页面

点击机器人运行模式按钮，将机器人自动模式改为手动模式：如下图所示：



图 2.11 模式切换

改为手动模式后，点击虚拟示教器使能按钮“Enable”就可以通过虚拟示教器摇杆控制机器人移动，如下所示：



图 2.12 虚拟示教器使能按钮

基于已建立的仿真系统解决方案，打开创建的解决方案，通过虚拟示教器控制，将机器人运动模式切换到手动模式，打开机器人电机使能按钮，切换到轴坐标运行。通过键盘 Ctrl+鼠标左键长按可以平移机器人工作站平面。Ctrl+Shift+鼠标左键长按可以旋转机器人工作站平面。在手动操纵菜单界面下切换到轴 4-6，通过移动摇杆将机器人的位姿调整为轴 1、轴 2、轴 3、轴 4、轴 6 角度都为 0°，轴 5 为 90°，其调整完的姿态如下，同时示教器中机器人轴坐标位置如下图所示。

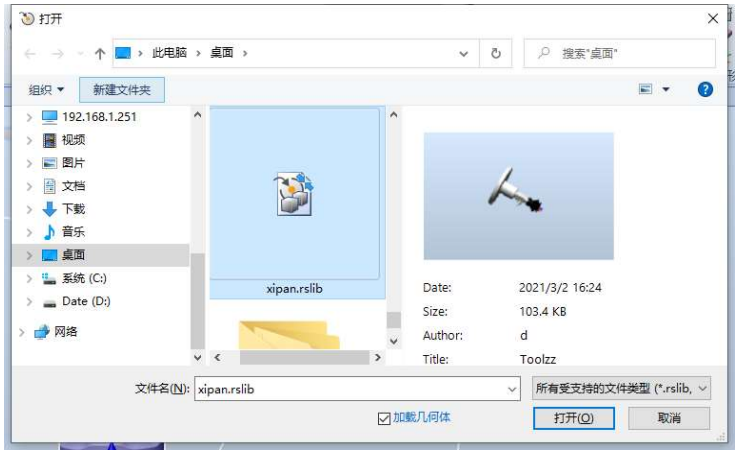


图 2.13 机器人轴坐标数据显示

为后面实验方便，需要增加一个吸盘模型，吸盘模型已提前创建好，直接添加即可，任务栏选择“基本”，选择导入模型库，选择浏览文件库，打开吸盘模型所在文件即可。操作如下图所示：



(a)



(b)

图 2.14 导入吸盘模型

打开吸盘模型文件后会在布局栏中显示导入的吸盘文件（导入其他工装夹具或在设备菜单中找各种工具、输送链、弧焊设备也类似），同时也会在创建的工作站中显示吸盘模型。右键选中吸盘模型，选择安装到对应的机器人模型上面，操作如下图所示：

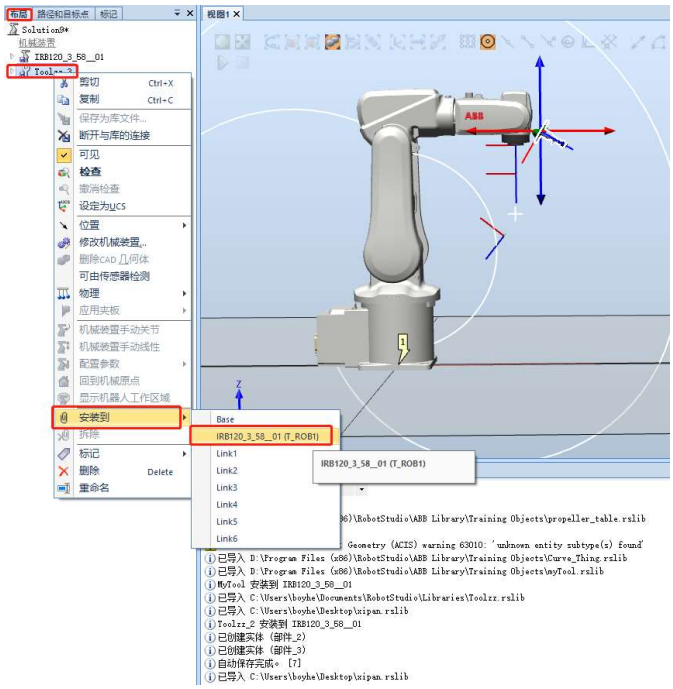


图 2.15 安装吸盘到法兰盘

弹出是否确认更新吸盘位置，选择“是”，吸盘就会安装到机器人法兰盘末端。操作如下图所示：

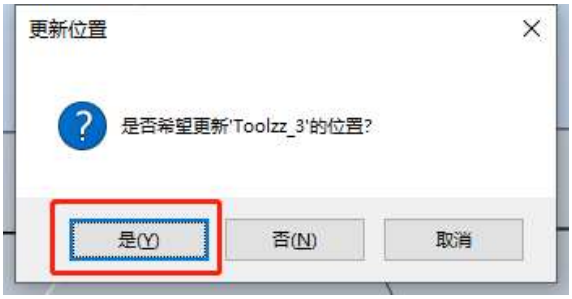


图 2.16 选择是否变更吸盘位置

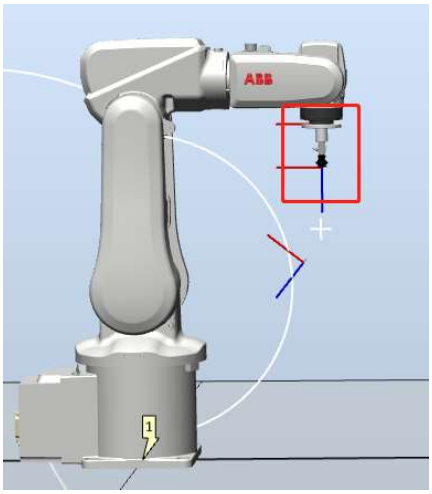


图 2.17 安装吸盘后的机械手

调整完机器人姿态和安装完吸盘后，在任务栏选中“建模”，鼠标右键选中创建的机器人模型，选中显示机器人工作区域，操作如下图所示：

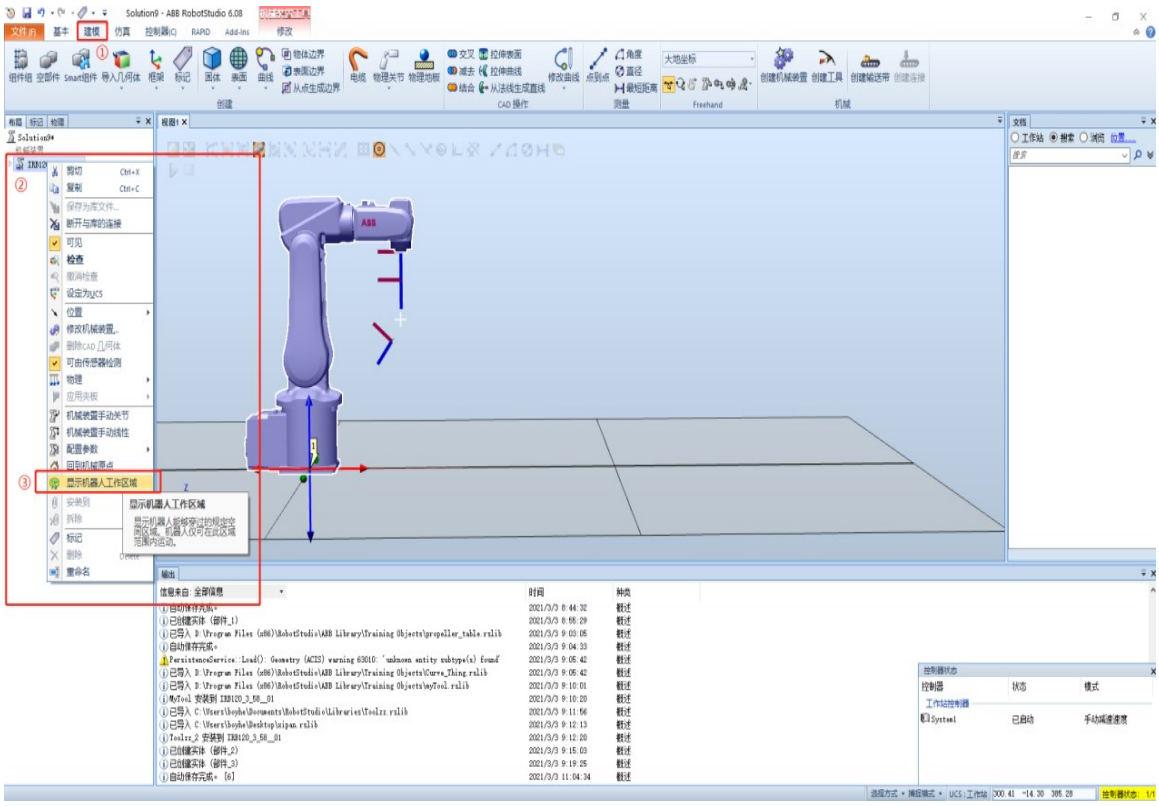


图 2.18 显示机器人工作区域

选择当前工具，勾选 2D 轮廓，3D 体积，勾选后机器人模型会显示当前以工具坐标为基点的机器人工具工作范围，操作如下图所示：

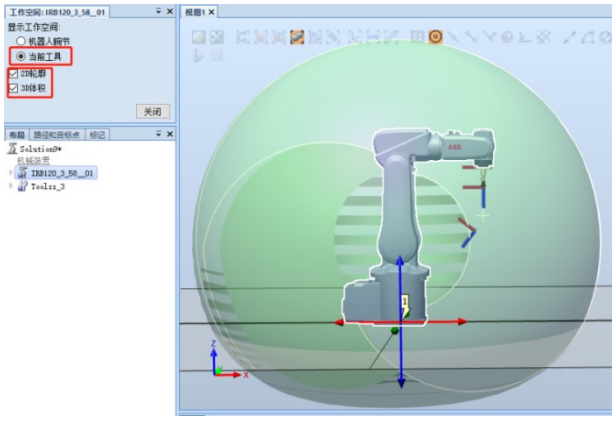


图 2.19 显示机器人工作区 3D 轮廓

确定机器人工具坐标范围后，创建一个机器人工具工作的矩形台体，打开任务栏“建模”，选择固体，点击创建矩形体，操作如下图所示：

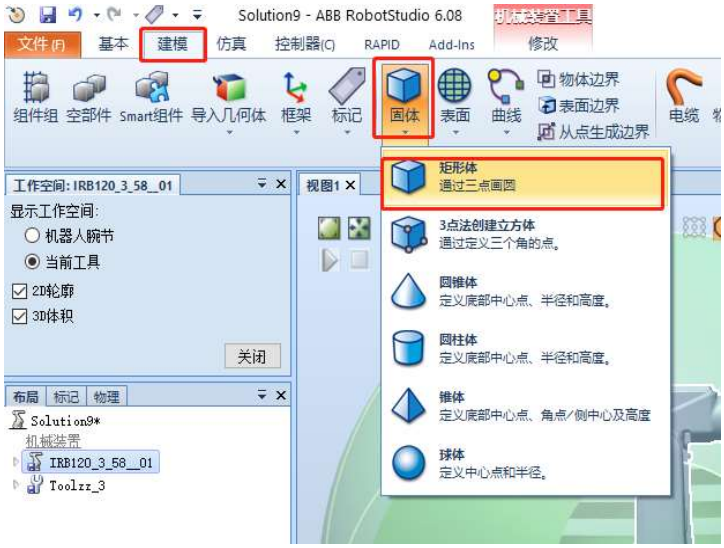


图 2.20 选择创建矩形体

创建一个方体，参考选择“world”坐标系，角点默认为 0，方向为 0，长度 400，宽度 300，高度 300，点击创建。操作如下图所示：

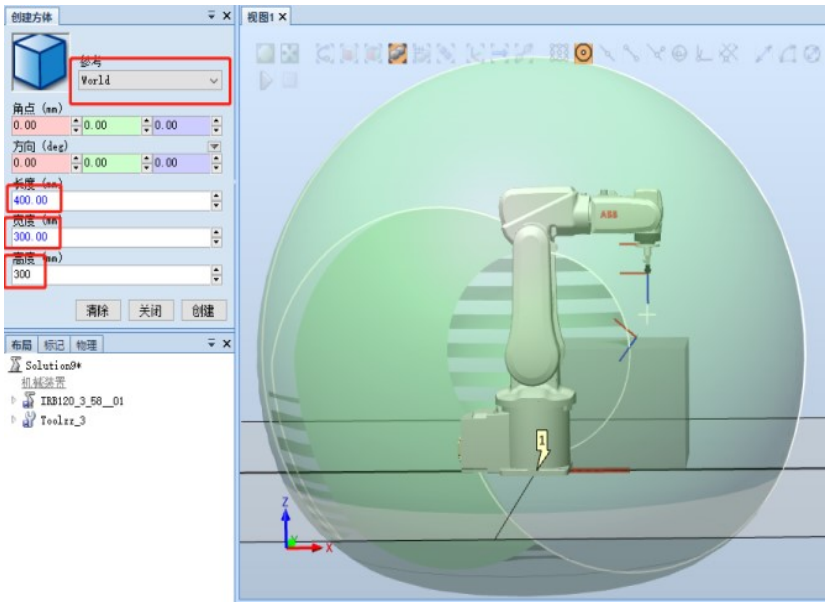


图 2.21 创建矩形体

创建成功后，通过快捷键 Ctrl+长按鼠标左键，Ctrl+Shift+长按鼠标左键，来平移和旋转调整工作站平面，选中创建的矩形台面，工作站中会显示一个上下左右移动的箭头，通过拖动箭头可以移动当前矩形桌面的位置，将当前矩形桌面的位置移动到机器人 3D 体积中，使其在机器人工具的工作范围内，操作如下图所示：

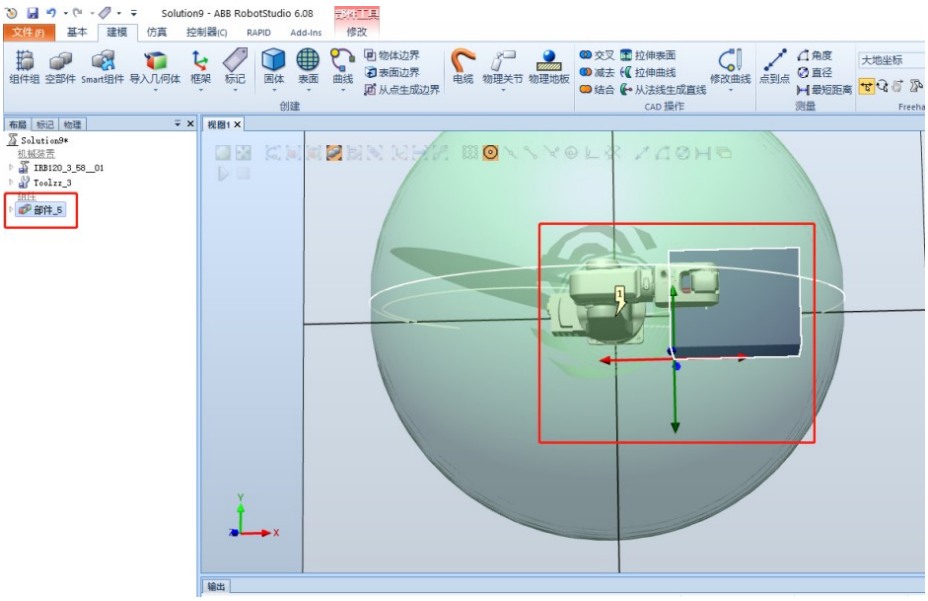


图 2.22 旋转平移工作站

打开虚拟示教器，点击菜单键，回到主界面，点击程序编辑器，新建一个程序模块，示教器会自动创建一个名为 MainModule 的程序模块,同时会自动创建该模块的主例行程序 main（）：



图 2.23 创建程序模块

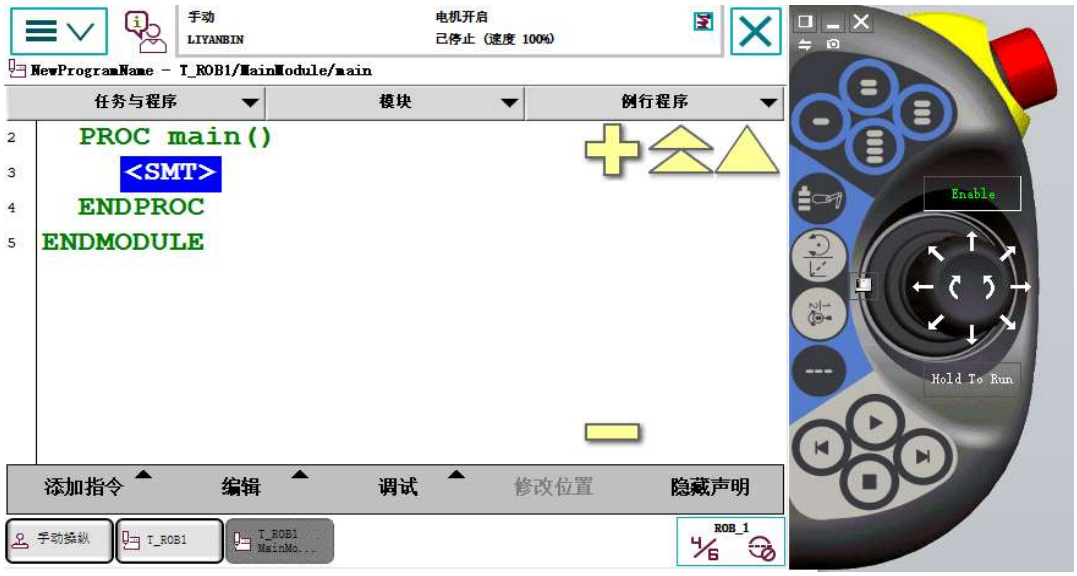


图 2.24 main 主程序

如果需要对示教器自动创建的模块名称及所属类型进行更改，点击程序界面“模块”选择当前需要更改的程序模块，点击“文件”，点击更改申明即可更改，操作如下所示：


手动
LIYANBIN
电机开启
已停止 (速度 100%)
NewProgramName - T_ROB1/MainModule/main
任务与程序
模块
例行程序
PROC main()
<SMT>
ENDPROC
ENDMODULE
添加指令
编辑
调试
修改位置
隐藏声明
手动操纵
T_ROB1
ROB_1

(a)


手动
LIYANBIN
电机开启
已停止 (速度 100%)
T_ROB1
模块
名称
类型
更改
1 到 3 共 3
BASE
系统模块
MainModule
程序模块
user
系统模块
新建模块...
加载模块...
另存模块为...
更改声明...
删除模块...
文件
刷新
显示模块
后退
手动操纵
T_ROB1
ROB_1

(b)


手动
LIYANBIN
电机开启
已停止 (速度 100%)
TestModule - T_ROB1
更改模块声明:
名称:
TestModule
ABC...
类型:
Program
确定
取消
手动操纵
T_ROB1
ROB_1

(c)



(d)

图 2.25 更改模块名称步骤

模块名称更改好后，点击“添加指令”添加一条运动指令 MoveJ：如下图所示：

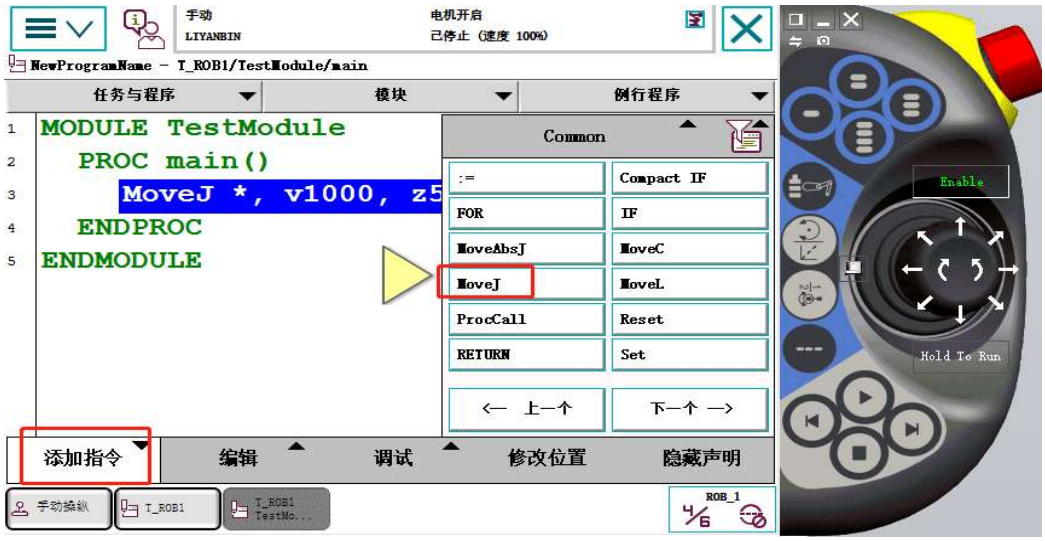


图 2.26 添加 MoveJ 指令

创建完运动指令后，选中“*”当前系统默认建立的坐标点名称，再点击“*”就可以对“*”名称重新建立一个名称 P1 进行替换，操作如下图所示：


手动
LIYANBIN
电机开启
已停止 (速度 100%)
NewProgramName - T_ROB1/TestModule/main
任务与程序
模块
例行程序
MODULE TestModule
PROC main()
MoveJ *, v1000, z50, tool0;
ENDPROC
ENDMODULE
添加指令
编辑
调试
修改位置
隐藏声明
ROB_1

(a)


手动
LIYANBIN
电机开启
已停止 (速度 100%)
更改选择
当前变量:
ToPoint
选择自变量值。
活动过滤器:
MoveJ *, v1000 , z50 , tool0;
数据
功能
1 到 2 共 2
新建
*
123...
表达式...
编辑
确定
取消
ROB_1

a

(b)


手动
LIYANBIN
电机开启
已停止 (速度 100%)
新数据声明
数据类型: robtarget
当前任务: T_ROB1
名称:
p1
范围:
全局
存储类型:
常量
任务:
T_ROB1
模块:
TestModule
例行程序:
<无>
维数
<无>
初始值
确定
取消
ROB_1

(c)



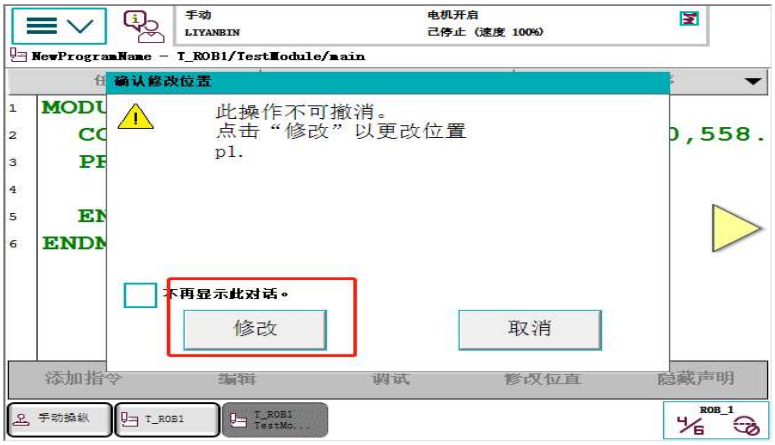
(d)

图 2.27 修改点位名称步骤

更改完位置名称后，点击“修改位置”就将当前机器人坐标示教到点位 P1，P1 点就是当前机器人位置坐标。其操作如下所示：



(a)



(b)

图 2.28 示教点位位置

创建完一条语句之后，机器人需要围着台面做一个矩形运动，就需要建立 4 个点 P2、P3、P4、P5，P1 点位作为运动的起始点，也可以称作为 HOME 点（机器人停止运动的安全点位），机器人移动路线就如图所示：

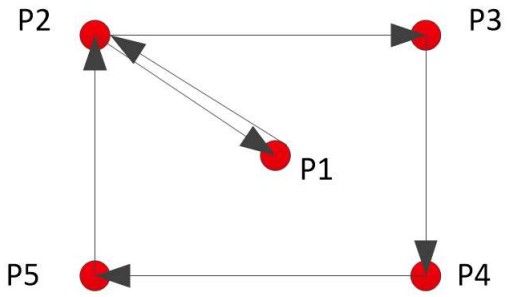


图 2.29 运动路径示意图

机器人从 P1 起始点开始到回到 P1 点的路径是：P1→P2→P3→P4→P5→P2→P1；机器人程序编写如图所示：



图 2.30 运动路径程序图

程序编写完毕，就是移动机器人到达各个点位进行修改位置来示教点位。操作如下：

通过平移和旋转调整机器人视角，用示教器摇杆机器人在线性模式下移动机器人到达点位，如下图所示：

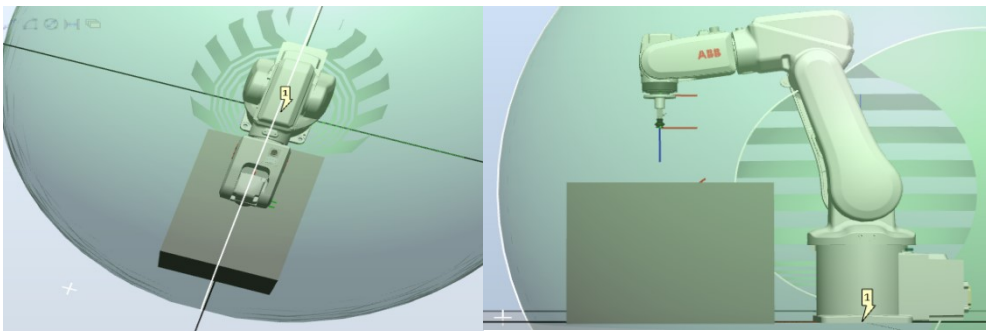
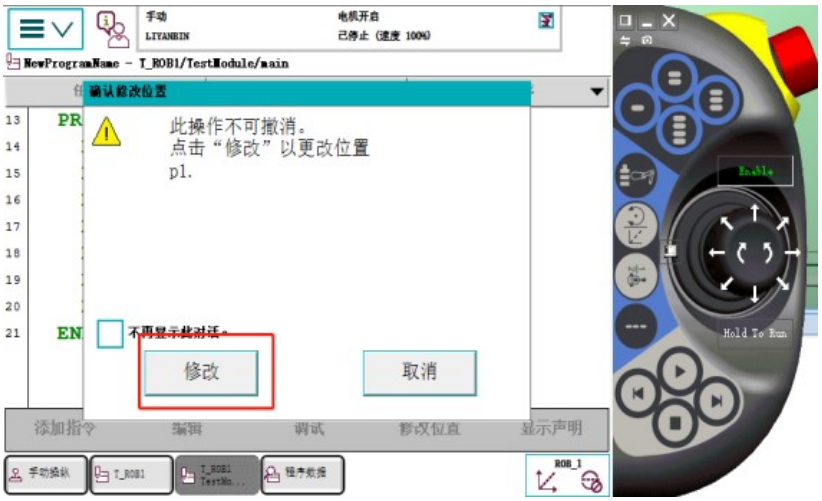


图 2.31　P1 点在仿真系统中的位置

调整好位置后就进行点位示教，点击程序的 P1 点，点击修改位置，点击是，P1 点的位置就示教完毕。



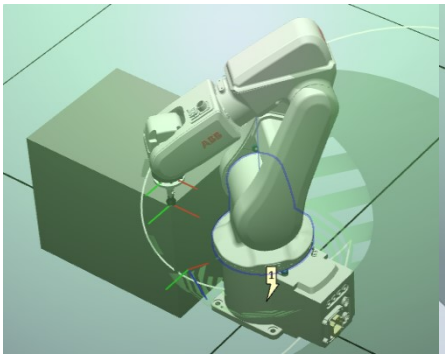
(a)



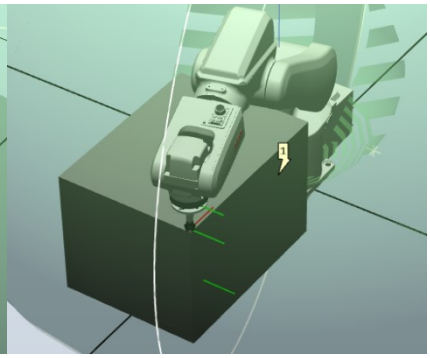
(b)

图 2.32　示教 P1 点位

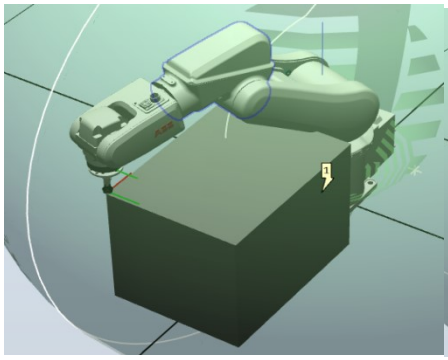
同理将机器人移动至 P2、P3、P4、P5 进行示教。如果在线性移动示教过程中，机器人移动位置超限，可以移动矩形台面的位置使其更加靠近机器人，使线性移动时能到达该点位，如下图所示：



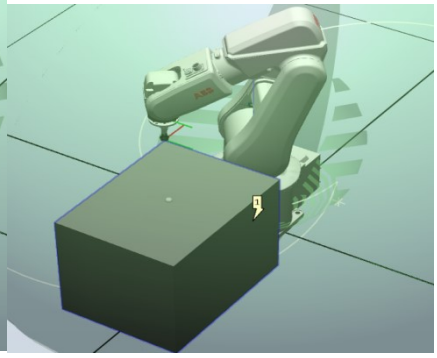
P2



P3



P4



p5

图 2.33　P2-P5 点位示教仿真图

示教完点后，电机使能，点击程序运行，机器人就会按照示教的点位进行移动了；机器人移动时会发现，机器人并没有完全到达每个点位，而是在点位前转弯，这是因为程序中设置了转弯半径 Z50，将 Z50 改为 fine，机器人就会完整按照示教点位精准运动。如下图所示：

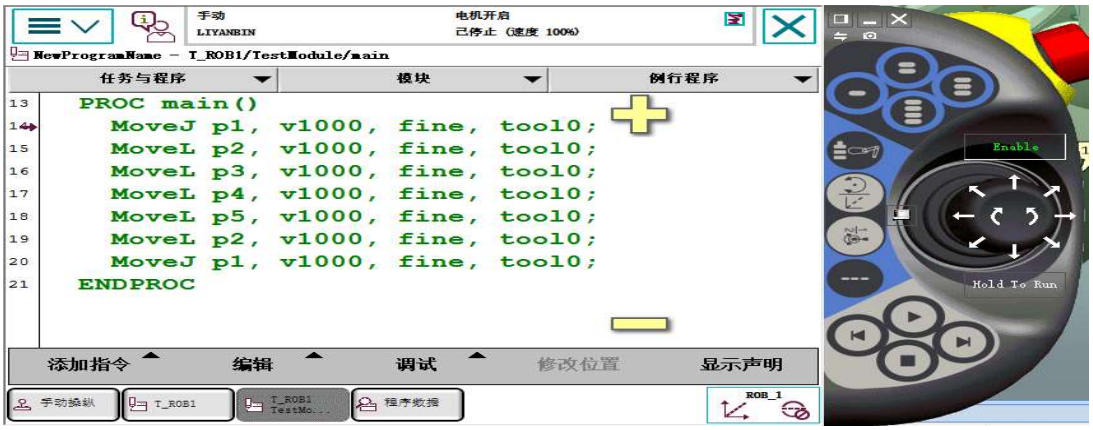


图 2.34 机器人运动程序图

实验练习：在原有的矩形台体上新建一个圆柱台体，利用 MoveC 指令编写程序，使机器人围绕圆柱台体表面做圆形运动。

附：MoveC 指令说明：使机械臂沿圆周移动

Movec 用于将工具中心点(TCP)沿圆周移动至给定目的地。移动期间，该周期的方位通常相对保持不变。

本指令仅可用于主任务 T_ROB1，或者如果在 MultiMove 系统中，则可用于运动任务中。

例 1

Movec p1, p2, v500,z30,tool2;

工具的 TCP tool2 沿圆周移动至位置 p2，其速度数据为 v500 且区域数据为 z30。根据起始位置、圆周点 p1 和目的点 p2，确定该循环。

例 2

Movec *, *, v500\T :=5,fine, grip3;

工具的 TCPgrip3 沿圆周移动至指令中储存的精点（标有第二个*)。同时将圆周点储存在指令中(标有第一个*)。完整的运动耗时 5 秒。

例 3

```
MoveL    p1, v500,fine, tool1;
Movec    p2, p3, v500,z20, tool1;
Movec    p4, p1, v500,fine, tool1;
```

本图显示了如何通过两个 Movec 指令，实施一个完整的周期。

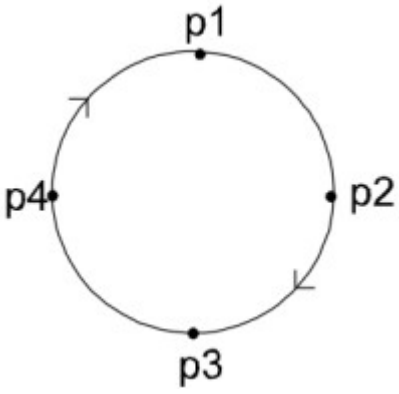


图 2.35　两个 Movec 指令运动曲线图

第三章　码垛实验仿真与实际操作实验

实验目的：学习码垛编程方法及实验台实际操作

实验要求：规划码垛路线，完成基本码垛指令方法，掌握示教器使用方法

实验设备：型号 MFTW-IR 和 CHL-JC-02-A 的工作站

实验步骤：

一、码垛实验需要用到的指令讲解

1. IO 控制指令：实验中码垛使用的抓取工具是机器人吸盘，机器人通过控制电磁阀来实现吸盘的抓和放。控制电磁阀使用的指令是 SetDo。

1.209 SetDO - 改变数字信号输出信号值

手册用法

无论是否存在时间延迟或同步，SetDO用于改变数字信号输出信号的值。

基本示例

以下实例介绍了指令SetDO：

例 1

```
SetDO do15, 1;
```

将信号do15设置为1.

例 2

```
SetDO weld, off;
```

将信号weld设置为off.

例 3

```
SetDO \SDelay := 0.2, weld, high;
```

将信号weld设置为high ，且时间延迟为0.2 s。通过下一指令，继续程序执行。

例 4

```
SetDO \Sync ,do1, 0;
```

将信号do1设置为0。程序执行进入等待，直至从物理上将信号设置为指定值。

图 3.1 SetDO 指令含义

在实验中机器人电磁阀 IO 的对应的点位是 Do0（型号 MFTW-IR 定义）；DO10_9（型号 CHL-JC-02-A 定义）

例：SetDO do0,1;（或 SetDO DO10_14,1）将数字输出 do0 或 DO10_14 的值设置为 1，表示打开吸盘。

WaitTime 0.5;程序等待 0.5 秒

SetDO do0,0; （或 SetDO DO10_14,0）将数字输出 do0 或 DO10_14 的值设置为 0，表示关闭吸盘。

2. 获取当前位置数据函数 CRobT。

2.45 CRobT - 读取当前位置（机器人位置）数据

手册用法

CRobT*(Current Robot Target)* 用于读取机械臂和外轴的当前位置。

该函数返回robtarget值以及位置（x、y、z）、方位（q1 ... q4）、机械臂轴配置和外轴位置。如果仅读取机械臂TCP（pos）的x、y和z值，则转而使用函数CPos。

基本示例

以下示例介绍了函数CRobT。

另请参阅第969页的更多示例

例 1

```
VAR robtarget p1;
MoveL *, v500, fine \Inpos := inpos50, tool1;
p1 := CRobT(\Tool:=tool1 \WObj:=wobj0);
```

将机械臂和外轴的当前位置储存在p1中。工具tool1和工件wobj0用于计算位置。

注意，在读取和计算位置前，机械臂静止不动。通过使用先前移动指令中位置精度inpos50内的停止点fine，实现上述操作。

返回值

数据类型： robtarget

最外面坐标系中的机械臂和外轴的当前位置，其将指定工具、工件和有效ProgDisp/ExtOffs坐标系纳入考虑。

图 3.2　CrobT 指令释义

3. 基于位置偏移指令 Offs。

2.108 Offs - 取代一个机械臂位置

手册用法

Offs用于在一个机械臂位置的工件坐标系中添加一个偏移量。

基本示例

以下示例介绍了函数Offs。

另请参阅第1085页的更多示例

例 1

```
MoveL Offs(p2, 0, 0, 10), v1000, z50, tool1;
```

将机械臂移动至距位置p2（沿z方向）10 mm的一个点。

例 2

```
p1 := Offs (p1, 5, 10, 15);
```

机械臂位置p1沿x方向移动5 mm，沿y方向移动10 mm，且沿z方向移动15 mm。

返回值

数据类型：robtarget

移动的位置数据。

图 3.3　Offs 指令释义

二、机器人码垛位置示教及逻辑讲解

码垛基本示意图如下图所示：

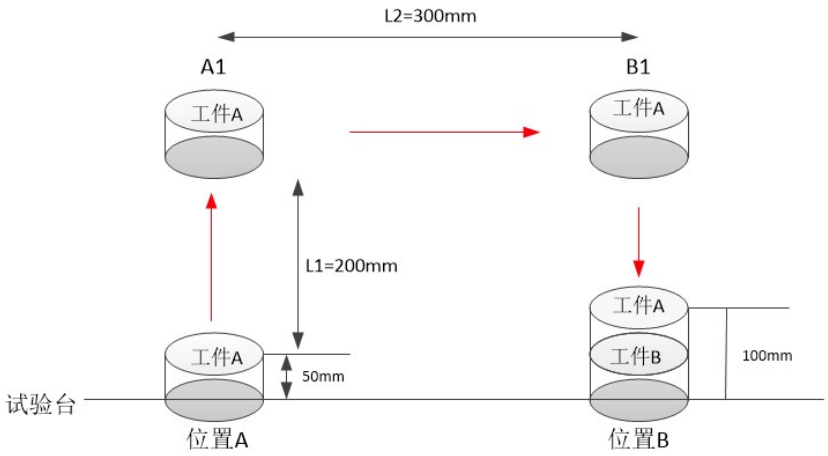


图 3.4 码垛示意图

从示意图中，需要将位置 A 处的工件 A 堆叠到同种工件 B 的上方，机器人移动到位置 A 打开吸盘抓取工件 A，抬升机械臂 L1 长度，到达位置 A1,再从 A1 经过平移 L2 距离到达 B1 位置，从 B1 垂直下降 L1-50mm 的高度关闭吸盘，将工件 A 码垛到工件 B 上，工件需要垂直抓取和垂直码垛的目的在于防止机器人与工件碰撞。

示教过程：首先将机器人移动到工件 A 表面中心点进行示教，更改名称为 Point_A，再将机器人移动到工件 B 表面中心点进行示教，更改名称为 Point_B。

开始编程实现上述过程：首先我们创建一个新的例行程序，点击例行程序，点击文件，新建例行程序，更改名称 MaDuo。



图 3.5 创建 MaDuo 例行程序步骤

三、编写程序

编写程序可以在示教器上进行开发，但是在示教器上面开发效率比较慢，对于复杂的逻辑结构编程，在示教器上操作比较困难，一般在 robotstudio 软件上进行编程，如下图所示。

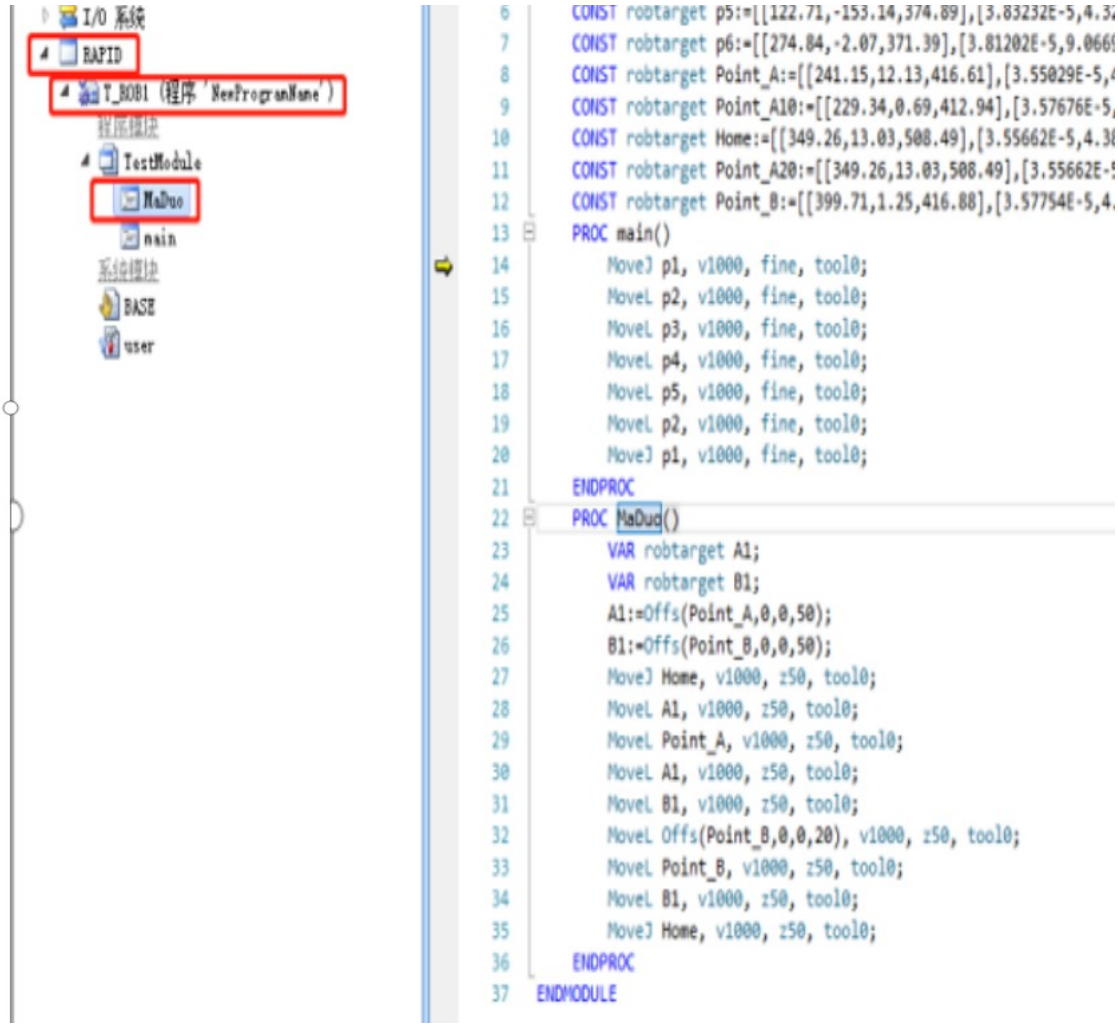


图 3.6　robotstudio 程序编辑界面

创建完系统，软件会自动创建机器人的系统文件，只需要在系统文件中的程序文件编写代码即可。如上图所示，在 T_ROB1 任务中创建了 TestModule 程序模块，TestModule 模块里面包含两个例程，一个是 main 程序，机器人执行主程序，一个是例行程序 MaDuo,双击 MaDuo 就可以在编辑器进行程序编辑了。编辑程序完成后，可以在手动模式下单独运行例行程序，但在自动模式下，机器人默认从 main 程序开始执行，如果在 main 程序中调用例行程序，用 procCall 命令调用。

码垛指导程序例程：

Var robtarget A1;声明一个 robtarget 数据类型的 A1 和 B1；

Var robtarget B1;

A1:=Offs(Point_A，0，0，200); 将距离 Point_A 点 Z 正方向 200mm 处的位置赋值给 A1;

B1:=Offs(Point_B，0，0，200); 将距离 Point_B 点 Z 正方向 200mm 处的位置赋值给 B1;

MoveJ Home, v500, z50, tool0; 从机器人的一个安全点Home位开始移动

MoveL A1, v500, fine, tool0;从 Point_A 正上方 A1 点开始直线下降到 Point_A

MoveL Point_A, v500, fine, tool0; 直线运动到 Point_A

SetDO do0,1;（或 SetDO DO10_14,1）表示打开吸盘

WaitTime 0.5;程序等待 0.5s 使吸盘稳定吸住工件

MoveL A1, v500, fine, tool0;机械臂抬升运动到 A1

MoveL B1, v500, fine, tool0; 直线运动到 B1

MoveL Offs(Point_B，0,0,50)，v500, fine, tool0;运动到距离 Point_B 点 Z 正方向上方 50mm 位置

SetDO do0,0;（或 SetDO DO10_14,0）表示关闭吸盘

WaitTime 0.5;程序等待 0.5s 使吸盘稳定放好工件

MoveL B1, v500, fine, tool0; 机械臂直线抬升运动到 B1

MoveJ Home, v500, z50, tool0;机器人回到安全点 Home

编写程序并示教好点位后就可以运行程序,运行程序操作是，点击调试，点击 PP 移至例行程序，选择编写好的例行程序，电机使能，按运行程序按钮就可以开始运行程序。

四、实际操作实验

A、示教器编程方法型号为 MFTW-IR 的实验台操作步骤如下：

1、将实验台总插座插到墙上插座里，将实验台后面电源开关切换到电源开档位。

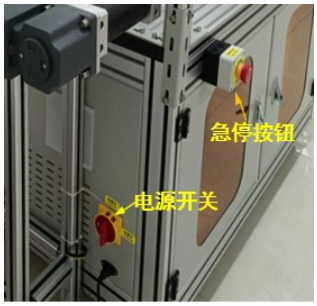


图 3.7 电源开关

2、打开左侧控制柜门并观测里面断路器是否跳闸，如果跳闸，请重新合上。





图 3.8 电源开关　　图 3.9 ABB 控制柜电源开关

3、打开右侧控制柜门，将 ABB 控制柜上的电源开关切换到开状态，手动和自动档位开关切换到手动状态。

4、将空气压缩机电源插座插到墙上插座里，手动拔起空气压缩机，使压缩机工作。

5、在示教器上程序数据→视图→全部数据类型→robtarget→新建

新建变量 A1，B1，Home，Point_A，Point_B（如果其他控制方法可以修改为其他变量名称如 P1,P2,P3,P4 等）

6、在示教器中依次选择菜单→程序编辑器→任务与程序→显示模块→点击 Module1 则可进入有 PROC main()这样开头的画面。当然 Module1 通过更改声明可以把名称修改为 TestModule。在 main()程序里可以直接增加程序，完成程序的编辑工作。也可以在 TestModule 模块里面，新建例行程序取名为 MaDuo，在 main()程序里，使用 procCall 命令调用 MaDuo 这个例行程序。

7、手动控制机械臂 6 轴运动，Home，Point_A，Point_B 的修改位置，满足工艺要求，示教时可以利用示教器上快捷按键实现示教工作。

方法如下：示教器上控制面板→配置可编程按键→按键 1 输出（类型：输出，按下按键：切换，允许自动模式;否）→选定数字输出信号（do0）→确定。

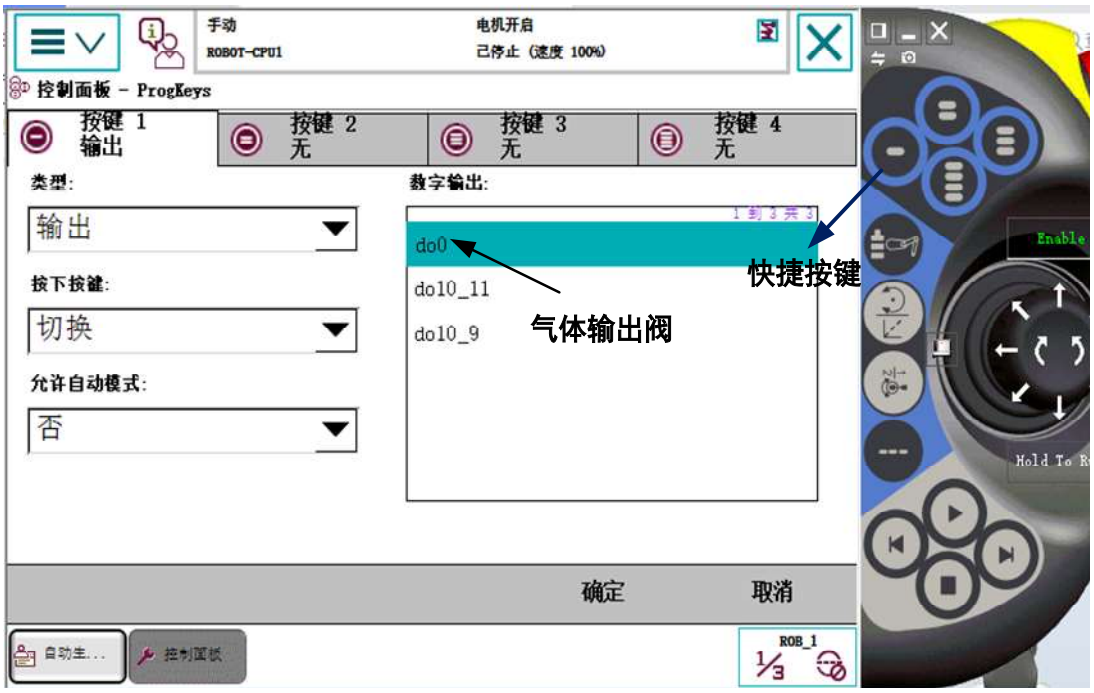


图 3.10　ABB 控制柜电源开关

这样在示教位置时，先将机器人移动到物体位置，按下快捷按键，气动阀门打开，吸盘吸取物体，根据实际控制需要物体移动到各个指定的位置进行示教位置修改，满足控制要求。

8、手动调试，点击 PP 移至例行程序，进行单步运行，满足控制要求。

9、切换到连续运行状态

10.将 ABB 控制柜里面手动模式切换到自动模式，电机按钮上电，示教器选择编号的自动运行程序，按下运行按钮，程序实现自动功能。

11、在运行和调试过程中，身体不能进入光栅，否则机器人停止运行。

备注：

1)该实验台 ABB 扩展的 I/O 模板主要参数如下：

A、使用来自模板的值选择：DSQC 652 24VDC I/O Device

B、 Name:d652　C、 Address ： 10

D、 Product Code ： 26 ;E、 Device Type： 7

F、 Connection Type ：　Change-of-State(COS)

g 、Connection output Size(byte):2(输出字节长度）

h 、Connection Input Size(byte):2(输入字节长度）

2)I/O 输入输出信号地址分配如下表：

定义名称	连接设备	信号类型(Type of Signal)	Device mapping	定义名称	连接设备	信号类型(Type of Signal)	Device mapping

di0	d652	Digtal Input	0	do0	d652	Digtal Output	0
di1	d652	Digtal Input	1	do1	d652	Digtal Output	1
di2	d652	Digtal Input	2	do2	d652	Digtal Output	2
di3	d652	Digtal Input	3	do3	d652	Digtal Output	3
di4	d652	Digtal Input	4	do4	d652	Digtal Output	4
di5	d652	Digtal Input	5	do5	d652	Digtal Output	5
di6	d652	Digtal Input	6	do6	d652	Digtal Output	6
				do7	do0	Digtal Output	7
				do8	d652	Digtal Output	8
				do9	d652	Digtal Output	9

B、示教器编程方法型号为 CHL-JC-02-A 的工作站的实验台操作步骤如下：

1、将实验台总插座插到墙上插座里，打开实验台右侧上方抽屉，将抽屉里 3 个断路器拔向上方。



图 3.11　CHL-JC-02-A 工作站电源开关

2、实验台触摸屏左侧有两个选择按钮，其中启动/停止切换按钮，选择启动功能，则会启动 ABB 控制柜及示教器电源，空气压缩机同时也启动。写字/示教和/流水线按钮，选择示教档位。电源指示灯亮则表示整个工作站已经供电。



图 3.12　为 CHL-JC-02-A 的工作站部分操作面板

3、 打开左侧控制柜门，将 ABB 控制柜上的电源开关切换到开状态，手动和自动档位开关切换到手动状态。

4、在示教器上程序数据→视图→全部数据类型→robtarget→新建

新建变量 A1，B1，Home，Point_A，Point_B（如果其他控制方法可以修改为其他变量名称如 P1,P2,P3,P4 等）

5、 在示教器中依次选择菜单→程序编辑器→任务与程序→显示模块→点击 Module1 则可进入有 PROC main()这样开头的画面。当然 Module1 通过更改声明可以把名称修改为 TestModule。在 main()程序里可以直接增加程序，完成程序的编辑工作。也可以在 TestModule 模块里面，新建例行程序取名为 MaDuo，在 main()程序里，使用 procCall 命令调用 MaDuo 这个例行程序。

6、 手动控制机械臂 6 轴运动，Home，Point_A，Point_B 的修改位置，满足工艺要求，示教时可以利用示教器上快捷按键实现示教工作。

方法如下：示教器上控制面板→配置可编程按键→按键 1 输出（类型：输出，按下按键：切换，允许自动模式:否）→选定数字输出信号(do10_14→确定)。



图 3.13　ABB 控制柜电源开关

这样在示教位置时，先将机器人移动到物体位置，按下快捷按键，气动阀门打开，吸盘吸取物体，根据实际控制需要物体移动到各个指定的位置进行示教位置修改，满足控制要求。

7、手动调试，点击 PP 移至例行程序，进行单步运行，满足控制要求。

8、切换到连续运行状态。

9、.将 ABB 控制柜里面手动模式切换到自动模式，电机按钮上电，示教器选择编号的自动运行程序，按下运行按钮，程序实现自动功能。

10、在运行和调试过程中，身体不能进入光栅，否则机器人停止运行。

ABB 在线编程方法

当然以上主程序编程和例行程序编程是基于示教器编程，还可以电脑在线编程和离线编程，离线编程将编写的程序用 U 盘拷贝到 ABB 实验台，进行调试。

在线编程首先确保电脑的防火墙关闭，确保实验台通电、ABB 控制器正常运行、网线正常连接情况下，设置电脑本地网络：打开“网络和 Internet”设置→网络共享中心→以太网→属性→Internet 协议版本 4（Tcp/IPV4）→IP 地址：10.70.2.***150***，子网掩码：255.255.255.0；默认网关：10.70.2.1,；DNS:211.67.208.1.其中 IP 地址第四位值根据每台电脑分配的 IP 地址确定，不能相同，否则出现地址冲突，无法正常连接机器人。输入机器人的 IP 地址，能够正常 ping 通，说明 ABB 机器人的控制柜能够与个人电脑进行在线编程。步骤如下：

1、 打开电脑桌面的 RobotStudio 软件，新建空工作站解决方案，进入开发界面选中控制器菜单，添加控制器下拉菜单中，选择添加控制器，在远程控制器中输入 ABB 控制器的 IP 地址，（如：10.70.2.252，网络组要求每台 ABB 的地址唯一性），点击添加按钮。如果能正常通讯，则在网络中的控制器中能显示系统名称、控制器名称、IP 地址和 RobotWare 版本。

双击选中该控制器，则能进行在线连接，如图所示：

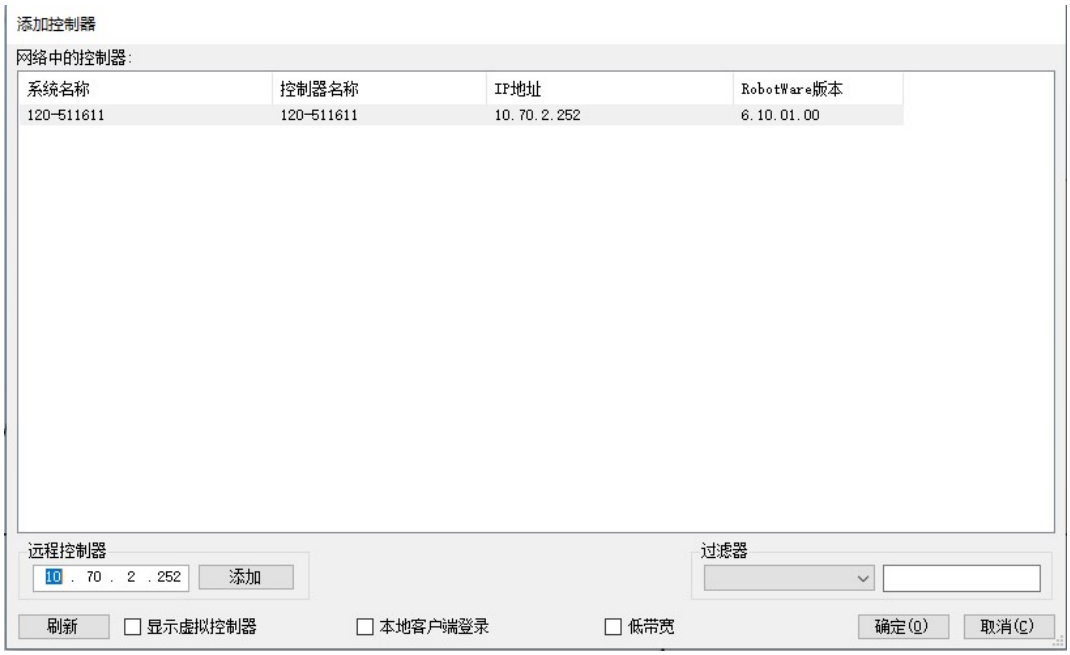


图 3.14　远程访问

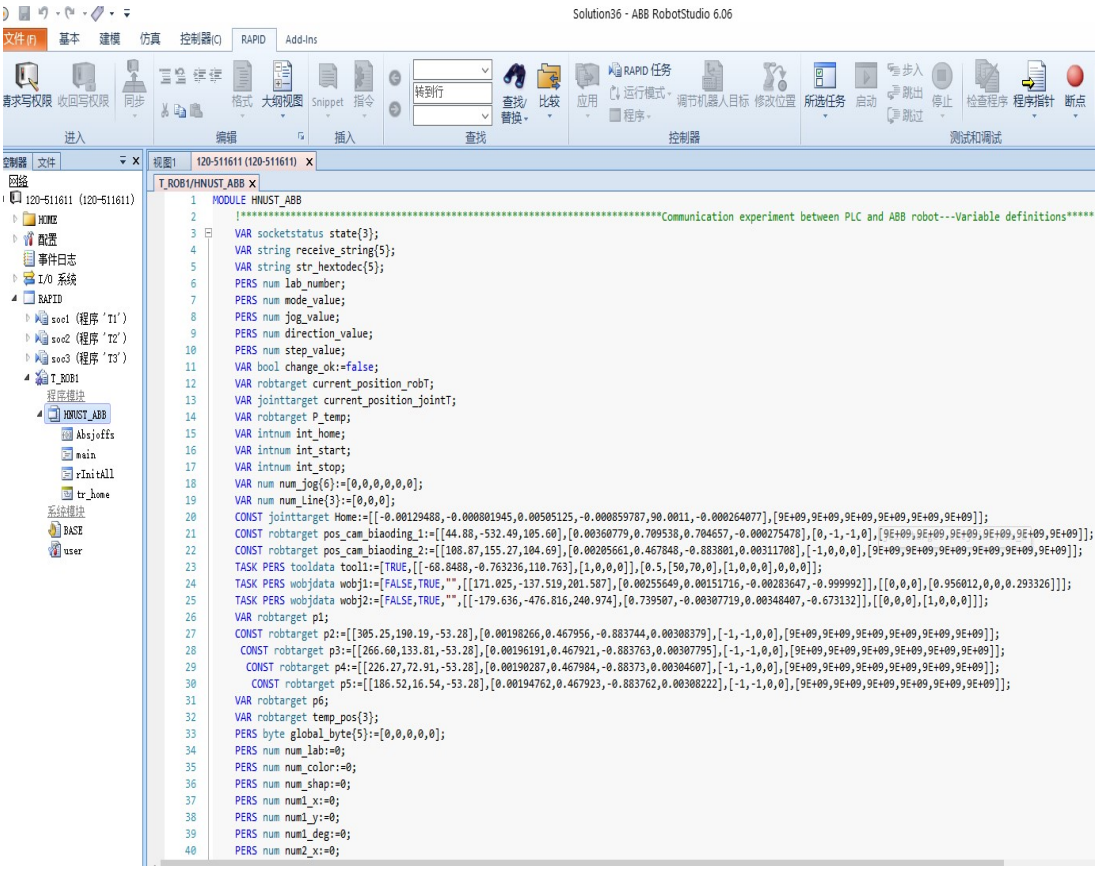


图 3.15　远程访问 ABB 程序

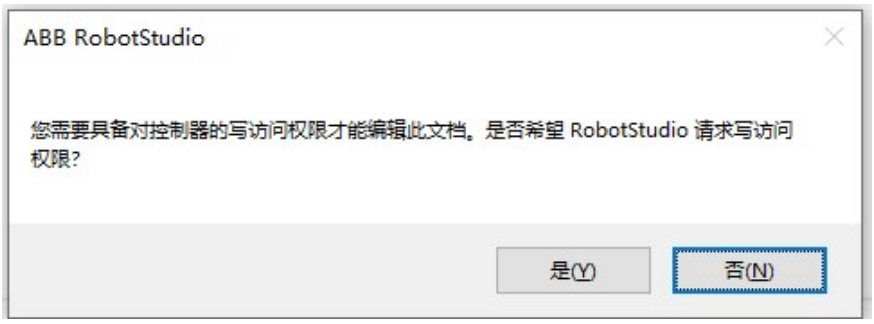


图 3.16　远程访问 ABB 请求访问权限

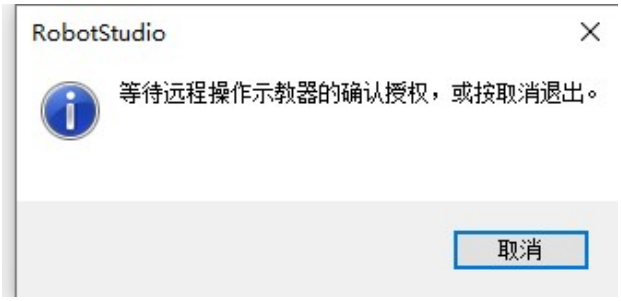


图 3.17 远程访问等待授权

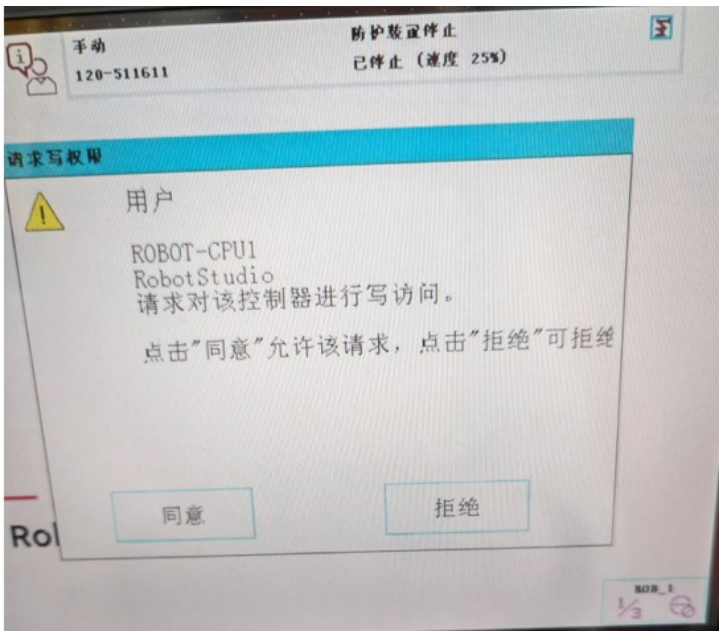


图 3.18 示教器上显示是否同意远程访问

实验练习：

根据以上例程使用 CrobT 指令实现拆垛程序，将堆叠的工件 A 和工件 B 拆垛抓取后放置到固定的点位。

第四章 机器人各坐标系定义实验

实验目的：熟悉机器人各坐标系建立方法

实验要求：掌握各坐标系创建的方法，

实验设备：型号 MFTW-IR 和 CHL-JC-02-A 的工作站

1、坐标系知识讲解

了解机器人的常用的坐标系：关节坐标系、基坐标系、工件坐标系、工具坐标系（TCP）；

关节坐标系：设定在机器人关节中的坐标系，它是每个轴相对其原点位置的绝对角度。

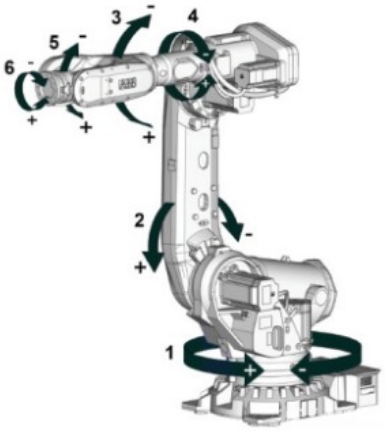


图 4.1 机器人轴坐标

基座标系：是机器人的基准坐标系（零位坐标系），机器人上除了关节坐标系以外，其他的坐标系都是由基座标系通过运动学变换而来，基座标系一般位于机器人安装底座几何中心位置。

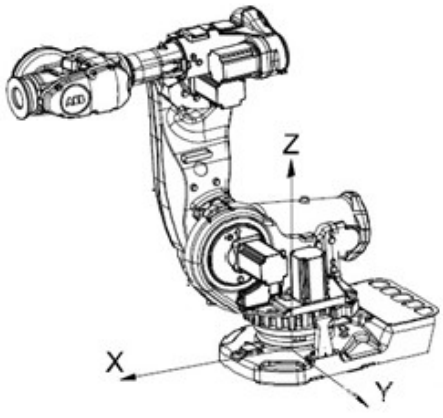
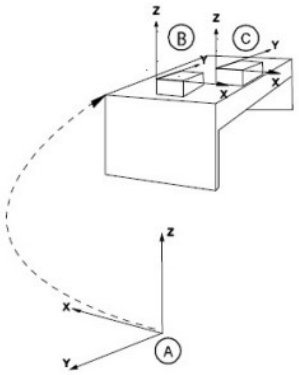


图 4.2 机器人基坐标

工件坐标系：它定义工件相对于大地坐标系（或其它坐标系）的位置。工件坐标系必须定义于两个框架：用户框架（与大地基座相关）和工件框架（与用户框架相关）。机器人可以拥有若干工件坐标系，或者表示不同工件，或者表示同一工件在不同位置的若干副本。



A	大地坐标系
B	工件坐标系 1
C	工件坐标系 2

图 4.3 机器人工件坐标

工具坐标系：将工具中心点设为零位，用来确定工具的位置和姿态。工具坐标系经常被缩写为 TCPF (Tool Center Point Frame)，而工具坐标系中心缩写为 TCP (Tool Center Point)。执行程序时，机器人就是将 TCP 移至编程位置。所有机器人在手腕处都有一个预定义工具坐标系，该坐标系被称为 tool0，也就是机器人法兰盘末端中点。这样就能将一个或多个新工具坐标系定义为 tool0 的偏移值。

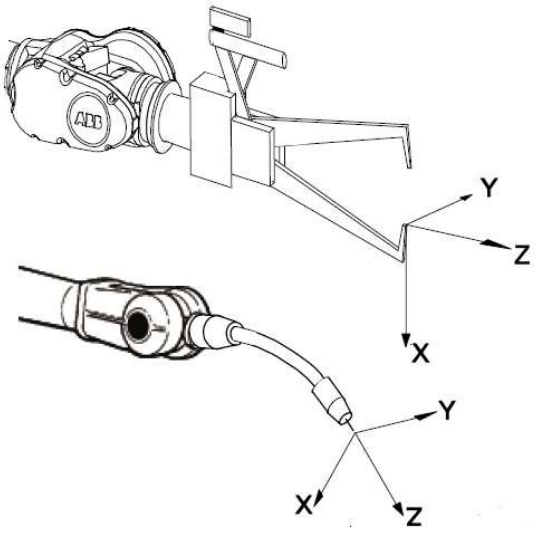


图 4.4 机器人工具坐标

2.工件和工具坐标系的建立

重新创建一个工作站或者基于原来的工作站基础上添加用于工件和工具坐标系仿真的模型。以下是基于原有工作站的基础上添加仿真模型进行仿真演示。

将 Solution9 工作站之前创建的部件删除，鼠标右键选中删除的部件，点击删除，操作如下图所示：

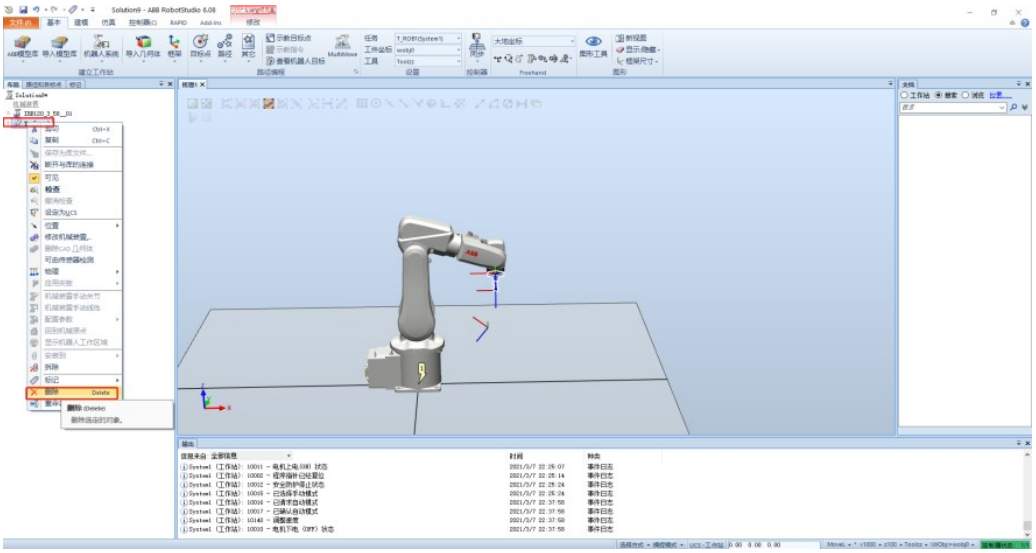


图 4.5 删除部件

在工作站中重新导入新焊枪的模型，和焊台，操作如下图所示。

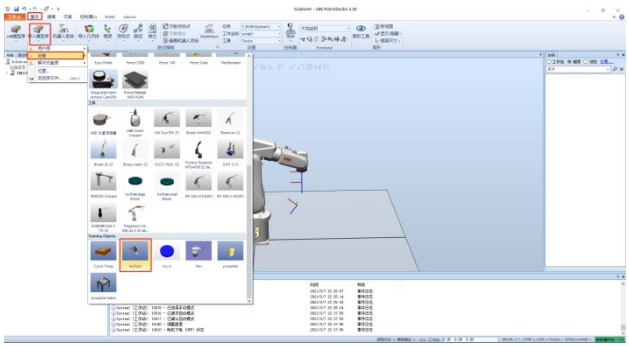


图 4.6 导入焊枪模型

将导入的焊枪安装到机器人手臂末端，其操作同前面实验操作一样，如下图所示：

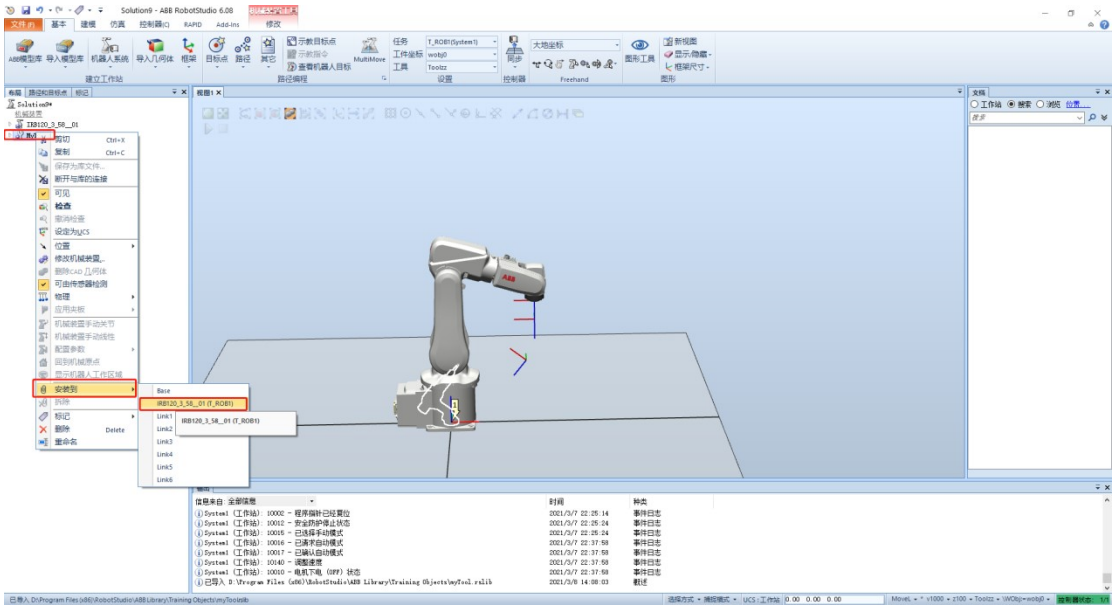


图 4.7 安装焊枪

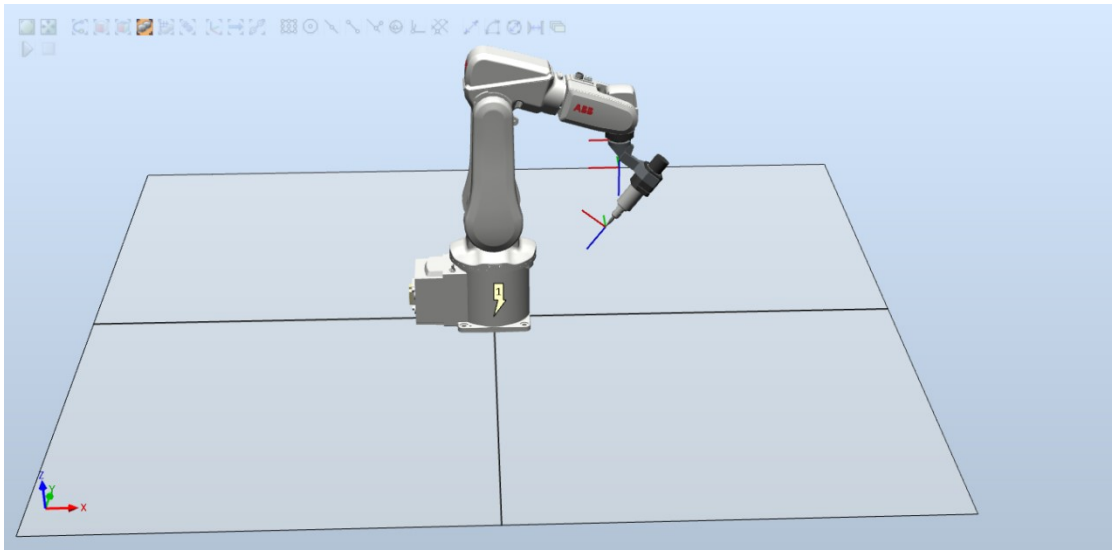


图 4.8 安装好的焊枪模型

同样的操作还需要导入一个焊接的操作台，如下图所示：

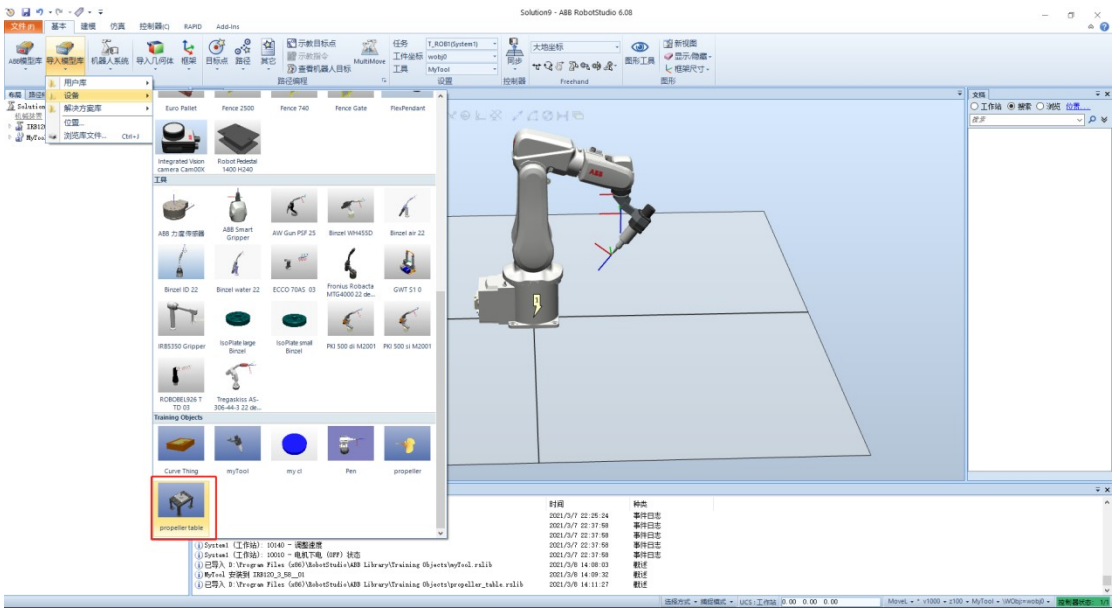


图 4.9 导入焊接实验桌

导入焊接台体后，通过旋转和平移将台体移动到机器人工作范围，移动到合适位置如下图所示：

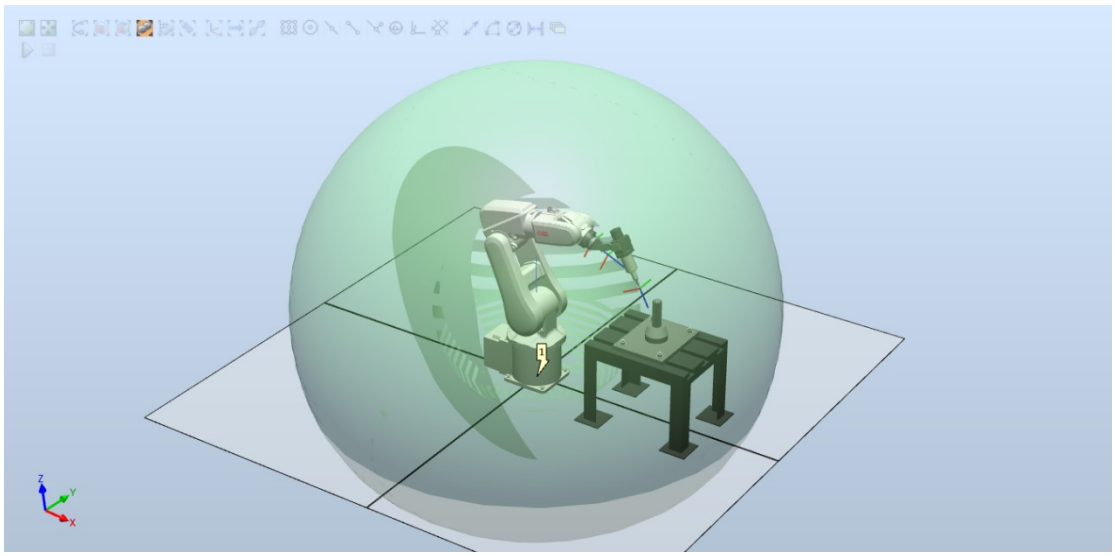


图 4.10 焊接台移动到机器人工作区域

3.工具坐标系的创建

工具数据（tooldata）用于描述安装在机器人第六轴上的工具的 TCP、质量、重心等参数数据。一般不同的机器人应用配置不同的工具，在执行机器人程序时，就是机器人将工具的中心点 TCP 移至编程位置。那么，如果要更改工具以及工具坐标系，机器人的移动也会随之改变，以便新的 TCP 到达目标。

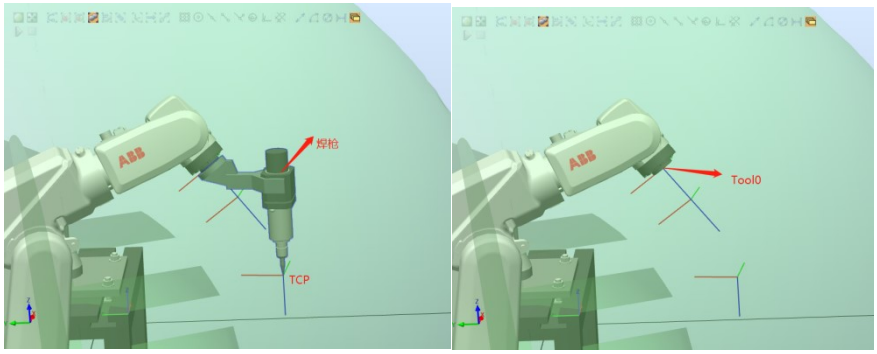


图 4.11 创建机器人工具数据

2、TCP 设定原理

（1）首先在机器人工作范围内找一个非常精确的固定点作为参考点；

（2）然后在工具上确定一个参考点（最好是工具中心点）；

（3）用手动操纵机器人的方法，去移动工具上的参考点，以四种以上不同的机器人姿态尽可能与固定点刚好碰上。为了获得更准确的 TCP，可以使用六点法进行操作，第四点是工具的参考点垂直于固定点，第五点是工具参考点从固定点向将要设定为 TCP 的 X 方向移动，第六点是工具参考点从固定点向将要设定为 TCP 的 Z 方向移动；

（4）机器人通过这几个位置点的位置数据计算求得 TCP 的数据，然后 TCP 的数据就保存在 tooldata 这个程序数据中被程序进行调用。

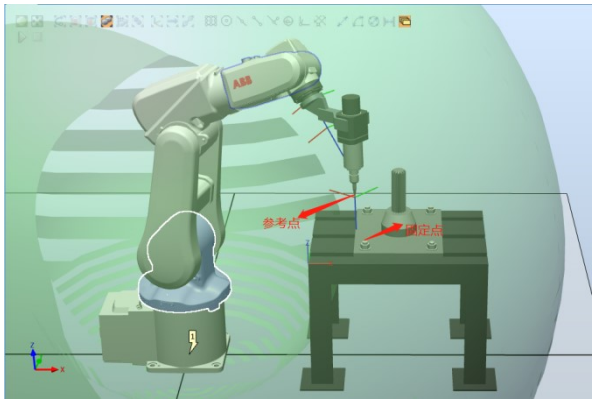


图 4.12 TCP 示教演示

3.工具数据 TCP 创建操作步骤

（1）单击“ABB”，选择“手动操纵”；



图 4.13 手动操作

（2）选择“工具坐标”。



图 4.14 工具坐标

（3）单击“新建...”，对它的属性进行过设定后，单击“确定”。



图 4.15 新建工具坐标

（4）为新建工具数据命名“tool1”。



图 4.16 更改工具坐标名称

（5）选中“tool1”后，单击“编辑”菜单中的“定义”选项。



图 4.17 工具坐标定义

（6）单击“方法”的下拉菜单，选择菜单中的“定义”选项，“TCP 和 Z，X”，使用 6 点法设定 TCP。



图 4.18　6 点法定义工具坐标

（7）选择合适的手动操纵模式，并按下使能键，使用摇杆将工具参考点靠上固定点，作为第一个点，在仿真工作站中也可以通过快捷操作方式，选中移动机器人的方式，鼠标拖动来快速移动机器人到达指定点位。



图 4.19 移动机器人快捷操作

（8）单击“修改位置”，将点 1 位置记录下来。



图 4.20 点 1 修改

（9）变化机器人工具姿态，让工具参考点以如图所示的姿态靠上固定点。

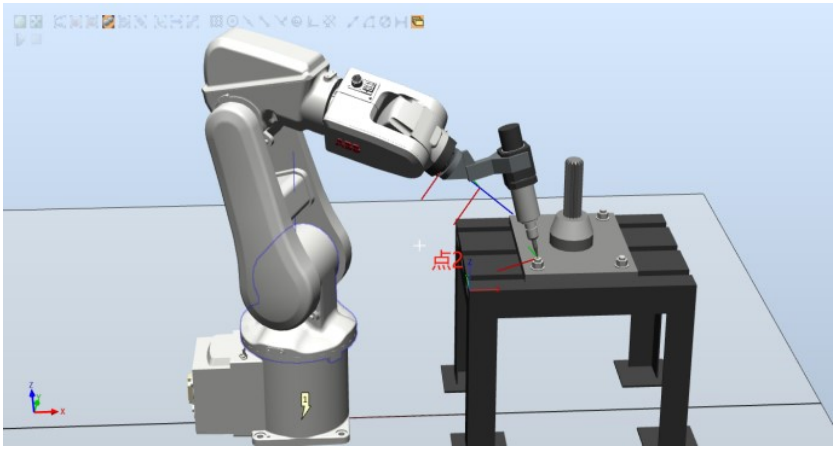


图 4.21 点 2 位置

（10）单击"修改位置"，将点 2 位置记录下来。



图 4.22 点 2 修改

（11）变化机器人工具姿态，让工具参考点换个姿态靠上固定点。

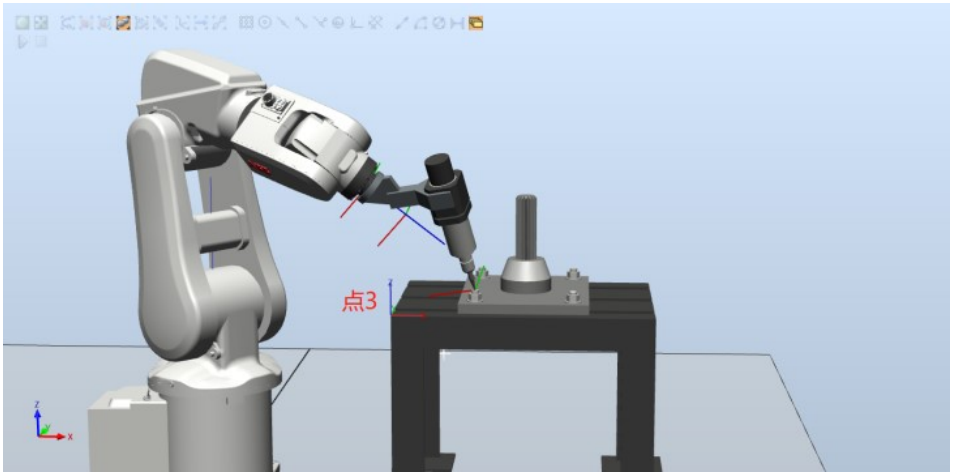


图 4.23 点 3 位置

（12）单击“修改位置”，将点 3 位置记录下来。



图 4.24 点 3 修改

（13）变化机器人工具姿态，让工具参考点以**垂直**于上固定点，这是第 4 个点。

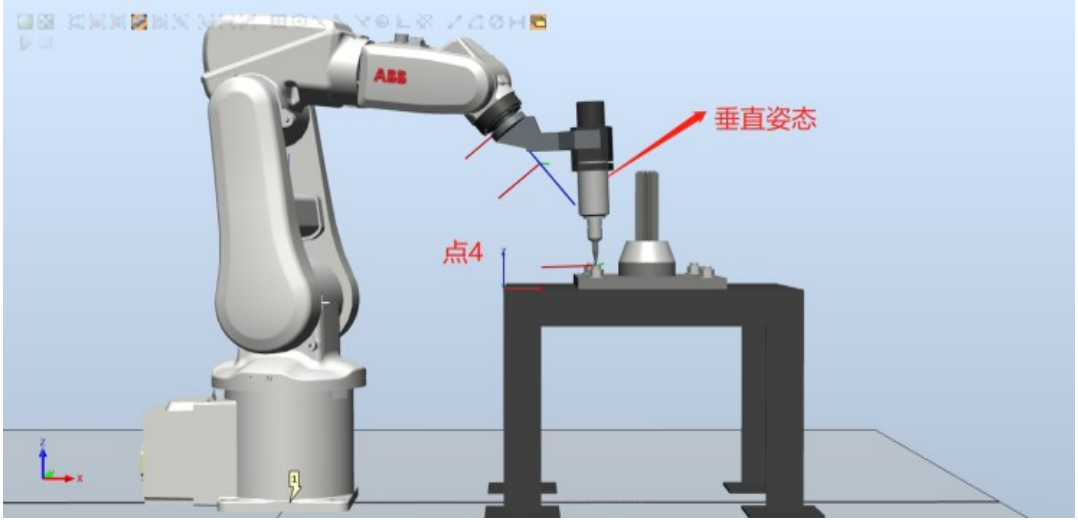


图 4.25 点 4 位置

（14）单击“修改位置”，将点 4 位置记录下来。



图 4.26 点 4 修改位置

（15）工具参考点以点 4 的姿态从固定点移动到工具 TCP 的+X 方向。

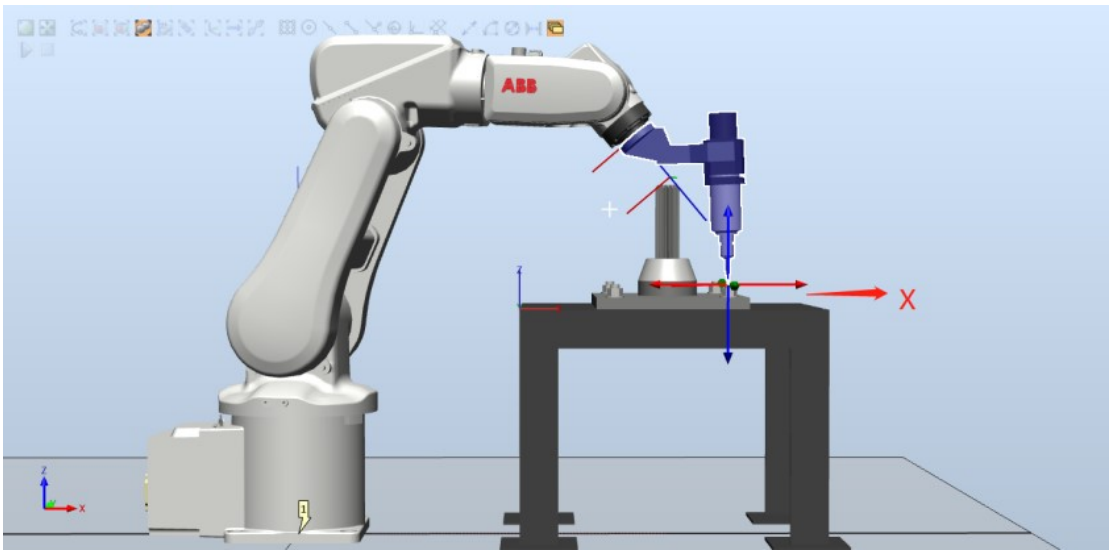


图 4.27　+X 方向

（16）单击“修改位置”，将延伸器点 X 位置记录下来。



图 4.28　+X 修改位置

（17）工具参考点以点 4 姿态移动到工具 TCP 的+Z 方向。

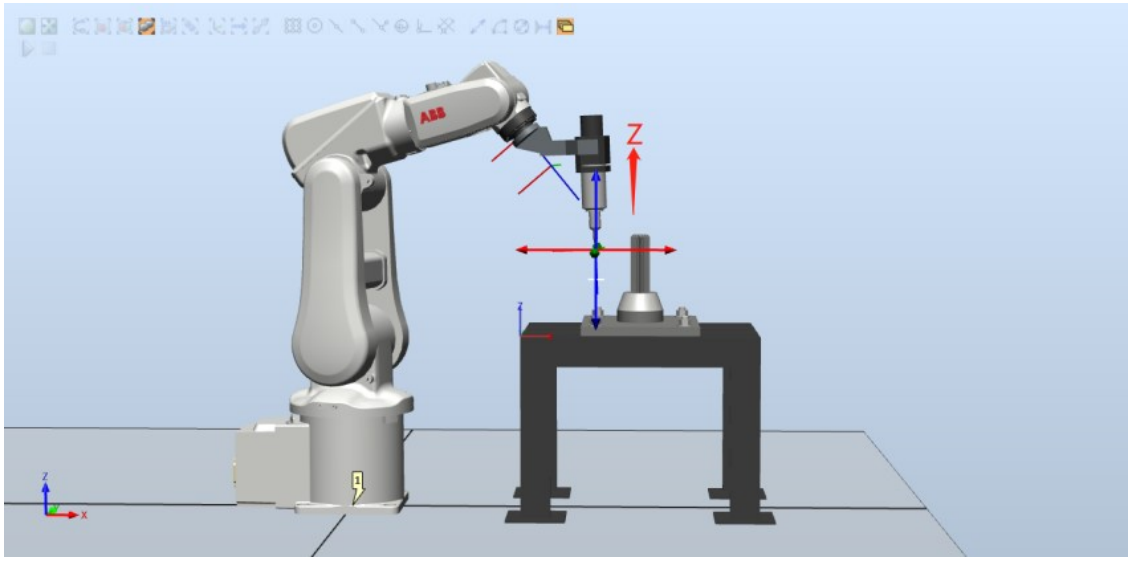


图 4.29　+Z 方向

（18）单击“修改位置”，将延伸器点 Z 位置记录下来，然后单击“确定”完成设定。



图 4.30　+Z 修改位置

（19）在误差确认界面，对误差进行确认，当然是越小越好，但也要以实际验证效果为准。



图 4.31　TCP 定义确定

（20）选中 tool1，然后打开编辑菜单选择“更改值”。



图 4.32　更改 Tool1 值

（21）单击右下角箭头向下翻页，找到有工具的质量 mass（单位 kg）的一栏，根据实际情况进行设定，我们可以将它更改为 1，工具重心 cog 根据情况设置，我们可以将 X 设置为 1，然后单击“确定”。



图 4.33 更改 Tool1 质量和重心

（22）设定好的工具数据 tool1，我们需要在重定位模式下手动操纵进行验证 TCP 是否精准，回到手动操纵界面，动作模式选定为“重定位”，坐标系选定为“工具”，工具坐标选定为“tool1”。

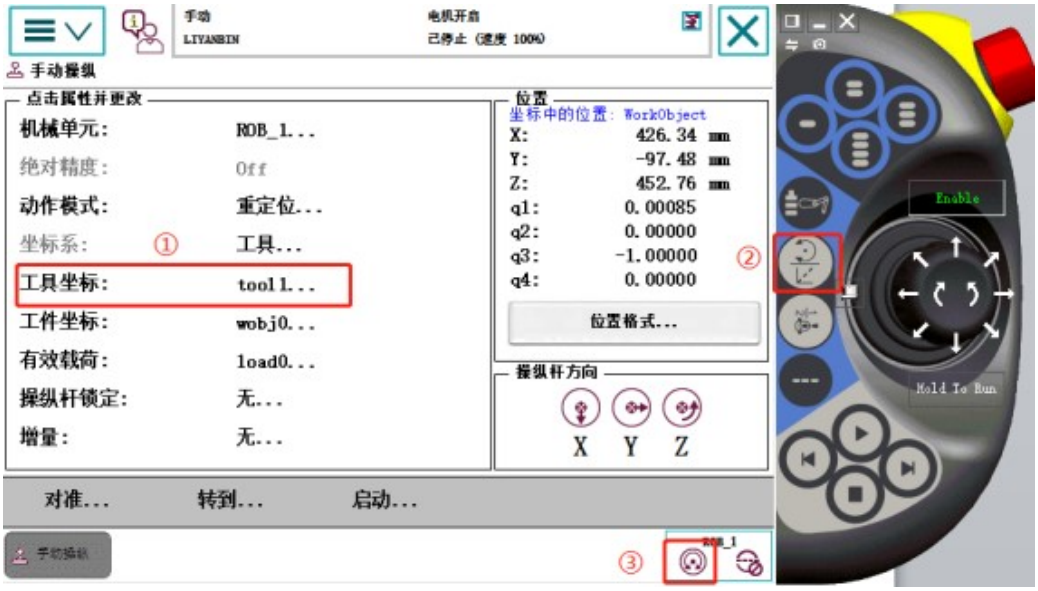


图 4.34　选择工具 Tool1

（23）手动操纵机器人将工具参考点靠上固定点，在重定位模式下，如果 TCP 设定精确的话，可以看到工具参考点与固定点始终保持接触，而机器人会根据重定位操作改变姿态。

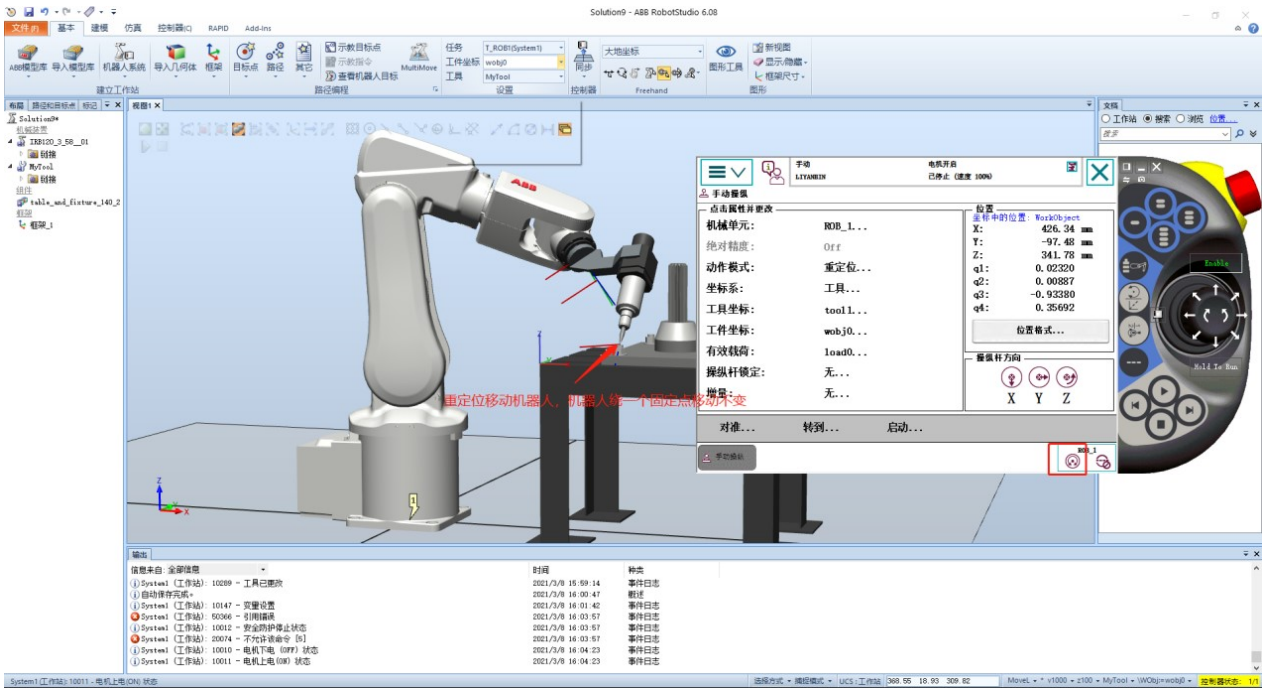


图 4.35 重定位运动验证 Tool1 精度

3.工件坐标系的创建

工件坐标对应工件，它定义工件相对于大地坐标（或其他坐标）的位置。对机器人进行编程时就是在工件坐标中创建目标和路径。重新定位工作站中的工件时，只需要更改工件坐标的位置，所有的路径将即刻随之更新。

例如，在下图中，定义好工件坐标 wobj1，并且对桌面工件的运动轨迹编程完成之后，如果桌子移动，只需要更改 wobj1 的值，之前的桌面工件运动轨迹就无需重新编程了。

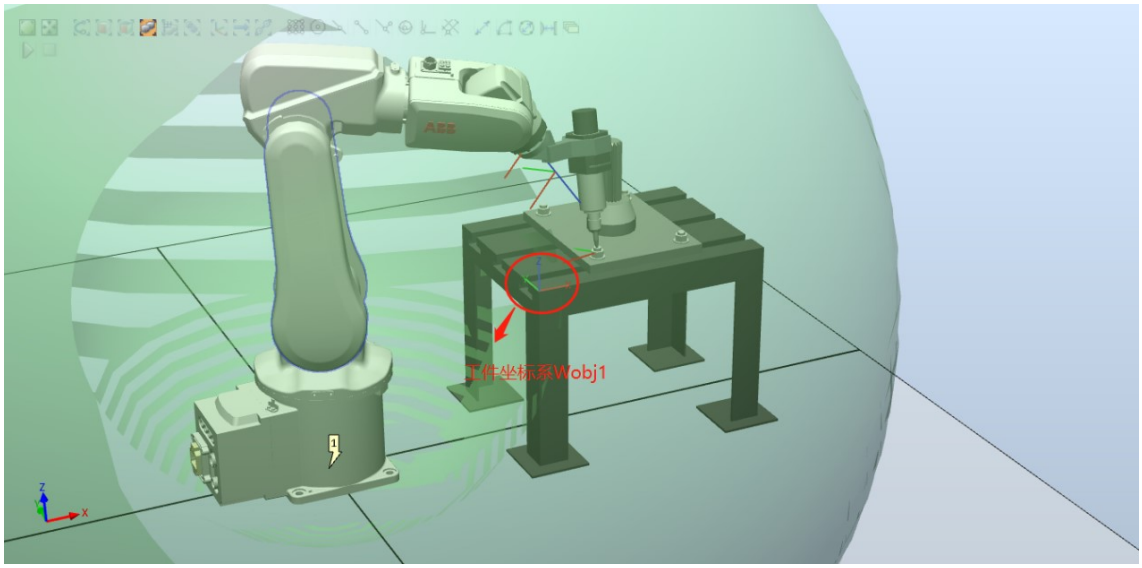


图 4.36 创建工件坐标系示意图

（1）工件坐标系的设定原理

在对象的平面上，只需要定义三个点，就可以建立一个工件坐标。

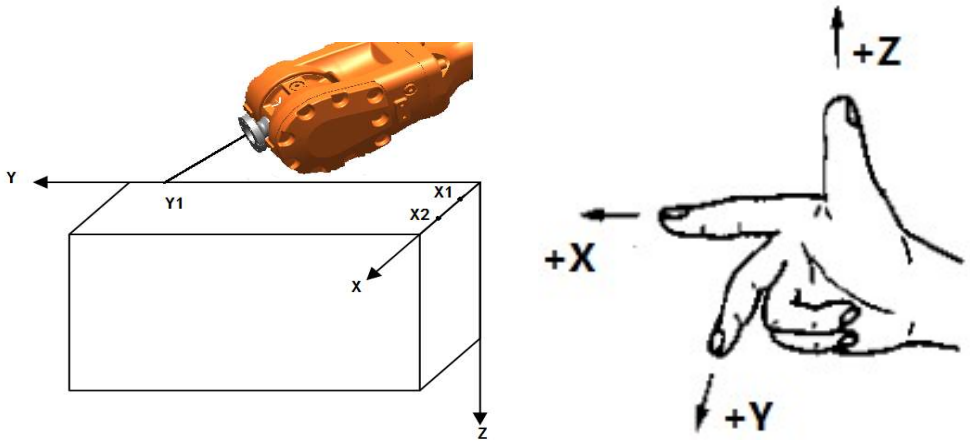


图 4.37 工件坐标系的设定原理

X1 点确定工件坐标原点，X1，X2 确定工件坐标 X 正方向，Y1 确定工件坐标 Y 正方向，最后 Z 的正方向根据右手定则得出。

（2）建立前面图中所示的工件坐标系 wobj1，步骤如下：

（注：操作界面与工具坐标系类似，只是将工具坐标更换成工件坐标）

A.点击“ABB”，选择“手动操纵”，在手动操纵画面中，选择“工件坐标”；

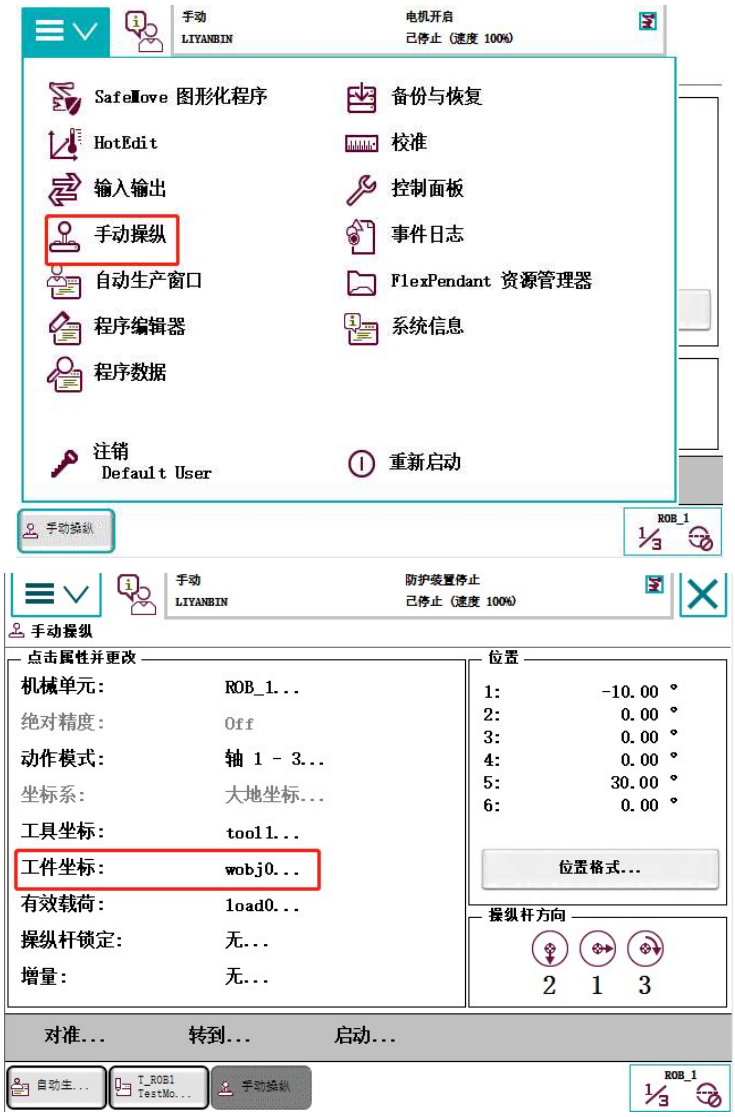


图 4.38 选择工件坐标系

B.单击“新建”；



图 4.39 创建工件坐标系

C.设定工件坐标数据属性，单击“确定”；



图 4.40 更改工件坐标系名称并确定创建

D.打开编辑菜单，选择“定义”；



图 4.41 定义工件坐标系

E.将用户方法设定为“3 点”；



图 4.42 3 点法创建工件坐标系

F.手动操纵机器人，让它的工具中心点靠近图示 X1 点，作为待设定工件坐标系的原点；

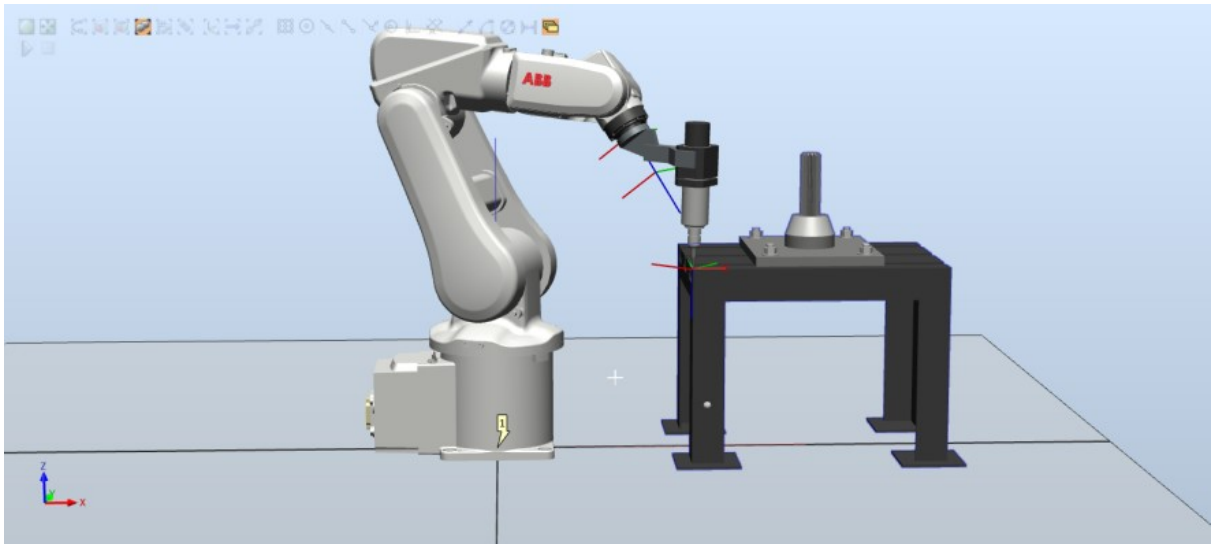


图 4.43　X1 点位置

G.单击“修改位置”，将 X1 点记录下来；



图 4.44 修改点 X1

H.沿着待定义工件坐标的 X 正向，手动操纵机器人的工具中心点靠近定义工件坐标的 X2 点。

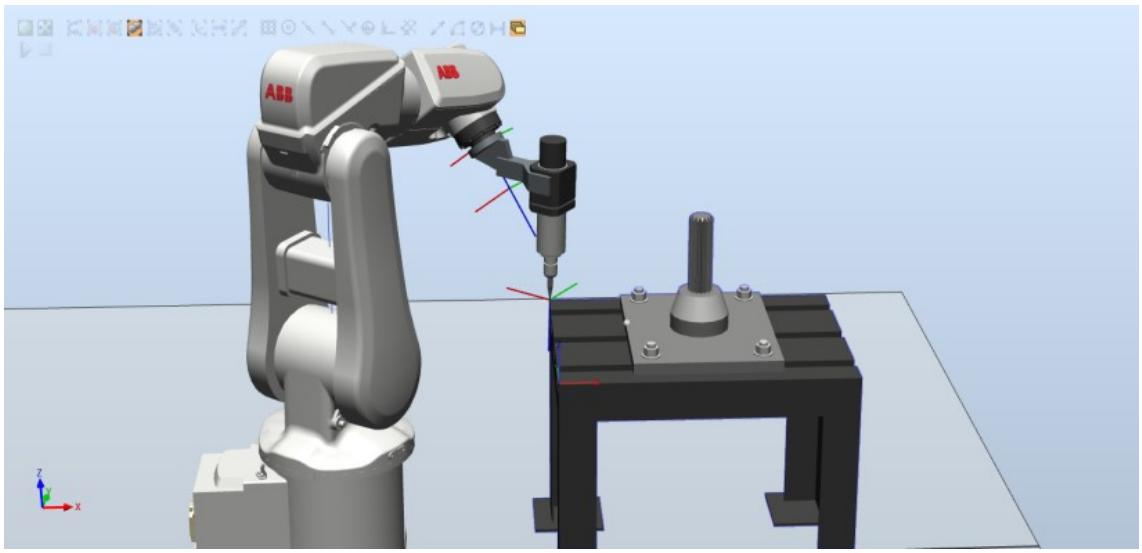


图 4.45 X2 点位置

I.单击“修改位置”，将 X2 点记录下来；



图 4.46 修改 X2 位置

J.手动操纵机器人的工具中心点靠近定义工件坐标的 Y1 点；

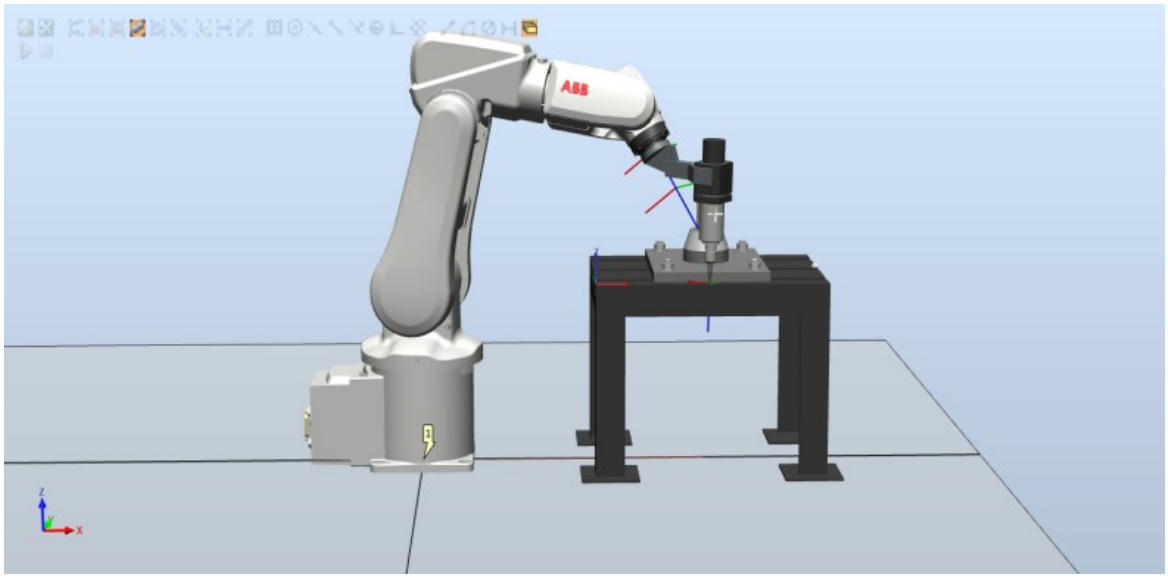


图 4.47 X3 位置

K.单击“修改位置”，将 Y1 点记录下来，然后点击“确定”；



图 4.48 修改 X3 位置

L.对自动生成的工件坐标数据进行确认后，单击“确定”；



图 4.49 确定创建工件坐标系 Wobj1

M.选择 wobj1 后，单击“确定”;



图 4.50 创建工件坐标系完毕

N.将机器人坐标系切换为“工件坐标系”选择对应的工件坐标系 Wobj1，就可以在 Wobj1 坐标系下运动了。



图 4.51 切换坐标系

实验练习：在示教器上创建工件坐标系和工具坐标系。

第五章 机器人轨迹示教编程实验

实验目的：学习机器人运动轨迹编程方法

实验要求：掌握机器人运动轨迹编程方法

实验软件：RobotStudio

实验步骤：

在工作站中创建一个用于轨迹示教编程的仿真模型，创建一个台体，台体的大小为 400*300*200mm,将台体放置到机器人工具工作范围内，再在台体上创建一个 200*100*50mm 矩形方块，两个直径 100*高 50mm 的圆柱体，将台体上三个物体组合到一起形成部件。操作如下：

1.首先创建一个 400*300*200mm 的工作台，将工作台放置到机器人工具工作范围内，操作方法同上节内容相同。

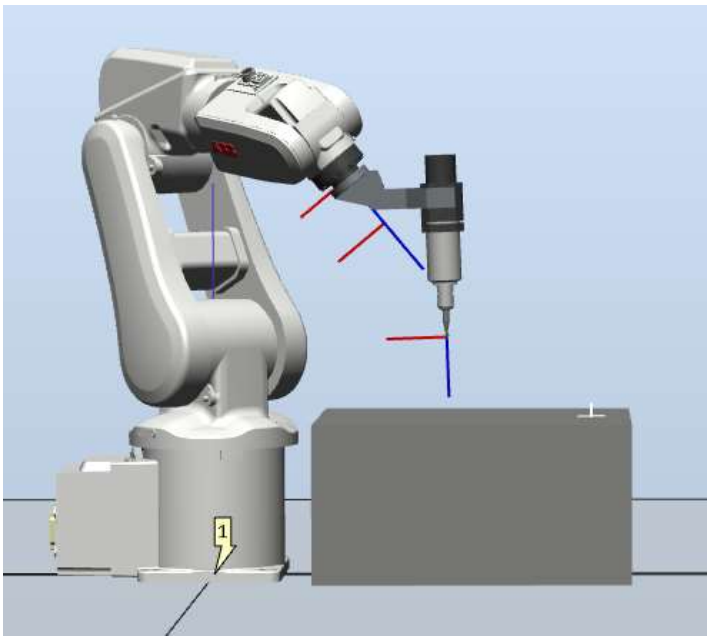


图 5.1 创建仿真工作台

2.创建 200*100*50 矩形方块和两个直径 100 高 50 的圆柱体。

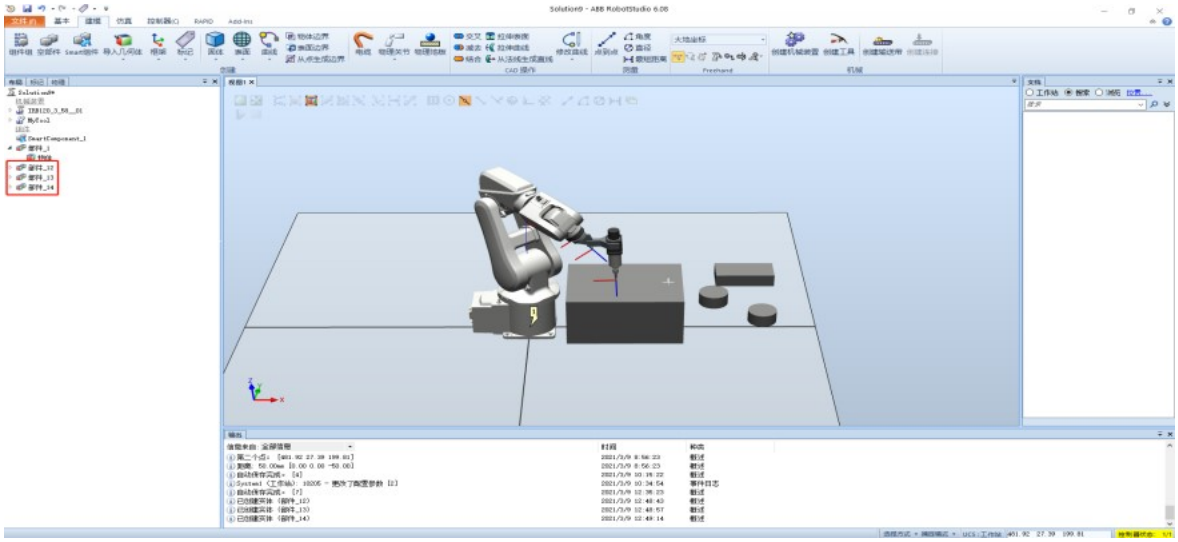


图 5.2 创建轨迹示教部件

3.将矩形方块放置到台体的中心，选中所要放置的矩形方块，右键选择位置，点击放置，选择一个点。

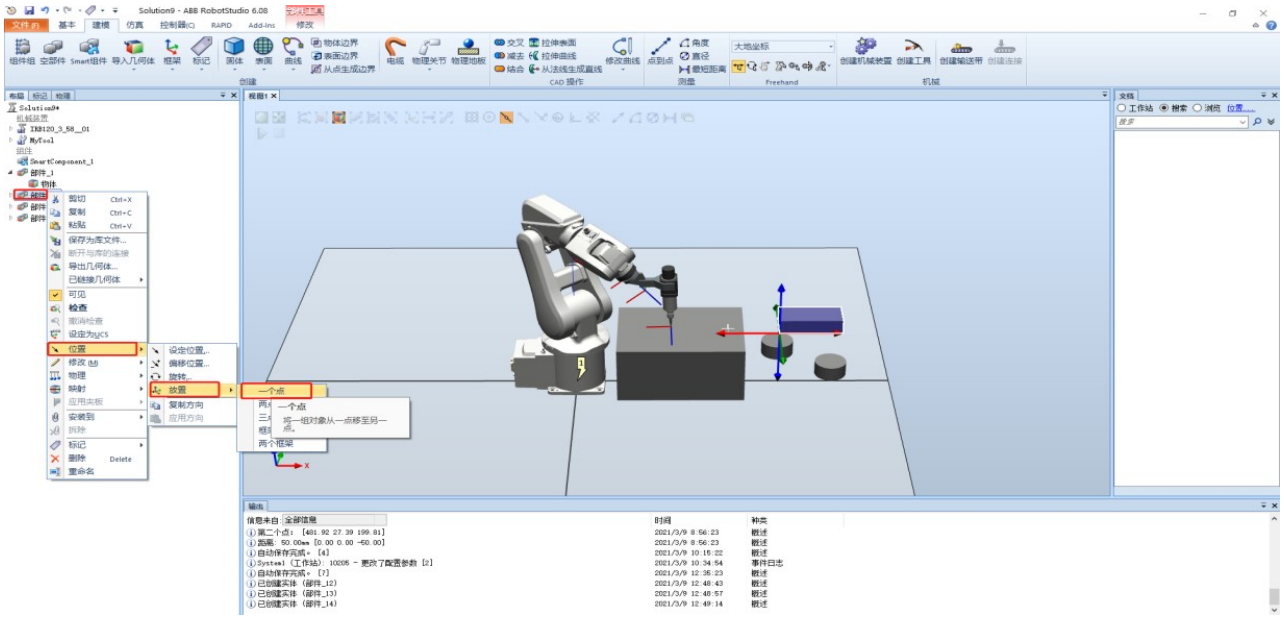


图 5.3 部件位置装配

4.点击主点-从输入框，选择自动捕捉中心，捕捉矩形方块中心。

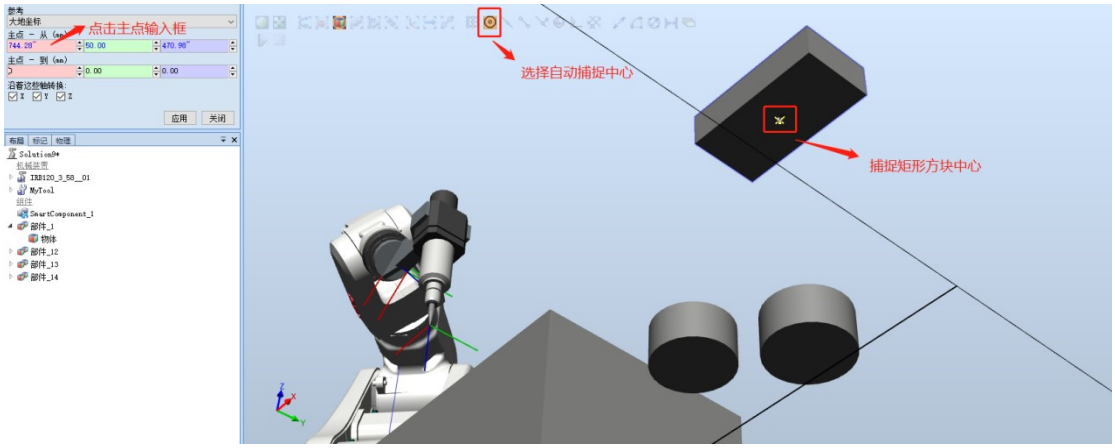


图 5.4“从”装配点选择

5.点击主点-到，选择台体的中点。

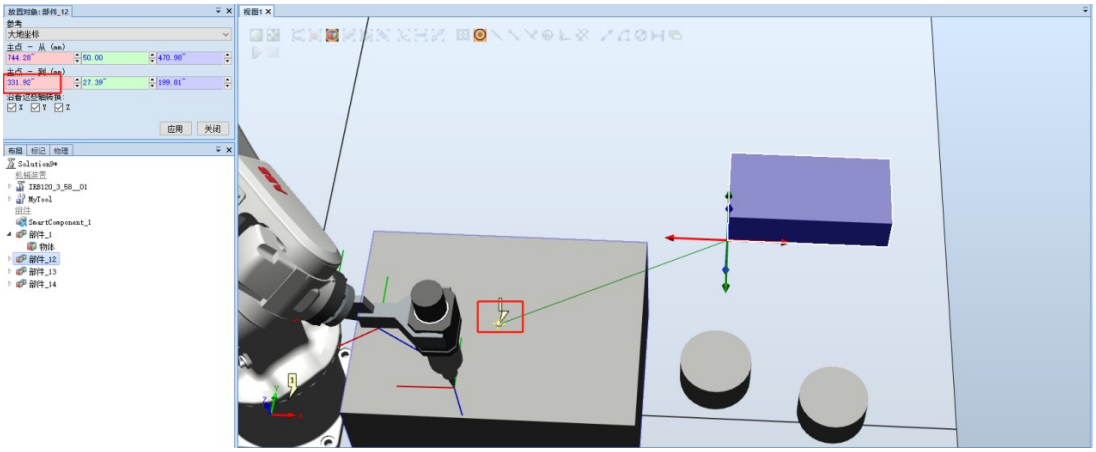


图 5.5“到”装配点选择

6.点击应用，就可以将矩形方块放置到台体的中心。

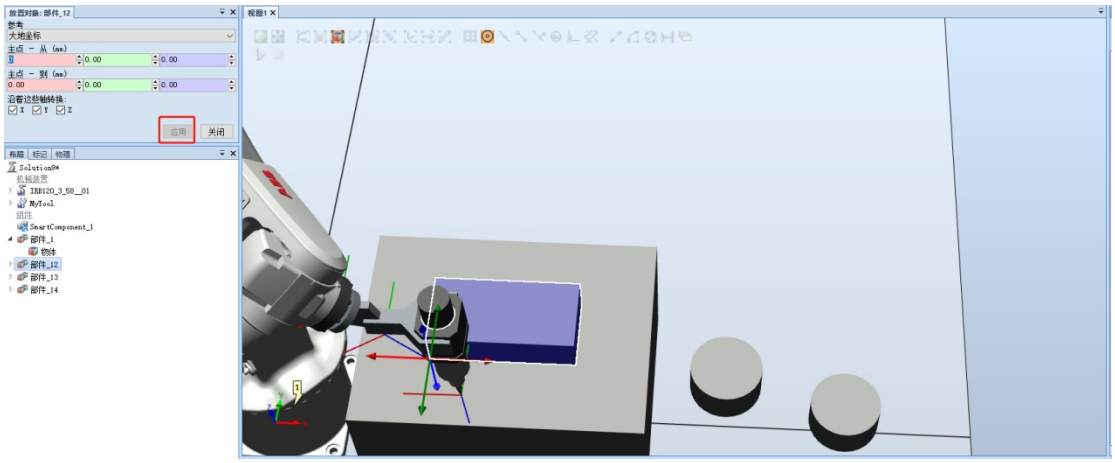


图 5.6 矩形方块装配完成

7.重复同样的操作原理，将两个圆柱体安装到矩形方块的两边。

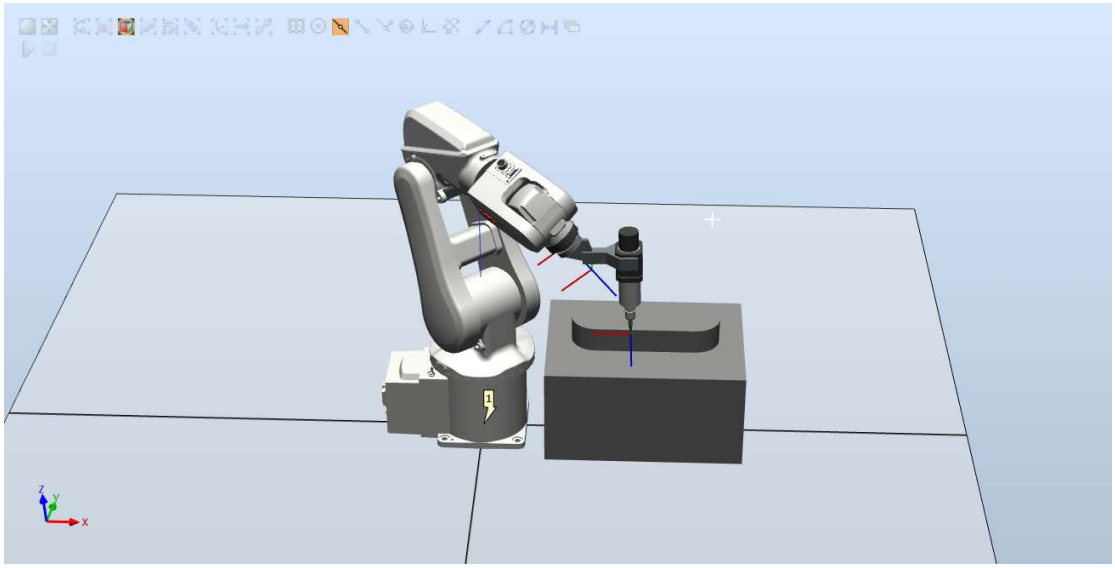


图 5.7 装配圆柱台体

8.放置好后，将三个物体组装起来，形成一个完整的新部件。选择结合，将部件_12 物体和部件_13 物体分别选择，去掉勾选保留初始位置，点击创建，就将两个物体组合到一起,生成一个新的部件_16。

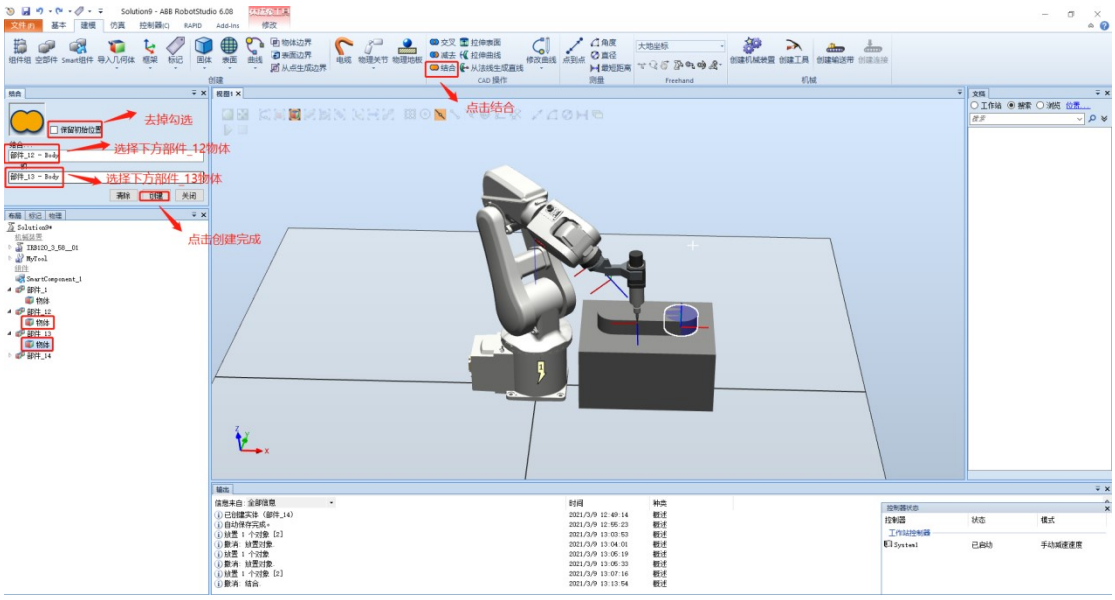


图 5.8 部件结合 1

9.重复上面的操作，将部件_16 和部件_14 组合到一起。

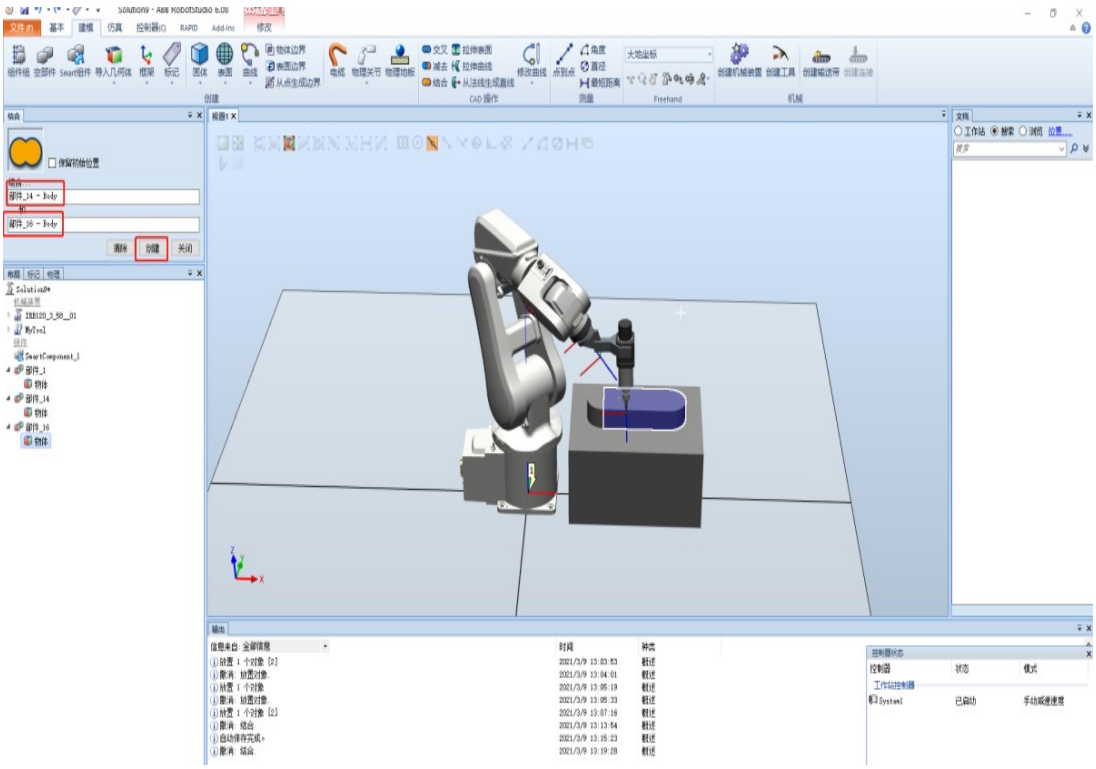


图 5.9 部件结合 2

10.得到一个新的部件_18,右键选择修改，点击设定颜色，将其颜色修改为红色。

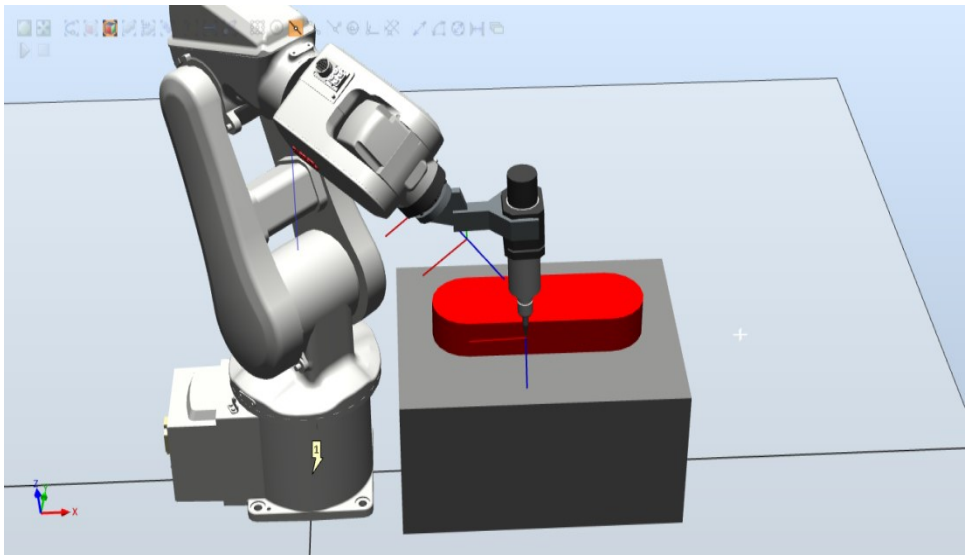


图 5.10 修改部件颜色

11.开始编写程序，使机器人焊枪末端的焊嘴沿部件_18 上边沿做焊接轨迹运动，轨迹示意图如下。

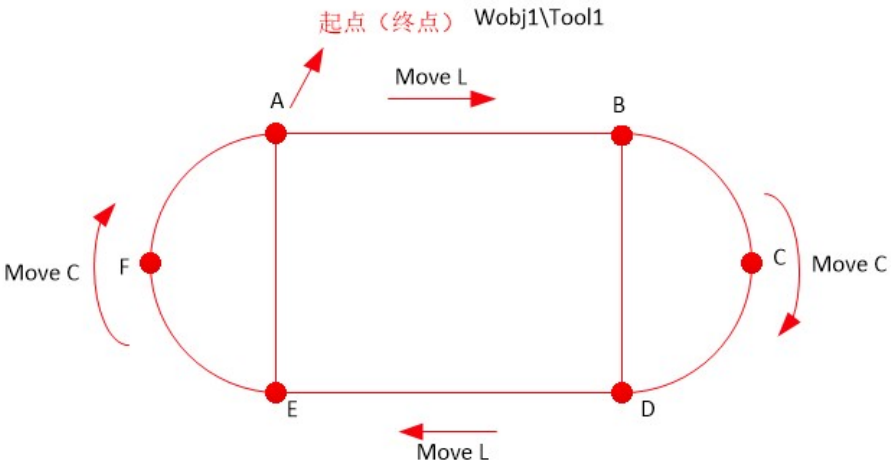


图 5.11 部件边缘轨迹示意图

12.编写上面的程序时，知道 A,B,C,D,E,F 之间的距离关系，只需要示教一个点位 A，就可以知道其他点位的位置，通过 offs 指令进行偏移就可以得到其他点位的坐标。这是其中的一种方法，另一种方法就是将机器人 A,B,C,D,E,F 轨迹上所有的点位进行示教，然后编写运动指令代码，也可以完成上面的轨迹，但是此方法比较笨重，需要示教很多点位。但是在实际生产过程中，不知道加工件的尺寸及位置关系就没法进行位置偏移，这个时候就需要示教每个点位进行轨迹运动。程序中运动位置都是基于 A 点进行偏移，所以可以将 A 点作为

一个常量进行定义。程序中添加了工件和工具坐标系，系统程序文件会自己生成数据如下图所示：

```
MODULE TestModule
    CONST robtarget p1:=[[293.59,-9.56,497.27],[3.94407E-5,8.94529E-6,1,-1.89021E-7],[-1,0,-1,0],[9E+9,9E+9,9E+9,9E+9,9E+9,9E+9]];
    CONST robtarget p2:=[[124.09,142.04,374.89],[3.83483E-5,9.05388E-6,1,-2.98581E-7],[0,-1,0,0],[9E+9,9E+9,9E+9,9E+9,9E+9,9E+9]];
    CONST robtarget p3:=[[516.88,142.04,374.89],[3.84595E-5,9.09881E-6,1,-3.01482E-7],[0,-1,0,0],[9E+9,9E+9,9E+9,9E+9,9E+9,9E+9]];
    CONST robtarget p4:=[[516.87,-153.14,374.89],[3.82592E-5,4.25901E-6,1,6.00488E-7],[-1,0,-1,0],[9E+9,9E+9,9E+9,9E+9,9E+9,9E+9]];
    CONST robtarget p5:=[[122.71,-153.14,374.89],[3.83232E-5,4.32312E-6,1,4.9544E-7],[-1,0,-1,0],[9E+9,9E+9,9E+9,9E+9,9E+9,9E+9]];
    CONST robtarget p6:=[[274.84,-2.07,371.39],[3.81202E-5,9.06695E-6,1,-2.31293E-7],[-1,0,-1,0],[9E+9,9E+9,9E+9,9E+9,9E+9,9E+9]];
    CONST robtarget Point_A:=[[241.15,12.13,416.61],[3.55029E-5,4.37354E-6,1,6.44499E-7],[0,-1,0,0],[9E+9,9E+9,9E+9,9E+9,9E+9,9E+9]];
    CONST robtarget Point_A10:=[[229.34,0.69,412.94],[3.57676E-5,4.41686E-6,1,6.94945E-7],[0,0,0,0],[9E+9,9E+9,9E+9,9E+9,9E+9,9E+9]];
    CONST robtarget Home:=[[349.26,13.03,508.49],[3.55662E-5,4.384E-6,1,6.59472E-7],[0,-1,0,0],[9E+9,9E+9,9E+9,9E+9,9E+9,9E+9]];
    CONST robtarget Point_A20:=[[349.26,13.03,508.49],[3.55662E-5,4.384E-6,1,6.59472E-7],[0,-1,0,0],[9E+9,9E+9,9E+9,9E+9,9E+9,9E+9]];
    CONST robtarget Point_B:=[[399.71,1.25,416.88],[3.57754E-5,4.38057E-6,1,6.47613E-7],[0,0,0,0],[9E+9,9E+9,9E+9,9E+9,9E+9,9E+9]];
    CONST robtarget A:=[[-5.56,225.91,-56.88],[0.00943377,-0.0918939,0.995715,-0.00418913],[0,-1,0,0],[9E+9,9E+9,9E+9,9E+9,9E+9,9E+9]];
    TASK PERS wobjdata wobj1:=[FALSE,TRUE,"",[[303.58,-147.313,308.935],[0.999938,-0.00562816,0.00930901,0.00169751]],[[0,0,0],[1,0,0,0]]];
    TASK PERS tooldata tool1:=[TRUE,[[32.7303,0.676437,229.185],[0.935327,-0.0234879,0.340979,-0.0913538]],[1,[1,0,0],[1,0,0,0],0,0,0]];
    PROC main()
        MoveJ p1, v1000, fine, tool0;
        MoveL p2, v1000, fine, tool0;
        MoveL p3, v1000, fine, tool0;
        MoveL p4, v1000, fine, tool0;
        MoveL p5, v1000, fine, tool0;
        MoveL p2, v1000, fine, tool0;
        MoveJ p1, v1000, fine, tool0;
    ENDPROC
    PROC MaDuo()
        VAR robtarget A1;
        VAR robtarget B1;
        A1:=Offs(Point_A,0,0,50);
        B1:=Offs(Point_B,0,0,50);
        MoveJ Home, v1000, z50, tool0;
        MoveL A1, v1000, z50, tool0;
        MoveL Point_A, v1000, z50, tool0;
        MoveL A1, v1000, z50, tool0;
        MoveL B1, v1000, z50, tool0;
        MoveL Offs(Point_B,0,0,20), v1000, z50, tool0;
        MoveL Point_B, v1000, z50, tool0;
        MoveL B1, v1000, z50, tool0;
        MoveJ Home, v1000, z50, tool0;
    ENDPROC
    PROC GUIJI()
        VAR robtarget B;
        VAR robtarget C;
        VAR robtarget D;
        VAR robtarget E;
        VAR robtarget F;
        B:=Offs(A,200,0,0);
        C:=Offs(A,250,-50,0);
        D:=Offs(A,200,-100,0);
        E:=Offs(A,0,-100,0);
        F:=Offs(A,-50,-50,0);
        MoveJ A, v1000, z50, tool1\WObj:=wobj1;
        WHILE TRUE DO
            MoveL A,v1000,fine,tool1\WObj:=wobj1;
            MoveL B,v1000,fine,tool1\WObj:=wobj1;
            MoveC C,D,v1000,fine,tool1\WObj:=wobj1;
            MoveL E,v1000,fine,tool1\WObj:=wobj1;
            MoveC F,A,v1000,fine,tool1\WObj:=wobj1;
        ENDWHILE
    ENDPROC
ENDMODULE
```

图 5.12 轨迹程序

```
VAR robtarget B;定义robtarget类型的位置B变量
VAR robtarget C;  定义robtarget类型的位置C变量
VAR robtarget D;  定义robtarget类型的位置D变量
VAR robtarget E;  定义robtarget类型的位置E变量
VAR robtarget F;  定义robtarget类型的位置F变量
B:=Offs(A,200,0,0);  给变量B的位置进行赋值，B的位置是基于A偏移
C:=Offs(A,250,-50,0);
D:=Offs(A,200,-100,0);
E:=Offs(A,0,-100,0);
F:=Offs(A,-50,-50,0);
MoveJ A, v1000, z50, tool1\WObj:=wobj1;
WHILE TRUE DO   死循环，使机器人一直绕边沿运动
    MoveL A, v1000, fine, tool1\WObj:=wobj1;
    MoveL B, v1000, fine, tool1\WObj:=wobj1;
    MoveC C,D,v1000, fine, tool1\WObj:=wobj1;
    MoveL E, v1000, fine, tool1\WObj:=wobj1;
    MoveC F,A,v1000, fine, tool1\WObj:=wobj1;
ENDWHILE
```

图 5.13 部分程序图

实验练习：在仿真软件中，创建台体的大小为 500*400*300mm，再在台体上创建一个 300*100*60mm 矩形方块，两个直径 100*高 60mm 的圆柱体，将台体上三个物体组合到一起形成部件，编写该部件边缘运动轨迹程序并调试出来。

第六章　基于 Socket 通讯控制机器人运动实验

实验目的：学习 Socket 通讯方法，字符数据的处理和 6 轴单独运动控制

实验要求：掌握 Socket 通讯方法，字符数据的处理和 6 轴单独运动控制

实验设备：型号 MFTW-IR 和 CHL-JC-02-A 的工作站

ABB 机器人通讯常用的通讯有两种方式，一种是 Socket 网络，一种是 Profinet，机器人能使用这两种通讯，需要机器人控制器本身具有 616-1 PC Interface 和 888-2 PROFINET Controller/Device 这两个功能选项。



图 6.1 选项功能

Socket 通讯是 TCP 网络通讯中的一种。Socket 通讯需要有服务端和客户端。服务端和客户端之间需要确定通讯的 IP 地址和端口号。根据固定的 IP 地址和端口号进行定向接收和发送数据。服务端与客户端的区别类似于主从关系，只有在服务端建立了一个固定的 IP 地址和端口号的服务，客户端才能连接服务端，才能建立通讯。如果服务端关闭服务，客户端就不能进行通讯连接，故服务端是主，客户端是从。一个服务端能同时允许很多客户端连接，但一个客户端只能连接一个服务端。根据这种关系和实际情况来创建所需要是客户端还是服务端。

例如：在 ABB 机器人控制试验中，需要通过视觉、PLC、树莓派等外部器件与机器人通讯，故需要机器人同时连接多台设备，机器人就作为服务端，视觉、

PLC、树莓派就作为客户端，这样机器人就能允许任何设备同时连接，通过 IP 地址和端口号，每个客户端在各自的信道通讯而相互不干扰。

在编写通讯程序之前，学习 ABB 机器人用于 socket 通讯的相关指令及函数；

1.服务端常用指令：

SocketCreat：创建一个新套接字；

SocketBind：将套接字的 IP 与端口号绑定；

SocketListen：监听客户端连接；

SocketAccept：接收客户端连接；

SocketReceive：接收数据；

SocketSend：发送数据；

SocketClose：关闭套接字；

2.客户端常用指令：

SocketCreat：创建一个套接字；

SocketConnect：连接服务端；

SocketReceive：接收数据；

SocketSend：发送数据；

SocketClose：关闭套接字；

指令的具体定义及用法见下表函数指令解释：

1.222 SocketCreate - 创建新套接字

手册用法

SocketCreate用于针对基于通信或非连接通信的连接，创建新的套接字。

带有交付保证的流型协议TCP/IP以及数据电报协议UDP/IP的套接字消息传送均得到支持。可开发服务器和客户端应用。针对数据电报协议UDP/IP，支持采用广播。

基本示例

以下实例介绍了指令SocketCreate：

例 1

```
VAR socketdev socket1;
...
SocketCreate socket1;
```

创建使用流型协议TCP/IP的新套接字设备，并分配到变量socket1。

例 2

```
VAR socketdev udp_sock1;
...
SocketCreate udp_sock1 \UDP;
```

创建使用数据电报协议UDP/IP的新套接字设备，并分配到变量udp_sock1。

变元

```
SocketCreate Socket [\UDP]
```

Socket

数据类型：socketdev

用于储存系统内部套接字数据的变量。

[\UDP]

数据类型：switch

指定套接字应当为数据电报型协议UDP/IP。

程序执行

本指令创建新的套接字设备。

套接字不得业已使用。套接字在SocketCreate和SocketClose之间使用。

限制

可声明任意数量的套接字，但是仅可能同时使用32个套接字。

通过SocketCreate ... SocketClose，避免快回路，因为在某一特定时间之前，套接字并未真正关闭（当使用TCP/IP功能时）。

语法

```
SocketCreate
  [ Socket ':=' ] < variable (VAR) of socketdev >
  [ '\' UDP ] ';'
```

下一页继续

图 6.2 SocketCreat 函数释义

1.219 SocketBind - 将套接字与我的IP地址和端口绑定

手册用法

SocketBind用于将套接字与指定服务器IP地址和端口号绑定。SocketBind仅可用于服务器应用。

基本示例

以下实例介绍了指令SocketBind：

例 1

```
VAR socketdev server_socket;

SocketCreate server_socket;
SocketBind server_socket, "192.168.0.1", 1025;
```

创建服务器套接字，并与地址为192.168.0.1的控制器网络上的端口1025绑定。现在，可在SocketListen指令中使用服务器套接字，以监听位于该端口和地址上的输入连接。

变元

```
SocketBind Socket LocalAddress LocalPort
```

Socket

数据类型：socketdev

有待绑定的服务器套接字。必须创建尚未绑定的套接字。

LocalAddress

数据类型：string

将服务器网络地址与套接字绑定。唯一有效的地址为所有公共WAN地址，或控制器服务端口地址192.168.125.1。

LocalPort

数据类型：num

将服务器端口号与套接字绑定。通常，可自由使用端口1025-4999。

程序执行

将服务器套接字与指定服务器端口和IP地址绑定。

如果已在使用指定端口，则会产生错误。

在启动程序的过程中，使用SocketBind和SocketListen指令，从而使局部地址与套接字相关联，随后，监听指定端口上的输入连接。针对用到的各套接字和端口（TCP/IP），仅建议实施上述操作一次。

如果通过SocketReceiveFrom（UDP/IP）来接收数据，则使用SocketBind指令。

错误处理

系统会生成下列可恢复错误，并在错误处理器中处理这些错误。系统变量ERRNO将被设置成：

ERR_SOCK_CLOSED	关闭套接字（已经关闭或未创建） 将SocketCreate用于创建新的套接字。

下一页继续

图 6.3 SocketBind 函数释义

1.223 SocketListen - 监听输入连接

手册用法

SocketListen用于开始监听输入连接，即开始充当服务器。SocketListen仅可用于服务器应用。

基本示例

以下实例介绍了指令SocketListen：

例 1

```
VAR socketdev server_socket;
VAR socketdev client_socket;
...
SocketCreate server_socket;
SocketBind server_socket, "192.168.0.1", 1025;
SocketListen server_socket;
WHILE listening DO;
  ! Waiting for a connection request
  SocketAccept server_socket, client_socket;
```

创建服务器套接字，并与地址为192.168.0.1的控制器网络上的端口1025绑定。在执行SocketListen后，服务器套接字开始监听位于该端口和地址上的输入连接。

变元

```
SocketListen Socket
```

Socket

数据类型：socketdev

应当开始监听输入连接的服务器套接字。必须已经创建和绑定套接字。

程序执行

服务器套接字开始监听输入连接。当指令就绪时，套接字便准备好接受输入连接。

在启动程序的过程中，使用SocketBind和SocketListen指令，从而使局部地址与套接字相关联，随后，监听指定端口上的输入连接。针对用到的各套接字和端口，仅建议实施上述操作一次。

错误处理

系统会生成下列可恢复错误，并在错误处理器中处理这些错误。系统变量ERRNO将被设置成：

ERR_SOCK_CLOSED	关闭套接字（已经关闭或未创建）。 将SocketCreate用于创建新的套接字。

语法

```
SocketListen
  [ Socket ':=' ] < variable (VAR) of socketdev > ';'
```

图 6.4 SocketListen 函数释义

1.218 SocketAccept - 接受输入连接

手册用法

SocketAccept用于接受输入连接请求。SocketAccept仅可用于服务器应用。

基本示例

以下实例介绍了指令SocketAccept：

另请参阅第563页的更多示例

例 1

```
VAR socketdev server_socket;
VAR socketdev client_socket;
...
SocketCreate server_socket;
SocketBind server_socket,"192.168.0.1", 1025;
SocketListen server_socket;
SocketAccept server_socket, client_socket;
```

创建服务器套接字，并绑定至地址为192.168.0.1的控制器网络上的端口1025。在执行SocketListen之后，服务器套接字开始监听位于该端口和地址上输入连接。SocketAccept等待所有输入连接，接受连接请求，并返回已建立连接的客户端套接字。

变元

```
SocketAccept Socket ClientSocket [\ClientAddress] [ \Time ]
```

Socket

数据类型：socketdev

正在等待输入连接的服务器套接字。套接字必须已经创建、绑定并准备进行监听。

ClientSocket

数据类型：socketdev

将按照接受的输入连接请求，对返回的新客户端套接字进行更新。

[\ClientAddress]

数据类型：string

将通过已接受输入连接请求的IP地址来进行更新的变量。

[\Time]

数据类型：num

程序执行等待输入连接的最长时间量。如果在任意输入连接之前耗尽时间，则将调用错误处理器，如果存在这样的情况，则采用错误代码ERR_SOCK_TIMEOUT。如果不存在错误处理器，则将停止执行。

如果未使用参数\Time，则等待时间为60秒。为了永久等待，则使用预定义常量WAIT_MAX。

程序执行

服务器套接字将等待所有输入连接请求。当接受输入连接请求时，本指令就绪，且返回的客户端套接字默认连接，并可用于SocketSend和SocketReceive指令。

下一页继续

图 6. 5SocketAccept 函数释义

1.224 SocketReceive - 接收来自远程计算机的数据

手册用法

SocketReceive用于从远程计算机接收数据。SocketReceive可用于客户端和服务器应用。

基本示例

以下实例介绍了指令SocketReceive：

另请参阅第577页的更多示例

例 1

```
VAR string str_data;
...
SocketReceive socket1 \Str := str_data;
```

从远程计算机接收数据，并将其储存在字符串变量str_data中。

变元

```
SocketReceive Socket [ \Str ] | [ \RawData ] | [ \Data ]
     [\ReadNoOfBytes] [\NoRecBytes] [\Time]
```

Socket

数据类型：socketdev

在套接字接收数据的客户端应用中，必须已经创建和连接套接字。

在套接字接收数据的服务器应用中，必须已经接受套接字。

[\Str]

数据类型：string

应当储存接收string数据的变量。可处理最多80个字符。

[\RawData]

数据类型：rawbytes

应当储存接收原始数据字节数据的变量。可处理最多1024个rawbytes。

[\Data]

数据类型：array of byte

应当储存接收字节数据的变量。可处理最多1024个byte。

仅可在同一时间使用可选参数\Str、 \RawData和\Data之一。

[\ReadNoOfBytes]

Read number of Bytes

数据类型：num

待读取的字节数。待读取字节的最小值为1，且最大数量为所用数据类型的规模，即如果使用数据类型string的变量，则为80个字节。

如果与始终发送固定字节数的客户端进行通信，则该可选参数可用于指定针对各SocketReceive指令所应读取的相同字节数量。

如果发送方发送原始数据，则接收方需要指定针对已发送的各rawbytes所应接收的4个字节。

下一页继续

图 6.6　SocketReceive 函数释义

1.226 SocketSend - 向远程计算机发送数据

手册用法

SocketSend用于向远程计算机发送数据。SocketSend可用于客户端和服务器应用。

基本示例

以下实例介绍了指令SocketSend：

另请参阅第585页的更多示例

例 1

```
SocketSend socket1 \Str := "Hello world";
```

将消息"Hello world"发送给远程计算机。

变元

```
SocketSend Socket [ \Str ] | [ \RawData ] | [ \Data] [ \NoOfBytes
    ]
```

Socket

数据类型：socketdev

在客户端应用中，必须已经创建和连接用于发送的套接字。

在服务器应用中，必须已经接受用于发送的套接字。

[\Str]

数据类型：string

将string发送到远程计算机。

[\RawData]

数据类型：rawbytes

将rawbytes数据发送到远程计算机。

[\Data]

数据类型：array of byte

将byte数组中的数据发送到远程计算机。

仅可在同一时间使用可选参数\Str、\RawData或\Data之一。

[\NoOfBytes]

数据类型：num

如果指定该参数，则仅将该字节数发送到远程计算机。如果\NoOfBytes大于待发送数据结构中的实际字节数，则SocketSend调用将失败。

如果未指定该参数，则将整个数据结构（rawbytes的有效部分）发送到远程计算机。

图 6.7　SocketSend 函数释义

1.220 SocketClose - 关闭套接字

手册用法

当不再使用套接字连接时，使用SocketClose。

在已经关闭套接字之后，不能将其用于除SocketCreate以外的所有套接字调用。

基本示例

以下实例介绍了指令SocketClose：

例 1

```
SocketClose socket1;
```

关闭套接字，且不能再进行使用。

变元

```
SocketClose Socket
```

Socket

数据类型：socketdev

有待关闭的套接字。

程序执行

将关闭套接字，并将释放其配置资源。

可在任意时间关闭任意套接字。关闭后，不能再使用套接字。在调用SocketCreate之后，可将其再次用于新的连接。

限制

在通过SocketSend发送数据后，随即关闭套接字连接，这会导致失去发送的数据。因为TCP/IP套接字具有在出现通信问题时重新发送数据的内置功能。

为避免有关数据丢失的此类问题，应在SocketClose之前进行以下操作：

- 交换关闭信号，或
- WaitTime 2

通过SocketCreate ... SocketClose，避免快回路，因为在某一特定时间之前，套接字并未真正关闭（TCP/IP功能）。

语法

```
SocketClose
  [ Socket ':=' ] < variable (VAR) of socketdev > ';'
```

相关信息

信息，关于	更多信息
一般的套接字通信	应用手册 - 控制器软件*IRC5*，套接字消息传送一节
创建新套接字	第*572*页的*SocketCreate* - 创建新套接字
连接远程计算机（仅客户端）	第*569*页的*SocketConnect* - 连接远程计算机
向远程计算机发送数据	第*584*页的*SocketSend* - 向远程计算机发送数据

下一页继续

图 6.8　SocketClose 函数释义

1.221 SocketConnect - 连接远程计算机

手册用法

SocketConnect用于将套接字与客户端应用中的远程计算机相连。

基本示例

以下实例介绍了指令SocketConnect：

另请参阅第569页的更多示例

例 1

```
SocketConnect socket1, "192.168.0.1", 1025;
```

尝试与ip地址192.168.0.1和端口1025处的远程计算机相连。

变元

```
SocketConnect Socket Address Port [\Time]
```

Socket

数据类型：socketdev

有待连接的服务器套接字。必须创建尚未连接的套接字。

Address

数据类型：string

远程计算机的地址。必须将远程计算机指定为一个IP地址。不可能使用远程计算机的名称。

Port

数据类型：num

位于远程计算机上的端口。通常，可自由使用端口1025-4999。可能已经采用了低于1025的端口。

[\Time]

数据类型：num

程序执行等待接受或否定连接的最长时间量。如果在满足条件之前耗尽时间，则将调用错误处理器，如果存在这样的情况，则采用错误代码ERR_SOCK_TIMEOUT。如果不存在错误处理器，则将停止执行。

如果未使用参数\Time，则等待时间为60秒。为了永久等待，则使用预定义常量WAIT_MAX。

程序执行

套接字试图连接指定地址和端口上的远程计算机。程序执行将等待，直至连接得以建立、失效或出现超时。

更多示例

有关于指令SocketConnect的更多例子阐述如下。

例 1

```
VAR num retry_no := 0;
VAR socketdev my_socket;
```

下一页继续

图 6.9　SocketConnect 函数释义

机器人作为服务端创建一个 Socket 程序，接收并发送字符；

```
PROC SocketServer()
    VAR socketdev server_socket1;定义一个服务端；
    VAR socketdev client_socket1;定义一个客户端；
    VAR string client_ip:="10.70.2.240";定义客户端的IP地址；
    VAR string receive_string;定一个存储接收的字符串；
    SocketClose server_socket1;在创建socket之前先关闭，保证每次都能创建成功；
    SocketClose client_socket1;
    SocketCreate server_socket1;创建服务端套接字；
    SocketBind server_socket1,"10.70.2.252",3000;将机器人服务IP和端口号绑定；
    SocketListen server_socket1;开启监听；
SocketAccept server_socket1,client_socket1\ClientAddress:=client_ip\Time:=WAIT_MAX;
```

开启接收客户端连接，连接的等待时间为无限长，直到客户端连接上，如果没有连接上一直等待连接；

```
    WHILE TRUE DO
        SocketReceive client_socket1\str:=receive_string\Time:=WAIT_MAX;用于接收
         客户端发来的字符串，等待时间无无限长；
        TPWrite "receive_string="+receive_string;将接收的字符串打印输出显示；
        SocketSend client_socket1\Str:="Hello World";给客户端发送“Hello Word”；
    ENDWHILE
ENDPROC
```



图 6.10 通讯程序图

运行示教器上的 SocketServer 例行程序；

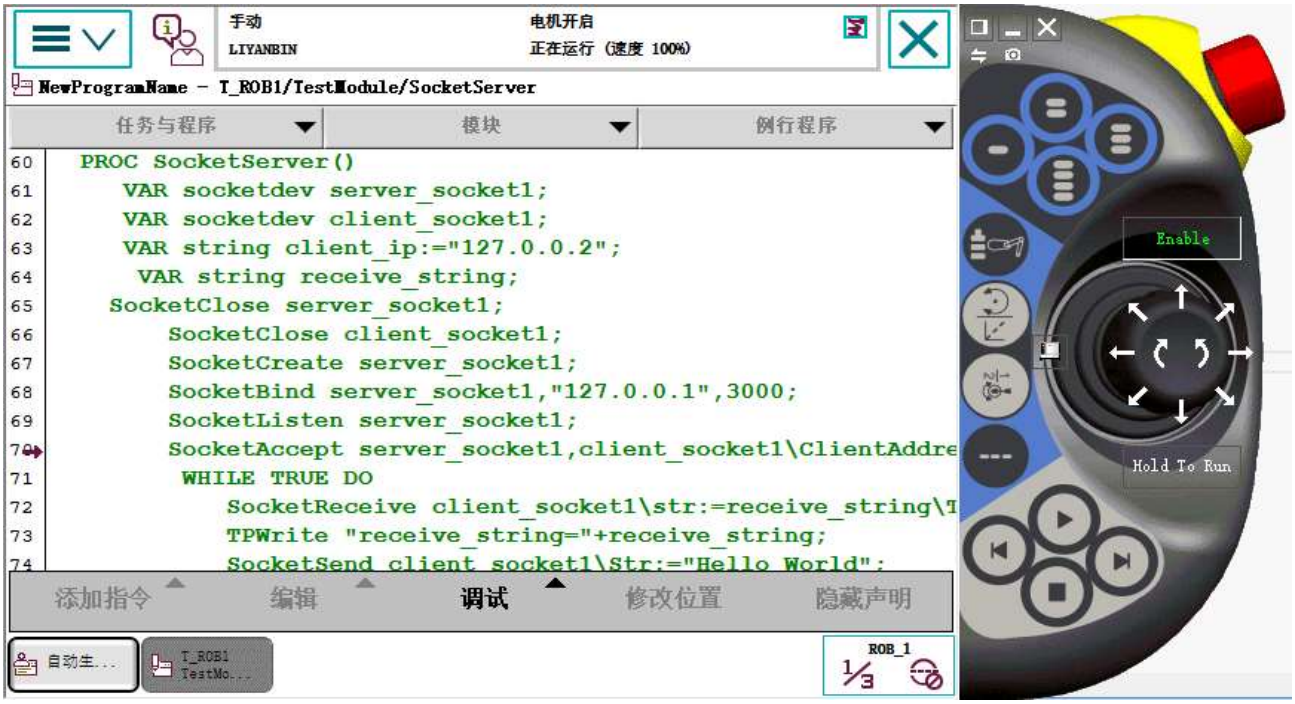


图 6.11 示教器通讯程序界面

打开网络调试助手，创建一个客户端，连接服务端，发送数据：

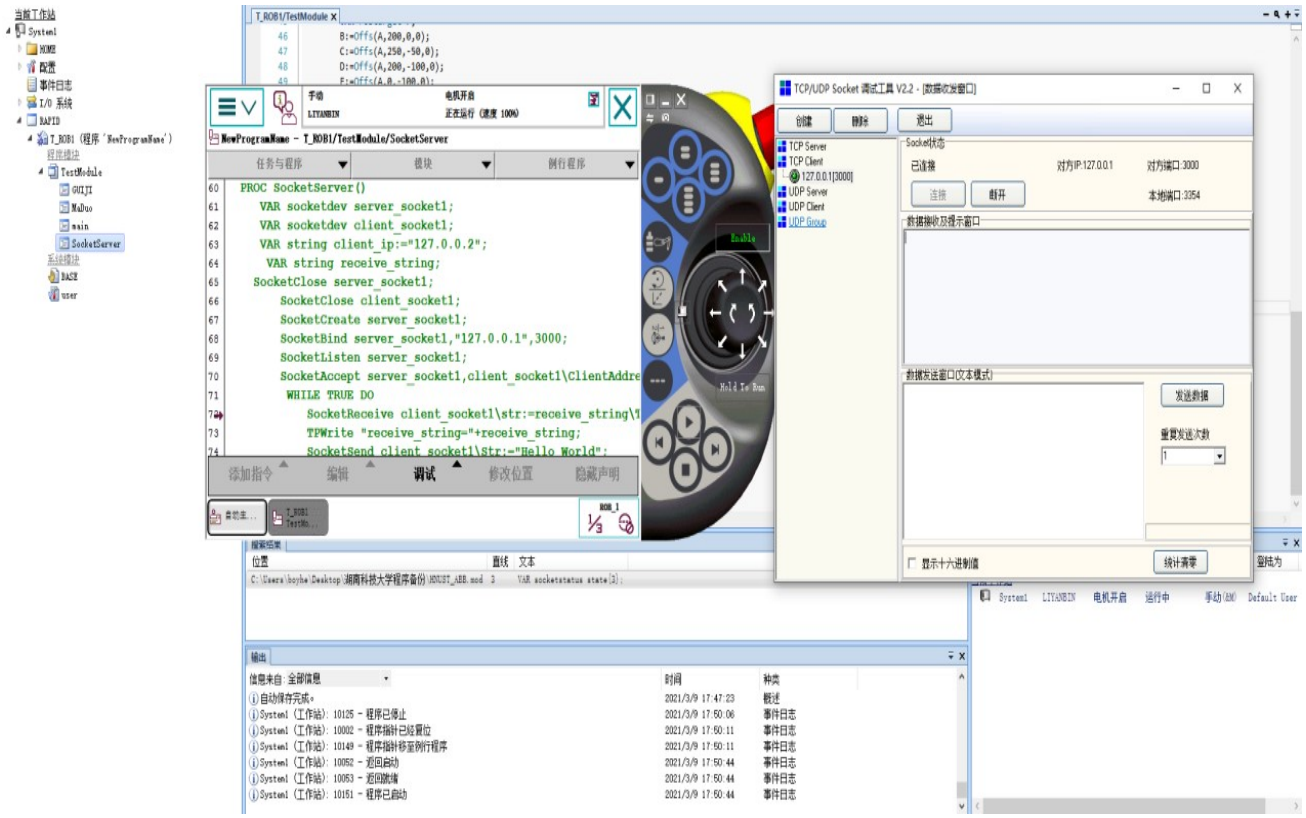


图 6.12 网络调试助手创建客户端连接

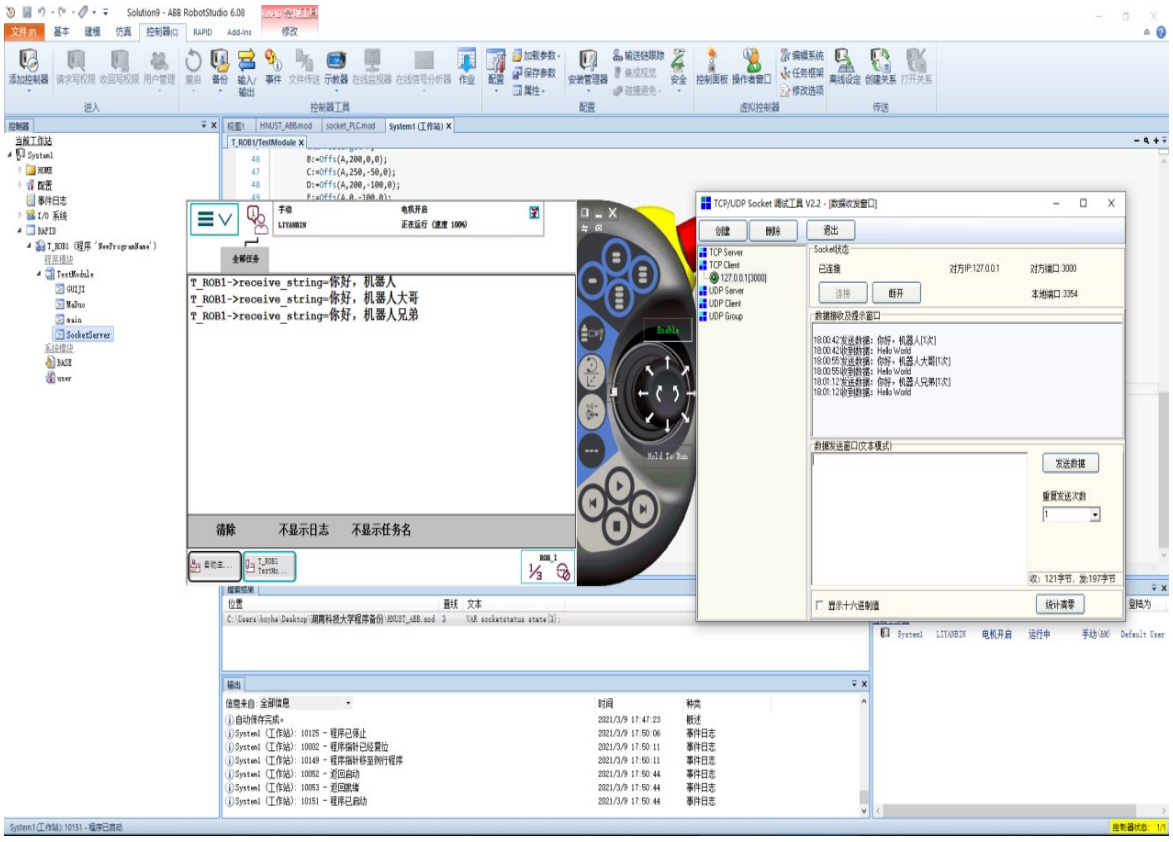


图 6.13 发送数据测试

注：在真实的机器人上面实现 socket 通讯，需要配置电脑 IP 和机器人 Server 网口（固定 IP：192.168.125.1）或者机器人 WAN 网口，LAN 网口的 IP 地址在同一网段。

示教器设置方法：打开控制面板，选择配置系统参数，主题选择 Communication，选择 IP Setting，点击添加，输入相应的 IP 地址，interface 选择定义为哪个网口的 IP（LAN，LAN3，WAN），设置好确定即可。



图 6.14 示教器网络端口配置

Robotstudio 软件设置方法：选择任务栏“控制器”，点击配置，选择 communication，选择 IP Setting，右键新建，输入 IP 地址，interface 选择对应网口，点击确定即可。

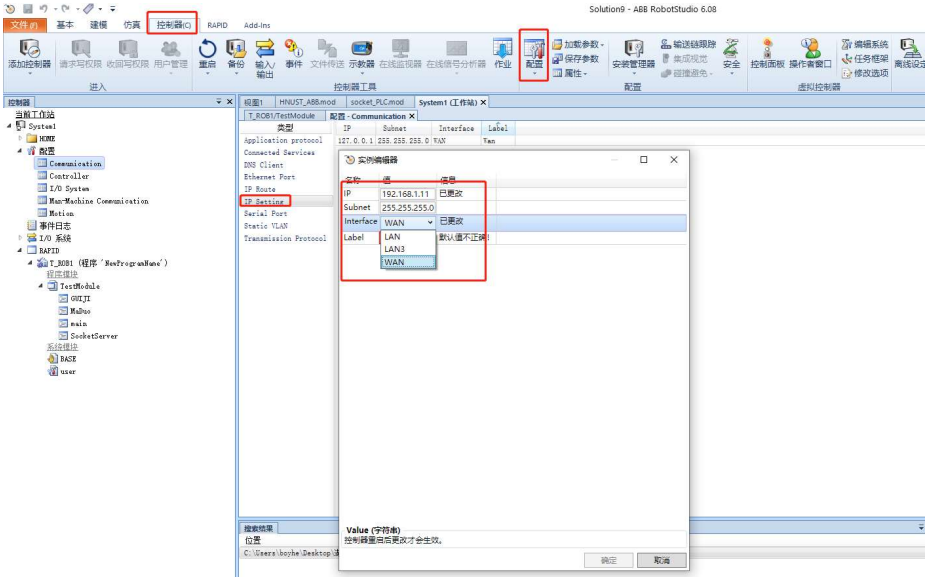


图 6.15 robotstudio 软件配置网络端口

任务：在机器人端创建客户端，用网络调试助手创建服务端，使机器人连接，并接收和发送数据。

基于 socket 通讯控制机器人运动可分为三个部分，一个是通讯程序，一个是数据处理，一个是控制指令。

数据处理有很多种方式，以下只讲解一种，其余的大家可以根据 ABB 指令手册去查找相关的指令，实现的方法很多，掌握其中的一种即可。

机器人接收到数据转换为值类型数据，如果控制机器人运动，就是需要将值类型的数据赋值给机器人点位坐标，机器人按照赋值后的点位数据进行移动，达到控制的目的。

机器人处理数据方法一：按位分割，将一连串任意字符的字符串按照取位数进行任意分组，比如-123a,456;，总共是 10 位字符组成的字符串，如果按照取位数为 2、3、4、1 的分组方式，第一组是“-1”，第二组是“23a”,第三组是“,456”，第四组是“;”。

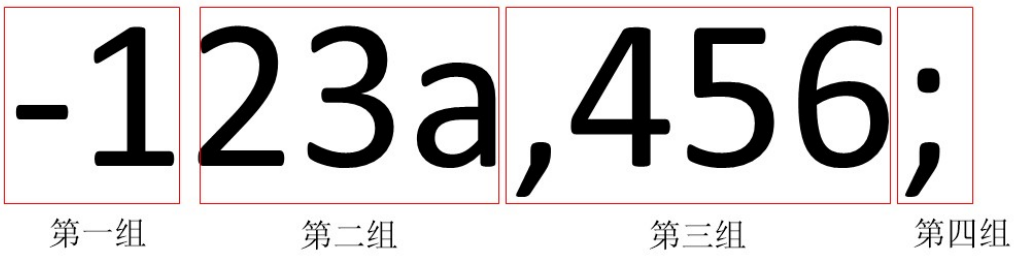


图 6.16 字符串按位分组示意图

机器人数据按位分割函数：StrPart；

2.164 StrPart - 寻找一部分字符串

手册用法

StrPart（*String Part*)用于寻找一部分字符串，以作为一个新的字符串。

基本示例

以下示例介绍了函数StrPart。

例 1

```
VAR string part;
part := StrPart("Robotics",1,5);
```

变量part被赋予值"Robot"。

返回值

数据类型：string

指定字符串的子串，其拥有规定的长度，并始于指定字符位置。

变元

```
StrPart (Str ChPos Len)
```

Str

String

数据类型：string

字符串，有待发现其组成部分。

ChPos

Character Position

开始字符位置。如果位于字符串以外，则产生运行时错误。

Len

长度

数据类型：num

字符串组成部分的长度。如果长度为负或大于字符串的长度，或者如果子串（部分地）位于字符串之外，则会产生运行时错误。

语法

```
StrPart '('
[ Str ':=' ] <expression (IN) of string> ','
[ ChPos ':=' ] <expression (IN) of num> ','
[ Len ':=' ] <expression (IN) of num> ')'
```

返回值的数据类型是 string 的函数。

相关信息

信息，关于	更多信息
字符串功能	技术参考手册 - *RAPID*语言概览
字符串的定义	*第1391页的string - 字符串*

下一页继续

图 6.17　StrPart 函数释义

6.1 控制机器人 6 个轴单独运动程序

基于上节程序进行更改如下：

```
PROC SocketServer()
    VAR socketdev server_socket1;
    VAR socketdev client_socket1;
    VAR string client_ip:="10.70.2.240";客户端的IP地址
    VAR string receive_string;
    VAR string string_j1:="";  定义用于分割取位后存储字符的字符串
    VAR string string_j2:="";
    VAR string string_j3:="";
    VAR string string_j4:="";
    VAR string string_j5:="";
    VAR string string_j6:="";
    VAR num num_j1:=0;定义将取位后的字符串转换成值类型的数据存储
    VAR num num_j2:=0;
    VAR num num_j3:=0;
    VAR num num_j4:=0;
    VAR num num_j5:=0;
    VAR num num_j6:=0;
    VAR bool change_ok:=false;  转换标志位
    VAR robtarget current_position_robT;  定义robtarget类型的坐标点
    VAR jointtarget current_position_jointT;  定义jointtarget类型的坐标点
        SocketClose server_socket1;
        SocketClose client_socket1;
        SocketCreate server_socket1;
        SocketBind server_socket1," 10.70.2.252",3000;机器人端IP地址和端口
        SocketListen server_socket1;
        SocketAccept
server_socket1,client_socket1\ClientAddress:=client_ip\Time:=WAIT_MAX;
        WHILE TRUE DO
```

```
                SocketReceive
client_socket1\str:=receive_string\Time:=WAIT_MAX;
!                TPWrite "receive_string="+receive_string;
!                SocketSend client_socket1\Str:="Hello World";
            string_j1:=StrPart(receive_string,1,6);对接收的字符串按位分割，每6位
                                                    为一组坐标（例：-321.1保留两位
                                                    小数，正好可以满足机器人坐标数
                                                    据使用。）
            string_j2:=StrPart(receive_string,7,6);第7位开始的6个字符串
            string_j3:=StrPart(receive_string,13,6);  第13位开始的6个字符串
            string_j4:=StrPart(receive_string,19,6);  第19位开始的6个字符串
            string_j5:=StrPart(receive_string,25,6);  第25位开始的6个字符串
            string_j6:=StrPart(receive_string,31,6);  第31位开始的6个字符串
            change_ok:=StrToVal(string_j1,num_j1);!j1将接收的字符串转换为值类
                                                    型的数据（值可以是整数也可
                                                    以是小数）
            change_ok:=StrToVal(string_j2,num_j2);!j2
            change_ok:=StrToVal(string_j3,num_j3);!j3
            change_ok:=StrToVal(string_j4,num_j4);!j4
            change_ok:=StrToVal(string_j5,num_j5);!j5
            change_ok:=StrToVal(string_j6,num_j6);!j6
            current_position_robT:= CRobT(\Tool:=tool1\wobj:=wobj1);获取当前坐
                                                    标点的数据赋值给current_position_robT点位。
            current_position_jointT:=CalcJointT(current_position_robT,tool1
\WObj:=wobj1);将robtarget型数据转换为jointtarget。
current_position_jointT:=Absjoffs(current_position_jointT,num_j1,num_j2,num_j3,n
um_j4,num_j5,num_j6);赋值函数
            MoveAbsJ current_position_jointT\NoEOffs, v1000, fine, tool1;
            ENDWHILE
        ENDPROC
```

```
FUNC jointtarget Absjoffs(jointtarget Point,num j10ffset,num j20ffset,num
J30ffset,num J40ffset,num J50ffset,num J60ffset)  偏移函数
    VAR jointtarget joint_temp;
    joint_temp:=Point;
    joint_temp.robax.rax_1:=joint_temp.robax.rax_1+J10ffset;对轴数据进行
                                                          赋值。
    joint_temp.robax.rax_2:=joint_temp.robax.rax_2+J20ffset;
    joint_temp.robax.rax_3:=joint_temp.robax.rax_3+J30ffset;
    joint_temp.robax.rax_4:=joint_temp.robax.rax_4+J40ffset;
    joint_temp.robax.rax_5:=joint_temp.robax.rax_5+J50ffset;
    joint_temp.robax.rax_6:=joint_temp.robax.rax_6+J60ffset;
    RETURN joint_temp;
ENDFUNC
```

```
16  PROC main
25  PROC MaDuo
40  PROC GUIJi
60  PROC SocketServer()
61    VAR socketdev server_socket1;
62    VAR socketdev client_socket1;
63    VAR string client_ip:="127.0.0.2";
64    VAR string receive_string;
65    VAR string string_j1:="";
66    VAR string string_j2:="";
67    VAR string string_j3:="";
68    VAR string string_j4:="";
69    VAR string string_j5:="";
70    VAR string string_j6:="";
71    VAR num num_j1:=0;
72    VAR num num_j2:=0;
73    VAR num num_j3:=0;
74    VAR num num_j4:=0;
75    VAR num num_j5:=0;
76    VAR num num_j6:=0;
77    VAR bool change_ok:=false;
78    VAR robtarget current_position_robT;
79    VAR jointtarget current_position_jointT;
80       SocketClose server_socket1;
81       SocketClose client_socket1;
82       SocketCreate server_socket1;
83       SocketBind server_socket1,"127.0.0.1",3000;
84       SocketListen server_socket1;
85       SocketAccept server_socket1,client_socket1\ClientAddress:=client_ip\Time:=WAIT_MAX;
86        WHILE TRUE DO
87          SocketReceive client_socket1\Str:=receive_string\Time:=WAIT_MAX;
88  !        TPWrite "receive_string="+receive_string;
89  !        SocketSend client_socket1\Str:="Hello World";
90        string_j1:=StrPart(receive_string,1,6);
91        string_j2:=StrPart(receive_string,7,6);
92        string_j3:=StrPart(receive_string,13,6);
93        string_j4:=StrPart(receive_string,19,6);
94        string_j5:=StrPart(receive_string,25,6);
95        string_j6:=StrPart(receive_string,31,6);
96        change_ok:=StrToVal(string_j1,num_j1);!j1
97        change_ok:=StrToVal(string_j2,num_j2);!j2
98        change_ok:=StrToVal(string_j3,num_j3);!j3
99        change_ok:=StrToVal(string_j4,num_j4);!j4
100       change_ok:=StrToVal(string_j5,num_j5);!j5
101       change_ok:=StrToVal(string_j6,num_j6);!j6
102       current_position_robT:= CRobT(\Tool:=tool1\wobj:=wobj1);
103       current_position_jointT:=CalcJointT(current_position_robT,tool1 \WObj:=wobj1);
104       current_position_jointT:=Absjoffs(current_position_jointT,num_j1,num_j2,num_j3,num_j4,num_j5,num_j6);
105       MoveAbsJ current_position_jointT\NoEOffs, v1000, fine, tool1;
106       ENDWHILE
107   ENDPROC
108      FUNC jointtarget Absjoffs(jointtarget Point,num j10ffset,num j20ffset,num J30ffset,num J40ffset,num J50ffset,num J60
109      VAR jointtarget joint_temp;
110      joint_temp:=Point;
111      joint_temp.robax.rax_1:=joint_temp.robax.rax_1+J10ffset;
112      joint_temp.robax.rax_2:=joint_temp.robax.rax_2+J20ffset;
113      joint_temp.robax.rax_3:=joint_temp.robax.rax_3+J30ffset;
114      joint_temp.robax.rax_4:=joint_temp.robax.rax_4+J40ffset;
115      joint_temp.robax.rax_5:=joint_temp.robax.rax_5+J50ffset;
116      joint_temp.robax.rax_6:=joint_temp.robax.rax_6+J60ffset;
117      RETURN joint_temp;
118   ENDFUNC
119 ENDMODULE
```

图 6.18 六轴控制程序图

例如;客户端发送 5.000000000000000000000000000000000000 机器人第一轴就会向正方向移动 5°,发送-10.000000000000000000000000000000000000 就会向第一轴负方向移动 10° 总共 36 位数字。

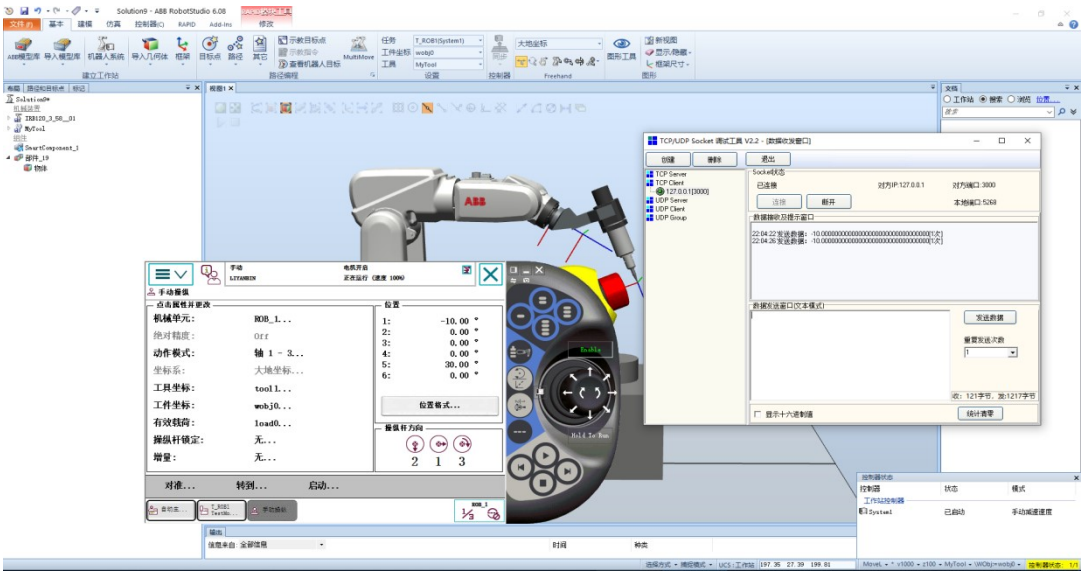


图 6.19 发送移动数据测试

实验练习：通过编程实现 socket 通讯发送数据控制机器人做 XYZ 线性运动。

提示内容：机器人线性运动坐标赋值方法。

```
current_position_robT:= CRobT(\Tool:=tool0\wobj:=wobj1);
current_position_robT.trans.x:=current_position_robT.trans.x+num_j1;
current_position_robT.trans.y:=current_position_robT.trans.y+num_j2;
current_position_robT.trans.z:=current_position_robT.trans.z+num_j3;
MoveL current_position_robT,v100,fine,tool1\wobj:=wobj1;
```

如果运行出现套接字错误，改 server_socket1 变为 server_socket4 或其他，client_socket1 变为 client_socket4 或其他，端口号 3002 改为 3010 或其他，其他的程序对应的变量依次修改。

第七章　手势识别控制机器人运动实验

1、实验目的

手势识别可以来自人的身体各部位的运动，但一般是手的运动。用户可以使用简单的手势来控制或与设备交互，让计算机理解人类的行为。其核心技术为手势分割、手势分析以及手势识别。

2、实验原理

通过 opencv 图像处理去除背景，提取手指轮廓，进行形态学滤波处理提取

区域。找到二值化的图像轮廓，对手指的轮廓面积和手指之间的角度、距离进行判断，从而计算出出手指的个数。

3、实验设备：型号 MFTW-IR 和 CHL-JC-02-A 的工作站。

4、树莓派系统手势识别网络数据格式定义：

如：0x02, 0x02, 0x03, 0x01, 0x0A（16 进制）

第一位：实验顺序。 0x02；0x02：这是第二个实验

第二位：控制机器人运行的模式。0x02；

第三位：控制某个轴动作。 0x01：控制 x 轴，0x02：控制 y 轴，0x03：控制 z 轴。

第四位：控制方向。 0x01：控制方向向上，0x02 控制方向向下

第五位：步长 10 个 0x0A 控制机械手运行的速度值 10 个步长

表 1-1 具体手势和执行动作指令

手势指令	手势	机器人动作	Socket 指令
手势 1		机械手上升	[0x02, 0x02, 0x03, 0x01, 0x0A]
手势 2		机械手下降	[0x02, 0x02, 0x03, 0x02, 0x0A]
手势 3		机械手左移	[0x02, 0x02, 0x01, 0x01, 0x0A]
手势 4		机械手右移	[0x02, 0x02, 0x01, 0x02, 0x0A]
手势 5		机械手前进	[0x02, 0x02, 0x02, 0x01, 0x0A]

4、实验步骤

1）确保系统配置中有 616-1 PC Interface 功能。

2）Robotstudio 软件中在线或离线编写程序

```
VAR socketdev server_socket2;      //定义变量，如果运行出现套接字错误，改server_socket2变为server_socket4或其他
```

```
VAR socketdev client_socket2;
 VAR string client_ip:="10.70.2.249"; //树莓派IP地址。
                                                    电脑虚拟调试IP地址为："127.0.0.2"
VAR byte receive_byte{5}:=[0,0,0,0,0];
PERS byte global_byte{5}:=[0,0,0,0,0];
VAR string receive_string{5};
VAR string str_hextodec{5};
PERS num lab_number:=2;
PERS num mode_value:=0;
PERS num jog_value:=0;
PERS num direction_value:=0;
PERS num step_value:=0;
VAR bool change_ok:=false;
VAR socketstatus state{3};
VAR robtarget current_position_robT;
VAR intnum int_home;
VAR num num_Line{3}:=[0,0,0];
PERS num num2_deg:=0;
VAR robtarget P_temp;
CONST jointtarget
Home:=[[-0.00129488,-0.000801945,0.00505125,-0.000859787,90.0011,-0.000264077],[9E+09,
9E+09,9E+09,9E+09,9E+09,9E+09]];
PROC main()
    SocketClose server_socket2;
    SocketClose client_socket2;
      client_ip:= "10.70.2.249";   //树莓派IP地址。
                                                    电脑虚拟调试IP地址："127.0.0.2"
    SocketCreate server_socket2;
       SocketBind server_socket2,"10.70.2.252",3002; //ABB机器人IP地址。
                                                  电脑虚拟调试ABB的IP地址："127.0.0.1"
    SocketListen server_socket2;
 SocketAccept server_socket2,client_socket2\ClientAddress:=client_ip\Time:=WAIT_MAX;
    WHILE TRUE DO
  SocketReceive client_socket2\Data:=receive_byte\Time:=WAIT_MAX;
        FOR i FROM 1 TO 5 DO
            global_byte{i}:=receive_byte{i};
        ENDFOR
        FOR i FROM 1 TO 5 DO
        receive_string{i}:=ByteToStr(global_byte{i});//16进制字转化成字符
        change_ok:=StrToVal(receive_string{1},lab_number); //字符转化成值
        change_ok:=StrToVal(receive_string{2},mode_value);
         change_ok:=StrToVal(receive_string{3},jog_value);
         change_ok:=StrToVal(receive_string{4},direction_value);
         change_ok:=StrToVal(receive_string{5},step_value);
```

```
            ENDFOR
            FOR i FROM 1 TO 5 DO
                TPWrite "receive_string="+receive_string{i};
            ENDFOR
        TPWrite "receive_string="+receive_string{5};   //在示教器上显示相应字符串
        TPWrite "lab_number="\Num:=+lab_number;
        TPWrite "mode_value="\Num:=+mode_value;
        TPWrite "jog_value="\Num:=+jog_value;
        TPWrite "direction_value="\Num:=+direction_value;
        TPWrite "step_value="\Num:=+step_value;
            IF lab_number=2 THEN     //实验号选择
                TEST mode_value      //模式测试
                CASE 2:
    current_position_robT:=CRobT(\Tool:=tool0\wobj:=wobj0); //机器人当前位置
                    FOR i FROM 1 TO 3 DO
                        IF (i=jog_value) THEN   //对控制轴进行判断  1=X,2=Y,3=Z
                        TEST direction_value         /对控制轴方向判断
                        CASE 1:
                            num_Line{i}:=step_value;     //方向向上
                        CASE 2:
                            num_Line{i}:=-step_value;     //方向向下
                        ENDTEST
current_position_robT.trans.x:=current_position_robT.trans.x+num_Line{1};
                            //机器人在当前位置X轴方向增加或减少的步长后的结果赋值
                current_position_robT.trans.y:=current_position_robT.trans.y+num_Line{2};
                            //  在当前位置Y轴方向增加或减少的步长后的结果赋值
current_position_robT.trans.z:=current_position_robT.trans.z+num_Line{3};
                                //  在当前位置Z轴方向增加或减少的步长后的结果赋值
        MoveL current_position_robT,v100,fine,tool0\wobj:=wobj0;
                                                    //机器人赋值后直线移动到该新位置
                        mode_value:=0; //复位为零
                        jog_value:=0;   //复位为零
                        direction_value:=0; //复位为零
                        step_value:=0;     //复位为零
                        FOR i FROM 1 TO 3 DO
                            num_Line{i}:=0; //数组复位为零
                        ENDFOR
                        FOR i FROM 1 TO 5 DO
                            global_byte{i}:=0; //数组复位为零
                        ENDFOR
                    Endif
                ENDFOR
            ENDTEST
        Endif
```

```
        ENDWHILE
    ENDPROC
ENDMODULE
```

3）调试

一、虚拟调试

虚拟调试就是利用电脑中 robotstudio 虚拟示教器手动运行程序时，在电脑中安装的网络调试助手(NetAssistV4.3)发数据给虚拟机器人,机器人接收到数据后经过程序解析，判断执行相应的动作。

a)ABB主程序中的**client_ip:= "10.70.2.249"； IP地址改为："127.0.0.2"，**

SocketBind server_socket2,"10.70.2.252",3002; IP地址改为："127.0.0.1"。

b)打开桌面上网络调试助手软件 NetAssistV4.3.29, 在操作界面上选择协议类型为:TCP Client ,远程主机地址：127.0.0.1；远程主机端口：3002；接收设置选择：HEX，按日志模式显示，接收区自动换行；发送设置选择：HEX，自动解析转义符，AT 指令自动回车；在数据发送框中键入所要发送的数据，如：02 02 03 02 0A，网络调试组手设置如图所示：



图 7.1 网络调试助手设置

c)编写好 ABB 程序后，手动运行程序，网络调试助手设置好参数后点击连接，然后数据发送框中输入十六进制数据，点击发送按钮，数据则传送给虚拟机器人，机器人机械臂会根据程序做出相应的动作。虚拟示教器上显示出发送过来的数据，如图所示，则说明程序调试成功。



图 7.2 虚拟示教器显示传送数据

二、与树莓派连接调试

a) 编写 ABB 程序，并修改好 IP 地址。

如果运行出现套接字错误，改 server_socket2 变为 server_socket4 或其他，client_socket2 变为 client_socket4 或其他，端口号 3002 改为 3010 或其他，其他的程序对应的变量依次修改。

b) 该实验需要在无复杂背景下去识别，最好是纯白色背景，否则可能会出现误识别现象。

c) 运行程序后伸出手指在摄像头前方不要移动，然后改变手指的个数，机械手会按照手指个数来执行机械手的预设动作，具体执行的串口动作指令请看串口数据格式定义。

d）树莓派软件操作

1、将机器人试验台供电，开启机器人和树莓派系统（图 7.3）



图 7.3 开启树莓派

2、找到树莓派桌面上的“手势识别.py”的实验程序。



图 7.4 手势识别

3. 鼠标右击系统桌面的手势识别软件，选择 Thonny Python IDE 程序打开识别软件。



图 7.5 Thonny Python IDE

4. 用 Thonny Python IDE 打开软件后，软件菜单栏中点击 Run，执行该手势识别程序。（图 7.6）

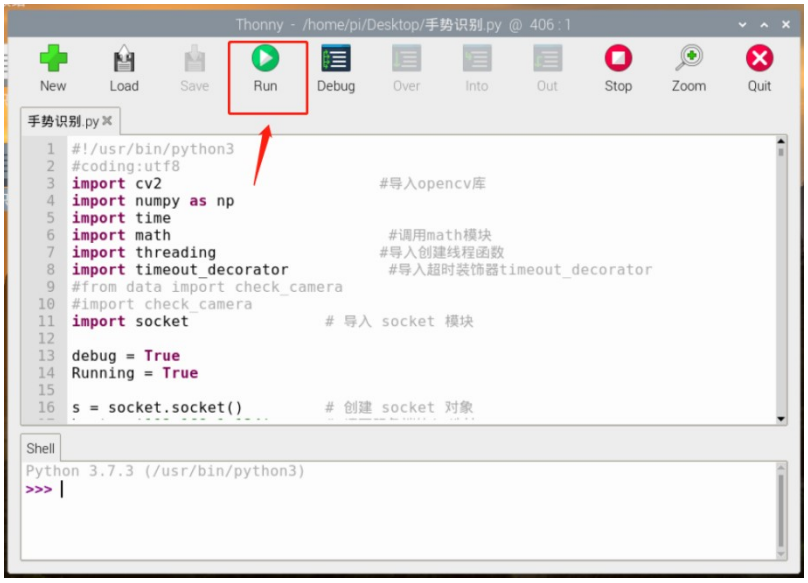


图 7.6 执行程序

5. 若在 shell 显示框打印出“连接不成功”、或 shell 显示框没有任何显示。可能是机器人的 socket 通讯程序未开启。（图 7.7）
此时需要开启机器人的 socket 服务端程序。



图 7.7 连接不成功

若在 shell 显示框打印出“连接到服务器”，且在桌面弹出一个手势识别的识别框。则表示程序运行成功。（图 7.8，图 7.9）（手势识别最好放置在白色背景下，这样识别准确率会更高）。



图 7.8 连接服务器成功

识别手势见图 7.9 表示

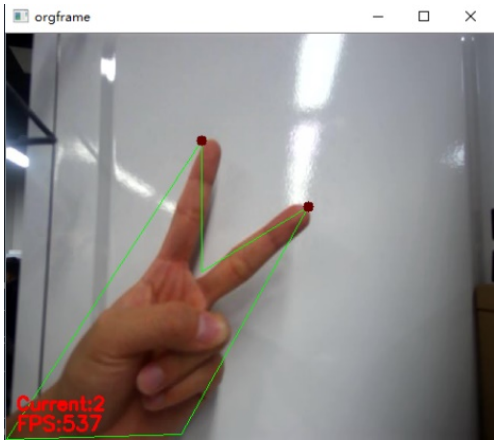


图7.9 识别界面（手势二）

6、根据手势动作情况，控制机器臂上升、下降、左移动、右移动、前进、后退。

树莓派手势识别 python 部分程序：

导入必要的模块

```
import cv2                          #导入 opencv 库
import numpy as np
import time
import math                          #调用 math 模块
import threading                    #导入创建线程函数
import timeout_decorator            #导入超时装饰器 timeout_decorator
from data import check_camera
import socket                        # 导入 socket 模块
```

树莓派连接机械手的 socket 通讯

```
s = socket.socket()                 # 创建 socket 对象
host = '10.70.2.252'                # 填写相应 abb 机器人服务端的 ip 地址
port = 3002                         # 设置端口号
try:
    s.connect((host, port))
    print("连接到服务器")
```

```
except:
    print("连接不成功")
    exit(1)
```

该手势识别中有两个线程函数

get_image 该线程是实时开启摄像头的函数

```
def get_image():
    global orgFrame
    global ret
    global Running
    global stream, cap
    while True:
        if Running:
            try:
                if cap.isOpened():                    #如果检测到摄像头开启
                    ret, orgFrame = cap.read()        #读取摄像头图像
                else:
                    time.sleep(0.01)                  #否则休眠 0.01
            except:
                cap = cv2.VideoCapture(stream)
                print('Restart Camera Successful!')
        else:
            time.sleep(0.01)
```

runAction 该线程是执行动作的函数。（大约在第 366 行附近）

num_mean 是指识别到的手指个数，如果识别到手指个数为 1，将向服务端发送上升的串口指令。

```
def runAction():                                      #动作函数
    global get_hand_num, action_finish                #全局定义
    global num_mean
    while True:                                       #开启程序后一直循环判断 num_mean 的值
        #time.sleep(1)
```

```
if get_hand_num:                                    #如果 get_hand_num 的值为真
    #print(1)
    get_hand_num = False    #将 get_hand_num 和 action_finish 设置为 False
    action_finish = False
    if num_mean == 1:                               #如果 num_mean 识别手指个数为 1
        a = [0x01, 0x02, 0x03, 0x01, 0x10]      #给串口发送上升动作的 16 进制
        b = ""
        for i in range(len(a)):
            b +=chr(a[i])               #将 a 的 16 进制数据格式转化成 ascii 码
        s. send(b. encode("utf-8"))                 #将数据发送给服务端
        print("1")
        num_maen = 0
        time. sleep(2)
        action_finish = True                #将 action_finish 的状态设置为 True
```

第八章　语音识别控制机器人运动实验

1、实验目的

语音识别模块是基于嵌入式的语音识别技术的模块，本实验目的主要是用语音控制机器人的基本动作，以及播放音乐等功能。

2、实验原理

采用IIC通信，用户只需要把识别的关键词语以字符串的形式传送进芯片，即可在下次识别中立即生效。

该模块有三种使用模式，用户可通过编程，设置两种不同的使用模式。

按钮检测模式：系统的主控MCU 在接收到外界一个触发后（比如用户按下按键）， 将会启动芯片上的一个定时识别过程（比如5 s），此时需要用户在这个定时过程中说出要识别的语音关键词语。当这个过程结束后，需要用户再次触

发才能再次启动一个识别过程。

循环检测模式：系统的主控MCU反复启动的识别过程。如果没有人说话就没有识别结果，则每次识别过程的定时到时后会再启动一个识别过程；如果有识别结果，则根据识别作相应处理后（比如播放某个声音作为回答）再启动一个识别过程。

口令检测模式：口令模式需要一个关键词来唤醒，唤醒后才可以进行识别，默认唤醒关键词为第一句，识别结束后，想再次进行识别，还需唤醒它，类似小艾同学。

3、实验设备：型号 MFTW-IR 和 CHL-JC-02-A 的工作站

4、实验步骤

1）在 ABB 系统中配置 616-1 PC Interface 功能。

2）Robotstudio 软件中在线或离线编写程序（ABB 程序与第七章相同）

```
VAR socketdev server_socket2;      //定义变量
VAR socketdev client_socket2;
 VAR string client_ip:="10.70.2.249"; //树莓派IP地址。
                                      电脑虚拟调试IP地址为："127.0.0.2"
VAR byte receive_byte{5}:=[0,0,0,0,0];
PERS byte global_byte{5}:=[0,0,0,0,0];
VAR string receive_string{5};
VAR string str_hextodec{5};
PERS num lab_number:=2;
PERS num mode_value:=0;
PERS num jog_value:=0;
PERS num direction_value:=0;
PERS num step_value:=0;
VAR bool change_ok:=false;
VAR socketstatus state{3};
VAR robtarget current_position_robT;
VAR intnum int_home;
VAR num num_Line{3}:=[0,0,0];
PERS num num2_deg:=0;
VAR robtarget P_temp;
CONST jointtarget
Home:=[[-0.00129488,-0.000801945,0.00505125,-0.000859787,90.0011,-0.000264077],[9E+09,
9E+09,9E+09,9E+09,9E+09,9E+09]];
PROC main()
    SocketClose server_socket2;
    SocketClose client_socket2;
```

```
        client_ip:= "10.70.2.249";   //树莓派IP地址。
                                                    电脑虚拟调试IP地址："127.0.0.2"
        SocketCreate server_socket2;
          SocketBind server_socket2,"10.70.2.252",3002; //ABB机器人IP地址。
                                                    电脑虚拟调试ABB的IP地址："127.0.0.1"
        SocketListen server_socket2;
SocketAccept server_socket2,client_socket2\ClientAddress:=client_ip\Time:=WAIT_MAX;
        WHILE TRUE DO
    SocketReceive client_socket2\Data:=receive_byte\Time:=WAIT_MAX;
            FOR i FROM 1 TO 5 DO
                global_byte{i}:=receive_byte{i};
            ENDFOR
            FOR i FROM 1 TO 5 DO
            receive_string{i}:=ByteToStr(global_byte{i});//16进制字转化成字符
            change_ok:=StrToVal(receive_string{1},lab_number); //字符转化成值
            change_ok:=StrToVal(receive_string{2},mode_value);
             change_ok:=StrToVal(receive_string{3},jog_value);
             change_ok:=StrToVal(receive_string{4},direction_value);
             change_ok:=StrToVal(receive_string{5},step_value);
            ENDFOR
            FOR i FROM 1 TO 5 DO
                TPWrite "receive_string="+receive_string{i};
            ENDFOR
        TPWrite "receive_string="+receive_string{5};   //在示教器上显示相应字符串
        TPWrite "lab_number="\Num:=+lab_number;
        TPWrite "mode_value="\Num:=+mode_value;
        TPWrite "jog_value="\Num:=+jog_value;
        TPWrite "direction_value="\Num:=+direction_value;
        TPWrite "step_value="\Num:=+step_value;
            IF lab_number=2 THEN     //实验号选择
                TEST mode_value      //模式测试
                CASE 2:
    current_position_robT:=CRobT(\Tool:=tool0\wobj:=wobj0); //机器人当前位置
                    FOR i FROM 1 TO 3 DO
                        IF (i=jog_value) THEN   //对控制轴进行判断  1=X,2=Y,3=Z
                        TEST direction_value        /对控制轴方向判断
                        CASE 1:
                            num_Line{i}:=step_value;    //方向向上
                        CASE 2:
                            num_Line{i}:=-step_value;    //方向向下
                        ENDTEST
current_position_robT.trans.x:=current_position_robT.trans.x+num_Line{1};
                    //机器人在当前位置X轴方向增加或减少的步长后的结果赋值
current_position_robT.trans.y:=current_position_robT.trans.y+num_Line{2};
```

```
                    // 在当前位置Y轴方向增加或减少的步长后的结果赋值
current_position_robT.trans.z:=current_position_robT.trans.z+num_Line{3};
                        // 在当前位置Z轴方向增加或减少的步长后的结果赋值
    MoveL current_position_robT,v100,fine,tool0\wobj:=wobj0;
                                        //机器人赋值后直线移动到该新位置
                    mode_value:=0; //复位为零
                    jog_value:=0;   //复位为零
                    direction_value:=0; //复位为零
                    step_value:=0;    //复位为零
                    FOR i FROM 1 TO 3 DO
                        num_Line{i}:=0; //数组复位为零
                    ENDFOR
                    FOR i FROM 1 TO 5 DO
                        global_byte{i}:=0; //数组复位为零
                    ENDFOR
                Endif
            ENDFOR
        ENDTEST
    Endif
  ENDWHILE
 ENDPROC
ENDMODULE
```

3）调试

一、虚拟调试

虚拟调试就是利用电脑中 robotstudio 虚拟示教器手动运行程序时，在电脑中安装的网络调试助手(NetAssistV4.3)发数据给虚拟机器人,机器人接收到数据后经过程序解析，判断执行相应的动作。

a)ABB主程序中的**client_ip:= "10.70.2.249"; IP地址改为："127.0.0.2",**

SocketBind server_socket2,"10.70.2.252",3002; IP地址改为："127.0.0.1"。

b)打开桌面上网络调试助手软件 NetAssistV4.3.29, 在操作界面上选择协议类型为:TCP Client ,远程主机地址：127.0.0.1；远程主机端口：3002；接收设置选择：HEX，按日志模式显示，接收区自动换行；发送设置选择：HEX，自动解析转义符，AT 指令自动回车；在数据发送框中键入所要发送的数据，如：02 02 03 02 0A，网络调试组手设置如图所示：



图 8.1 网络调试助手设置

c)编写好 ABB 程序后，手动运行程序，网络调试助手设置好参数后点击连接，然后数据发送框中输入十六进制数据，点击发送按钮，数据则传送给虚拟机器人，机器人机械臂会根据程序做出相应的动作。虚拟示教器上显示出发送过来的数据，如图所示，则说明程序调试成功。



图 8.2 虚拟示教器显示传送数据

二、与树莓派连接调试

1) 编写 ABB 程序，并修改好 IP 地址。

如果运行出现套接字错误，改 server_socket2 变为 server_socket4 或其他，client_socket2 变为 client_socket4 或其他，端口号 3002 改为 3010 或其他，其他的程序对应的变量依次修改。

2）开启机器人和树莓派系统（图 8.3）

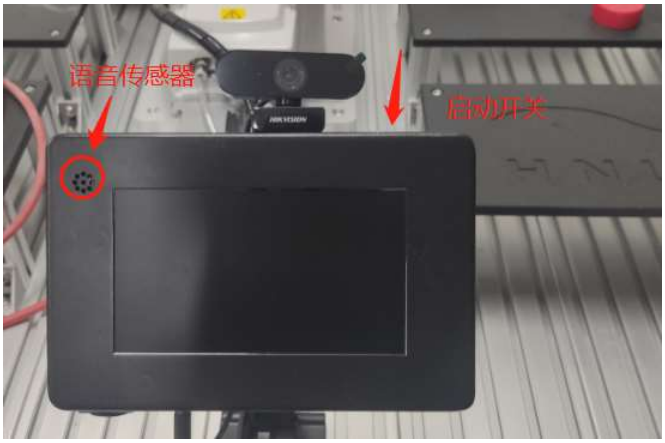


图 8.3 开启树莓派



3）找到树莓派桌面上的“语音识别.py”的实验程序。

图 8.4 语音识别

4）鼠标右击系统桌面的语音识别软件，选择Thonny Python IDE程序打开识别软件。（图8.4）

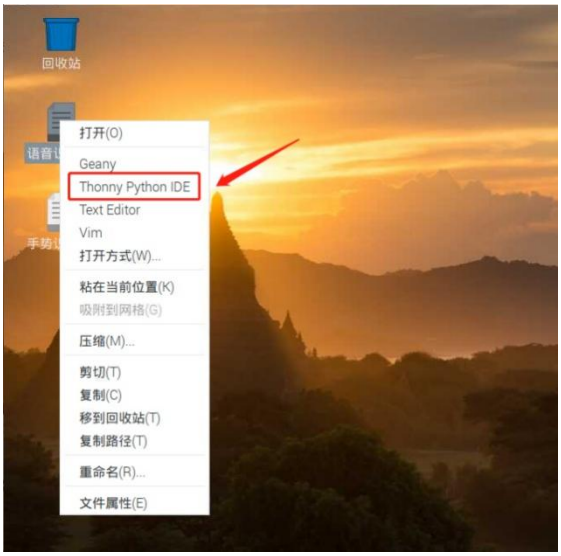


图 8.5 Thonny Python IDE

5）用 Thonny Python IDE 打开软件后,软件菜单栏中点击 Run，执行该语音识别程序。

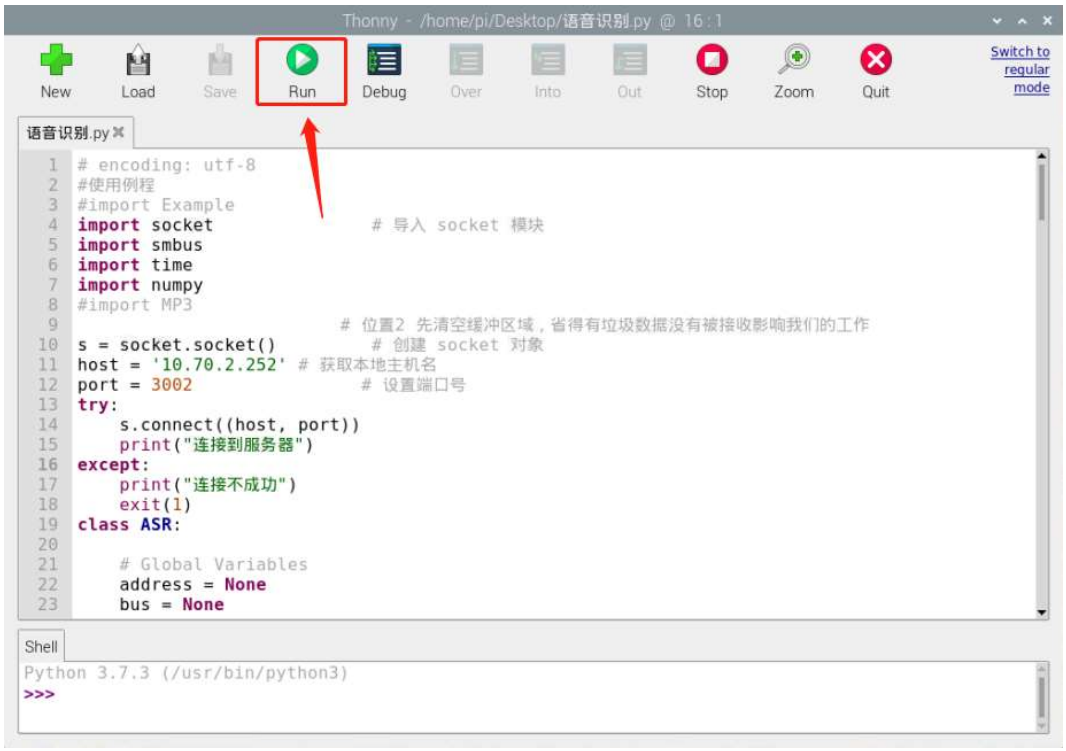


图8.6 执行程序

6）若在 shell 显示框打印出“连接不成功”、或 shell 显示框没有任何显示。可能是机器人的 socket 通讯程序未开启。
此时需要开启机器人的 socket 服务端程序。

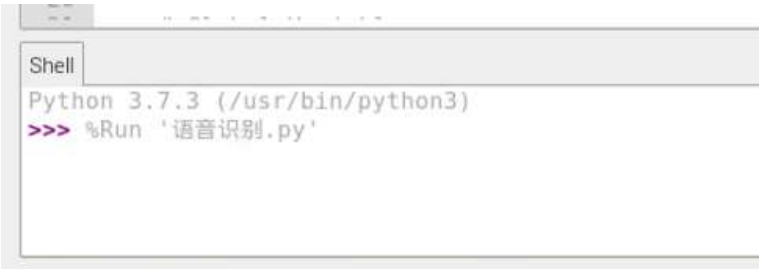


图 8.7 未连接成功

若在shell显示框打印出“连接到服务器”，则表示程序运行成功。



图 8.8 连接服务器成功

7）语音识别程序成功启动后，传感器一直保持监听状态，并在Shell显示框返回result。如果返回值是“0”，表示未接收到指令，则不执行任何动作。

当对语音模块下达“下降”指令，系统接收到“下降”的指令后，在屏幕Shell显示框输出result:2。并执行下降的动作。以下是检测与执行的代码（如图8.9）

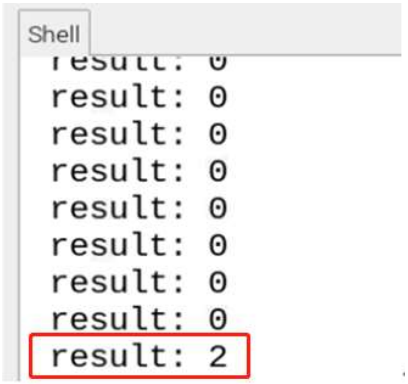


图 8.9 信号输出

8）具体指令和执行动作（表8-1）

表 8-1 语音具体指令

语音指令	机器人动作	Socket 指令

上升	机械手上升	[0x02,0x02,0x03,0x01,0x0A]
下降	机械手下降	[0x02,0x02,0x03,0x02,0x0A]
左边	机械手左边	[0x02,0x02,0x01,0x01,0x0A]
右边	机械手右边	[0x02,0x02,0x01,0x02,0x0A]
前进	机械手前进	[0x02,0x02,0x02,0x01,0x0A]
后退	机械手后退	[0x02,0x02,0x02,0x02,0x0A]

9）Socket通讯指令介绍：

[0x02,0x02,0x03,0x01,0x0A]

第一位：实验顺序。　　0x02：第二个实验。

第二位：控制机器人运行的模式。　　0x02：第二个运行模式。

第三位：控制某个轴动作。 0x01：控制 x 轴，0x02:控制 y 轴，0x03:控制 z 轴。

第四位：控制方向。　　0x01：控制方向向上，0x02：控制方向向下。

第五位：步长为 0A　　0x0A：控制机械手运行的速度值为 10 个步长。

10）树莓派语音识别部分代码：

1.导入一些必要的库

```
import socket                    # 导入 socket 通讯模块
import smbus                     #smbus 通讯协议
import time                      #导入时间模块
import numpy
```

#NumPy 是 Python 语言的一个扩展程序库，支持大量的维度数组与矩阵运算，此外也针对数组运算提供大量的数学函数库。

以下是 python 中 socket 客户端的连接方法

```
s = socket.socket()                    # 创建 socket 对象
host = '10.70.2.252'                   # 填写服务端的 ip 地址
port = 3002                            # 填入设置好的端口号
try:                                   #异常的用法
    s.connect((host, port))            #去连接服务器端的 ip 和端口
    print("连接到服务器")
except:
    print("连接不成功")
    exit(1)
```

1.运行该 python 文件后会执行 dd()这个函数。

```
if __name__ == "__main__":
    dd()
```

2.首先是连接 IIC 的语音传感器

```
addr = 0x79                    #连接语音传感器 iic 地址
asr = ASR(addr)                #进行实例化
```

3.接下来监听所设置词条指令和对应的词条地址

```
if 1:
    asr.eraseWords()                    #擦出词条
    asr.setMode(1)                              #识别模式为 1
    asr.addWords(1, 'shang shen')       #说上升口令时，在程序中赋予词条的地址是 1
    asr.addWords(2, 'xia jiang')        #说下降口令时，在程序中赋予词条的地址是 2
```

```
asr.addWords(3, 'zuo bian')          #说左边口令时，在程序中赋予词条的地址是 3
asr.addWords(4, 'you bian')          #说右边口令时，在程序中赋予词条的地址是 4
asr.addWords(5, 'qian jing')         #说右边口令时，在程序中赋予词条的地址是 5
asr.addWords(6, 'hou tui')           #说后退口令时，在程序中赋予词条的地址是 6
asr.addWords(7, 'tui chu')           #说退出口令时，在程序中赋予词条的地址是 7
```

4.当触发指令后，词条地址会赋值给 data，随后程序进入 while 循环。

（程序会发送 data 值所对应的串口指令给机器人，随后机器人执行相应的动作）

```
while 1:
    data = asr.getResult()                    #将得到的识别结果赋值给 data
    print("result:", data)                    #在终端打打印出 data 的值
    if data==1:                               #如果 data 值等于 1
        a = [0x02,0x02,0x03,0x01,0x10]        #给串口发送上升动作的 16 进制
        b = ""
        for i in range(len(a)):
            b +=chr(a[i])                     #将 a 的 16 进制数据格式转化成 ascii 码
        s.send(b.encode("utf-8"))             #将数据发送给服务端
        time.sleep(0.1)                       #发送成功后程序休眠一定时间
        # print("shang shen")
```

第九章　基于视觉识别控制机器人定位抓取实验

1、实验目的

基于视觉识别软件判断工件的形状颜色，通过 Socket 通讯的方式，发送识别数据给机器人，机器人按照数据进行定位抓取，完成对工件按照颜色，形状进行

分类的实验。此实验主要使学生掌握视觉与机器人之间的工作原理，如何基于视觉辅助控制机器人完成控制动作。

2、实验原理

通过视觉软件拍照，经过图像处理技术得到工件中心像素坐标，通过手眼标定原理，计算出图像坐标系与机器人坐标系之间的矩阵关系，通过像素坐标得到机器人坐标，基于 Socket 通讯，将图像识别数据发送给机器人，机器人接收后对数据进行处理，即可完成定位抓取的实验。

3、实验设备：型号 MFTW-IR 和 CHL-JC-02-A 的工作站

4、实验过程

1） 视觉软件的使用说明

视觉实验部分，相机固定安装。视觉软件直接输出颜色，形状，大小，角度，机器人坐标视觉软件作为客户端，机器人做服务端，机器人做服务端如 IP：10. 70. 2. 252，端口号：3003。

2） 数据的定义

视觉拍照信号是通过 socket 通讯完成，机器人发送数字 1 定义为拍照信号，发送拍照信号后机器人等待视觉返回识别信息，如果识别信息无任何数据，发送 null。

在软件中能对颜色，形状进行映射，例如颜色对应 0-9，形状对应 a-z；红色映射为 0，形状映射为 a，映射完的数字，字母具有唯一性，不能再映射为其他工件。工件大小默认是小对应 0，大对应 1。

如果没有角度默认发 0。发送数据的格式是，一组数据内每项用逗号（,）分割，结束用分号（;）举例说明：识别到红色大圆形工件发送的数据格式是：0，a，1，-12，325. 6，-128. 2；

由于实验关系，视觉软件中已经将工件形状及颜色的定义固定，其定义如下：

第一位：颜色

0-红色、1-黄色、2-白色，出现其它颜色工件，默认不处理。

第二位：形状

a-圆形、b-方形，出现其它形状工件，默认不处理。

第三位：大小

0-是小 1-大，出现其它尺寸工件，默认不处理。

第四位：X

第五位：Y

第六位：旋转角度

3） 视觉与机器人工作逻辑

实验前视觉软件已经同机器人完成手眼标定，机器人做服务端，视觉做客户端，故需要先运行机器人程序，视觉才能连接上机器人服务端。机器人运行程序，与视觉连接成功，移动到工件拍照位置，给机器人发送拍照信号“1”，视觉接收到拍照信号“1”进行拍照，机器人等待视觉拍照后返回的数据，视觉将当前工件进行识别，发送识别后的数据

“0, a, 1, 325. 23, -125. 23, 0;”给机器人，机器人接收到数据后按照实验定义的方法进行数据处理，坐标赋值，按照坐标位置进行移动，对工件进行分类，实验完成。其视觉实验界面如下图所示：

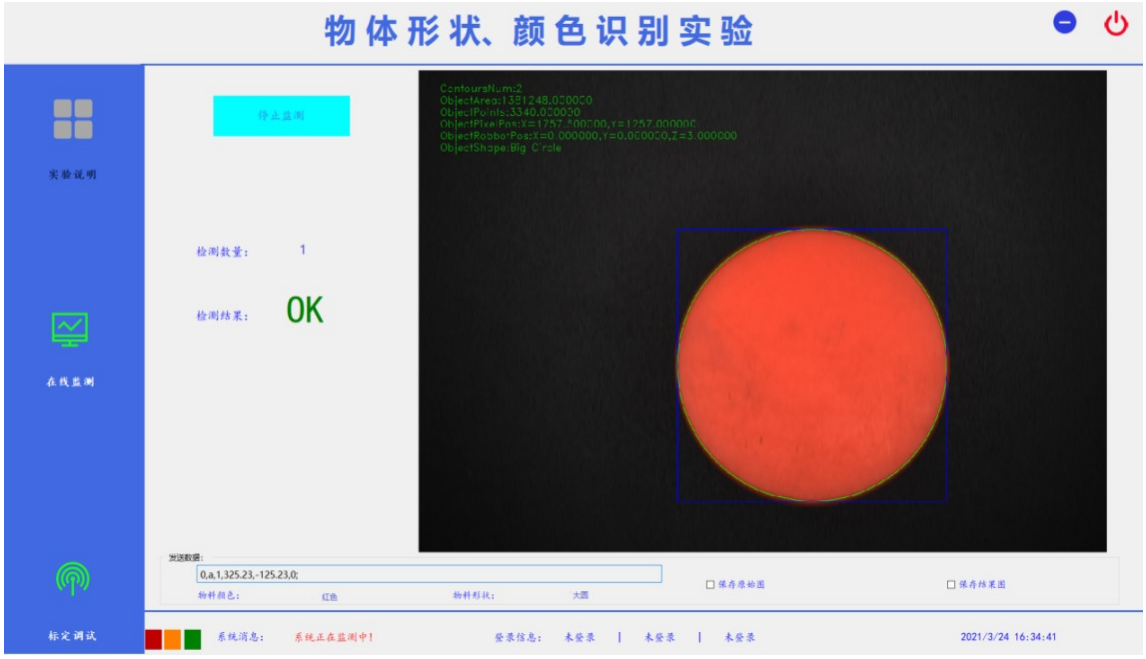


图 9.1 视觉软件识别结果图

4） 机器人编程

PROC Vision_Control()

Const robtarget

robot_2D_position_biaoding:=[[-87.50,100,-42.42],[0.715273,0.00325326,0.00315491,-0.69883],[0,0,0,0],[9E+09,9E+09,9E+09,9E+09,9E+09,9E+09]];拍照的基准位置，常量类型，

其值不可改变，不可进行坐标赋值操作

VAR robtarget robot_2D_position_wait:=[[-87.50,100,-42.42],[0.715273,0.00325326,0.00315491,0.69883],[0,0,0,0],[9E+09,9E+09,9E+09,9E+09,9E+09,9E+09]];拍照的基准位置，变量类型，其值可改变，可进行坐标赋值操作

VAR robtarget A:=[[103.51,-340.92,-212.06],[0.715356,0.002804,0.00528505,-0.698735],[0,0,0,0],[9E+09,9E+09,9E+09,9E+09,9E+09,9E+09]];放置工价的位置，变量类型

```
VAR string receive_string;                          定义接收数据的字符串
PERS string    data{6}:=["","","","","","",""];     用于存储分割后的字符串数组
VAR bool change_ok;                                 字符类型转换标志位
VAR num color;                                      工件颜色
VAR string shape;                                   工件形状
VAR num size;                                       工件大小
VAR num num_x;                                      工件图像坐标转为机器人的X坐标
VAR num num_y;                                      工件图像坐标转为机器人的Y坐标
VAR num deg;                                        工件角度
VAR socketdev server_socket1;                       定义服务端socket
VAR socketdev client_socket1;                       定义客户端socket
VAR string client_ip:="10.70.2.251";                相机IP地址
SocketClose server_socket1;                         关闭服务端socket
  SocketClose client_socket1;                       关闭客户端socket
  SocketCreate server_socket1;                      创建服务端socket
 SocketBind server_socket1,"10.70.2.252",3003;      机器人绑定IP地址和端口号
   SocketListen server_socket1;                     服务端开启监听
   SocketAccept server_socket1,client_socket1\ClientAddress:=client_ip\Time:=WAIT_MAX;
                                                    等待接收数据
  WHILE TRUE DO                                     主循环体
  Movej robot_2D_position_biaoding,vmax,z20,tool0\wobj:=wobj1; 移动到拍照位置。
```

SocketSend client_socket1\Str:="1"; 机器人发送拍照指令相机拍照

SocketReceive client_socket1\str:=receive_string\Time:=WAIT_MAX;等待接收视觉处理后的数据

SplitStr receive_string,",",data; 将接收的视觉数据进行分割处理，见SplitStr函数

TPWrite "receive_string="+receive_string; 打印输出接收的字符串

change_ok:=StrToVal(data{1},color); 将字符串转为值类型数据

shape:=data{2};

change_ok:=StrToVal(data{3},size);

change_ok:=StrToVal(data{4},num_x);

change_ok:=StrToVal(data{5},num_y);

change_ok:=StrToVal(data{6},deg);

robot_2D_position_wait.trans.x:=num_x; 将视觉计算的X坐标值赋值给拍照位的基准坐标X

robot_2D_position_wait.trans.y:=num_y; 将视觉计算的Y坐标值赋值给拍照位的基准坐标Y

Movej robot_2D_position_wait,vmax,z20,tool0\wobj:=wobj1; 机器人按照赋值后的坐标平移

robot_2D_position_wait.trans.z:=robot_2D_position_biaoding.trans.z-300;

基于偏移后的基准位置Z轴下降300mm，300mm指机器人吸盘到工件表面的高度，300mm前面是“-”或者“+”需要根据右手定则确定机器人Z轴的方向确定正负号，也可以根据示教器上面坐标数据看出。

Movej robot_2D_position_wait,vmax,z10,tool0\wobj:=wobj1; 移动到工件表面

SetDO do0,1; 打开吸盘（或SetDO Do10_14,1）

WaitTime 0.5; 等待0.5s

robot_2D_position_wait.trans.z:=robot_2D_position_wait.trans.z+150; 抓取工件抬升150mm

Movej robot_2D_position_wait,v500,z20,tool0\wobj:=wobj1; 移动到抬升位置

TEST color 判断工件属于哪一种类型进入到对应位置

CASE 0: 红色物块

IF shape="a" THEN 圆形

TEST size

CASE 0: 小尺寸

Movej A1,v500,z20,tool0\wobj:=wobj1; A1位置自己示教不同工件摆放不同位置

```
            case 1:                          大尺寸
MoveL Offs(A2,50,0,0), v500, z50, tool0;                    A2位置自己示教
            ENDTEST
        ENDIF
        IF shape="b" THEN                                    方形
          TEST size
            CASE 0:                                          小尺寸
          MoveL Offs(A3,100,0,0), v500, z50, tool0;          A3位置自己示教
            case 1:                                          大尺寸
          MoveL Offs(A4,150,0,0), v500, z50, tool0;          A4位置自己示教
            ENDTEST
        ENDIF
      CASE 1:                                                黄色物块
        IF shape="a" THEN                                    圆形
            TEST size
            CASE 0:                                          小尺寸
              MoveL Offs(A5,0,50,0), v500, z50, tool0; A5位置自己示教
            case 1:                                          大尺寸
            MoveL Offs(A6,0,100,0), v500, z50, tool0; A6位置自己示教
            ENDTEST
        ENDIF
        IF shape="b" THEN                                    方形
          TEST size
            CASE 0:                                          小尺寸
            MoveL Offs(A7,0,150,0), v500, z50, tool0; A7位置自己示教
            case 1:                                          大尺寸
            MoveL Offs(A8,50,50,0), v500, z50, tool0; A8位置自己示教
            ENDTEST
        ENDIF
      CASE 2:                                                白色物体
```

```
                IF shape="a" THEN                    圆形
                    TEST size
                    CASE 0:                          小尺寸
                  MoveL Offs(A9,50,100,0), v500, z50, tool0; A9位置自己示教
                    case 1:                          大尺寸
                MoveL Offs(A10,50,150,0), v500, z50, tool0; A10位置自己示教
                    ENDTEST
                ENDIF
                IF shape="b" THEN                    方形
                  TEST size
                    CASE 0:                          小尺寸
                MoveL Offs(A11,100,50,0), v500, z50, tool0; A11位置自己示教
                    case 1:                          大尺寸
               MoveL Offs(A12,100,100,0), v500, z50, tool0; A12位置自己示教
                    ENDTEST
                ENDIF
            ENDTEST
            SetDO do0,0;关闭吸盘（或SetDO Do10_14,0）
            WaitTime 0.5;等待0.5s
        ENDWHILE
    ENDPROC
PROC SplitStr( string inputstr,string sp,inout string data{*})
                字符串分割函：inputstr输入字符串，sp分割符，data{*}存入数据的数组
    VAR string s1;
    VAR string s2;
    VAR num count:=1;
    VAR bool flag1;
    FOR i FROM 1 TO StrLen(inputstr) DO      循环遍历输入字符串
        s1:=StrPart(inputstr,i,1);           每次取一位分割符字符
        IF s1=sp THEN
```

flag1:=StrToVal(s2,data{count});

如果当前字符是分隔符，则将之前的字符串转化为数据并存入data{count}中

s2:="";

Incr count;

ELSE

s2:=s2+s1; 如果当前字符不是分割字符，则字符串拼接

ENDIF

flag1:=StrToVal(s2,data{count});

ENDFOR

ENDPROC

```
133  PROC SplitStr( string inputstr,string sp,inout string data{*})
134      VAR string s1;
135      VAR string s2;
136      VAR num count:=1;
137      VAR bool flag1;
138      FOR i FROM 1 TO StrLen(inputstr) DO
139          s1:=StrPart(inputstr,i,1);
140          IF s1=sp THEN
141              flag1:=StrToVal(s2,data{count});
142              s2:="";
143              Incr count;
144          ELSE
145              s2:=s2+s1;
146          ENDIF
147          flag1:=StrToVal(s2,data{count});
148      ENDFOR
149  ENDPROC
```

分割函数

```
PROC Vision_Control()
    Const robtarget  robot_2D_position_biaoding:=[[-87.50,100,-42.42],[0.715273,0.00325326,0.00315491,-0.69883],[0,0,0,0],[9E+09,9E+09,9E+09,9E+09,9E+09,9E+09]];
    VAR robtarget  robot_2D_position_wait:=[[-87.50,100,-42.42],[0.715273,0.00325326,0.00315491,-0.69883],[0,0,0,0],[9E+09,9E+09,9E+09,9E+09,9E+09,9E+09]];
    VAR robtarget A:=[[103.51,-340.92,-212.06],[0.715356,0.002804,0.00528505,-0.698735],[0,0,0,0],[9E+09,9E+09,9E+09,9E+09,9E+09,9E+09]];
    VAR string receive_string;
    PERS string  data{6}:=["","","","","",""];
    VAR bool change_ok;
    VAR num color;
    VAR string shape;
    VAR num size;
    VAR num num_x;
    VAR num num_y;
    VAR num deg;
    VAR socketdev server_socket1;
    VAR socketdev client_socket1;
    VAR string client_ip:="127.0.0.2";
    SocketClose server_socket1;
       SocketClose client_socket1;
       SocketCreate server_socket1;
       SocketBind server_socket1,"127.0.0.1",3003;
       SocketListen server_socket1;
       SocketAccept server_socket1,client_socket1\ClientAddress:=client_ip\Time:=WAIT_MAX;
        WHILE TRUE DO
          Movej robot_2D_position_biaoding,vmax,z20,tool0\wobj:=wobj1;
          SocketSend client_socket1\Str:="1";
          SocketReceive client_socket1\str:=receive_string\Time:=WAIT_MAX;
          SplitStr receive_string,",",data;
          TPWrite "receive_string="+receive_string;
          change_ok:=StrToVal(data{1},color);
            shape:=data{2};
          change_ok:=StrToVal(data{3},size);
          change_ok:=StrToVal(data{4},num_x);
          change_ok:=StrToVal(data{5},num_y);
          change_ok:=StrToVal(data{6},deg);
            robot_2D_position_wait.trans.x:=num_x;
            robot_2D_position_wait.trans.y:=num_y;
            Movej robot_2D_position_wait,vmax,z20,tool0\wobj:=wobj1;
            robot_2D_position_wait.trans.z:=robot_2D_position_biaoding.trans.z-300;
            Movej robot_2D_position_wait,vmax,z10,tool0\wobj:=wobj1;
            SetDO do0,1;
            WaitTime 0.5;
            robot_2D_position_wait.trans.z:=robot_2D_position_wait.trans.z+150;
```

```
207             Movej robot_2D_position_wait,v500,z20,tool0\wobj:=wobj1;
208             TEST color
209               CASE 0:
210                 IF shape="a" THEN
211                    TEST size
212                    CASE 0:
213                      Movej A,v500,z20,tool0\wobj:=wobj1;
214                    case 1:
215                      MoveL Offs(A,50,0,0), v500, z50, tool0;
216                    ENDTEST
217                 ENDIF
218                 IF shape="b" THEN
219                   TEST size
220                    CASE 0:
221                     MoveL Offs(A,100,0,0), v500, z50, tool0;
222                    case 1:
223                     MoveL Offs(A,150,0,0), v500, z50, tool0;
224                    ENDTEST
225                 ENDIF
226               CASE 1:
227                 IF shape="a" THEN
228                    TEST size
229                    CASE 0:
230                      MoveL Offs(A,0,50,0), v500, z50, tool0;
231                    case 1:
232                      MoveL Offs(A,0,100,0), v500, z50, tool0;
233                    ENDTEST
234                 ENDIF
235                 IF shape="b" THEN
236                   TEST size
237                    CASE 0:
238                      MoveL Offs(A,0,150,0), v500, z50, tool0;
239                    case 1:
240                      MoveL Offs(A,50,50,0), v500, z50, tool0;
241                    ENDTEST
242                 ENDIF
243               CASE 2:
244                 IF shape="a" THEN
245                    TEST size
246                    CASE 0:
247                      MoveL Offs(A,50,100,0), v500, z50, tool0;
248                    case 1:
249                      MoveL Offs(A,50,150,0), v500, z50, tool0;
250                    ENDTEST
251                 ENDIF
252                 IF shape="b" THEN
253                   TEST size
```

图 9.2 机器人视觉定位抓取程序图

第十章　基于 Profinet 通讯控制机器人速度实验

实验目的：学习 Profinet 通讯配置方法，学习中断使用方法
实验要求：掌握 Profinet 通讯配置，掌握中断使用方法
实验设备：型号 MFTW-IR 和 CHL-JC-02-A 的工作站
实验步骤：
A、西门子 S7-1200 PLC 与机器人 Profinet 通讯配置流程
1、获取配置文件（GSDML）（机器人或其他设备驱动文件）

步骤一：安装目录下查找。

步骤二：软件中查找。

步骤三：在示教器系统中查找:

在示教器中查找方法：资源管理器→硬盘驱动→1200—510076—2019→products→robotware→6.07.1011→utility→service→GSDML→GSDML-V2.33-ABB-ROBOTICS-ROBOT-DEVICE-20

(GSDML-V2.33-ABB-ROBOTICS-ROBOT-DEVICE-20 这个驱动为 888-2/888-3 选项的文件。

B、 PLC 组态软件配置(博途软件中的配置）

1.在西门子博图软件中导入 GSDML 文件。

2.设备组态:

挂 ABB 机器人的设备组态：其他现场设备→profinet I/O→I/O→ABB ROBOTICS→ROBOT DEVICE→前端模块→BASIC V1.2。

3.分配 IP 地址，站号（PROFINET Station Name）

3.1.PLC 设备 IP 分配

1）.IP 地址,如 192.168.1.3 ，子网 255.255.255.0

2）.端口→端口互联→伙伴端口，选择任何伙伴。

3.2.博图软件中机器人的设置

1）.ip 地址,如 192.168.1.4 ，子网 255.255.255.0

2）.从站名称：profinet 设备名称如：pn1（最好改名不要自动生成 profinet 设备名称）。

3.3.通讯数据分配

在博图软件中→网路视图→选中 ABB 机器人图标→设备视图→硬件目录中选择模块下拉菜单→选择 DI 128/16/256/32/64/8

bytes　DO 128/16/256/32/64/8 bytes ,这么多字节选项中，可根据需要选择数据大小。如选择 DI 64bytes,DO 64bytes，生成通讯数据分配的组态在 robotbasicIO profnet interface 下，可根据编程需要对地址进行修改，如 DI 地址 68-131，DO 地址也可以改成 68-131。64 字节为 512 位，一个字节为 8 位。1）.定义的 DI 68-131 的地址中，表示 PLC 接收 ABB 机器人传输过来的数据，也就是 512 个位的数据。2）.在定义的 DO 68-131 的地址中，表示 PLC 发出数据给 ABB 机器人也是 512 个位的数据。

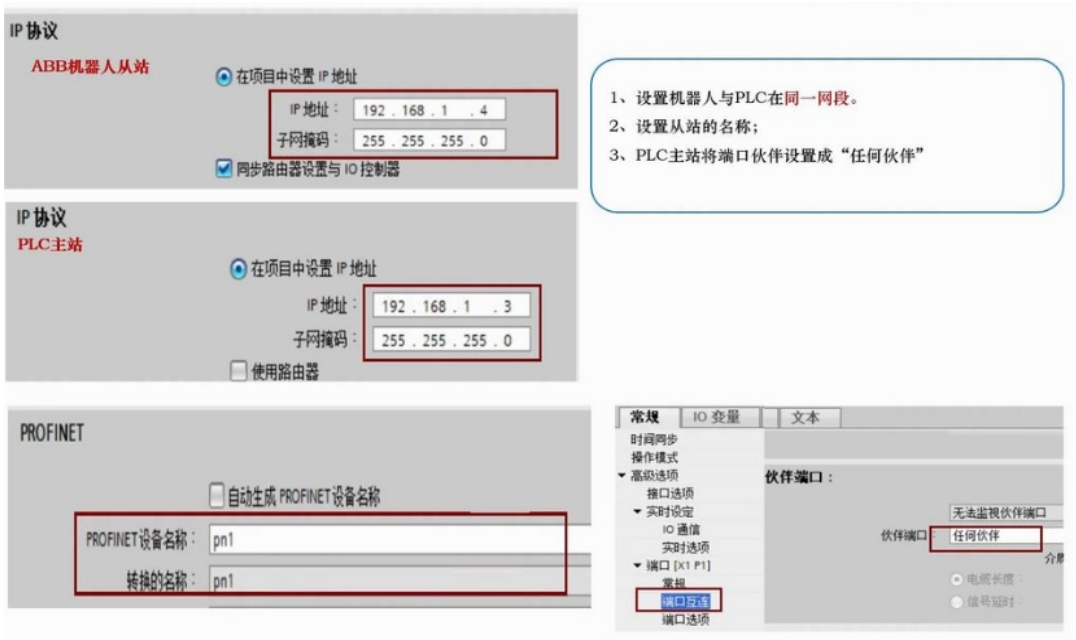


图 10.1 PLC 中 IP 地址设置

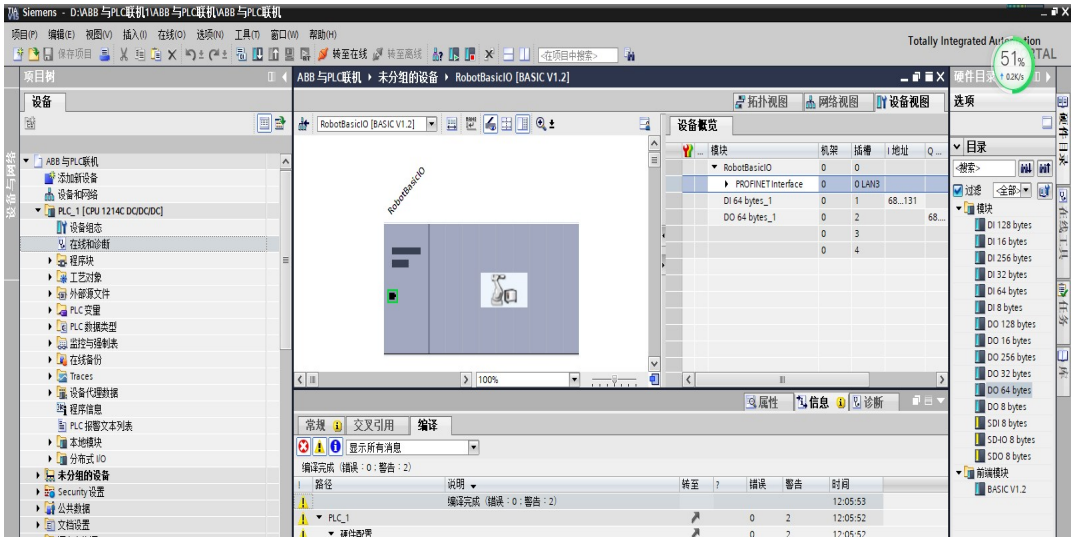


图 10.2　与机器人组态

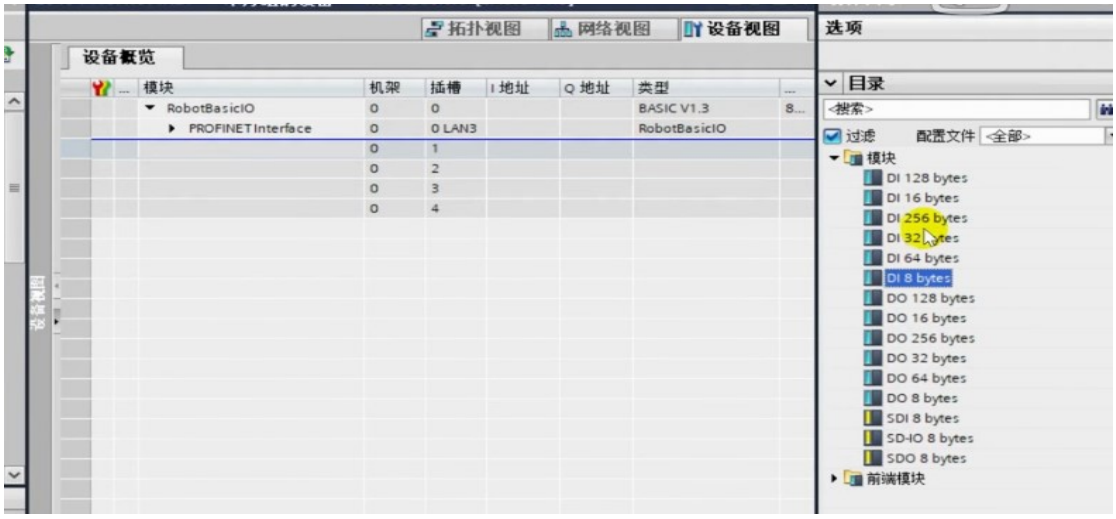


图 10.3 通讯数据为分配

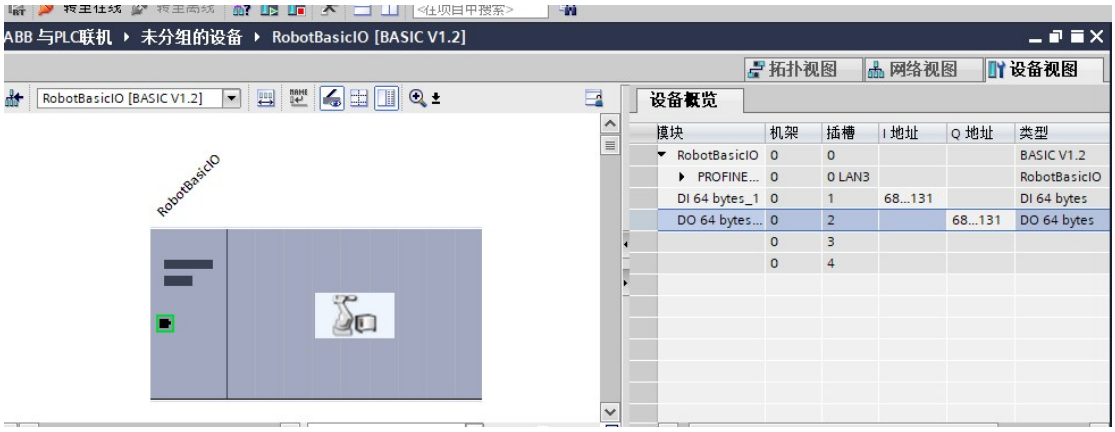


图 10.4 通讯数据为地址

C、 ABB 机器人设备上 PROFINET 通讯配置

1. 设置 IP

1）. ABB 示教器在手动模式下操作：控制面板→控制器设置→左下角的‘设置’→网络→设置 WAN 口 IP 地址 A. B. C. X。

2）. ABB 示教器在手动模式下操作：控制面板→配置→Communication→IP Setting → PROFINET Network → （IP 值： A. B. C. X ；Subnet：255. 255. 255. 0；Interface：WAN；Label：PROFINET Network）。

2. 新建 PROFINET 工业网络

ABB 示教器在手动模式下操作：控制面板→配置→主题→I/O→Industrial

Network→PROFINET→单击添加（来自模块的值：选择 PROFINET ，Connection:PROFINET Network;PROFINET Staion Name:pn1 ）其中 PROFINET Staion Name 中设定的名称要和 PLC 组态 ABB 从站的站名一样（**ABB 从站的改名方法：1. 进入博图软件中左边设备与网络界面 2. 双击 ABB 模块上的名称改名**）。

3. 建立设备

ABB 示教器在手动模式下操作：控制面板→配置→主题→I/O→PROFINET Internal Device→PN_Internal_Device→单击添加(自模块的值：选择 PN_Internal_Device;Network:PROFINET;Input Size :64;Output Size :64)其中 Input Size 和 Output Size 具体字节数据根据 PLC 中 ABB 组态中 DI 和 DO 数据的长度进行一一对应设置。

4. 通讯数据分配对应

ABB 示教器在手动模式下操作：控制面板→配置→主题→I/O→Signal→单击显示全部→添加（Name:PN_GI0;Type of Signal:Group Input;Assigned to Device:PN_Internal_Device;Device Mapping :0-7）这个信号设置为组输入信号 0-7，即为 PLC 组态中 ABB 数据区 DO 端前 0-7 位数据传输人 ABB 设备中名字为 PN_GI0 存储。根据实际需要添加完所有的组输入和组输出信号，便于数据的发送和接收。

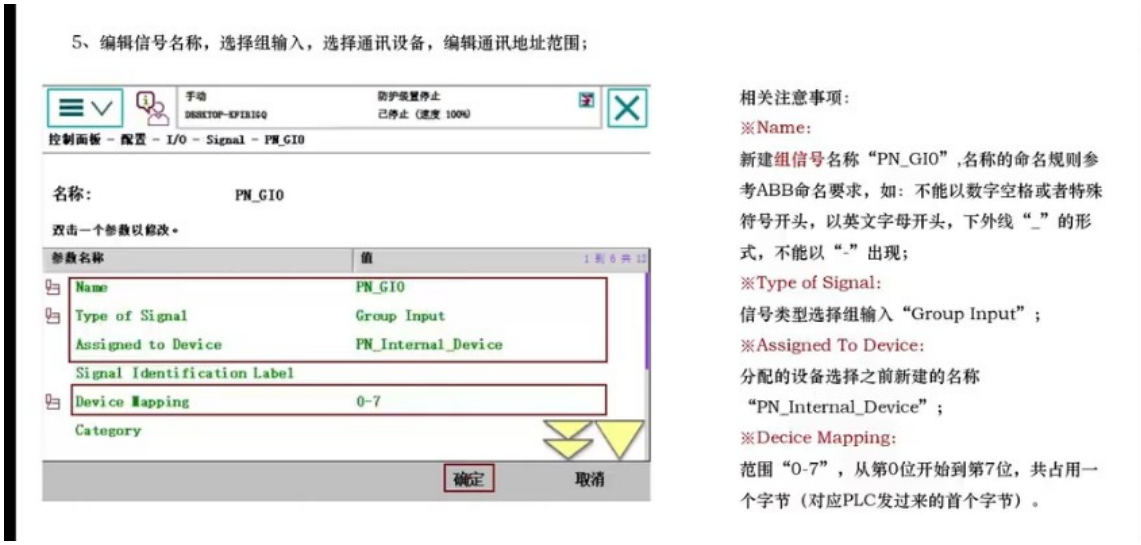


图 10.5 机器人组态 Io 地址与 PLC 对应分配

表 1：PLC 与机器人通讯地址输出位对应表

在 PLC 中 ABB 设置的地址（PLC 发送信号）	Mapping	ABB 机器人组名称（机器人接收 PLC 的组信号）
QB68（发送起始）	0-7	PN_GI0
QB69	8-15	PN_GI1
QB70	16-23	PN_GI2
---	---	---
QB131	504-511	PN_GI63

表 2：PLC 与机器人通讯地址输入位对应表

在 PLC 中 ABB 设置的地址（PLC 接收信号）	Mapping	ABB 机器人组名称（机器人发送到 PLC 的组信号）
IB68（接收起始）	0-7	PN_GO0
IB69	8-15	PN_GO1
IB70	16-23	PN_GO2
---	---	---
IB131	504-511	PN_GO63

D、PLC 通过 Profinet 通讯控制调节机器人速度示例

1.机器人中断程序创建

程序中断解释:中断是指机器人执当前行任务过程中，中间突然需要去执行另一个任务，需要暂停当前的任务去完成另一个任务，当另一个任务完成后再继续执行当前任务，类似 C 语言中 Goto 用法。

新建例行程序 tr_speed,设置为中断程序操作如下所示



图 10.6 新建例行程序



图 10.7 新建中断程序

新建完中断程序后，程序类型就是中断类型:



图 10.8 中断程序类型

2.在机器人添加组输入信号。

配置好 Profinet 通讯后，添加组输入信号，组输入信号命名为 gi_speed，创建完成后重启控制器，操作如下：



图 10.9 添加组信号



图 10.10 添加组信号定义

3.编写速度控制程序

编写程序时需要用到以下指令，其释义如下：

1.232 SpeedRefresh - 更新持续运动速度覆盖

手册用法

SpeedRefresh用于改变当前运动程序任务中进行中的机械臂移动的速度。通过该指令，可能创建关于某些传感器输入的某种粗速度适应。

本指令仅可用于主任务T_ROB1，或者如果在*MultiMove*系统中，则可用于任何运动任务中。

基本示例

以下实例介绍了指令SpeedRefresh：

例 1

```
VAR num change_speed:=70;
SpeedRefresh change_speed;
```

这将把当前速度覆盖改变为70%。

变元

```
SpeedRefresh Override
```

Override

数据类型：num

范围0 ... 100 %内的速度覆盖值。

程序执行

将更新当前运动程序任务中关于机械臂和外部单元进行中的移动的实际速度覆盖值。

当前运动任务中任意机械单元的所有速度数据分量均将受到影响。

> **注意**
>
> 根据SpeedRefresh设置的速度覆盖，不等于根据FlexPendant示教器而设置速度。这是两种不同的值。这两种值的结果以及编程的速度将成为移动中使用的速度。

如果完成PP到Main或已加载新的程序，则将重置通过SpeedRefresh设置的速度，并将应用根据FlexPendant示教器而设置的速度。

更多示例

有关于指令SpeedRefresh的更多例子阐述如下。

例 1

```
VAR intnum time_int;
VAR num override;
...
PROC main()
  CONNECT time_int WITH speed_refresh;
  ITimer 0.1, time_int;
  ISleep time_int;
  ...
  MoveL p1, v100, fine, tool2;
  ! Read current speed override set from FlexPendant
```

下一页继续

1.88 IDelete - 取消中断

手册用法

IDelete（中断删除）用于取消（删除）中断预定。
如果中断仅临时禁用，则应当使用指令ISleep或IDisable。

基本示例

以下实例介绍了指令IDelete：

例 1

```
IDelete feeder_low;
```

取消中断feeder_low。

变元

```
IDelete Interrupt
```

Interrupt

数据类型：intnum
中断识别号。

程序执行

彻底擦除中断的定义。为了再次进行定义，必须首先将其重新连接至软中断程序。
建议从停止点开始IDelete。否则，在达到运动路径终点之前，中断将停用。
无须擦除中断；在以下情形时，中断会自动停用：

- 加载新的程序
- 从起点重启程序
- 将程序指针移动到程序起点

语法

```
IDelete [ Interrupt ':=' ] < variable (VAR) of intnum > ';'
```

1.40 CONNECT - 将中断与软中断程序相连

手册用法

CONNECT用于发现中断识别号，并将其与软中断程序相连。
通过下达中断事件指令并规定其识别号，确定中断。因此，当出现该事件时，自动执行软中断程序。

基本示例

以下实例介绍了指令CONNECT：

例 1

```
VAR intnum feeder_low;
PROC main()
  CONNECT feeder_low WITH feeder_empty;
  ISignalDI di1, 1 , feeder_low;
  ...
```

创建中断识别号feeder_low，并将其与软中断程序feeder_empty相连。当输入di1变高时，将会出现中断。换句话说，当信号变高时，执行feeder_empty软中断程序。

变元

```
CONNECT Interrupt WITH Trap routine
```

Interrupt

数据类型：intnum
被分配以中断识别号的变量。不得在程序（程序数据）内进行声明。

Trap routine

Identifier
软中断程序的名称。

1.110 ISignalGI - 下达一组数字信号输入信号中断的指令

手册用法

ISignalGI（中断信号组数字信号输入）用于下达和启用一组数字信号输入信号中断的指令。

基本示例

以下实例介绍了指令ISignalGI：

例 1

```
VAR intnum sig1int;
PROC main()
  CONNECT sig1int WITH iroutine1;
  ISignalGI gi1,sig1int;
```

当数字信号输入组信号变更数值时，下达中断指令。

！控制主程序

PROC Profinet_Control()

VAR intnum sig1int;！声明一个中断识别号

IDelete sig1int;！连接中断之前先取消中断，防止连接失败

CONNECT sig1int WITH tr_speed;！将中断识别号同中断程序相连

ISignalGI gi_speed, sig1int;！组输入信号变化时，执行中断指令

WHILE TRUE DO ！主程序循环

SocketServer;

ENDWHILE

ENDPROC

TRAP tr_speed

SpeedRefresh gi_speed; ！执行中断程序就能控制机器人速度；

ENDTRAP

ENDMODULE

```
52  PROC Profinet_Control()
53      VAR intnum sig1int;
54      IDelete sig1int;
55      CONNECT sig1int WITH tr_speed;
56      ISignalGI gi_speed,sig1int;
57      WHILE TRUE DO
58          SocketServer;
59          !主程序循环
60      ENDWHILE
61  ENDPROC
62
63  TRAP tr_speed
64      SpeedRefresh gi_speed;
65  ENDTRAP
66
67  ENDMODULE
```

第十一章 简单码垛生产线实验

实验目的

1. 学习 ABB 工业机器人的控制语句的使用。
2. 学习 ABB 工业机器人中对于 I/O 信号的操作。
3. 初步认识 ABB 工业机器人生产线硬件的设计。
4、初步认识 ABB 工业机器人生产线程序的编写。

实验要求

根据生产线，利用机器人完成搬运、送料、运送、冲压、检测、码放成品的任务。

实验设备：型号 CHL-JC-02-A 的工作站

预习要点

1. 预习 ABB 工业机器人编程指令中控制指令的用法。
2. 预习 ABB 工业机器人编程指令中 I/O 操作指令的用法。
3. 了解实验室 ABB 工业机器人的 I/O 口接入。

A、控制系统整体架构

设计的码垛生产线控制系统主要由以下 7 个部分组成：ABB Robot 控制器、机械臂、PLC、示教器、工作台、触摸屏以及电脑。其中以 ABB Robot 控制器和 PLC 为控制部分，机械臂和工作台为执行任务的部分，示教器和电脑作为可编程终端，设计机器人控制程序，触摸屏可控制 PLC,进行人机交互。在 ABB Robot 控制器上扩展有 DSQC652（简称 d652 信号板）输入输出信号板，该信号板为 16 点数字量输入和 16 点数字量输出，如图 11.1 所示。

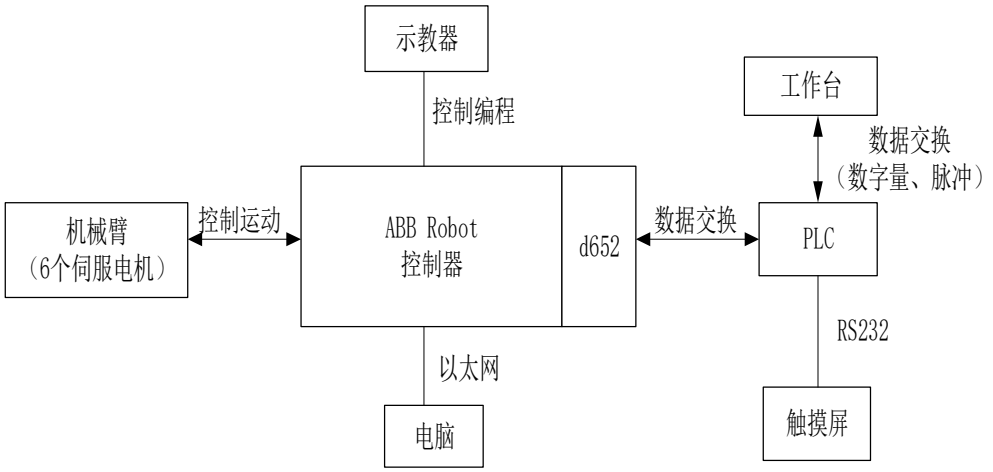


图 11.1 整体架构图

机器人与 PLC I/O 通信接线图

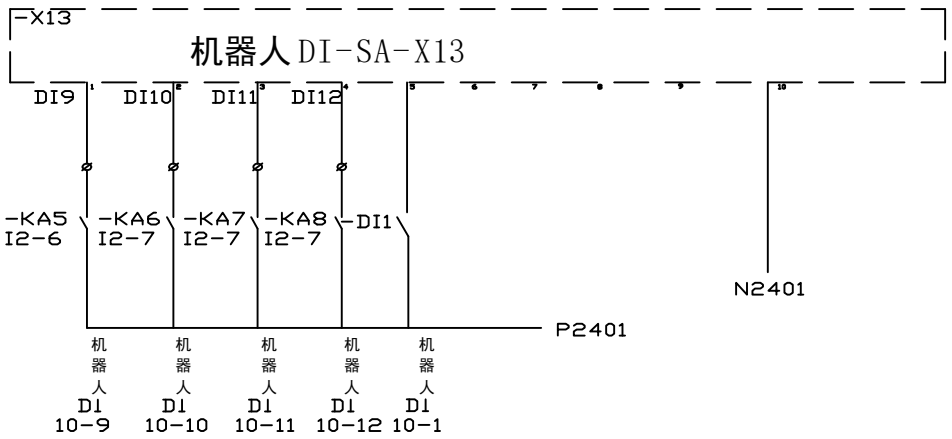


图 11.2 机器人输入端接线图

其中：1、DI10_9 检测物体是否传输到输送带末端信号。

2、DI10_10 检测冲压是否完成信号。

3、DI10_11 与 DI10_1 检测是否开启流水线工作模式信号。

4、DI10_12 作为备用部分。

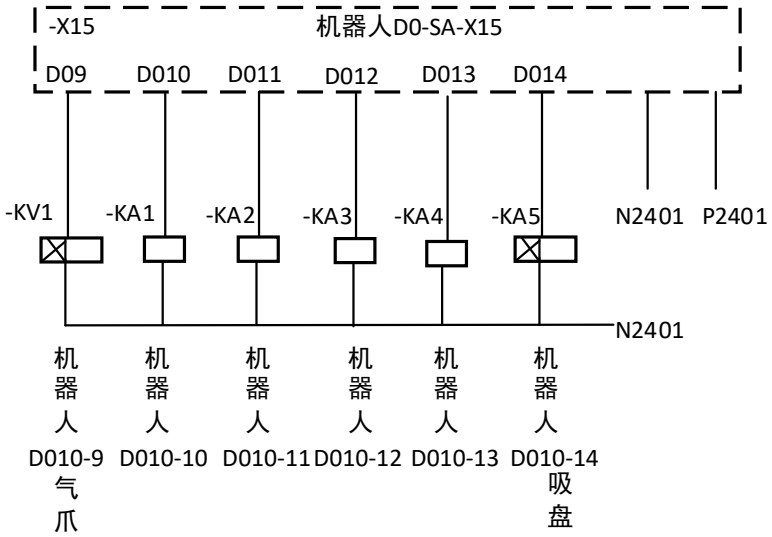


图 11.3 机器人输出端接线图

其中：1、DO10_9 控制机械臂上夹爪的闭合与松开。

2、DO10_10 控制启动冲压信号。

3、DO10_11 控制开始模拟检测信号。

4、DO10_12 与机器人 DO10_13 作为备用部分。

5、DO10_14 控制机械臂吸盘吸起物体或放下物体。

为了实现机器人生产线所要求的任务，机器人与 PLC 之间需要进行信号交换，信号交换特别说明。

（1）信号 DI10_9，当物块被传送带运输到末端，皮带侧安装的光电传感器检测到物块时，PLC 数字量输出端 Q1.0 就会给机器人端口 DI10_9 发送信号，机器人接收到信号后控制机械臂夹取物块放至冲压前区域。

（2）信号 DO10_10，当物块被机器人放至到冲压前区域时，机器人信号板输出端 DO10_10 给 PLC 数字量输入端 I0.5 发送信号，此时冲压前光电传感器检测到该物块，工作台启动冲压功能。

（3）信号 DI10_10，当物块冲压完成后,光电传感器检测到该物块，PLC 数字量输出端 Q1.1 发送信号到机器人输入端 DI10_10，机器人接收到信号后控制机械臂移动至冲压后区域并夹取物块。

（4）信号 DO10_11,机器人控制机械臂夹取物块后，移动至成品检测区，完成该任务后，机器人信号板输出端 DO10_11 给 PLC 发送信号，PLC 开关量输入端 I0.6 接收到信号后，对该物块进行模拟成品检测。

（5）信号 DO10_14 控制机械臂上的吸盘信号。当此信号被置位时，吸盘吸起物体；当此信号被复位时，吸盘松开物体。

（6）信号 DI10_11 和 DI10_1 为开启流水线模式的信号。当信号 DI10_11 为 1，信号 DI10_1 为 0 时，流水线模式才能运行。

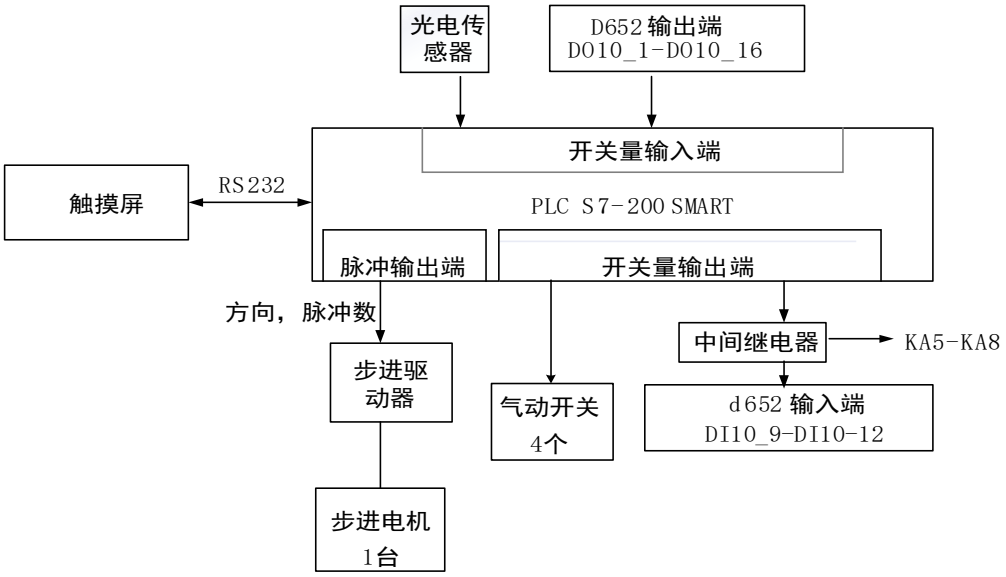


图 11.4 PLC 整体框架图

如图 11.4 所示，整个机器人码垛生产线以 PLC 控制器为核心，PLC 控制器采集光电传感器的信号、机器人信号板输出的信号和触摸屏信号，根据生产工艺需要控制步进电机、气动开关、中间继电器工作。其中中间继电器工作后，输出的开关量信号进入机器人信号板输入端，控制机器人工作，满足生产工艺要求。

7.4.2 PLC 的 I/O 分配表

（1）PLC 输入地址分配

符号	地址	注释
流水线	I0.0	
启动	I0.2	
停止	I0.3	
机器人信号 1	I0.5	机器人信号板接口 DO10_10 给 PLC 发送信号
机器人信号 2	I0.6	机器人信号板接口 DO10_11 给 PLC 发送信号
原料检测	I1.3	
送料气缸前限位	I1.4	
送料气缸后限位	I1.5	
冲压 1 气缸前限位	I1.6	
冲压 1 气缸后限位	I1.7	

冲压 2 气缸前限位	I2.0	
冲压 2 气缸后限位	I2.1	
冲压 3 气缸前限位	I2.2	
冲压 3 气缸后限位	I2.3	
皮带光电	I2.4	
冲压前检测	I2.5	
冲压完成检测	I2.6	
检测成品	I2.7	

（2）PLC 输出地址分配

符号	地址	注释
机器人 1 信号	Q1.0	PLC 给机器人信号板接口 DI10_9 发送信号
机器人 2 信号	Q1.1	PLC 给机器人信号板接口 DI10_10 发送信号
送料，气缸	Q1.4	
冲压气缸 1	Q1.5	
冲压气缸 2	Q1.6	
冲压气缸 3	Q1.7	

步进驱动器与步进电机

将电脉冲转化为角位移的执行机构称为步进驱动器。当它接收到一个脉冲信号，就驱动步进电机按照设定好的方向转动一个固定的角度，它的转动是以固定的角度一步一步运行的。图 11.5 为步进驱动器与步进电机之间的接线图，其中 Q0.0 为脉冲信号，Q0.1 为方向信号。

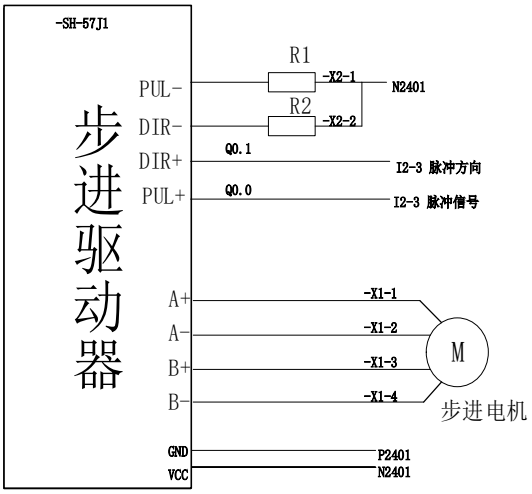


图 11.5 步进驱动器与步进电机接线图

B、 动力电源接线图及 PLC 电气原理图

机器人生产线设备的启动只需将 QF 断路器与 QS 隔离开关闭合，再将机器人电源插头与空压机电源插头插入插座，最后按下启动按钮，便可启动整套设备并亮起电源指示灯，图 11.6 为机器人生产线设备的动力电源接线图。

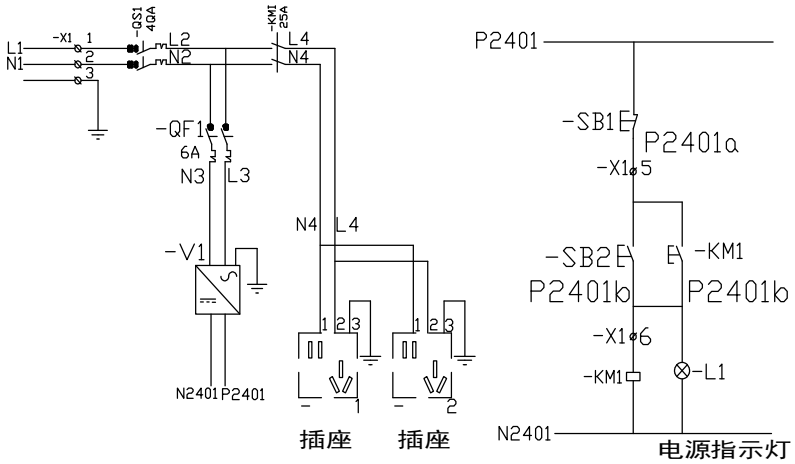


图 11.6 动力电源接线图

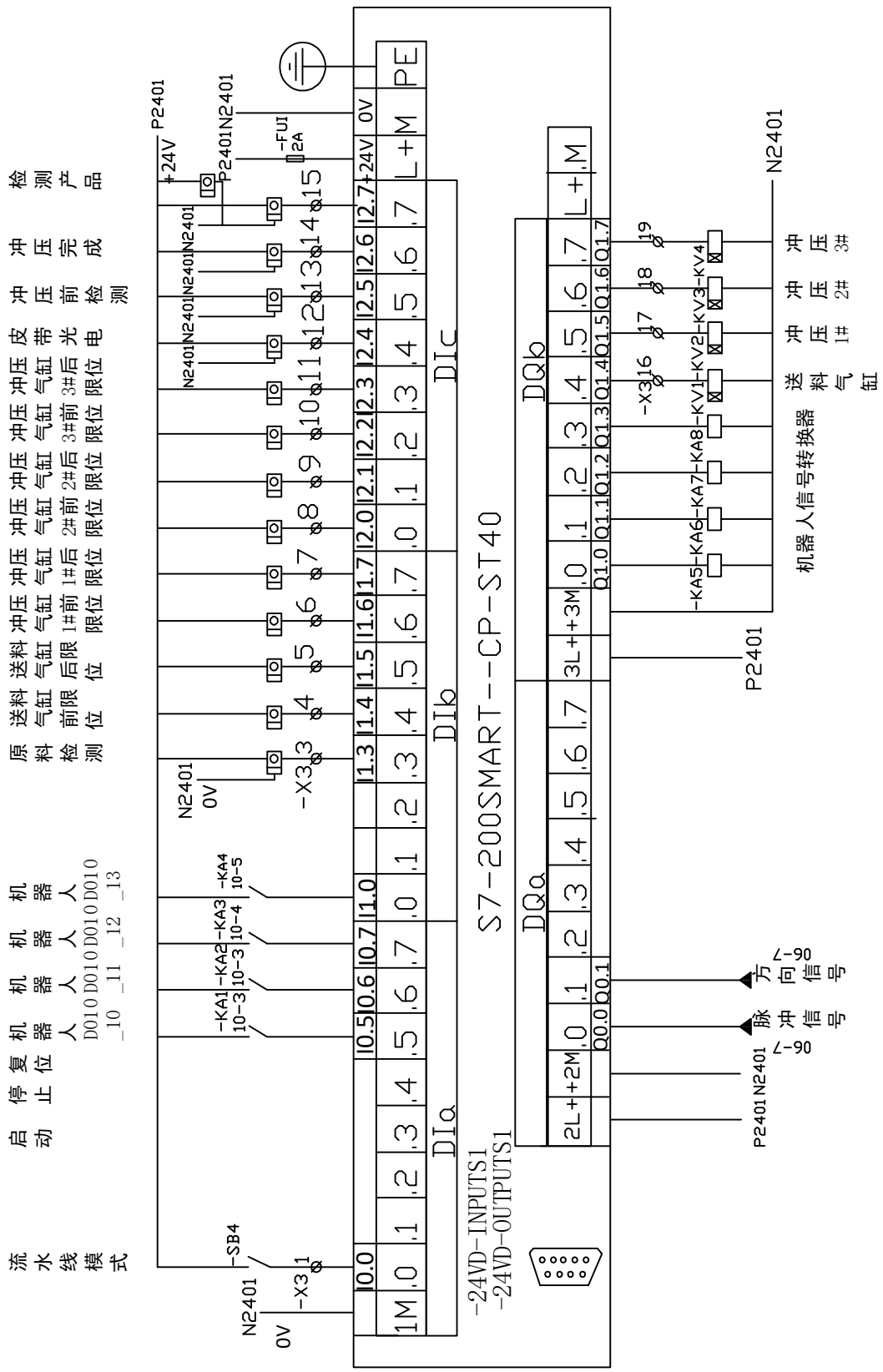


图 11.7 PLC 电气原理图

C、参考程序流程图

1）生产线工艺流程图

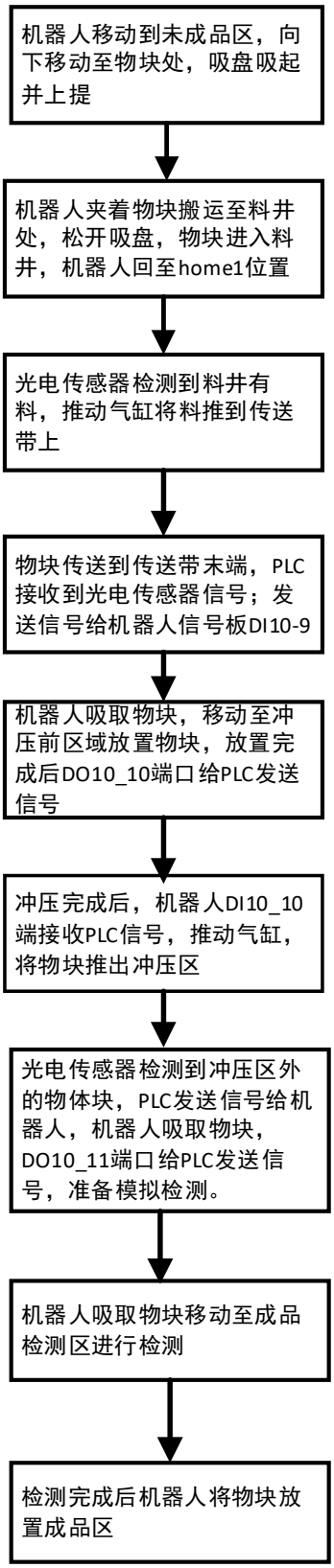


图 11.8 生产线工艺流程

2）未成品区和码垛成品区物块位置码垛图及计算方法

1、物块为尺寸相同的长方体物块，尺寸为:长×宽×高=60×30×30（单位：毫米）

2、未成品区和码垛成品区物块码垛的托盘尺寸相同，如图 11.9 所示。

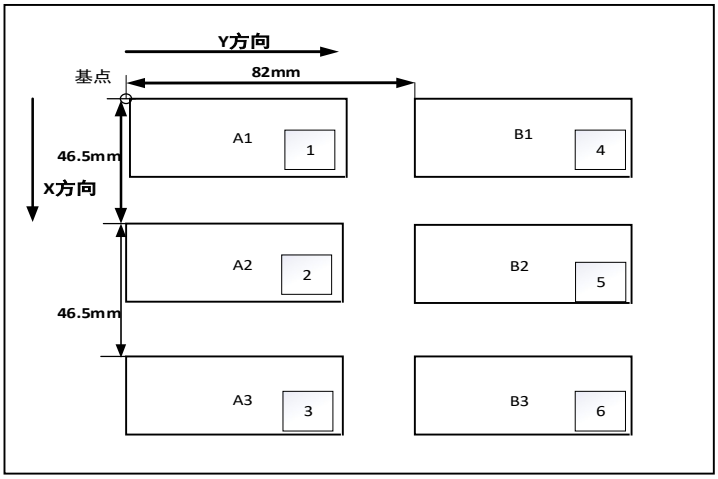


图 11.9 托盘主要尺寸图

该托盘有 6 小格，可摆放 6 块长方体物块。如果以基点为坐标作偏移，则 A1,A2...B3 左上角的坐标值如下表表示：

名称	X 方向坐标	Y 方向坐标	名称	X 方向坐标	Y 方向坐标
A1	0mm	0mm	B1	0mm	82mm
A2	46.5mm	0mm	B2	46.5mm	82mm
A3	93mm	0mm	B3	93mm	82mm

3、如果未成品区每个小格已码垛好物块，先从 A1 区域拿取物块，经过工艺流程码垛到另一个托盘上，码垛位置同样为 A1 区域，机器人在未成品区拿取物块的顺序依次为 A1,A2,A3,B1,B2,B3,码垛的顺序同样为 A1,A2,A3,B1,B2,B3，则可编子程序实现码垛取和码垛放子程序。

码垛相关子程序算法包含两个二维数组，分别为 count1{a,1}（X 轴方向偏移的值）和 count1{a,2}（Y 轴方向偏移的值），本次共夹取 6 个物块，每完成一个物块的整个环节，a 值便会加 1，则机械臂下次移至未成品区的具体位置与上次位置不同。

X 轴与 Y 轴偏移量计算公式如下：

$$x=x_0+465i;\quad y=y_0+82j$$

i 的取值为 0-2，j 的取值为 0-1

X 轴的偏移公式为内循环，Y 轴的偏移公式为外循环，即 j 先取值为 0，Y 轴的偏移量为 0，i 依次取值为 0,1,2，X 轴的偏移量分别为 0,46.5,93；j 再取值为 1,Y 轴的偏移量为 82,i 依次取值为 0,1,2,X 轴的偏移量分别为 0,46.5,93。

由以上公式可得以下 12 个二维数组：

count1{1,1}的值为 0，count1{1,2}的值为 0

count1{2,1}的值为 46.5，count1{2,2}的值为 0

count1{3,1}的值为 93，count1{3,2}的值为 0

count1{4,1}的值为 0，count1{4,2}的值为 82

count1{5,1}的值为 46.5，count1{5,2}的值为 82

count1{6,1}的值为 93，count1{6,2}的值为 82

4）参考程序操作步骤如下：

1、定义 I/O 变量（PLC 与机器人通信）

1.1 在示教器或虚拟示教器上打开控制面板→配置系统参数 →主题 →I/O→ Devicenet Device→ 查看是否有 d652,如果没有请添加该模块，过程如下：打开控制面板→配置系统参数 →主题 →I/O→ Devicenet Device→ 添加

需要修改的主要参数如下：

1）使用来自模板的值选择：DSQC 652 24VDC I/O Device

2）Name:d652

3）Address ： 10

4）Product Code ： 26

5）Device Type： 7

6）Connection Type ： Change-of-State(COS)

7）Connection output Size(byte):2(输出字节长度）

8）Connection Input Size(byte):2(输入字节长度）

1.2 控制面板→配置系统参数 →主题 →I/O→signal→添加

名称	连接设备	信号类型(Type of Signal)	Device mapping	名称	连接设备	信号类型(Type of Signal)	Device mapping
DI10_1	d652	Digtal Input	0	DO10_1	d652	Digtal Output	0
DI10_2	d652	Digtal Input	1	DO10_2	d652	Digtal Output	1
DI10_3	d652	Digtal Input	2	DO10_3	d652	Digtal Output	2
DI10_4	d652	Digtal Input	3	DO10_4	d652	Digtal Output	3
DI10_5	d652	Digtal Input	4	DO10_5	d652	Digtal Output	4
DI10_6	d652	Digtal Input	5	DO10_6	d652	Digtal Output	5
DI10_7	d652	Digtal Input	6	DO10_7	d652	Digtal Output	6
DI10_8	d652	Digtal Input	7	DO10_8	d652	Digtal Output	7
DI10_9	d652	Digtal Input	8	DO10_9	d652	Digtal Output	8
DI10_10	d652	Digtal Input	9	DO10_10	d652	Digtal Output	9
DI10_11	d652	Digtal Input	10	DO10_11	d652	Digtal Output	10
DI10_12	d652	Digtal Input	11	DO10_12	d652	Digtal Output	11
DI10_13	d652	Digtal Input	12	DO10_13	d652	Digtal Output	12

DI10_14	d652	Digtal Input	13	DO10_14	d652	Digtal Output	13
DI10_15	d652	Digtal Input	14	DO10_15	d652	Digtal Output	14
DI10_16	d652	Digtal Input	15	DO10_16	d652	Digtal Output	15

2、定义程序数据

2.1 定义 num 类型数据

程序数据→视图→全部数据类型→num→新建

2.1.1 数据名称 a,初始值为 7，全局，可变量。

数据名称 b,初始值为 1，全局，可变量。

数组名称：count1，全局，可变量，维数，2，{6,2}（系统自带软数字输入键，不可电脑键盘输入）。

组件（数组）	值	组件（数组）	值	组件（数组）	值
{1,1}	0	{3,1}	93	{5,1}	46.5
{1,2}	0	{3,2}	0	{5,2}	82
{2,1}	46.5	{4,1}	0	{6,1}	93
{2,2}	0	{4,2}	82	{6,2}	82

2.2 定义 BOOL 类型数据

程序数据→视图→全部数据类型→bool→新建

数据名称 Finish ，初始值为 TRUE，全局，可变量。

2.2 定义 robtarget 类型数据

程序数据→视图→全部数据类型→robtarget→新建

数据名称： Area_1，全局，常量 （物块在未成品区位置 1 的示教）

Area_2 ，全局，常量;（物块在物料口位置 2 的示教）

Area_3 ，全局，常量;（物块在输送皮带末端位置 3 的示教）

Area_4 ，全局，常量;（物块在冲压前区位置 4 的示教）

Area_5 ，全局，常量;（物块在冲压后区位置 5 的示教）

Area_6 ，全局，常量;（物块在检验区位置 6 的示教）

Area_7 ，全局，常量;（物块在成品区位置 7 的示教）

Home1 ，全局，常量;（机器人返回到归位区位置 8 的示教）

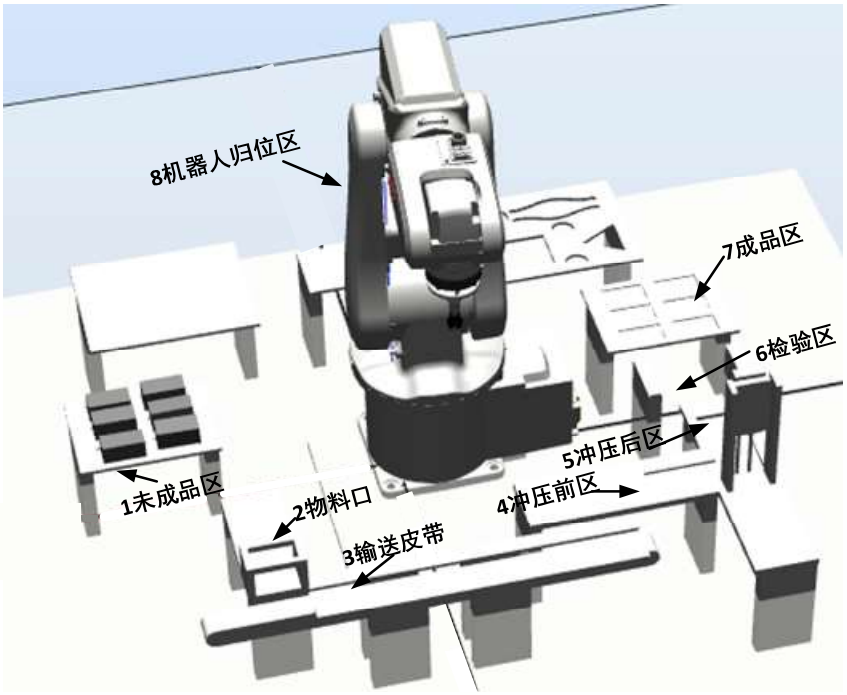


图 11.10 机器人各位置标定示意图

3、编写程序

启动 ABB 工业机器人，将操作模式选到手动操作模式下，才能在示教器上或虚拟示教器上编写主程序和子程序，主程序调用子程序。

子程序编辑：

1、在示教器中依次选择菜单→程序编辑器→任务与程序→显示模块→文件→新建模块。新模块名称：shenchanxian,类型：Program。

双击 shenchanxian，点击例行程序→文件→新建例行程序，在例行程序声明中，名称：chushihua，类型：程序，参数：无，模块：shenchanxian；

重复这样的步骤，分别建立 4 个子程序：chushihua，quhuo，banyun，fanghuo。

双击 chushihua，则进入编程界面，4 个子程序参考如下：

```
PROC chushihua()  //初始化程序
    VAR robtarget here;
        here:=CRobT();
        MoveL offs(here,0,0,home1.trans.z-here.trans.z),v1000,z50,tool0;
        MoveJ home1,v1000,z50,tool0;
        a:=1;
        b:=1;
        Reset DO10_9;
        Reset DO10_14;
        Finish:=TRUE;
    ENDPROC
PROC quhuo()          //机器人取物体程序（偏移量算法）
        MoveJ Offs(Area_1 ,count1{a,1},count1{a,2},80),v1000,fine,tool0;
         Movel Offs(Area_1 ,count1{a,1},count1{a,2},0),v1000,fine,tool0;
        Set DO10_14;
        WaitTime 1;
```

```
            Movel Offs(Area_1 ,count1{a,1},count1{a,2},300),v1000,fine,tool0;
    ENDPROC

PROC banyun()   //搬运程序
          MoveJ Offs(Area_2,0,0,50),v1000,fine,tool0;
          Movel Area_2,v1000,fine,tool0;
         Reset DO10_14;
         waittime 1;
        Movel offs(Area_2,0,0,50),v1000,fine,tool0;
        MoveJ home,v1000,z20,tool0;
        WaitDI DI10_9,1;
        MoveJ Offs(Area_3,0,0,250),v1000,fine,tool0;
        Movel Area_3,v1000,fine,tool0;
        set DO10_14;
        waittime 1;
       Movel offs(Area_3,0,0,200), v1000, fine, tool0;
       MoveJ offs(Area_4,0,0,200),v1000,fine,tool0;
       Movel Area_4,v1000,fine,tool0;
       reset DO10_14;
       waittime 1;
       Movel Offs(Area_4,0,0,250),v1000,fine,tool0;
       PulseDO\PLength:=0.5, DO10_10;
       WaitDI DI10_10,1;
       MoveJ Offs(Area_5,0,0,250),v1000,fine,tool0;
       MoveJ offs(Area_5,0,0,100),v1000,fine,tool0;
       Movel Area_5,v1000,fine,tool0;
       set DO10_14;
       waittime 1;
       Movel offs(Area_5,0,0,200),v1000,fine,tool0;
       PulseDO\PLength:=0.5, DO10_11;
       MoveJ offs(Area_6,0,0,100),v1000,fine,tool0;
       Movel Area_6,v1000,fine,tool0;
       WaitTime 1;
       Movel offs(Area_6,0,0,200),v1000,fine,tool0;
 ENDPROC

    PROC fanghuo()   //码垛程序
        MoveJ Offs(Area_7,count1{a,1},count1{a,2},200),v1000,z50,tool0;
            Movel Offs(Area_7,count1{a,1},count1{a,2},0),v1000,fine,tool0;
            reSet DO10_14;
            WaitTime 1;
            Movel Offs(Area_7,count1{a,1},count1{a,2},200),v1000,fine,tool0;
            Incr a;
    ENDPROC
```

2、主程序编辑：

在示教器中依次选择菜单→程序编辑器→任务与程序→显示模块→点击Module1则可进入有PROC main()这样开头的画面。当然Module1通过更改声明可以把名称修改为其他名称。

```
MODULE Module1（主程序）chushihua，quhuo，banyun，fanghuo
  PROC main2()
        chushihua;
        WHILE DI10_11=1 AND DI10_1=0 AND Finish=TRUE DO
        quhuo;
        banyun;
        fanghuo;
        MoveJ home1,v1000,z20,tool0;
        IF a=7then
         stop;
        ENDWHILE
        WHILE DI10_12=1 AND DI10_1=0 AND Finish=TRUE DO
        MoveJ home1,v1000,z20,tool0;
        TPWrite "Please install the tool on the jig";
        Break;
        set DO10_14;
        Break;
        MoveJ home1,v1000,z20,tool0;
        TPWrite "Please put the jig on the tool";
        Break;
        Reset DO10_14;
        Break;
        Finish :=FALSE;
        ENDWHILE
        IF Finish=FALSE TPWrite"Failed to initialize";
      IF DI10_11=0 AND DI10_12=0 TPWrite"The current in the teaching mode";
        IF DI10_1=1 TPWrite"you should turn off the DI10_1";
  ENDPROC
ENDMODULE
```

3、调试

程序编程完成后，利用示教器进行手动调试，运行过程中出现报警，根据报警提示进行修改，直到报警解除。在生产线运行的各个阶段示教robtarget中变量的位置，即修改Area_1，Area_2，Area_3，Area_4，Area_5，Area_6，Area_7及home1。

连续运行

手动调试结束，一切都正常后，将ABB控制箱按钮切换到自动状态，电

机按钮上电，PLC 触摸屏上启动 PLC，设置输送带速度，6 块物块放置在托盘上，在示教器上打开程序编辑器，进入主程序 Module1，将 PP 移至 main，设置运动速度，通常设置 50%左右，启动示教器上右下方按钮运行按钮▲，程序连续运行，如出现故障按下停止按钮■，程序停止运行。

D、工作台工作流程

工作台上要执行将物块推出料井，运输，冲压，成品检测的任务，具体流程见图 11.11

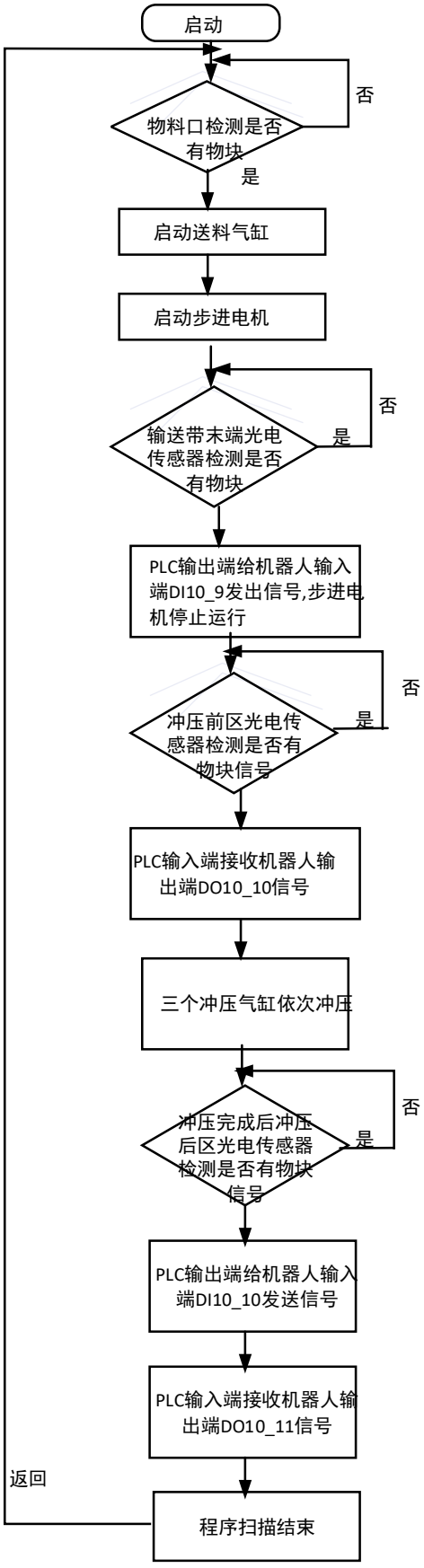


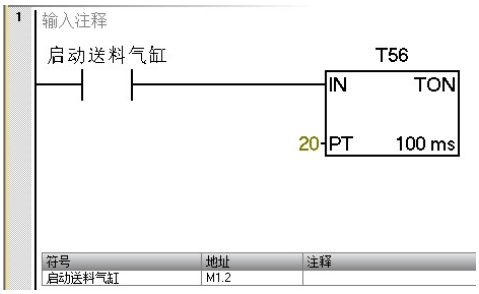
图 11.11 工作台工作流程

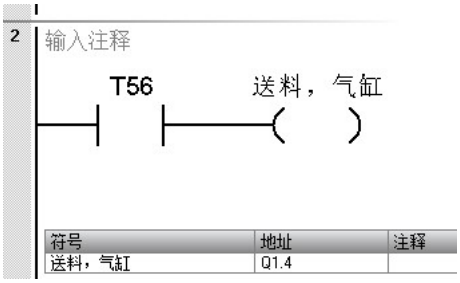
1） PLC 梯形图设计

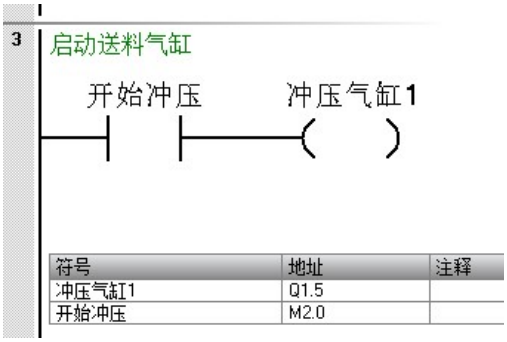
PLC 程序设计分为主程序和子程序。

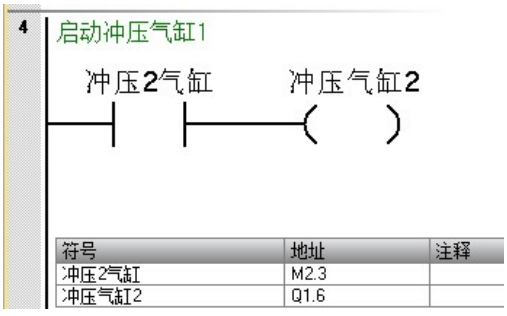
子程序有 2 个，分别命名为 shuchu，bujindianji。

1）子程序 shuchu

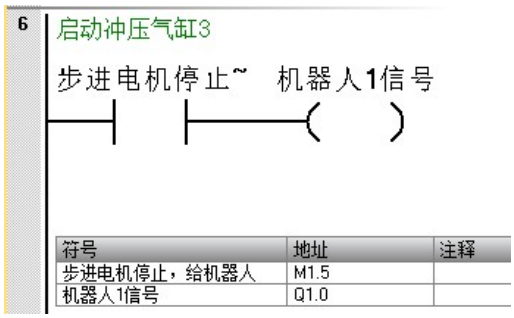


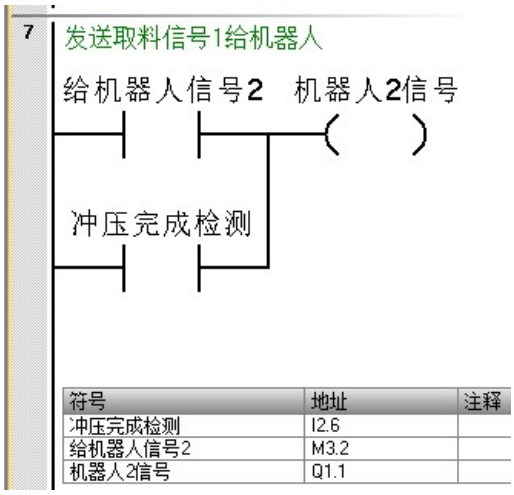


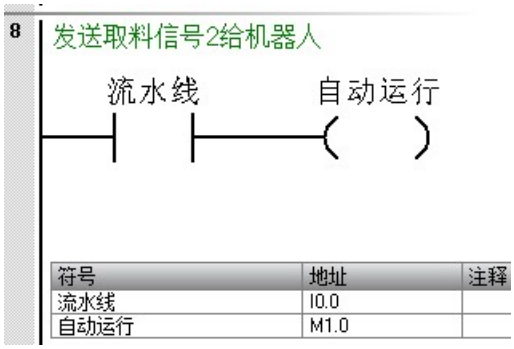


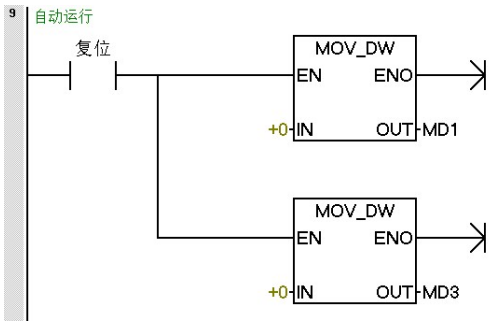




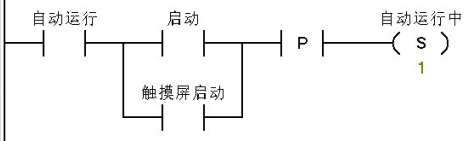






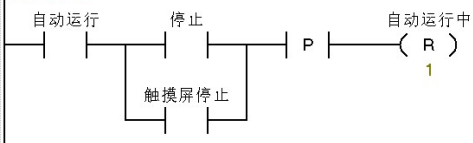


10 复位



符号	地址	注释
触摸屏启动	M0.1	
启动	I0.2	
自动运行	M1.0	
自动运行中	M1.1	

11 启动自动运行



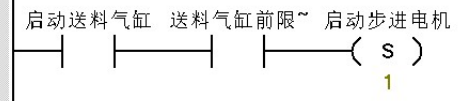
符号	地址	注释
触摸屏停止	M0.2	
停止	I0.3	
自动运行	M1.0	
自动运行中	M1.1	

12 停止自动运行



符号	地址	注释
启动送料气缸	M1.2	
送料气缸后限位	I1.5	
原料检测	I1.3	
自动运行中	M1.1	

13 启动送料气缸



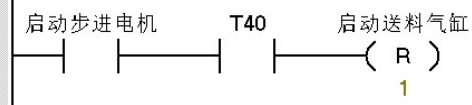
符号	地址	注释
启动步进电机	M1.3	
启动送料气缸	M1.2	
送料气缸前限位	I1.4	

14 启动步进电机



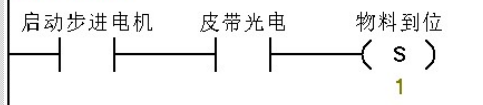
符号	地址	注释
启动步进电机	M1.3	

15 T40



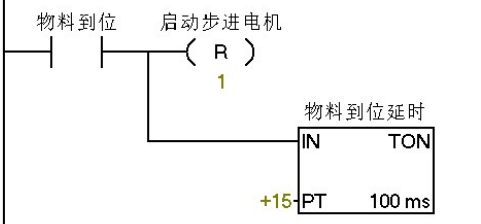
符号	地址	注释
启动步进电机	M1.3	
启动送料气缸	M1.2	

16 送料气缸缩回



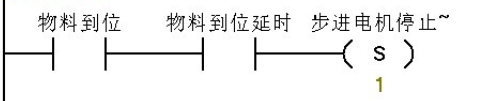
符号	地址	注释
皮带光电	I2.4	
启动步进电机	M1.3	
物料到位	M1.4	

17 物料到位



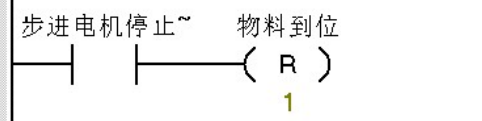
符号	地址	注释
启动步进电机	M1.3	
物料到位	M1.4	
物料到位延时	T41	

18 复位步进电机



符号	地址	注释
步进电机停止，给机器人	M1.5	
物料到位	M1.4	
物料到位延时	T41	

19 发送取料信号给机器人



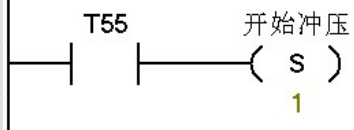
符号	地址	注释
步进电机停止，给机器人	M1.5	
物料到位	M1.4	

20 复位物料到位信号



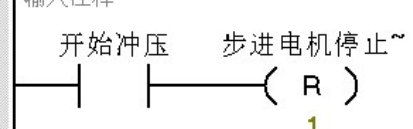
符号	地址	注释
步进电机停止，给机器人	M1.5	
冲压1气缸后限位	I1.7	
冲压2气缸后限位	I2.1	
冲压3气缸后限位	I2.3	
冲压前检测	I2.5	
机器人信号1	I0.5	抓料完成

21 冲压1启动



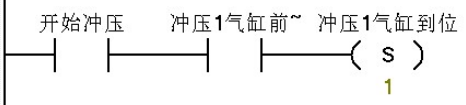
符号	地址	注释
开始冲压	M2.0	

22 输入注释



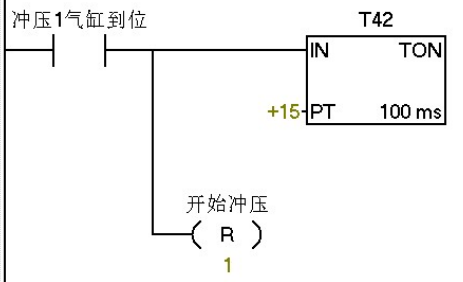
符号	地址	注释
步进电机停止，给机器人	M1.5	
开始冲压	M2.0	

23 输入注释



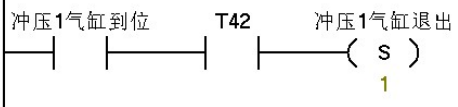
符号	地址	注释
冲压1气缸到位	M2.1	
冲压1气缸前限位	I1.6	
开始冲压	M2.0	

24 冲压1到位



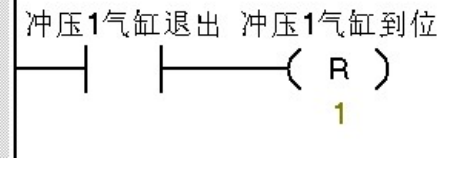
符号	地址	注释
冲压1气缸到位	M2.1	
开始冲压	M2.0	

25 T42

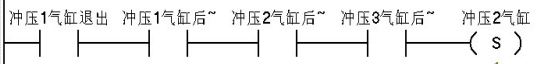


符号	地址	注释
冲压1气缸到位	M2.1	
冲压1气缸退出	M2.2	

26 冲压1缩回

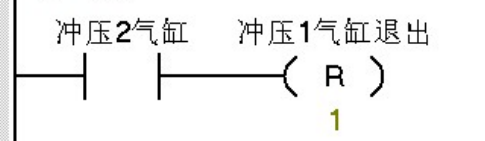


27 复位冲压1到位



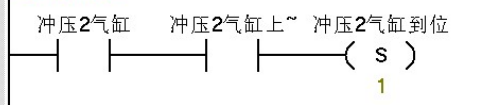
符号	地址	注释
冲压1气缸后限位	I1.7	
冲压1气缸退出	M2.2	
冲压2气缸	M2.3	
冲压2气缸后限位	I2.1	
冲压3气缸后限位	I2.3	

28 冲压2启动



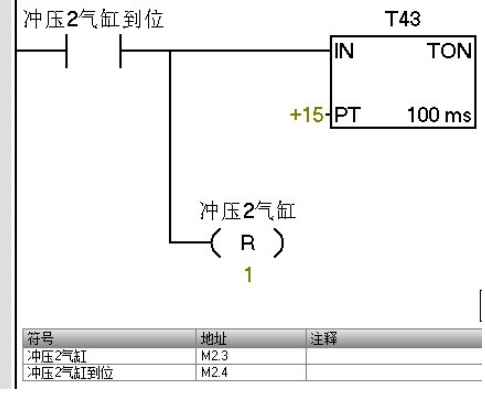
符号	地址	注释
冲压1气缸退出	M2.2	
冲压2气缸	M2.3	

29 复位冲压1缩回



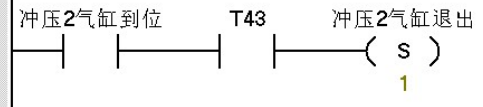
符号	地址	注释
冲压2气缸	M2.3	
冲压2气缸到位	M2.4	
冲压2气缸上限位	I2.0	

30 冲压2到位



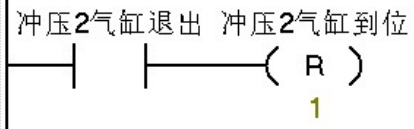
符号	地址	注释
冲压2气缸	M2.3	
冲压2气缸到位	M2.4	

31 T43



符号	地址	注释
冲压2气缸到位	M2.4	
冲压2气缸退出	M2.5	

32 冲压2缩回



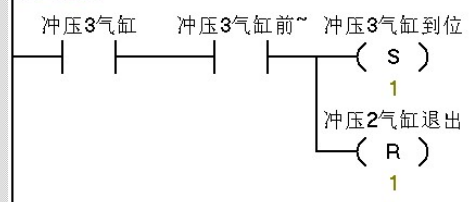
符号	地址	注释
冲压2气缸到位	M2.4	
冲压2气缸退出	M2.5	

33 复位冲压2到位



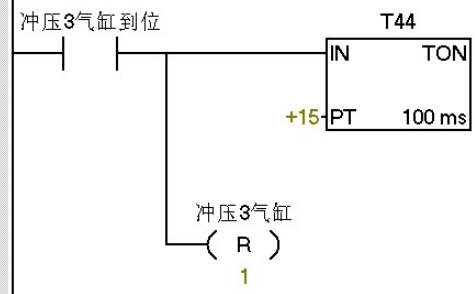
符号	地址	注释
冲压1气缸后限位	I1.7	
冲压2气缸后限位	I2.1	
冲压2气缸退出	M2.5	
冲压3气缸	M2.7	
冲压3气缸后限位	I2.3	

34 冲压3启动



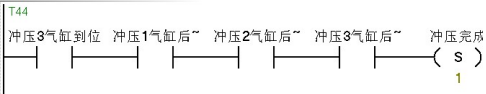
符号	地址	注释
冲压2气缸退出	M2.5	
冲压3气缸	M2.7	
冲压3气缸到位	M3.0	
冲压3气缸前限位	I2.2	

35 冲压3到位，复位冲压2缩回



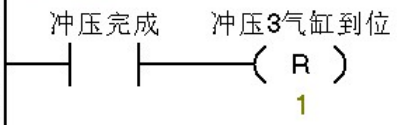
符号	地址	注释
冲压3气缸	M2.7	
冲压3气缸到位	M3.0	

36 T44



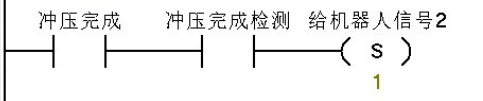
符号	地址	注释
冲压1气缸后限位	I1.7	
冲压2气缸后限位	I2.1	
冲压3气缸到位	M3.0	
冲压3气缸后限位	I2.3	
冲压完成	M3.1	

37 冲压3完成



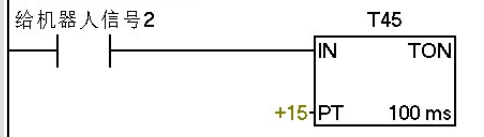
符号	地址	注释
冲压3气缸到位	M3.0	
冲压完成	M3.1	

38 复位冲压3到位



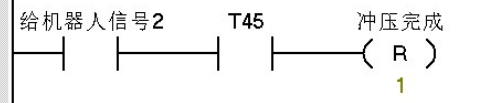
符号	地址	注释
冲压完成	M3.1	
冲压完成检测	I2.6	
给机器人信号2	M3.2	

39 发送取料信号2给机器人



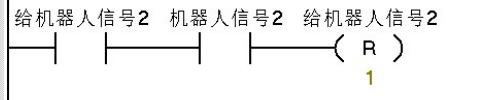
符号	地址	注释
给机器人信号2	M3.2	

40 T45



符号	地址	注释
冲压完成	M3.1	
给机器人信号2	M3.2	

41 复位冲压完成信号



符号	地址	注释
给机器人信号2	M3.2	
机器人信号2	I0.6	

子程序 shuchu 编程说明：

传感器检测到物块后，原料检测开关闭合，接通延时继电器（T55）1.5 秒后接通启动送料气缸，送料气缸将物块推送至传送带上，送料气缸前限位开关接通，步进电机被启动，接通延时继电器（T40）1.5 秒后将送料气缸复位，皮带光电传感器检测到物块时，复位步进电机，接通物料到位延时并进行下一步。

（2）物料到位延时接通 1.5 秒后，步进电机停止同时给机器人发送 1 信号，机械臂会将物块从传送带末端搬运到冲压前区域，准备开始冲压。

（3）物块到达冲压前位置，光电传感器检测到物块并接通冲压前检测开关，开始第一次冲压，第一次冲压到位后，接通延时继电器（T42）1.5 秒后复位冲压 1 气缸，开始第二次冲压。

（4）物块到达第二次冲压位置，物块被检测到后，开始第二次冲压，第二次冲压结束后，将要开始第三次冲压。

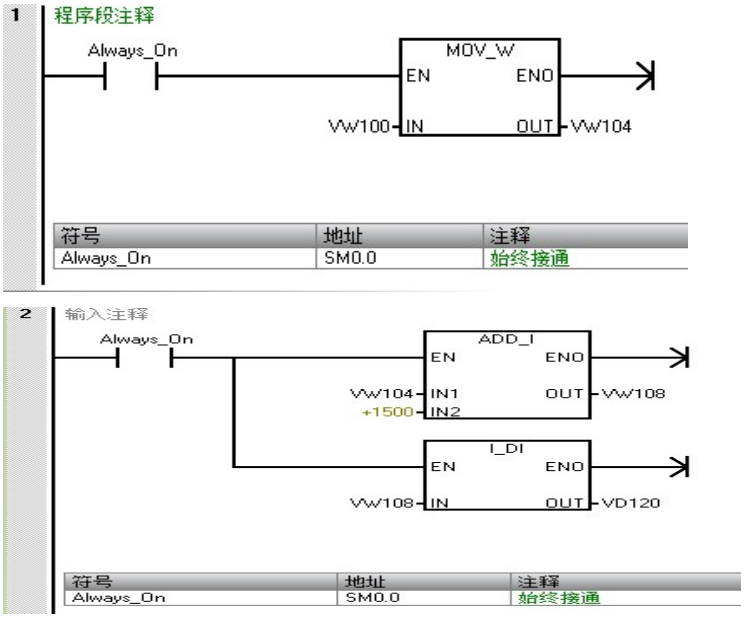
（5）第三次冲压完成后，物块被推到冲压后区域，光电传感器检测到物块，PLC 给机器人发送 2 信号。

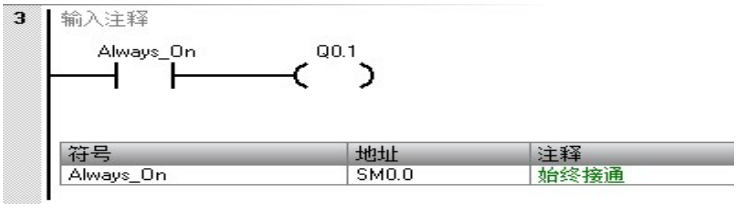
（6）机器人接收到 2 信号后，机械臂将物块从冲压后区域搬运至成品检测区并进行检测。

（7）整条生产流水线的启动和停止可通过触摸屏来进行控制。

（8）系统复位程序。

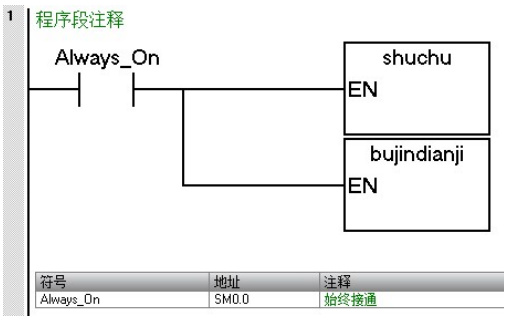
2）子程序 bujindianji

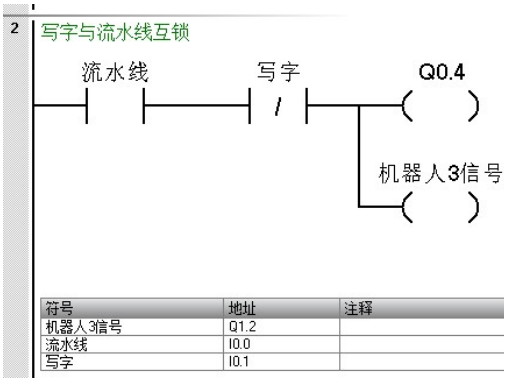


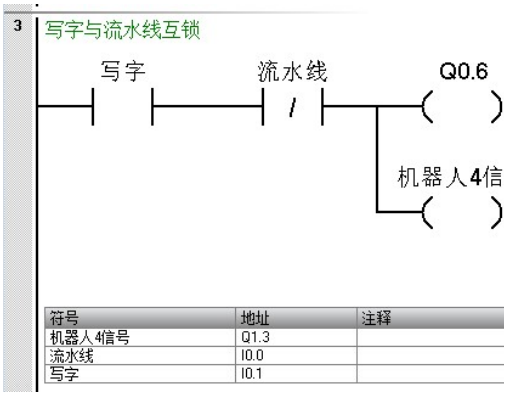


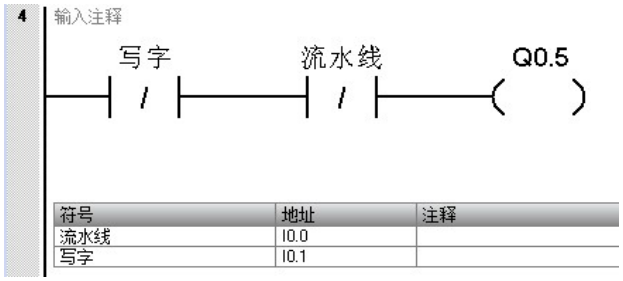
子程序 bujindianji 程序主要功能是利用触摸屏变量 VW100 输入步进电机的速度数值，速度数值加上 1500 后，输出的整数值转化成双精度整数值存入到 VD120 中，VD120 数据为控制步进电机的运动速度值，Q0.1 为步进电机方向， Q0.1=1 为正方向，Q0.1=0 为反方向。Q0.0 为步进电机脉冲输出端。

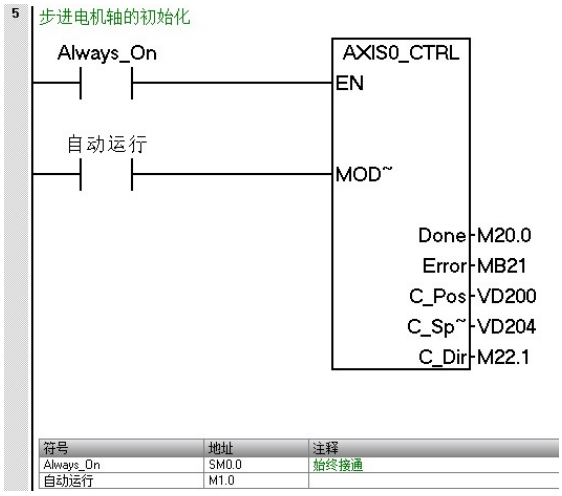
3）主程序 MAIN

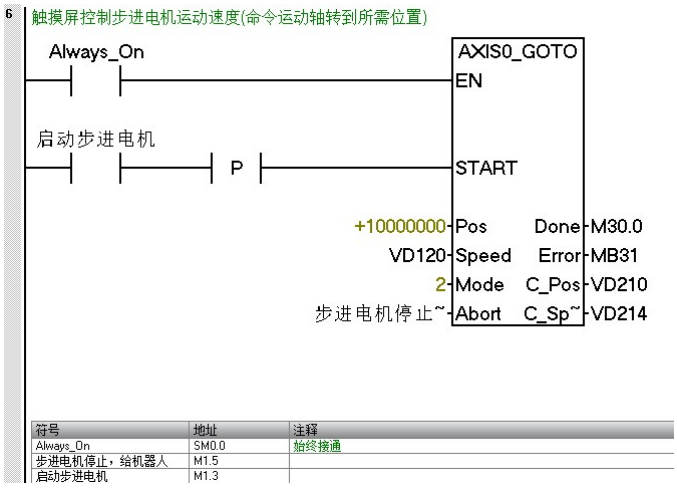


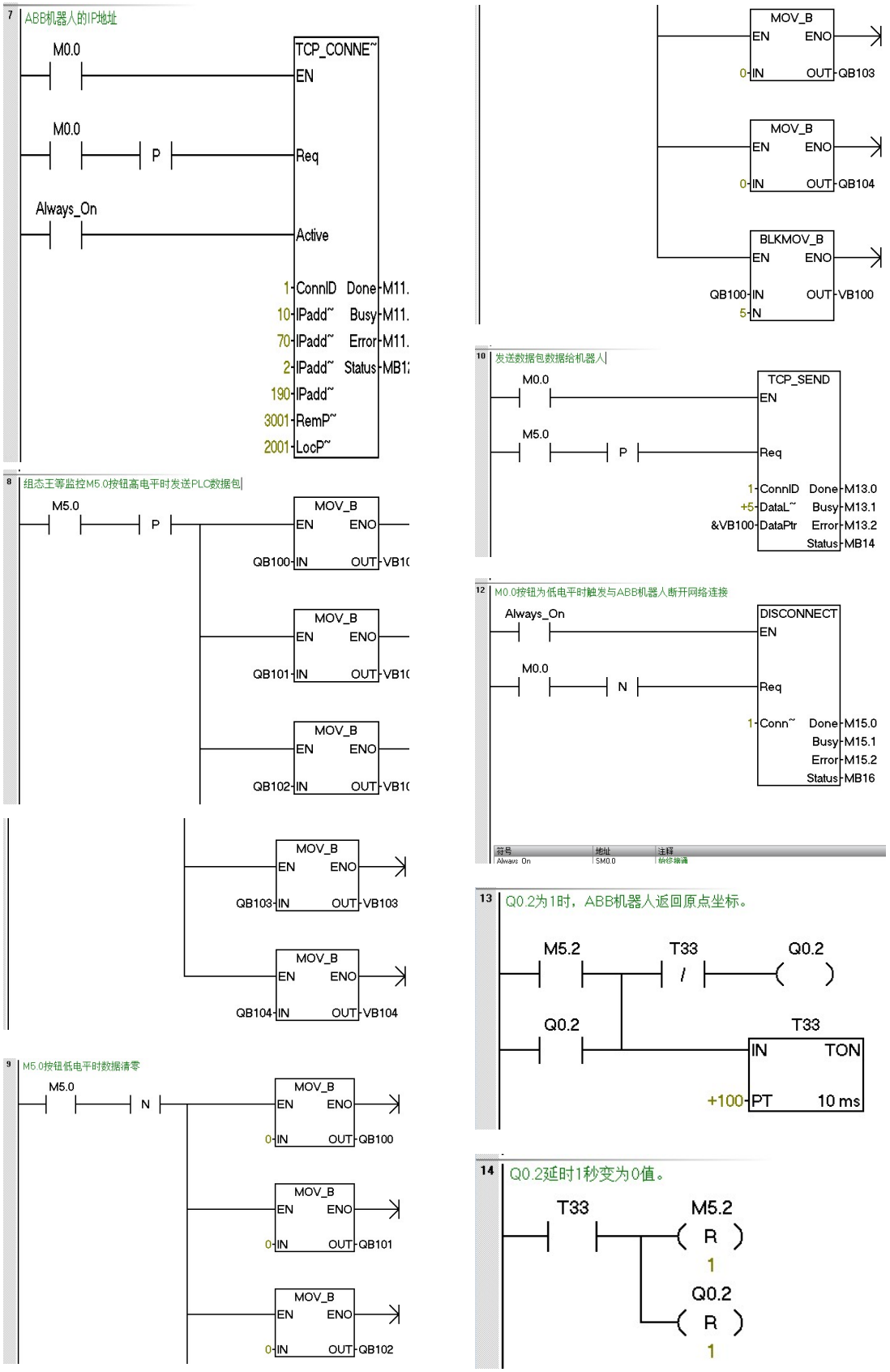












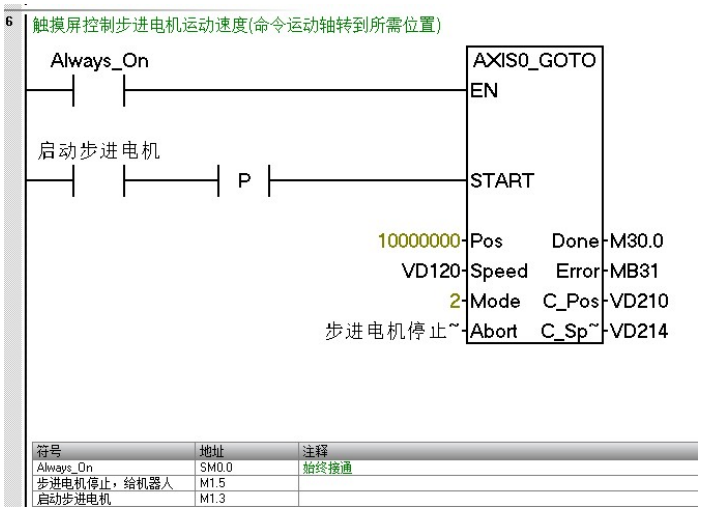
A）MOD，EN 参数必须开启，才能启用其它运动控制子例程向运动轴发送命

令。如果 MOD，EN 参数关闭，则运动轴将中止进行中的任何指令并执行减速停止。

B）当运动轴完成任何一个子例程时，Done 参数会开启。

C）Error 参数包含该子例程的结果:C_Pos 参数表示运动轴的当前位置。根据测量单位，该值是脉冲数 (DINT) 或工程单位数 (REAL)；C_Speed 参数提供运动轴的当前速度。如果您针对脉冲组态运动轴的测量系统，C_Speed 是一个 DINT 数值，其中包含脉冲数/每秒。如果您针对工程单位组态测量系统，C_Speed 是一个 REAL 数值，其中包含选择的工程单位数/每秒 (REAL)。

D）C_Dir 参数表示电机的当前方向：信号状态 0 为 正向，信号状态 1 为反向。



A）开启 EN 位会启用此子例程。确保 EN 位保持开启，直至 DONE 位指示子例程执行已经完成。

B）开启 START 参数会向运动轴发出 GOTO 命令。 对于在 START 参数开启且运动轴当前不繁忙时执行的每次扫描，该子例程向运动轴发送一个 GOTO 命令。 为了确保仅发送了一个 GOTO 命令，请使用边沿检测元素用脉冲方式开启 START 参数。

C）Pos 参数包含一个数值，指示要移动的位置（绝对移动）或要移动的距离（相对移动）。 根据所选的测量单位，该值是脉冲数 (DINT) 或工程单位数 (REAL)。

D）Speed 参数确定该移动的最高速度。根据所选的测量单位，该值是脉冲数/每秒 (DINT) 或工程单位数/每秒 (REAL)。

Mode 参数选择移动的类型：0 为绝对位置；1 为相对位置；2 为单速连续正向旋转；3 为单速连续反向旋转。

当运动轴完成此子例程时，Done 参数会开启。

G)开启 Abort 参数会命令运动轴停止执行此命令并减速，直至电机停止。
H)Error 参数包含该子例程的结果: C_Pos 参数包含运动轴的当前位置。根据测量单位，该值是脉冲数 (DINT) 或工程单位数 (REAL);C_Speed 参数包含运动轴的当前速度。根据所选的测量单位，该值是脉冲数/每秒 (DINT) 或工程单位数/每秒 (REAL)。
主程序 MAIN 编程功能说明：
调用两个子程序：shuchu 和 bujindianji
写字、流水线或其他的按钮选择，其中流水线才能控制有效，写字和其他无效。
步进电机的初始化 AXIS0_CTRL 以及步进电机速度和方向 AXIS0_GOTO 控制设置。
I)运动控制设置：在 200SMART PLC 编程软件中向导→运动控制，对应控制步进驱动器的参数设置如下：

名称	功能设置
映射	I/O 映射表 轴 0；类型：P0；地址 Q0.0 轴 1；类型：P1；地址 Q0.2
测量系统	相对脉冲
方向控制	双相（2）输出，极性正（P0 为正向运动脉冲输出）
电机速度	最大电机速度 5000 个脉冲/s，最小电机速度 20 个脉冲/s 电机启动/停止速度 1000 个脉冲/s。
JOG	电机的点动速度 1000 个脉冲/s，增量 1 个脉冲。
储存器分配	VB218-VB310
组件	AXIS0-DATA 组态置于 VB218-VB310 AXIS0-SYS 为此组态创建符号表 AXIS0-CTRL 子程序初始化位置模块 AXIS0-CFG 子程序重新加载组态 AXIS0-DIS 子程序激活 DIS 输出 AXIS0-GOTO 子程序转到移动 AXIS0-LDOFF 子程序加载参考点偏移量 AXIS0-LDPOS 子程序加载位置 AXIS0-MAN 子程序手动模式 AXIS0-RSEEK 子程序查找参考点位置 AXIS0-RUN 子程序运行曲线

第十二章 组态王与 PLC 控制机器人实验

一、实验目的

1. 学习组态王与 PLC 之间的通信。
2. 学习 PLC 通过以太网控制 ABB 机器人。

二、要求

利用组态王软件设计界面，发送数据包控制机器人动作。

三、 实验设备：型号 MFTW-IR 和 CHL-JC-02-A 的工作站

四、 实验步骤

a、实验台 PLC 设置及程序情况（已编好程序）



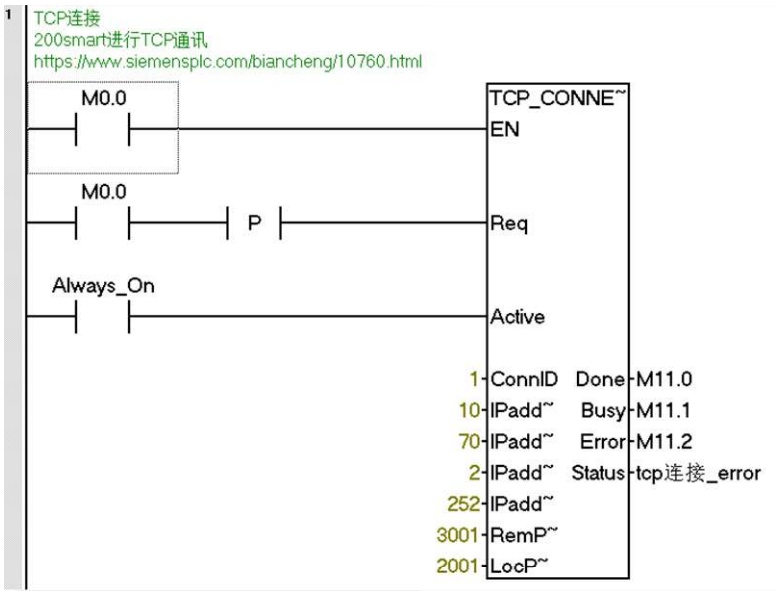
图 12.1 PLC 模块通信设置

由于每台设备对应的 PLC 地址不一样，根据现场进行修改。

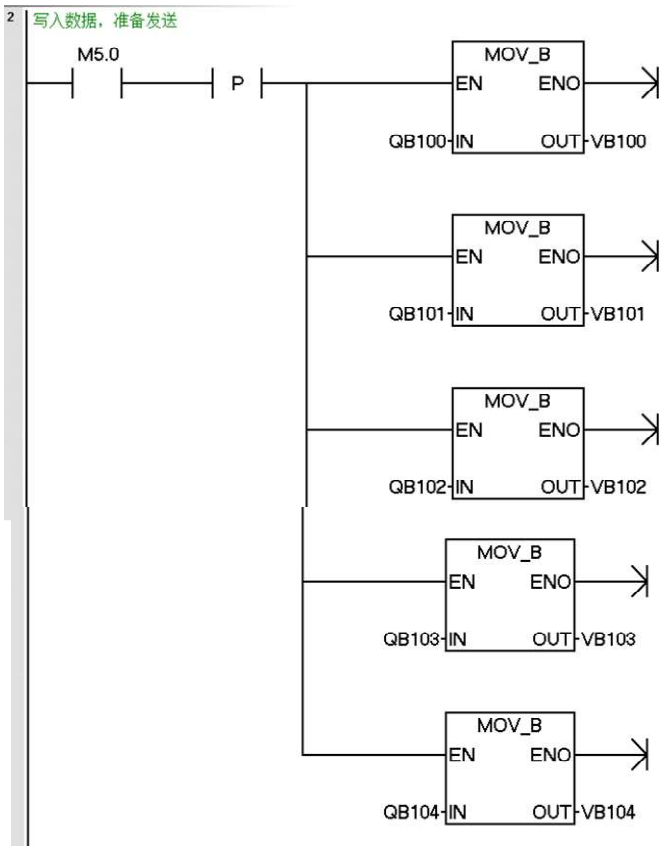
符号表

	符号	地址
1	CPU_输入0	I0.0
2	CPU_输入1	I0.1
3	CPU_输入2	I0.2
4	CPU_输入3	I0.3
5	CPU_输入4	I0.4
6	CPU_输入5	I0.5
7	CPU_输入6	I0.6
8	CPU_输入7	I0.7
9	CPU_输入8	I1.0
10	CPU_输入9	I1.1
11	CPU_输入10	I1.2
12	CPU_输入11	I1.3
13	CPU_输出0	Q0.0
14	CPU_输出1	Q0.1
15	CPU_输出2	Q0.2
16	CPU_输出3	Q0.3
17	CPU_输出4	Q0.4
18	CPU_输出5	Q0.5
19	CPU_输出6	Q0.6
20	CPU_输出7	Q0.7

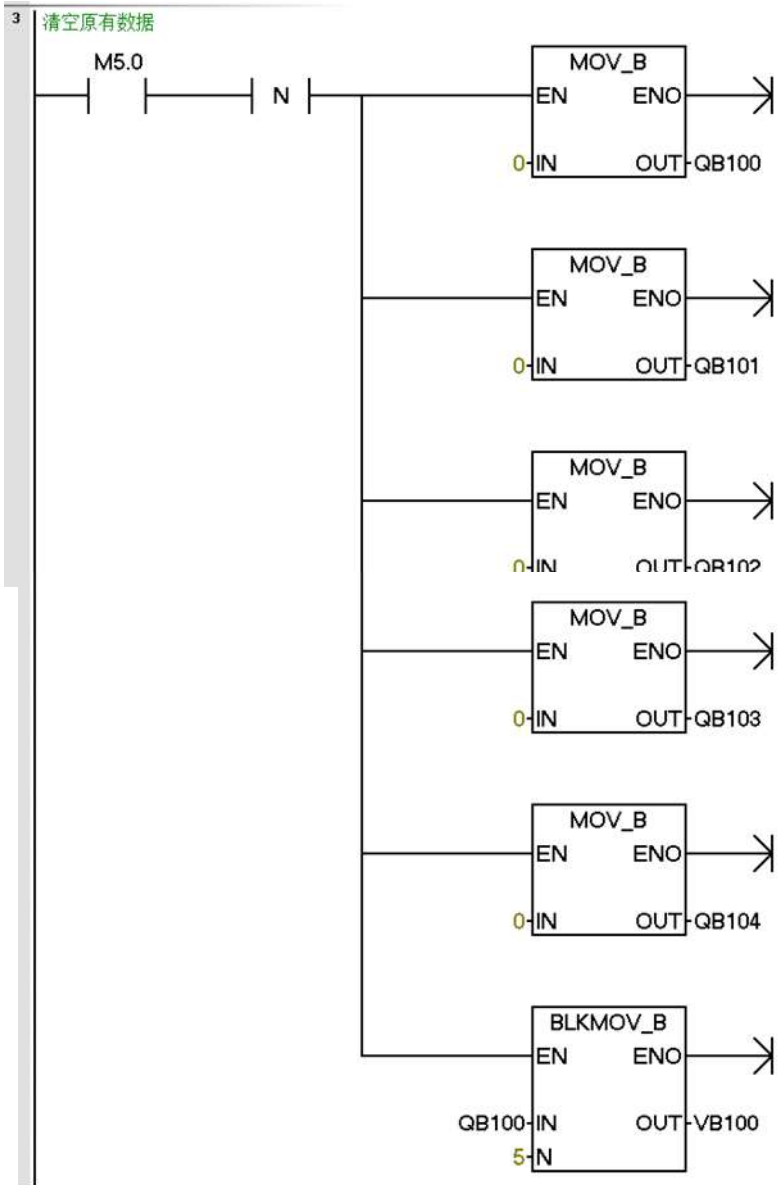
图 12.2 I/O 端口部分符号表

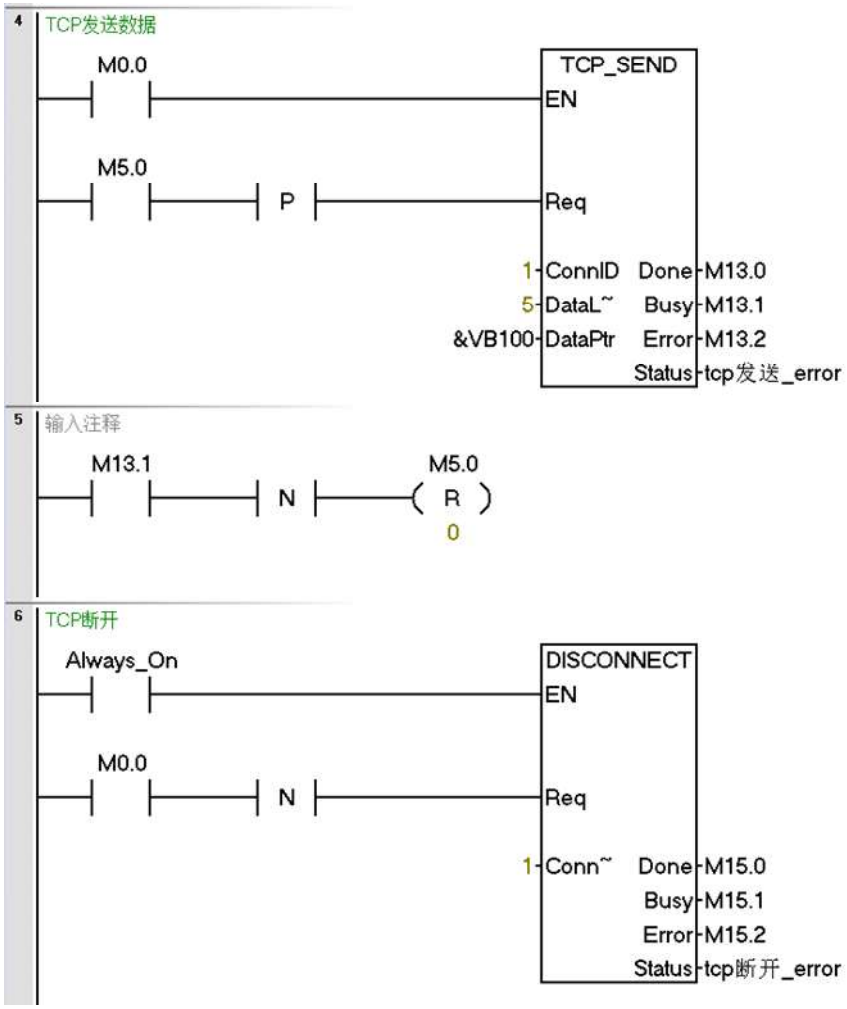


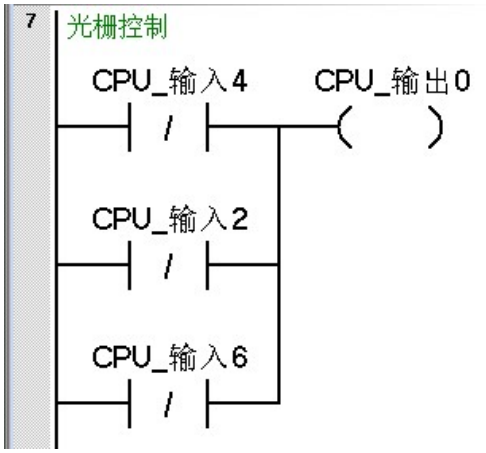
机器人的 IP 地址

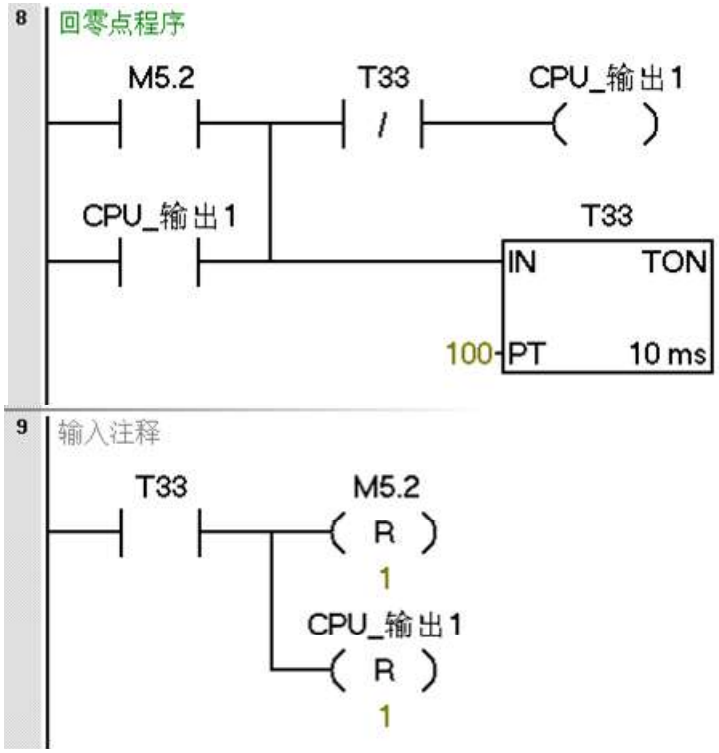


PLC 发送通过组态王或其他界面发送数据给 ABB 机器人执行动作


3
清空原有数据
M5.0
N
MOV_B
EN
ENO
0
IN
OUT
QB100
MOV_B
EN
ENO
0
IN
OUT
QB101
MOV_B
EN
ENO
MOV_B
EN
ENO
0
IN
OUT
QB103
MOV_B
EN
ENO
0
IN
OUT
QB104
BLKMOV_B
EN
ENO
QB100
IN
OUT
VB100
5
N


4
TCP发送数据
M0.0
M5.0
P
TCP_SEND
EN
Req
1 ConnID
5 DataL~
&VB100 DataPtr
Done M13.0
Busy M13.1
Error M13.2
Status tcp发送_error
5
输入注释
M13.1
N
M5.0
R
0
6
TCP断开
Always_On
M0.0
N
DISCONNECT
EN
Req
1 Conn~
Done M15.0
Busy M15.1
Error M15.2
Status tcp断开_error


7
光栅控制
CPU_输入4
CPU_输出0
CPU_输入2
CPU_输入6



B、组态王界面开发

1、定义设备

组态王 KingView7.5 界面中，根据每台设备的具体 IP 定义设备，如下图：

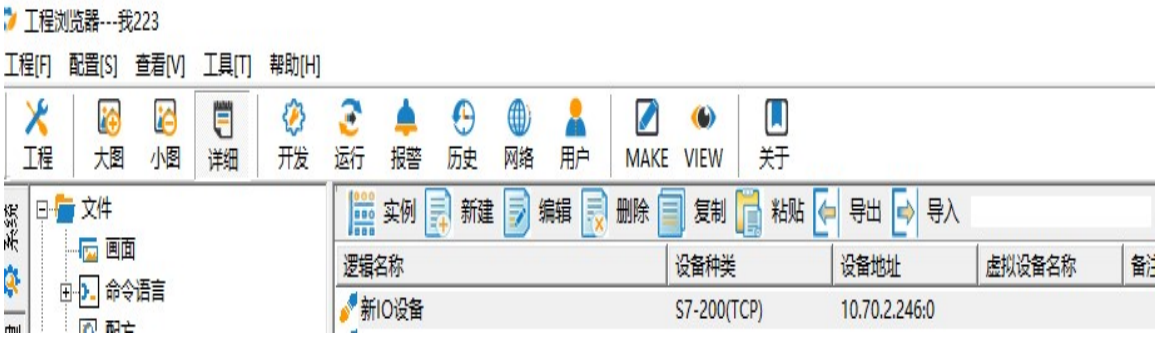


图 12.3 组态王 PLC 设备地址

2、定义数据词典

数据词典如下图：

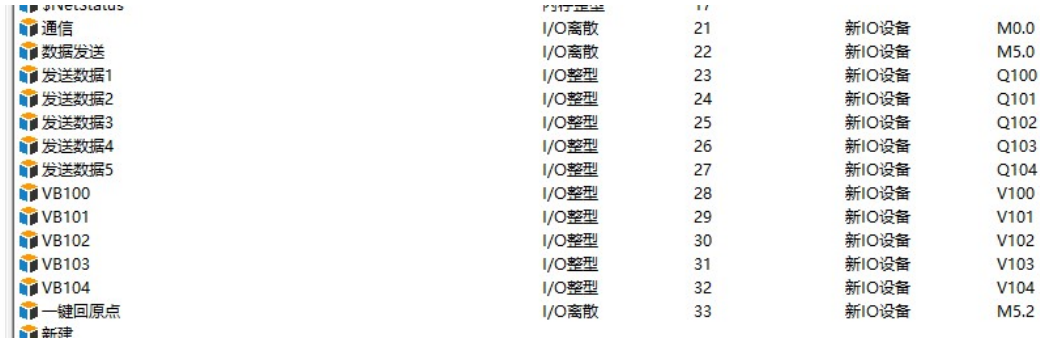


图 12.3 组态王数据定义

C、界面开发和动画连接

主画面如下图：

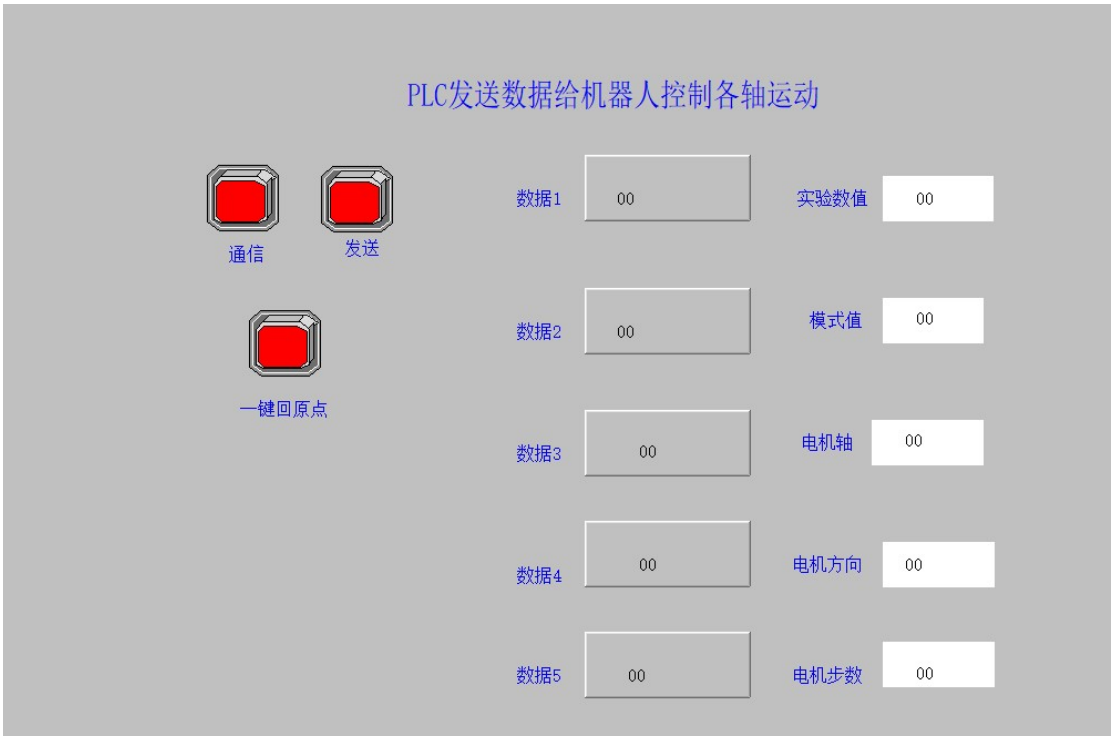


图 12.3 组态王画面定义

动画连接变量表

名称	动画连接变量	名称	动画连接变量
数据 1	Q100	实验数值	V100
数据 2	Q101	模式值	V101
数据 3	Q102	电机轴	V102
数据 4	Q103	电机方向	V103
数据 5	Q104	电机步数	V104
通信	M0.0	发送	M5.0
一键回原点	M5.2		

数据 1 的数值为 1，2 可选（实验数据）

数据 2 为 1 （模式值）

数据 3 为 1，2，3，4，5，6 可选（电机 6 个轴中选择一个轴）

数据 4 为 1，2（电机方向 1 为正转，2 为反转）

数据 5 为 0-20 值（电机行走的步数）

当在组态王运行时，打开通信，从数据 1 到数据 5 分别输入 5 个数据，点击发送按钮，可控制机械臂中 6 个轴中的任意一个轴做正向或反向运动，点击一键回原点时电机回到原点。

PLC 控制机器人的程序 Robotstudio 参考程序：

```
VAR socketdev server_socket2;
VAR socketdev client_socket2;
VAR string client_ip:="10.70.2.250";
VAR byte receive_byte{5}:=[0,0,0,0,0];
PERS byte global_byte{5}:=[0,0,0,0,0];
VAR string receive_string{5};
VAR string str_hextodec{5};
PERS num lab_number:=2;
PERS num mode_value:=0;
PERS num jog_value:=0;
PERS num direction_value:=0;
PERS num step_value:=0;
VAR bool change_ok:=false;
VAR socketstatus state{3};
VAR robtarget current_position_robT;
VAR jointtarget current_position_jointT;
VAR intnum int_home;
VAR num num_jog{6}:=[0,0,0,0,0,0];
PERS num num2_deg:=0;
VAR robtarget P_temp;
CONST jointtarget
Home:=[[-0.00129488,-0.000801945,0.00505125,-0.000859787,90.0011,-0.00026407
7],[9E+09,9E+09,9E+09,9E+09,9E+09,9E+09]];

PROC main2()
    SocketClose server_socket2;
    SocketClose client_socket2;
      client_ip:= "10.70.2.250";
    SocketCreate server_socket2;
       SocketBind server_socket2,"10.70.2.252",3002; //ABB机器人IP地址。
    SocketListen server_socket2;
SocketAccept server_socket2,client_socket2\ClientAddress:=client_ip\Time:=WAIT_MAX;
    WHILE TRUE DO
  SocketReceive client_socket2\Data:=receive_byte\Time:=WAIT_MAX;
        FOR i FROM 1 TO 5 DO
            global_byte{i}:=receive_byte{i};
        ENDFOR
        FOR i FROM 1 TO 5 DO
        receive_string{i}:=ByteToStr(global_byte{i});//16进制字转化成字符
        change_ok:=StrToVal(receive_string{1},lab_number); //字符转化成值
        change_ok:=StrToVal(receive_string{2},mode_value);
         change_ok:=StrToVal(receive_string{3},jog_value);
         change_ok:=StrToVal(receive_string{4},direction_value);
```

```
            change_ok:=StrToVal(receive_string{5},step_value);
        ENDFOR
        FOR i FROM 1 TO 5 DO
            TPWrite "receive_string="+receive_string{i};
        ENDFOR
    TPWrite "receive_string="+receive_string{5};   //在示教器上显示相应字符串
    TPWrite "lab_number="\Num:=+lab_number;
    TPWrite "mode_value="\Num:=+mode_value;
    TPWrite "jog_value="\Num:=+jog_value;
    TPWrite "direction_value="\Num:=+direction_value;
    TPWrite "step_value="\Num:=+step_value;
        IF lab_number=1 OR lab_number=2 THEN     //实验号选择
            TEST mode_value      //模式测试
            CASE 1:
    current_position_robT:=CRobT(\Tool:=tool0\wobj:=wobj0); //机器人当前位置
current_position_jointT:=CalcJointT(current_position_robT,tool0\WObj:=wobj0);
                    FOR i FROM 1 TO 6 DO
                        IF (i=jog_value) THEN
                            TEST direction_value/对控制轴方向判断
                            CASE 1:
                                num_jog{i}:=step_value;      //方向向上
                            CASE 2:
                                num_jog{i}:=-step_value;     //方向向下
                            ENDTEST
current_position_jointT:=Absjoffs(current_position_jointT,num_jog{1},num_jog{2},
num_jog{3},num_jog{4},num_jog{5},num_jog{6});

            MoveAbsJ current_position_jointT\NoEOffs,v300,fine,tool0;
                    lab_number:= 0;
                    mode_value:=0; //复位为零
                    jog_value:=0;   //复位为零
                    direction_value:=0; //复位为零
                    step_value:=0;    //复位为零
                    FOR i FROM 1 TO 6 DO
                        num_jog{i}:=0; //数组复位为零
                    ENDFOR
                    FOR i FROM 1 TO 5 DO
                        global_byte{i}:=0; //数组复位为零
                    ENDFOR
                Endif
            ENDFOR
            ENDTEST
        Endif
    ENDWHILE
```

```
    ENDPROC
FUNC jointtarget Absjoffs(jointtarget Point,num j10ffset,num j20ffset,num
J30ffset,num J40ffset,num J50ffset,num J60ffset)   //自定义函数
        VAR jointtarget joint_temp;
        joint_temp:=Point;
        joint_temp.robax.rax_1:=joint_temp.robax.rax_1+J10ffset;
        joint_temp.robax.rax_2:=joint_temp.robax.rax_2+J20ffset;
        joint_temp.robax.rax_3:=joint_temp.robax.rax_3+J30ffset;
        joint_temp.robax.rax_4:=joint_temp.robax.rax_4+J40ffset;
        joint_temp.robax.rax_5:=joint_temp.robax.rax_5+J50ffset;
        joint_temp.robax.rax_6:=joint_temp.robax.rax_6+J60ffset;
        RETURN joint_temp;
    ENDFUNC
ENDMODULE
```

如果运行出现套接字错误，改 server_socket2 变为 server_socket4 或其他，client_socket2 变为 client_socket4 或其他，端口号 3002 改为 3010 或其他，其他的程序对应的变量依次修改。

第十三章 工业相机物体识别码垛生产线实验

实验目的

1. 掌握 ABB 工业相机的基本使用方法。
2. 掌握 ABB 工业机器人与 PLCI/O 端口信号交互功能。
3. 掌握码垛生产线程序的编写方法。

实验要求

在实验平台上随意摆放 6 块尺寸大小相同的物体,利用工业相机完成 6 块物体的分拣、搬运、送料、运送、冲压、检测、码垛成品的任务。

实验设备：型号 CHL-JC-02-A 的工作站

预习要点

1. 预习 ABB 工业机器人编程指令中通信指令的用法。
2. 预习 ABB 工业机器人编程指令中 I/O 操作指令的用法。
3. 了解工业相机图像处理方法。

A、控制系统整体架构

设计的工业相机物体识别码垛生产线控制系统主要由以下 9 个部分组成：ABB Robot 控制器、机械臂、PLC、示教器、工作台、触摸屏、电脑、工业交换机和工业相机，如图 13.1 所示。其中 ABB Robot 控制器、PLC、电脑和工业相机连接到工业以太网，该 4 台设备之间以太网通信方式进行数据交换。ABB Robot 控制器上的扩展模块 D652 有 16 路开关量输入和 16 路开关量输出,可分别与 PLC 开关量输出、开关量输入进行数据交换。实现机器人与 PLC 控制的生产线之间进行数据交换，满足码垛生产线工艺要求。触摸屏上主要有简单的启动、停止复位按钮、输送带速度输入窗口等。

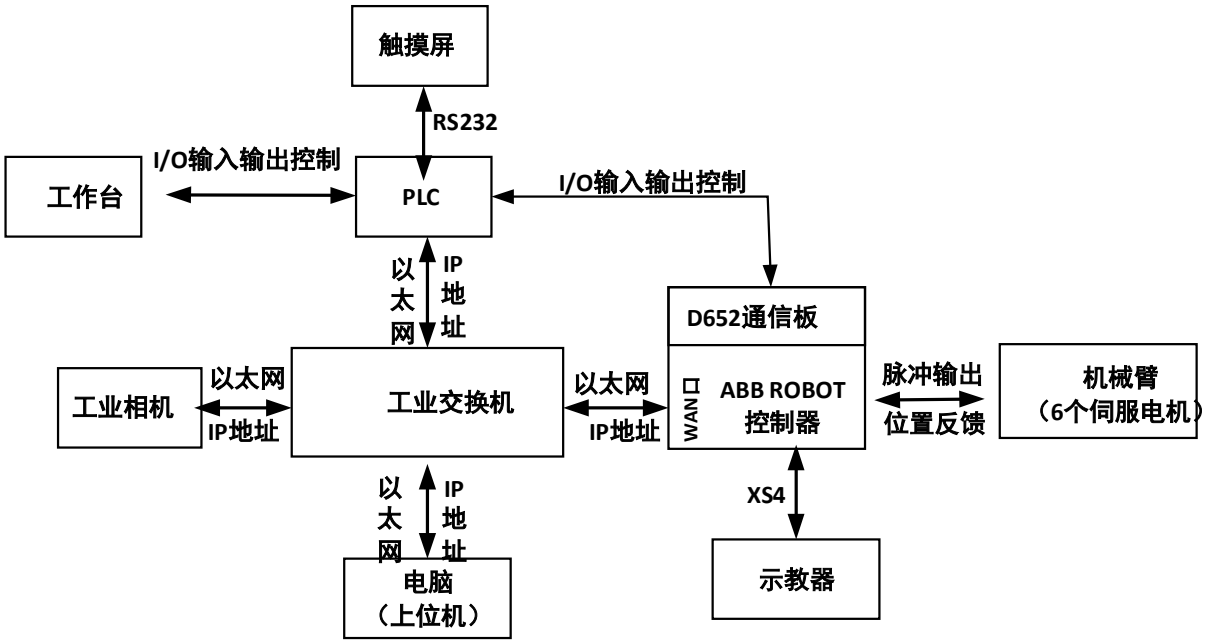


图 13.1 整体架构图

第十四章 视觉工程项目软件架构及源码说明

一、软件基本框架

视觉工程项目软件采用 C#和 C++联合开发，前端 UI 界面基于 C#开发，UI 界面功能包括软件前端操作控件、相机图像采集部分、网络通讯等，后端图像处理算法部分采用 C++动态库形式开发。此种架构开发是机器视觉工程项目中常用、高效的方法，前端 UI 和后端图像处理算法之间通过接口函数进行传参。

为什么采用这种架构进行开发而不使用一种语言进行开发呢？因为在工程项目中 C#在开发图形界面时比较方便和高效，只需要花费比较少的时间就可以完成一个简单的工程项目的 UI 设计，而且很多常用的库都有基于.NET 平台，开发起来大大提高了工程项目的调试效率。图像处理算法基于 C++开发，是因为最常用的开源图像处理算法库 OpenCV 是基于 C++开发，调用起来非常方便，再者 C++适合做底层算法开发，运行效率高，这一点在实际工程项目中非常的重要。

当然这只是其中的一种开发架构，对于喜欢研究的同学可以尝试不同的方法，比如基于 Python+C++开发，基于 Java+C++,全部基于 C++或者全部基于 Python 开发都是可以的。只要能满足实际工程应用项目，采用任何开发方法都是可以的，一般都是基于自己最擅长的方式进行开发。所以在以下介绍时基于 C#和 C++联合开发。

注意确定自己的开发平台及版本，以下所有开发是基于 Microsoft Visual Studio 2015 进行开发。

二、软件工程文件说明

在开发此项目时，将所有的工程文件都分类放在一个项目中，目的是方便移植和调试，因为我们在自己电脑开发完成需要移植到别的电脑上去，这个时候就有可能出现软件运行环境不适配等各种问题，导致软件打开失败，奔溃等问题。这个时候就需要找各种依赖文件，各种库，非常的麻烦和费时间而且调试不方便。所以我们在开发的起始就需要在我们的工程项目中把软件开发需要用的库，环境等搭建好，这样就不会出现上述的情况。下面将从本项目实际开发介绍。

在电脑中创建一个 ABB_Develop – dll 文件夹，文件夹的名字自己定义，自己方便知道含义即可。文件夹 ABB_Develop – dll 里面包含了基于视觉定位抓取项目所有的工程文件，如下图所示：



图 1.工程文件分类

ABBVisionLab 文件夹是前端 UI 工程项目文件，包括 UI 控件，相机图像采集，通讯等源代码；include 文件夹是项目中用到的 OpenCV 版本对应的函数引用库所有文件；LabPictures 是保存测试图片的文件夹；lib 文件夹是项目中用到的 OpenCV 版本的静态链接库所有文件。Public 文件夹是软件编译生成可执行文件及所有依赖项集合，也是最后使用的可执行文件 exe 文件夹，我们可以在编译的 IDE 中将生成的可执行文件的路径进行自定义改变默认的 bin 文件夹路径变为 public 文件夹路径具体方法参考网上说明或者通过项目工程属性查看具体设置。RT_CV_Recognition 是创建的 C++动态链接库所在的工程文件。

三、UI 界面及相机采集

首先需要了解 C#的基本语法和开发过程。这个部分内容需要自己学习，可通过网上教学视频和书籍等资料学习。

在对 C#语法有一定的了解后可以开始相机的采集程序的编写及调试。相机采集程序是每个嵌入式工业相机的开发接口说明及例程，因为工业相机底层是不会开放，每个厂家的工业相机都是进过封装后开放了调用接口，对照每个相机提供的开发文档和基本开发例程就可以进行工业相机的二次开发。基本上每个工业相机都会提供 C#,C++,Python 等版本的开发例程，选择合适的即可。

当前视觉工程项目所使用的工业相机是海康威视生产的。我们在其官网下载官方相机的软件包 MVS，下载链接：[海康机器人-机器视觉-下载中心 (hikrobotics.com)](https://www.hikrobotics.com/cn/machinevision/service/download?module=0) 进入该链接下载海康威视机器视觉工业相机客户端 MVS V3.3.1 最新版的即可；



图 2.海康威视工业相机下载

下载好后再 Windows 电脑安装该软件，安装好之后就可以通过该软件连接相机，进行采图，相机调试等操作。

相机的 SDK 在相机安装目录中..\MVS\Development\Samples 文件家中，如下图所示：

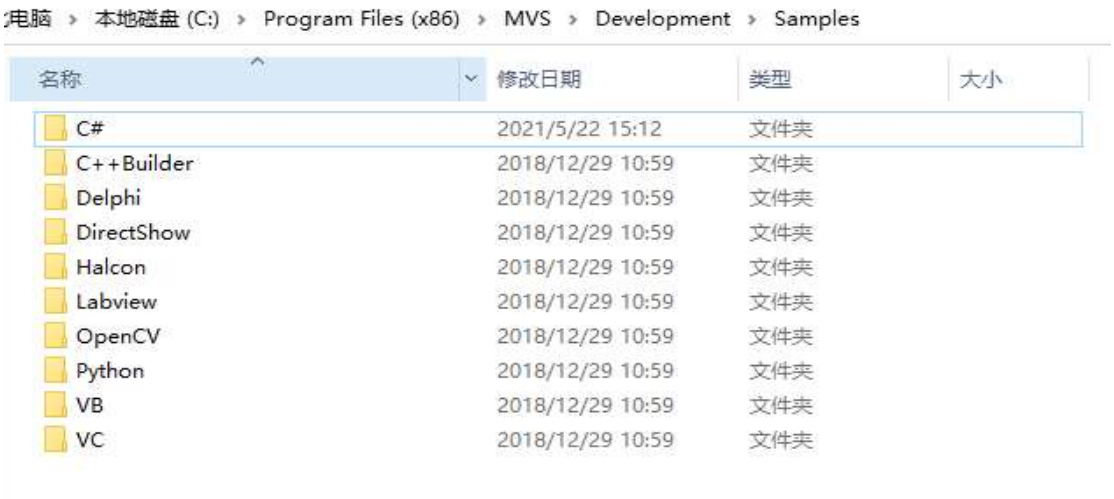


图 3.相机 SDK 采集程序文件夹路径

打开 C#文件运行相机采集程序解决方案 CSharpDotNetSamples_VS2008.sln 界面如下图所示：

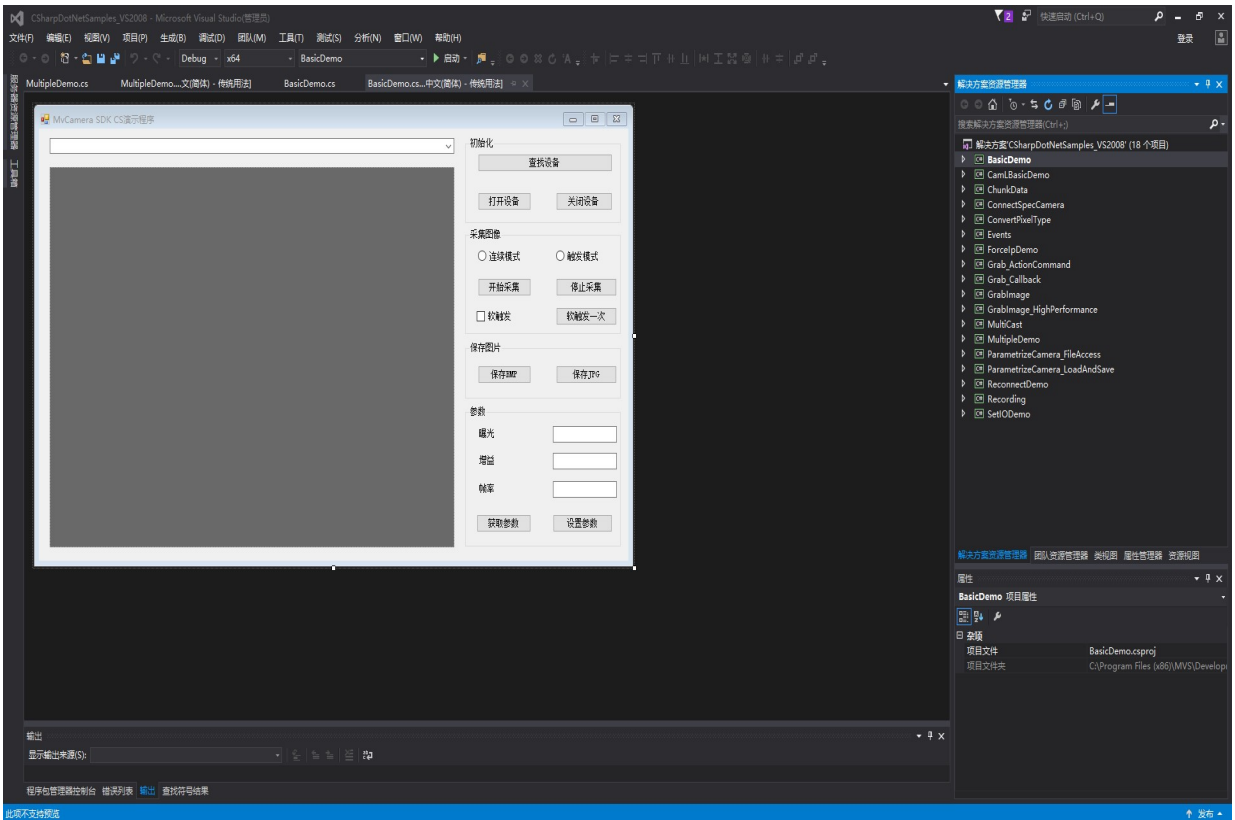


图 4.相机采集 SDK 源码

工程文件的左边是对应项目的代码显示区，右边是工程文件的项目栏，里面包含了海康威视提供的相机二次开发所使用的所有的 demo 程序。通过查看代码就可以知道相机采集程序的流程。以第一个工程 BasicDemo 项目为例此项目是单相机采集的基本开发流程例程。展开 BasicDemo 项目，双击 BasicDemo.cs 打开软件的 UI 编辑界面，如下图所示：

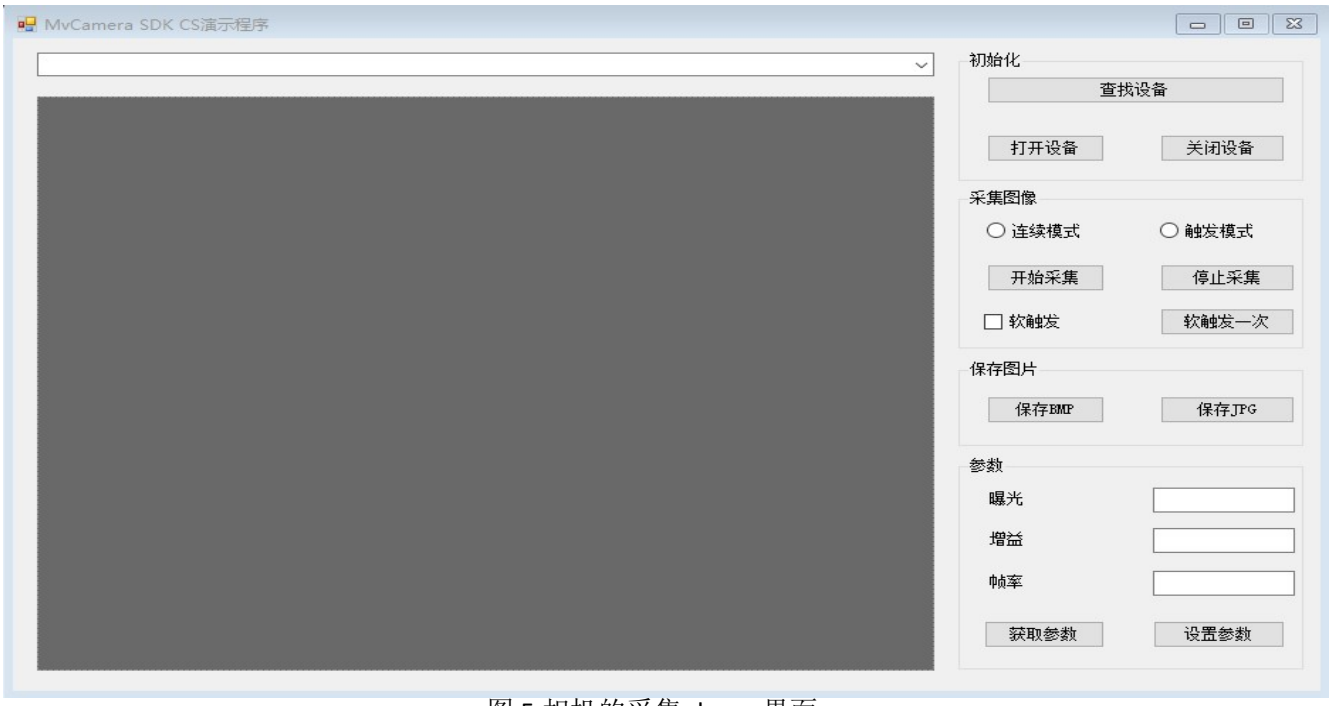


图 5.相机的采集 demo 界面

从图中可以看到相机采集界面的基本控件，每个控件对应一个功能，从控件的顺序及命名基本上可以知道控件的作用。

界面按照从上到下，从左到右的顺序查看，右边第一个控件是 ComboBox,是一个列表下拉框用于显示搜索到的相机列表名称。右边第二个是一个 PictureBox 控件，用于显示采集到的图像。左边第一个是控件集合，是相机进行初始化相关操作的控件，初始化中第一个控件是查找设备，即搜索连接的相机，第二个控件是打开设备，即搜索到相机后打开相机设备，第三个是关闭设备。第二个控件集合是图像采集相关的控件，第一个是 Radiobutton 用于选择的控件，在属性里面设置了默认为选中状态为相机连续模式。点击开始采集按钮，此时 picturebox 控件就有图像显示了。到此整个图像的采集过程完成。总结相机采集整个过程的

起始：1.相机进行初始化操作，搜索相机设备；2.搜索到相机设备后打开相机设备；3.打开相机设备后设置相机的采集模式；4.开始采集图像；基本上分为四个步骤。

1.相机进行初始化采集操作

我们查看 UI 界面的代码程序，在窗口的构造函数 public Form1()中看窗口在初始化过程中做了哪些操作；

```
public Form1()
        {
            InitializeComponent();//初始化先关操作，比如控件大小，属性等设置，变量的获取
                                  等窗口显示前的先关初始化，自动完成。
            m_pDeviceList = new MyCamera.MV_CC_DEVICE_INFO_LIST();//实例化申明的相机对象
            m_bGrabbing = false;//图像数据获取的标志位，用控制什么时候获取图像
            DeviceListAcq();//查找相机
        }
```

可以看到在程序中看到界面在初始化过程中就已经连接搜索了相机。相机查找函数是 DeviceListAcq()，通过此函数可以查找相机，查找到相机后将相机的名称显示在下拉列表

```
// ch:在窗体列表中显示设备名 | en:Display device name in the form list
for (int i = 0; i < m_pDeviceList.nDeviceNum; i++)
{
    MyCamera.MV_CC_DEVICE_INFO device = (MyCamera.MV_CC_DEVICE_INFO)Marshal.PtrToStructure(m_pDeviceList.pDeviceInfo[i], typeof(MyCamera.MV_CC_DEVICE_INFO));
    if (device.nTLayerType == MyCamera.MV_GIGE_DEVICE)
    {
        IntPtr buffer = Marshal.UnsafeAddrOfPinnedArrayElement(device.SpecialInfo.stGigEInfo, 0);
        MyCamera.MV_GIGE_DEVICE_INFO gigeInfo = (MyCamera.MV_GIGE_DEVICE_INFO)Marshal.PtrToStructure(buffer, typeof(MyCamera.MV_GIGE_DEVICE_INFO));
        if (gigeInfo.chUserDefinedName != "")
        {
            cbDeviceList.Items.Add("GigE: " + gigeInfo.chUserDefinedName + " (" + gigeInfo.chSerialNumber + ")");
        }
        else
        {
            cbDeviceList.Items.Add("GigE: " + gigeInfo.chManufacturerName + " " + gigeInfo.chModelName + " (" + gigeInfo.chSerialNumber + ")");
        }
    }
    else if (device.nTLayerType == MyCamera.MV_USB_DEVICE)
    {
        IntPtr buffer = Marshal.UnsafeAddrOfPinnedArrayElement(device.SpecialInfo.stUsb3VInfo, 0);
        MyCamera.MV_USB3_DEVICE_INFO usbInfo = (MyCamera.MV_USB3_DEVICE_INFO)Marshal.PtrToStructure(buffer, typeof(MyCamera.MV_USB3_DEVICE_INFO));
        if (usbInfo.chUserDefinedName != "")
        {
            cbDeviceList.Items.Add("USB: " + usbInfo.chUserDefinedName + " (" + usbInfo.chSerialNumber + ")");
        }
        else
        {
            cbDeviceList.Items.Add("USB: " + usbInfo.chManufacturerName + " " + usbInfo.chModelName + " (" + usbInfo.chSerialNumber + ")");
        }
    }
}
```

图 6.搜索相机并显示相机列表

同样双击“查找设备按钮”里面运行的函数也是 DeviceListAcq()，与界面初始化时搜索相机一致。

双击“打开设备”按钮查看里面的代码做了什么,请查看注释。

```
private void bnOpen_Click(object sender, EventArgs e)//打开设备按钮点击事件
```

```
        {
            if (m_pDeviceList.nDeviceNum == 0 || cbDeviceList.SelectedIndex == -1)
            {
                ShowErrorMsg("No device, please select", 0);
                return;
            }//判断有没有搜索到相机
            int nRet = -1;

            // ch:获取选择的设备信息 | en:Get selected device information
            MyCamera.MV_CC_DEVICE_INFO device =

(MyCamera.MV_CC_DEVICE_INFO)Marshal.PtrToStructure(m_pDeviceList.pDeviceInfo[cbDeviceLi
st.SelectedIndex],
typeof(MyCamera.MV_CC_DEVICE_INFO));//获取连接的相机设备
            // ch:打开设备 | en:Open device
            if (null == m_pMyCamera)
            {
                m_pMyCamera = new MyCamera();//实例化相机对象
                if (null == m_pMyCamera)
                {
                    return;
                }
            }
            nRet = m_pMyCamera.MV_CC_CreateDevice_NET(ref device);//创建搜索到信息的相机
设备，返回创建息值，等于0表示创建正常
            if (MyCamera.MV_OK != nRet)//判断有没有创建成功，没有则返回
            {
                return;
            }
            nRet = m_pMyCamera.MV_CC_OpenDevice_NET();//开启相机设备
```

```
            if (MyCamera.MV_OK != nRet) //判断有没有开启成功，没有则销毁创建的设备
            {
                m_pMyCamera.MV_CC_DestroyDevice_NET();//开启失败，销毁创建的设备信息，防止下次重新创建被占用
                ShowErrorMsg("Device open fail!", nRet);//显示错误日志，相机打开失败原因
                return;
            }
            // ch:探测网络最佳包大小(只对GigE相机有效) | en:Detection network optimal package size(It only works for the GigE camera)
            if (device.nTLayerType == MyCamera.MV_GIGE_DEVICE)
            {
                int nPacketSize = m_pMyCamera.MV_CC_GetOptimalPacketSize_NET();
                if (nPacketSize > 0)
                {
                    nRet = m_pMyCamera.MV_CC_SetIntValue_NET("GevSCPSPacketSize", (uint)nPacketSize);
                    if (nRet != MyCamera.MV_OK)
                    {
                        Console.WriteLine("Warning: Set Packet Size failed {0:x8}", nRet);
                    }
                }
                else
                {
                    Console.WriteLine("Warning: Get Packet Size failed {0:x8}", nPacketSize);
                }
            }
            // ch:设置采集连续模式 | en:Set Continues Aquisition Mode
            m_pMyCamera.MV_CC_SetEnumValue_NET("AcquisitionMode", 2);// ch:工作在连续模式 | en:Acquisition On Continuous Mode
```

```
            m_pMyCamera.MV_CC_SetEnumValue_NET("TriggerMode", 0);    // ch:连续模式 |
en:Continuous

            bnGetParam_Click(null, null);// ch:获取参数 | en:Get parameters
            // ch:控件操作 | en:Control operation
            SetCtrlWhenOpen();
        }
```

双击“开始采集”按钮查看里面的代码

```
  private void bnStartGrab_Click(object sender, EventArgs e)//开始采集控件点击事件
        {
            int nRet;
            // ch:开始采集 | en:Start Grabbing
            nRet = m_pMyCamera.MV_CC_StartGrabbing_NET();//开始采集
            if (MyCamera.MV_OK != nRet)
            {
                ShowErrorMsg("Trigger Fail!", nRet);
                return;
            }
            // ch:控件操作 | en:Control Operation
            SetCtrlWhenStartGrab();
            // ch:标志位置位true | en:Set position bit true
            m_bGrabbing = true;//控制图像数据的获取与否
            // ch:显示 | en:Display
            nRet = m_pMyCamera.MV_CC_Display_NET(pictureBox1.Handle);//传入窗口的句柄进
行显示
            if (MyCamera.MV_OK != nRet)
            {
                ShowErrorMsg("Display Fail! ", nRet);
            }
            }
```

以上操作就是相机采集的一般过程，里面包含了相机采集所用到的基本函数及相机采集过程中处理方法。对照上面的注释说明，其他控件里面的代码同理，根据相机命名的函数名称就可以知道当前函数的作用是什么。

四、自己创建相机采集工程项目

在理解了上述代码后需要自己动手根据海康相机的 SDK 程序，写一个相机采集，按照上述的方式及操作完成相机采集的基本程序，并显示图像成功，可以在程序界面对相机的基本参数（曝光、增益、白平衡、触发模式等）进行设置。自己在开发中，需要引用海康相机的动态库 MvCameraControl.Net.dll，这个动态库封装了海康相机的所有接口功能函数，所以开发时一定要注意。

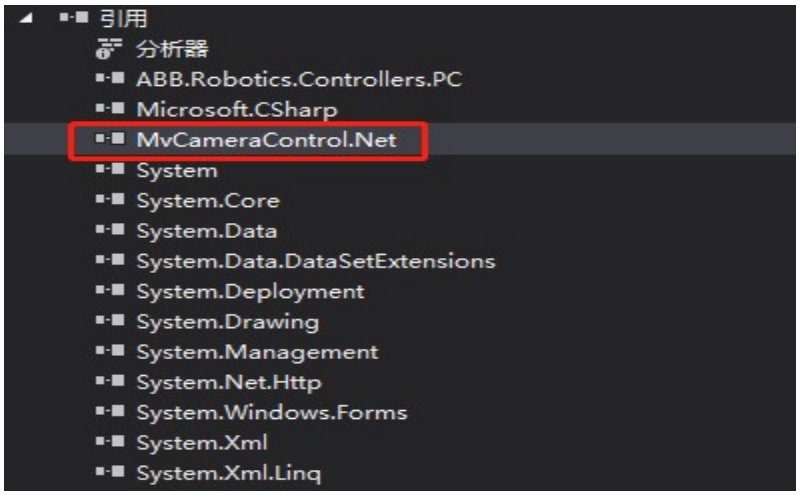


图 7.引用相机的动态链接库

在开发过程中可以参考 ABBVisionLab.sln 的代码进行开发，实现的效果如下图界面，实现的方法都是一样的，在 ABBVisionLab.sln 解决方案中，将相机的相关的采集、设置参数等相关的参数使用函数进行了整理，汇总在 HIKCameraHelper.cs 类文件中，大家在开发项目时可以直接引用改类，调用里面的函数直接开发使用，大大缩小时间和工作量。



图 8.相机参数界面

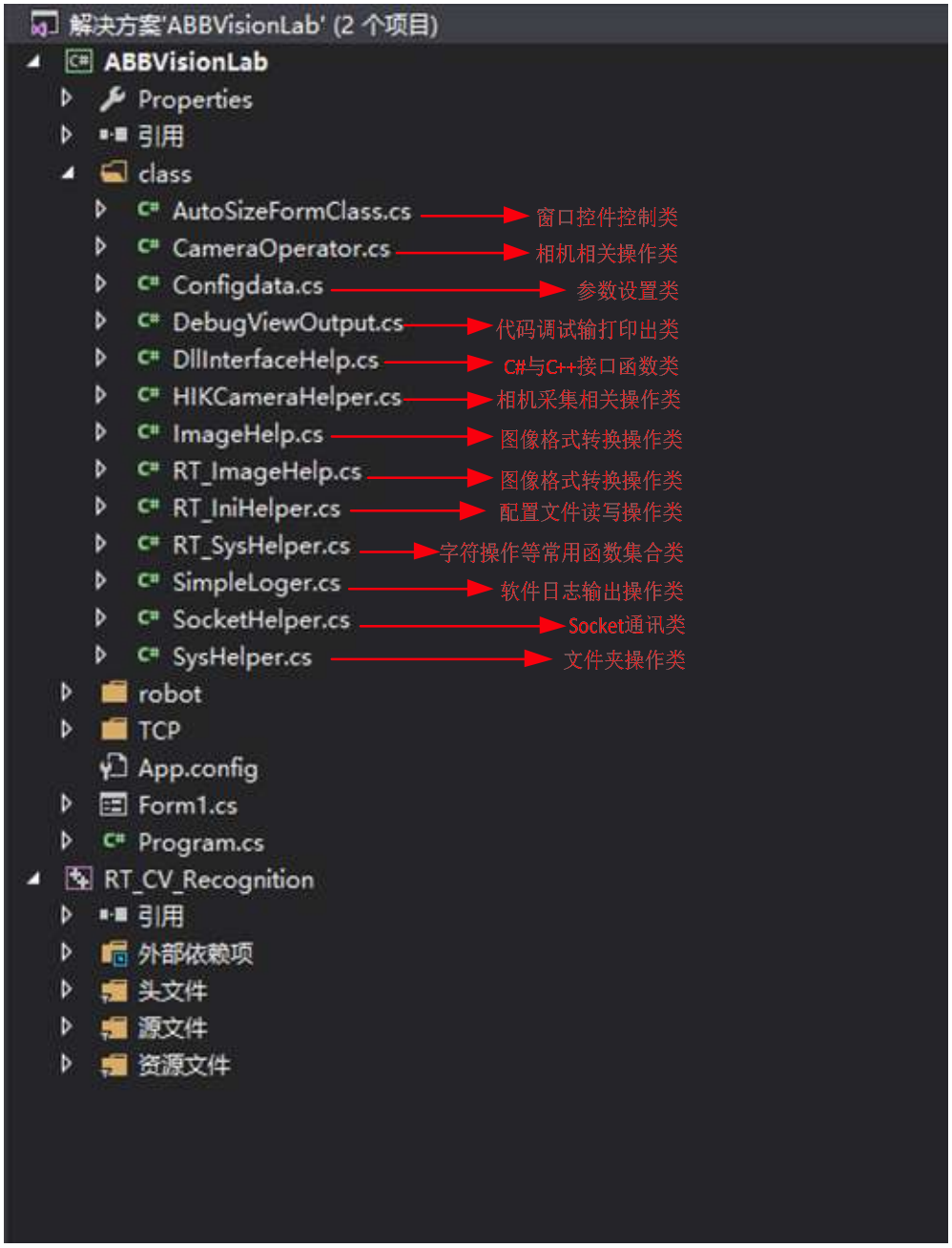

解决方案'ABBVisionLab' (2 个项目)
ABBVisionLab
Properties
引用
class
AutoSizeFormClass.cs
窗口控件控制类
CameraOperator.cs
相机相关操作类
Configdata.cs
参数设置类
DebugViewOutput.cs
代码调试输打印出类
DllInterfaceHelp.cs
C#与C++接口函数类
HIKCameraHelper.cs
相机采集相关操作类
ImageHelp.cs
图像格式转换操作类
RT_ImageHelp.cs
图像格式转换操作类
RT_IniHelper.cs
配置文件读写操作类
RT_SysHelper.cs
字符操作等常用函数集合类
SimpleLoger.cs
软件日志输出操作类
SocketHelper.cs
Socket通讯类
SysHelper.cs
文件夹操作类
robot
TCP
App.config
Form1.cs
Program.cs
RT_CV_Recognition
引用
外部依赖项
头文件
源文件
资源文件

图 9 项目常用类说明

同学们在开发过程中可以查看项目中常用“类”，看类中的函数是做什么用的，函数都做了说明，或者根据函数名即可知道函数的功能。这样将实现某一块功能的各种函数整合到类中是一种常用的类的封装，方便代码的调试、查看、整理。如果所有的函数写在同一个文件中，各种功能函数掺杂在一起没有分类，代码可读性大大降低，后期也不好维护，出了问题不知道在哪里，所以“类”的封装是一种常用的开发手段，也是后期代码量增加时，更容易维护和调试的前提，所以一定要学会这种方法，后期维护调试会大大的方便很多。

五、C++图像处理

C++图像处理部分是基于开源的计算机视觉算法库 OpenCV，本项目开发是基于 OpenCV3.1 版本进行开发，同学们可以自己选择相应的版本。在基于 OpenCV 开发前需要在工程项目中配置 OpenCV，才能调用 OpenCV 的函数，配置的方法同学们参考这篇文章：VS2017 配置 OpenCV 4.3.0（https://blog.csdn.net/weixin_44737922/article/details/105375574）项目开发中使用的 VS2015+OpenCV3.1 的配置方法同连接的一致，只需要对应相应的版本即可。

在配置好 OpenCV 后引用 OpenCV 库的头文件即可调用 OpenCV 的函数了；



图 1.引用 OpenCV 库头文件

引用完 OpenCV 库头文件即可进行图像算法的开发了，一般开发流程是，图像读入-图像预处理-图像二值化-根据二值化效果再判断处理调参得到满意的二值效果-提取轮廓-选择轮廓-求取轮廓的中心点（最小外接矩形等）。

图像处理一般的开发流程大概就以上几点，同学们在掌握后再做其他深入的研究增加算法的鲁棒性。

以下以流水线白色方块检测中心点及角度代码为例，说明图像处理的一般方法及过程；

```
  //方块中心像素点及角度检测函数
void RT_CV_Recognition::Shap_check(Mat src, float str_center[])
{
    try
    {
        Mat m_cImageProcess = src.clone();//将原图像分别克隆给多个Mat中间对象，目的是不改变原图像。
        Mat m_cImageColor = src.clone();
        Mat m_cImageRgb = src.clone();
        cv::Mat imgGray, img_gray;
        cv::cvtColor(m_cImageProcess, img_gray, cv::COLOR_RGB2GRAY);//将原图像转换为灰度图
       GaussianBlur(img_gray, img_gray, Size(5, 5), 2); //高斯滤波
```

```
        Mat element = getStructuringElement(MORPH_RECT, Size(30, 30));//将图像变为30*30像素大小的结构，是图像腐蚀大小的尺度
        erode(img_gray, img_gray, element);//图像腐蚀，目的是将粘连的方块分开
        threshold(img_gray, img_gray, 110, 180, cv::THRESH_BINARY); //阈值化
        imgGray = img_gray.clone();
        Mat distShow;
        distShow = Mat::zeros(imgGray.size(), CV_8UC1); //将每个通道像素值变为0，图像初始化，目的是将轮廓显示在一张纯黑的图像上，方便查看。
        std::vector<cv::Vec4i> hierarchy;
        std::vector<std::vector<cv::Point>> Contours;
cv::findContours(imgGray, Contours, hierarchy, CV_RETR_LIST, CV_CHAIN_APPROX_SIMPLE);//找轮廓
        int Numbers = (int)Contours.size();
        Scalar color = Scalar(0, 255, 0);
        int index = -1, maxArea = 0;
        for (int i = 0; i < Numbers; i++)//遍历轮廓，根据限制条件查找需要的轮廓
        {
            int area = contourArea(Contours[i]);//轮廓的面积
            if (area > 10000 && area < 30000)
            {
                if (area > maxArea)
                {
                    index = i;
                    maxArea = area;
                }
            }
        }
        //drawContours(m_cImageProcess, Contours, i, color, 3, 8, hierarchy);//将最大区域的轮廓画出来（后面有画，这里就不重复）
        cv::Rect boundRect;
        RotatedRect Rect;
        Rect = minAreaRect(Contours[index]);//求取轮廓的最小外接矩形
        Point2f rect_points[4];
        Rect.points(rect_points);
        str_center[0] = Rect.center.x;//将最小外接矩形的中心像素点赋值给传参变量
        str_center[1] = Rect.center.y;
        //最小外接矩阵的角度，根据角度对最小外接矩阵旋转
        float angle_rotation;
        if ((Rect.size.width/ Rect.size.height)<1)
            angle_rotation = 90+ Rect.angle;
        else
            angle_rotation = Rect.angle;
        str_center[2] = angle_rotation;//将获得旋转角度赋值给传参变量，顺时针为负，逆时针为正。
```

```
            circle(m_cImageProcess, Point(Rect.center.x, Rect.center.y), 8, CV_RGB(255, 0, 0),
CV_FILLED);//画出中心像素点
            for (int j = 0; j < 4; j++)
            {
                cv::line(m_cImageProcess, rect_points[j], rect_points[(j + 1) % 4], Scalar(0,
0, 255), 2, 8);  //绘制最小外接矩形每条边

            }
            cv::putText(m_cImageProcess, to_string(index), cvPoint(Rect.center.x + 10,
Rect.center.y + 10), CV_FONT_HERSHEY_COMPLEX, 1, cv::Scalar(0, 0, 255), 1, 5, 0);//将满
足条件轮廓的索引号写在矩形中心像素点旁边
            cv::putText(m_cImageProcess, to_string(index) + ":X=" + to_string(Rect.center.x)
+ "," + "Y=" + to_string(Rect.center.y) + "," + "Angle=" + to_string(angle_rotation),
cvPoint(20, 50), CV_FONT_HERSHEY_COMPLEX, 1, cv::Scalar(0, 0, 255), 1, 5, 0);//将矩形的
中心坐标及角度写在图像左上角上
            p_RT_CV_Recognition->zy_Camera_Display(m_cImageProcess, m_cImageProcess.rows,
m_cImageProcess.cols);//将结果图像显示
            int m_Src_count_1 = 1;
            char*pImageName = (char*)malloc(20);
            sprintf_s(pImageName, 20, "Pictures//%03d.jpg", m_Src_count_1);
            imwrite(pImageName, m_cImageProcess); //保存结果图像
            m_Src_count_1++;
        }
        catch (cv::Exception cvExc)
        {
            std::cout << cvExc.msg << std::endl;//如果程序中断输出错误信息
        }
        }
```

以上就是检测方块中心点像素坐标及角度的方法和过程。

六、C#与 C++接口函数及传参

前面我们讲了图像处理部分得到矩形方块的像素中心坐标及矩形方块的角度，那么这个得到的数据是在 C++中申明的变量存储，如何传参到 C#中使用呢？这就用到了 C++与 C#事先定义好的接口函数，C++定义好外部输出的函数接口，C#定义好外部输入的函数接口，函数的类型需要一样即函数的返回值及参数类型一致即可。这样就可以通过函数参数进行传值，C++具体操作如下代码所示：

```
//-------------------------------OPENCV函数END-------------------------------

/// ...
ZYDLL bool __stdcall   EX_zy_System_Init(void* m_hDispWindow, int icam, int mode){ ... }
/// ...
ZYDLL void __stdcall   SetCallback(Camera_PResHook callback, int iIndex){ ... }

ZYDLL bool __stdcall   EX_RT_calibrateCam(float *camCoordinates, float *handCoordinates, float _transMat[]){ ... }
// ...
ZYDLL bool __stdcall   EX_RT_initcalibrateCamParam(float _transMat[]){ ... }
// ...
ZYDLL bool __stdcall   EX_RT_transCoord(float *camCoordinates, float handCood[]){ ... }
```

以上折叠代码函数表示定义的外部接口函数，格式：宏定义 ZYDLL+返回值类型+_stdcall+函数（参数 1，参数 2，参数 3）；里面是对函数进行了封装，用到了之前图像处理的函数，形成一个完整的功能进行封装。

为了外部调用需要在头文件中对这些封装函数进行申明，申明如下：

```
extern "C" RT_EXPORTS_API void RT_CV_DLL_Shap_check(unsigned char* imgPtr, long dataLength, int h, int w, float str_center[]);
extern "C" RT_EXPORTS_API void RT_CV_DLL_MarkLocCenter(unsigned char* imgPtr, long dataLength, int h, int w, float str_Markcenter[]);
//dll外部函数声明，并在该函数中调用C#中传递过来的委托
#pragma region dll外部函数
EXTERN_C ZYDLL void __stdcall    SetCallback(Camera_PResHook callback, int iIndex);
EXTERN_C ZYDLL bool __stdcall    EX_zy_System_Init(void* m_hDispWindow, int icam, int mode);
EXTERN_C ZYDLL bool __stdcall    EX_RT_calibrateCam(float *camCoordinates, float *handCoordinates, float _transMat[]);
EXTERN_C ZYDLL bool __stdcall    EX_RT_initcalibrateCamParam(float _transMat[]);
EXTERN_C ZYDLL bool __stdcall    EX_RT_transCoord(float *camCoordinates, float handCood[]);
#pragma endregion
#endif
```

同样需要在头文件进行宏定义外部输入输出函数：

```
#ifndef RT_CV_Recognition_H
#define RT_CV_Recognition_H

#define RT_EXPORTS_EXPORT

#ifdef RT_EXPORTS_EXPORT
#define RT_EXPORTS_API extern "C" _declspec(dllexport)
#else
#define RT_EXPORTS_API extern "C" _declspec(dllimport)
#endif
#define ZYDLL __declspec(dllexport)
#else
#define ZYDLL __declspec(dllimport)
#endif
```

C#接口函数都在 DllInterfaceHelp.cs 类中进行了定义，定义格式如下所示：

```
[DllImport(@"RT_CV_Recognition.dll", EntryPoint = "RT_CV_DLL_Shap_check", CallingConvention = CallingConvention.Cdecl, CharSet = CharSet.Ansi)]
1 个引用
public static extern unsafe void RT_CV_DLL_Shap_check([In, Out] [MarshalAs(UnmanagedType.LPArray)] byte[] imgData, long dataLength, int height, int width, ref float str_center);
1 个引用
public unsafe void CV_DLL_Shap_check(byte[] imgData, long dataLength, int height, int width, ref float str_center)
{
    unsafe
    {
        try
        {
            RT_CV_DLL_Shap_check(imgData, dataLength, height, width, ref str_center);

        }
        catch (System.Exception ex)
        {
            MessageBox.Show(ex.ToString());
        }

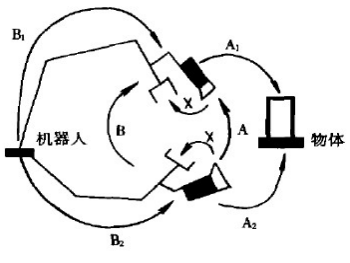
    }
}
```

[DllImport(@"RT_CV_Recognition.dll", EntryPoint = "RT_CV_DLL_Shap_check", CallingConvention = CallingConvention.Cdecl, CharSet = CharSet.Ansi)]//定义好动态链接库的名称和接口函数名称与 C++中函数返回值与参数一致。

将需要传参的数据通过在参数前加上“ref”按引用传递值。

七、像素坐标与机器人坐标标定方法

图像坐标与机器人坐标的标定是建立像素坐标与机器人坐标的矩阵转换关系 AX=XB，通过求解 X 就知道图像坐标与机器人坐标的转换关系。A 表示图像坐标矩阵，B 表示机器人坐标矩阵，X 就是转换关系矩阵。



关于标定的详细数学理论知识参考以下几个博客：

http://blog.csdn.net/PINBODEXIAOZHU/article/details/27495951 机器人 工具坐标系的标定
http://blog.csdn.net/yunlinwang/article/details/51871520 经典手眼标定算法之 Navy 的 OpenCV 实现

http://blog.csdn.net/yunlinwang/article/details/51622143 经典手眼标定算法之 Tsai-Lenz 的 OpenCV 实现

在项目中标定函数如下（去掉了打印输出代码）：

```
bool RT_CV_Recognition::calibrateCam(float *camCoordinates, float *handCoordinates, float
_transMat[])
{
    try
    {
        Point2f camPoints[3], handPoints[3];
        for (int i = 0; i < 3; i++)
        {
            camPoints[i] = Point2f(camCoordinates[2 * i], camCoordinates[2 * i + 1]);
//像素坐标 (3组)
            handPoints[i] = Point2f(handCoordinates[2 * i], handCoordinates[2 * i + 1]);
//机器人坐标 (3组)
        }
        Mat tmpMat = getAffineTransform(camPoints, handPoints);//矩阵仿射变换
        tmpMat.convertTo(tmpMat, CV_32FC1);
        for (int i = 0; i < tmpMat.rows; i++)
            for (int j = 0; j < tmpMat.cols; j++)
            {
                _transMat[i*tmpMat.cols + j] = tmpMat.at<float>(i, j);
            }
        transMat = tmpMat.clone();
        return true;
    }

}
```

手眼转换函数(去掉打印输出部分)：

```
void RT_CV_Recognition::transCoord(float*camCood, float handCood[])
{
        if (transMat.data == NULL)
```

```
            return;
            Mat camMat(3, 1, CV_32FC1, camCood);
            Mat handMat = transMat * camMat;//将相机坐标转换为机器人坐标
            handCood[0] = handMat.at<float>(0, 0);
            handCood[1] = handMat.at<float>(1, 0);
}
```

通过以上关键函数代码就可以了解整个标定的过程，第一部分是机器人坐标与像素坐标的关系求解，第二部分就是得到了像素坐标通过关系解计算出机器人坐标，发送给机器人就可以按照位置进行移动。

八、Socket 通讯

Socket 通讯是 TCP/IP 中的一种网络通讯方式，需要有客户端和服务端，两者之间需要先握手（建立通讯属性关系）才能相互收发数据，主要是稳定性高，但是不适合传输大文件，因为频繁握手效率低。所以 socket 的通讯机制大家一定要了解清楚，还有 UDP，一种广播的形式，没有握手机制，速度快，适用于传输大文件，但是稳定性安全性不高，因为没有握手机制。这些常用的网络通讯大家自己网上查阅，区别和用法讲的比我好太多，我知道怎么用，要说原理性的知识，在下实在是学术浅薄，不好意思了各位同学。如果你能坚持看到这里，并且将我所说所写的点理解到位，那么恭喜你机器视觉入门，你基本会了。按照我的方法可以开发常用的工业项目。大体的框架都差不多，只是需要做的处理的多少，写程序不是简单的考虑功能的实现，还是有软件操作的易用性，稳定性，后期调试维护等等，那么考虑到这些情况的时候，就需要额外增加很多处理机制在里面，废话不多说，下面开始 socket 通讯的讲解。

Socket 通讯我在程序中已经添加好了类 SocketHelper.cs（网上 copy 的），直接调用函数即可，知道怎么用就行。

将 SocketHelper.cs 类中的各个函数看一遍后，根据解释及函数名大概知道函数是做什么用的，这里就不再一一解释各个函数了，如果学到这里函数都看不懂，建议回头再学习，如果前面讲的都会了，不用说就知道，不要懒，自己多学多看多写就会了，你目前所有不懂的问题都可以在网上查找到，不要简单一句话看不懂就不管，那不是你会，而是你的懒惰，懒的去查找问题与解决问题。

首先在窗口载入的时候需要实例化客户端对象。（如果你问为什么要实例化对象，说明你前面的 C#白学了）。在程序中为了使用了一个定时器进行相关参数的初始化，使用定时器的目的是在于初始化的过程中不会出问题，因为相机初始化连接需要一定的时间，如果程序没做处理，相机没上就进行下面的相关操作就有可能出问题。

```
private void Form1_Load(object sender, EventArgs e)
        {
            label_Title.BringToFront();
            //this.WindowState = FormWindowState.Maximized;
            System.Timers.Timer t = new System.Timers.Timer(2000);//实例化Timer类, 设置间隔时间为1000毫秒 就是1秒;
            t.Elapsed += new System.Timers.ElapsedEventHandler(timer_init);//到达时间的时候执行事件;
            t.AutoReset = false;//设置是执行一次 (false) 还是一直执行(true);
            t.Enabled = true;//是否执行System.Timers.Timer.Elapsed事件;
```

```
            send_datas_list.Clear();
            tabControl1.SelectedTab = tabControl1.TabPages[0];
            }
```

在 timer_init 函数中进行了通讯的相关操作，创建 socket 连接，实例化客户端对象，绑定 IP 和端口号。

```
public void timer_init(object source, System.Timers.ElapsedEventArgs e)
{
      #region 通讯初始化
           SocketHelper.pushSockets = new SocketHelper.PushSockets(Rec);//建立socket连接
            client = new SocketHelper.TcpClients();
            ip = configdata.server_Ip;
            port = configdata.server_Port;
            txtIP.Text= configdata.server_Ip;
            txtPort.Text= configdata.server_Port;
            try
            {
                client.InitSocket(ip, int.Parse(port));//绑定端口与IP
                client.Start();
                lxb_show.Items.Add("连接服务器成功.!");
            }
            catch (Exception ex)
            {
                lxb_show.Items.Add(string.Format("启动失败!原因: {0}", ex.Message));
            }
            #endregion
            System_Init_Main();//获取主显示句柄
            }
    }
```

以上就是建立 socket 通讯的相关操作，建立通讯后就可以发送数据，函数如下所示：

```
private void Send_Robot_Datas()
        {
            while (true)
            {
                if (receive_robot_datas=="1")//如果机器人发"1"就发送数据
                {
                    if (send_datas_list.Count()>0)
                    {
                        receive_robot_datas = "0";
                        string[] data = send_datas_list.ElementAt(0);
                        send_datas_list.RemoveAt(0);
                        try
                        {
                            client.SendData(data[0] + "," + data[1] + "," + data[2]);//
发送数据
```

```
                    lxb_show.Items.Add(data[0] + "," + data[1] + "," + data[2]);
                }
                catch (Exception)
                {
                    if (lxb_show.InvokeRequired)
                    {
                        this.Invoke(new EventHandler(delegate
                        {
                            lxb_show.Items.Add("机器人没有打开通信端口");
                        }));
                    }
                    else
                    {
                        lxb_show.Items.Add("机器人没有打开通信端口");
                    }
                }
            }
        }
        Thread.Sleep(10);
    }

}
```

总结：以上就是 C++与 C#联合开发的机器视觉项目。开发的架构，希望大家了解，对以后自己进行工业项目开发有实际的帮助。以上的先关知识搞懂后，不管是上位机开发，界面开发，图像处理算法开发都可以基于此种方式开发。这只是简单的机器视觉入门，但是大的框架是不变的。所以在学习完以上的知识后，能运用此框架在自己的项目中后，就着手重点知识的学习，比如数字图像处理的相关知识，涉及到 C++的开发非常多，而且比较复杂，这需要自己去沉淀。懂了相机 SDK 的开发，其他的相关硬件的开发都是差不多，不管是相机还是 PLC，机器人（ABB 也有上位机控制程序，通过调用 SDK 直接控制机器人）开发的方式都差不多，官网提供先关的 SDK 开发说明照着写代码即可。所以这部分的知识是应用，知道怎么用就行。希望同学们好好学习，对你们以后科研或是工作都有极大的帮助，写的都是实用的东西，就是工业常用的，所以你们学习后，就可以进行相关项目的开发。

附件一 树莓派语音识别源代码

```
# encoding: utf-8

import socket                    # 导入 socket 通讯模块

import smbus                     #smbus 通讯协议
```

```
import time

import numpy        #NumPy 是 Python 语言的一个扩展程序库，支持大量的维度数组与矩阵运算，此外也针对数组运算提供大量的数学函数库。

s = socket.socket()                    # 创建 socket 对象

host = '10.70.2.252'                   # 填写服务端的 ip 地址

port = 3002                            # 填入设置好的端口号

try:                                   #如果当 try 后的语句执行时发生异常，python 就跳回到 try 并执行第一个匹配该异常的 except 子句，异常处理完毕，控制流就通过整个 try 语句（除非在处理异常时又引发新的异常）。

    s.connect((host, port))            #去连接服务器端的 ip 和端口

    print("连接到服务器")

except:

    print("连接不成功")

    exit(1)

class ASR:                             #一个 ASR 的类

    # Global Variables

    address = None

    bus = None

    ASR_RESULT_ADDR = 100

    #识别结果存放处，通过不断读取此地址的值判断是否识别到语音，不同的值对应不同的语音

    ASR_WORDS_ERASE_ADDR = 101

    #擦除所有词条
```

ASR_MODE_ADDR = 102

#识别模式设置，值范围 1~3

#1：循环识别模式。状态灯常亮（默认模式）

#2：口令模式，以第一个词条为口令。状态灯常灭，当识别到口令词时常亮，等待识别到新的语音,并且读取识别结果后即灭掉

#3：按键模式，按下开始识别，不按不识别。支持掉电保存。状态灯随按键按下而亮起，不按不亮

ASR_ADD_WORDS_ADDR = 160

#词条添加的地址，支持掉电保存

```
def __init__(self, address, bus=1):
        self.address = address
        self.bus = smbus.SMBus(bus)
          def readByte(self):
        return self.bus.read_byte(self.address)
    def writeByte(self, val):
        value = self.bus.write_byte(self.address, val)
        if value != 0:
              return False
        return True
def writeData(self, reg, val):
        self.bus.write_byte(self.address,    reg)
        self.bus.write_byte(self.address,    val)
    def getResult(self):                                    #识别结果函数
```

```
        if ASR.writeByte(self, self.ASR_RESULT_ADDR):
            return -1
        value = self.bus.read_byte(self.address)
        return value
    '''
    *  添加词条函数，
    * idNum：词条对应的识别号，1~255 随意设置。识别到该号码对应的词条语音时，
    *          会将识别号存放到 ASR_RESULT_ADDR 处，等待主机读取，读取后清 0
    * words：要识别汉字词条的拼音，汉字之间用空格隔开
    *  执行该函数，词条是自动往后排队添加的。
    '''
    def addWords(self, idNum, words):        #添加词条地址函数
        buf = [idNum]
        for i in range(0, len(words)):
            buf.append(eval(hex(ord(words[i]))))
        self.bus.write_i2c_block_data(self.address, self.ASR_ADD_WORDS_ADDR, buf)
        time.sleep(0.05)
    def eraseWords(self):    #擦出词条函数
        result = self.bus.write_byte_data(self.address, self.ASR_WORDS_ERASE_ADDR,
0)      time.sleep(0.06)
        if result != 0:
            return False
        return True
```

```
 def setMode(self, mode):     #识别模式函数
        result = self.bus.write_byte_data(self.address, self.ASR_MODE_ADDR, mode)
        if result != 0:
            return False
        return True
def dd():
    addr = 0x79                  #连接语音传感器 iic 地址
    asr = ASR(addr)              #进行实例化
    if 1:
        asr.eraseWords()         #擦出词条
        asr.setMode(1)           #识别模式为 1
        asr.addWords(1, 'shang shen')    #说上升口令时，在程序中赋予词条的地址是 1
        asr.addWords(2, 'xia jiang')     #说下降口令时，在程序中赋予词条的地址是 2
        asr.addWords(3, 'qian jing')     #说左边口令时，在程序中赋予词条的地址是 3
        asr.addWords(4, 'hou tui')       #说右边口令时，在程序中赋予词条的地址是 4
        asr.addWords(5, 'zuo bian')      #说右边口令时，在程序中赋予词条的地址是 5
        asr.addWords(6, 'you bian')      #说后退口令时，在程序中赋予词条的地址是 6
        asr.addWords(7, 'tui chu')       #说退出口令时，在程序中赋予词条的地址是 7
#第一位：实验顺序  第二位：运行模式   第三位：控制 z 轴动作  第四位：控制方向向上  第五位：步长 10 个
    while 1:
        data = asr.getResult()                          #将得到的识别结果赋值给 data
        print("result:", data)                          #在终端打打印出 data 的值
```

```
if data==1:                                    #如果 data 值等于 1
    a = [0x02,0x02,0x03,0x01,0x10]             #给网口发送上升动作的 16 进制
    b = ""
    for i in range(len(a)):
        b +=chr(a[i])                          #将 a 的 16 进制数据格式转化成 ascii 码
    s.send(b.encode("utf-8"))                  #将数据发送给服务端
    time.sleep(0.1)                            #发送成功后程序休眠一定时间
    # print("shang shen")
elif data==2:                                  #如果 data 的值等于 2
    a = [0x02,0x02,0x03,0x02,0x10]             #给串口发送下降动作的 16 进制
    b = ""
    for i in range(len(a)):
        b +=chr(a[i])                          #将 a 的 16 进制数据格式转化成 ascii 码
    s.send(b.encode("utf-8"))                  #将数据发送给服务端
    time.sleep(0.1)
    # print("xia jiang")
elif data==3:                                  #如果 data 的值等于 3
    a = [0x02,0x02,0x02,0x01,0x10]             #给串口发送下降动作的 16 进制
    b = ""
    for i in range(len(a)):
        b +=chr(a[i])                          #将 a 的 16 进制数据格式转化成 ascii 码
    s.send(b.encode("utf-8"))                  #将数据发送给服务端
```

```
        time.sleep(0.1)
        # print("qian jing")
    elif data==4:                                   #如果 data 的值等于 4
        a = [0x02,0x02,0x02,0x02,0x10]              #给串口发送右边动作的 16 进制
        b = ""
        for i in range(len(a)):
            b +=chr(a[i])                           #将 a 的 16 进制数据格式转化成 ascii 码
        s.send(b.encode("utf-8"))                   #将数据发送给服务端
        time.sleep(0.1)
        #print("hou tui")
    elif data==5:
        a = [0x02,0x02,0x01,0x01,0x10]              #给串口发送左边动作的 16 进制
        b = ""
        for i in range(len(a)):
            b +=chr(a[i])
        s.send(b.encode("utf-8"))
        time.sleep(0.1)
        # print("zuo bian")
    elif data==6:
        a = [0x02,0x02,0x01,0x02,0x10]              #给串口发送右边动作的 16 进制
        b = ""
        for i in range(len(a)):
```

```
                b +=chr(a[i])

            s.send(b.encode("utf-8"))

            time.sleep(0.1)

            # print("you bian")

        elif data==7:

            #s.send("4".encode("utf-8"))

            break;                                  #如果 data 的值是 7。退出当前循环

            print("tui chu")

            time.sleep(0.1)

        time.sleep(0.2)

if __name__ == "__main__":

    dd()
```

附件二 树莓派手势识别源代码

```
#!/usr/bin/python3
#coding:utf8
import cv2
import numpy as np
import time
import math
import threading
import timeout_decorator
from data import check_camera
import socket                    # 导入 socket 模块
debug = True
Running = True
s = socket.socket()              # 创建 socket 对象
host = '10.70.2.252' # 获取本地主机名
port = 3002                      # 设置端口号
try:
    s.connect((host, port))
```

```
    print("连接到服务器")
except:
    print("连接不成功")
    exit(1)
#print (s.recv(1024))

#摄像头默认分辨率 640x480,处理图像时会相应的缩小图像进行处理，这样可以加快运行速度
#缩小时保持比例 4：3,且缩小后的分辨率应该是整数
c = 80
width, height = c*6, c*5

class Point(object):#一个坐标点
    x = 0
    y = 0
    def __init__(self, x=0, y=0):
        self.x = x
        self.y = y
class Line(object):#一条线
    def __init__(self, p1, p2):
        self.p1 = p1
        self.p2 = p2
def GetCrossAngle(l1, l2):
    '''
    求两直线之间的夹角
    :param l1:
    :param l2:
    :return:
    '''
    arr_0 = np.array([(l1.p2.x - l1.p1.x), (l1.p2.y - l1.p1.y)])
    arr_1 = np.array([(l2.p2.x - l2.p1.x), (l2.p2.y - l2.p1.y)])
    cos_value = (float(arr_0.dot(arr_1)) / (np.sqrt(arr_0.dot(arr_0)) * np.sqrt(arr_1.dot(arr_1))))  # 注意转成浮点数运算
    return np.arccos(cos_value) * (180/np.pi)
def two_distance(start, end):
    """
    计算两点的距离
    :param start: 开始点
    :param end: 结束点
    :return: 返回两点之间的距离
    """
    s_x = start[0]
    s_y = start[1]
    e_x = end[0]
```

```
    e_y = end[1]
    x = s_x - e_x
    y = s_y - e_y
    return math.sqrt((x**2)+(y**2))
#cv2.namedWindow('YCC_frame')
#cv2.namedWindow('Cr')
#cv2.namedWindow('Cr_inrange')
#cv2.namedWindow('opend')
cv2.namedWindow('orgframe')
#cv2.moveWindow('YCC_frame',0, 100)
#cv2.moveWindow('Cr',0, 100 + height + 75)
#cv2.moveWindow('Cr_inrange', (width + 125), 100)
#cv2.moveWindow('opend', (width + 125), 100 + height + 75)
#cv2.moveWindow('orgframe', 2*(width + 125), 100)
cv2.moveWindow('orgframe', 500, 500)
def image_process(image):#hsv
    '''
    # 光线影响，请修改 cb 的范围
    # 正常黄种人的 Cr 分量大约在 140~160 之间
    识别肤色
    :param image: 图像
    :return: 识别后的二值图像
    '''
    YCC_frame = cv2.cvtColor(image, cv2.COLOR_BGR2YCR_CB)  # 将图片转化为
YCrCb
    #cv2.imshow('YCC_frame', YCC_frame)
    Y, Cr, Cb = cv2.split(YCC_frame)  # 分割 YCrCb
    #cv2.imshow('Cr', Cr)
    Cr_inrange = cv2.inRange(Cr, 132, 175)
    #cv2.imshow('Cr_inrange', Cr_inrange)
    # 开运算，去除噪点
    open_element = cv2.getStructuringElement(cv2.MORPH_RECT, (5, 5))
    opend = cv2.morphologyEx(Cr_inrange, cv2.MORPH_OPEN, open_element)
    #cv2.imshow('opend', opend)
    return opend
def get_defects_far(defects, contours, img):
    '''
    获取凸包中最远的点
    '''
    if defects is None and contours is None:
        return None
    far_list = []
    for i in range(defects.shape[0]):
        s, e, f, d = defects[i, 0]
```

```
            start = tuple(contours[s][0])
            end = tuple(contours[e][0])
            far = tuple(contours[f][0])
            # 求两点之间的距离
            a = two_distance(start, end)
            b = two_distance(start, far)
            c = two_distance(end, far)
            # print('a=', a)
            # print('b= ', b)
            # print('c= ', c)
            # 求出手指之间的角度
            angle = math.acos((b ** 2 + c ** 2 - a ** 2) / (2 * b * c)) * 180 / math.pi
            # 手指之间的角度一般不会大于 100 度
            # 小于 90 度
            if angle <= 75:   # 90:
                # cv.circle(img, far, 10, [0, 0, 255], 1)
                far_list.append(far)
    return far_list
def get_max_coutour(cou, max_area):
    '''
    找出最大的轮廓
    根据面积来计算，找到最大后，判断是否小于最小面积，如果小于侧放弃
    :param cou: 轮廓
    :return: 返回最大轮廓
    '''
    max_coutours = 0
    r_c = None
    if len(cou) < 1:
        return None
    else:
        for c in cou:
            # 计算面积
            temp_coutours = math.fabs(cv2.contourArea(c))
            if temp_coutours > max_coutours:
                max_coutours = temp_coutours
                cc = c
        # 判断所有轮廓中最大的面积
        if max_coutours > max_area:
            r_c = cc
        return r_c
def find_contours(binary, max_area):
    '''
    mode    提取模式.
    CV_RETR_EXTERNAL -  只提取最外层的轮廓
```

CV_RETR_LIST - 提取所有轮廓，并且放置在 list 中

CV_RETR_CCOMP - 提取所有轮廓，并且将其组织为两层的 hierarchy: 顶层为连通域的外围边界，次层为洞的内层边界。

CV_RETR_TREE - 提取所有轮廓，并且重构嵌套轮廓的全部 hierarchy

method 逼近方法 (对所有节点, 不包括使用内部逼近的 CV_RETR_RUNS).

CV_CHAIN_CODE - Freeman 链码的输出轮廓. 其它方法输出多边形(定点序列).

CV_CHAIN_APPROX_NONE - 将所有点由链码形式翻译(转化）为点序列形式

CV_CHAIN_APPROX_SIMPLE - 压缩水平、垂直和对角分割，即函数只保留末端的象素点;

CV_CHAIN_APPROX_TC89_L1,

CV_CHAIN_APPROX_TC89_KCOS - 应用 Teh-Chin 链逼近算法. CV_LINK_RUNS - 通过连接为 1 的水平碎片使用完全不同的轮廓提取算法

```
    :param binary: 传入的二值图像
    :return: 返回最大轮廓
    '''
    # 找出所有轮廓
    contours = cv2.findContours(binary, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_NONE)[-2]
    # 返回最大轮廓
    return get_max_coutour(contours, max_area)
def get_hand_number(binary_image, rgb_image):
    '''
    返回手指的个数
    :param binary_image:
    :param rgb_image:
    :return:
    '''
    # # 2、找出手指尖的位置
    # # 查找轮廓，返回最大轮廓
    x = 0
    y = 0
    contours = find_contours(binary_image, 1000)
    coord_list = []
    if contours is not None:
        # 周长 0.035 根据识别情况修改，识别越好，越小
        epsilon = 0.025 * cv2.arcLength(contours, True)
        # 轮廓相似
        approx = cv2.approxPolyDP(contours, epsilon, True)
        #cv2.approxPolyDP()的参数 2(epsilon)是一个距离值，表示多边形的轮廓接近实际轮廓的程度，值越小，越精确；参数 3 表示是否闭合
        cv2.polylines(rgb_image, [approx], True, (0, 255, 0), 1)#画多边形

        if approx.shape[0] >= 3:  # 有三个点以上#多边形最小为三角形，三角形需要三个点
```

```
                approx_list = []
                for j in range(approx.shape[0]):#将多边形所有的点储存在一个列表里
                    # cv2.circle(rgb_image, (approx[j][0][0],approx[j][0][1]), 5, [255, 0, 0], -1)
                    approx_list.append(approx[j][0])
                approx_list.append(approx[0][0])      # 在末尾添加第一个点
                approx_list.append(approx[1][0])      # 在末尾添加第二个点

                for i in range(1, len(approx_list) - 1):
                    p1 = Point(approx_list[i - 1][0], approx_list[i - 1][1])      # 声明一个点
                    p2 = Point(approx_list[i][0], approx_list[i][1])
                    p3 = Point(approx_list[i + 1][0], approx_list[i + 1][1])
                    line1 = Line(p1, p2)      # 声明一条直线
                    line2 = Line(p2, p3)
                    angle = GetCrossAngle(line1, line2)        # 获取两条直线的夹角
                    angle = 180 - angle        #
                    # print angle
                    if angle < 42:   # 求出两线相交的角度，并小于 37 度的
                        #cv2.circle(rgb_image, tuple(approx_list[i]), 5, [255, 0, 0], -1)
                        coord_list.append(tuple(approx_list[i]))
            # 去除手指间的点
            # 1、获取凸包缺陷点，最远点点
            # cv.drawContours(rgb_image, contours, -1, (255, 0, 0), 1)
            hull = cv2.convexHull(contours, returnPoints=False)
            # 找凸包缺陷点 。返回的数据， 【起点，终点， 最远的点， 到最远点的近似距离】
            defects = cv2.convexityDefects(contours, hull)
            # 返回手指间的点
            hand_coord = get_defects_far(defects, contours, rgb_image)
            # print 'h', hand_coord
            # print 'c', coord_list
            # 2、从 coord_list 去除最远点
            new_hand_list = []   # 获取最终的手指间坐标
            alike_flag = False
            if len(coord_list) > 0:
                for l in range(len(coord_list)):
                    for k in range(len(hand_coord)):
                        if (-10 <= coord_list[l][0] - hand_coord[k][0] <= 10 and
                                -10 <= coord_list[l][1] - hand_coord[k][1] <= 10):      # 最比较 X,Y 轴，相近的去除
                            alike_flag = True
                            break     #
                    if alike_flag is False:
                        new_hand_list.append(coord_list[l])
                    alike_flag = False
```

```
                # print new_hand_list
                #  获取指尖的坐标列表并显示
                for i in new_hand_list:
                    cv2.circle(rgb_image, tuple(i), 5, [0, 0, 100], -1)
            if new_hand_list is []:
                return 0
            else:
                return len(new_hand_list)
        else:
            return 0
@timeout_decorator.timeout(0.5, use_signals=False)
def Camera_isOpened():
    global stream, cap
    cap = cv2.VideoCapture(stream)
orgFrame = None
Running = True
ret = False
#stream = "http://127.0.0.1:8080/?action=stream?dummy=param.mjpg"
stream = 0
try:
    Camera_isOpened()
    cap = cv2.VideoCapture(stream)
except:
    print('Unable to detect camera! \n')
    check_camera.CheckCamera()

def get_image():
    global orgFrame
    global ret
    global Running
    global stream, cap
    while True:
        if Running:
            try:
                if cap.isOpened():
                    ret, orgFrame = cap.read()
                else:
                    time.sleep(0.01)
            except:
                cap = cv2.VideoCapture(stream)
                print('Restart Camera Successful!')
        else:
            time.sleep(0.01)
th1 = threading.Thread(target = get_image)
```

```
th1.setDaemon(True)
th1.start()
#执行动作组
get_hand_num = False
action_finish = True
def runAction():
    print("laile")
    global get_hand_num, action_finish
    global num_mean
    while True:
        #time.sleep(1)
        if get_hand_num:
            #print(1)
            get_hand_num = False
            action_finish = False
            if num_mean == 1:
                a = [0x01,0x02,0x03,0x01,0x10]
                b = ""
                for i in range(len(a)):
                    b +=chr(a[i])
                s.send(b.encode("utf-8"))
                print("1")
                num_maen = 0
                time.sleep(2)
                action_finish = True

            elif num_mean == 2:
                a = [0x02,0x02,0x03,0x02,0x10]
                b = ""
                for i in range(len(a)):
                    b +=chr(a[i])
                s.send(b.encode("utf-8"))
                print("2")
                num_mean = 0
                time.sleep(2)
                action_finish = True

            elif num_mean == 3:
                a = [0x03,0x02,0x01,0x01,0x10]
                b = ""
                for i in range(len(a)):
                    b +=chr(a[i])
                s.send(b.encode("utf-8"))
                print("3")
```

```
                num_mean = 0
                time.sleep(2)
                action_finish = True
            elif num_mean == 4:
                a = [0x04,0x02,0x01,0x02,0x10]
                b = ""
                for i in range(len(a)):
                    b +=chr(a[i])
                s.send(b.encode("utf-8"))
                print("4")
                num_mean = 0
                time.sleep(2)
                action_finish = True
            elif num_mean == 5:
                a = [0x0,0x02,0x02,0x01,0x10]
                b = ""
                for i in range(len(a)):
                    b +=chr(a[i])
                s.send(b.encode("utf-8"))
                print("5")
                num_mean = 0
                time.sleep(2)
                action_finish = True
            else:
                action_finish = True
                time.sleep(0.01)
        else:
            #print(2)
            time.sleep(0.01)

#启动动作在运行线程
th2 = threading.Thread(target=runAction)
th2.setDaemon(True)
th2.start()


num = []
num_mean = 0
while True:
  if orgFrame is not None and ret:
    t1 = cv2.getTickCount()
    orgframe = cv2.resize(orgFrame, (width,height), interpolation = cv2.INTER_CUBIC) #
将图片缩放
    frame = orgframe
```

```
        binary = image_process(frame)#处理图像
        hand_num = get_hand_number(binary, frame)#获取手指个数
        num.append(hand_num)
        if len(num) == 10:
            num_mean = int(round(np.mean(num)))
            num = num[1:]
            if action_finish:
                get_hand_num = True
        t2 = cv2.getTickCount()
        time_r = (t2 - t1) / cv2.getTickFrequency()
        fps = 1.0/time_r
        cv2.putText(orgframe, "FPS:" + str(int(fps)),
                    (10, orgframe.shape[0] - 10), cv2.FONT_HERSHEY_SIMPLEX, 0.65, (0, 0,
255), 2)     #(0, 0, 255)BGR
        cv2.putText(orgframe, "Current:" + str(num_mean),
                    (10, orgframe.shape[0] - 30), cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 0,
255), 2)#显示当前检测到的手指个数，(0, 0, 255)BGR
        cv2.imshow("orgframe", orgframe)
        if cv2.waitKey(1) & 0xff == ord('q'):
            break;

        #cv2.waitKey(1)
    else:
        time.sleep(0.01)
cap.release()
cv2.destroyAllWindows()
exit(1)
```